

版本信息:

版本 REV2022

时间 04/26/2022

米联客 2022 版 AXI4 总线专题

Milianke 2022 Ultrascale+ MPSOC Course

电子版自学资料

常州一二三电子科技有限公司

溧阳米联电子科技有限公司

版权所有

米联客(MILIANKE)04QQ 群: 516869816

米联客(MILIANKE)03QQ 群: 543731097

米联客(MILIANKE)02QQ 群: 86730608

米联客(MILIANKE)01QQ 群: 34215299

微信扫码注册米联客技术论坛www.uisrc.com 免费享受更多资源



扫码关注微信公众平台“米联客(MILIANKE)”掌握更多信息动态



版本	时间	描述
Rev2016	2016-07-03	第一版
Rev2019	2019-05-09	第二版
Rev2020	2020-05-23	第三版
Rev2022	2022-04-26	第四版

序 1:

FPGA 芯片是硬件技术而 FPGA 编程又称为硬件编程语言和流行的各类软件编程语言 C/C++、JAVA、python 等相比, 掌握基础的硬件编程语言不是难事, 难点是 FPGA 在每个专业领域的应用, 只有充分理解了 FPGA, 并且具有对自己所处行业专业背景认知, 才能真正理解 FPGA 应该用在什么场合更加合适。

从业多年来, 亲身经历了 FPGA 的发展历程, 也深刻体会到未来 FPGA 应用领域可能发生的深刻变革, FPGA 从简单的逻辑门, 发展到现在具备很多高速通信接口, 而且最新 SOC 中也集成了 FPGA 单元, 实现了 ARM 和 FPGA 单芯片。目前 XILINX 代表了业内领先的 FPGA 技术, 已经可以把 FPGA\ARM\GPU\RFDID 等集成到单芯片。FPGA 的逻辑资源也是达到了前所未有的密度以及超大容量。

很多人问我学习 FPGA 是否有前途, 这个问题着实难以回答。我们可以一起来探讨以下几种情况, 会是 FPGA 发挥作用的场合:

1)、数字 IC 设计工作

数字 IC 设计还是主要以硬件编程语言去设计数字 IC 芯片, 通过硬件编程语言, 软件可以把语言翻译成电路, 自动布线工具可以完成布局布线, 之后流片。由于流片费用非常贵, 前期也可以用 FPGA 芯片模拟设计的数字 IC 芯片的功能。

2)、高速模数字信号采集分析

高速的 ADC,DAC 的数模信号处理的领域也是必然需要用到 FPGA, 在无线通信、雷达信号处理等领域也都会用到。

3)、数字信号高速通信

FPGA 具备的高速接口也非常适合用于高速通信, 比如 PCIE 通信、光通信、以太网通信

4)、视频图像

包括图像的拼接、缩放、高效的实时传输等, 4K 视频、8K 视频领域

5)、硬件加速算法

FPGA 的硬件加速领域也是目前的热门研究, 也是 FPGA 未来最有前景的一个应用领域, 已经有很多公司利用 FPGA 的硬件加速实现了很好的经济效应, 但是目前 FPGA 的加速还没有做到普及, 和传统的 GPU 相比, 主要难度还是在开发难度上, 一般小公司很难有实力取得突破。

6)、通用 CPU GPU 无法完成的工作

如果通用的 CPU 和 GPU 无法完成的工作任务, 可以考虑下 FPGA 或者带 FPGA 的 SOC.

FPGA 到目前为止依然是一个小众的领域, 如果专门为了学习 FPGA 而学习 FPGA 而不知道如何应用 FPGA, 那么这是非常悲哀的一件事情, 学习 FPGA 只是学习一门技能, 而结合自己专业背景选择最合适的解决方案, 解决问题才是最终的目的。

序 2:

米联客团队励志在 FPGA 领域可以贡献一份自己的力量, 让 FPGA 从业者入门槛降低、FPGA 应用难度减少, 为中小型 FPGA 团队提供必要有价值的技术资料 and 硬件支持。我们希望能和广大客户形成紧密的合作伙伴关系, 一起创造共赢, 各自实现自己的价值目标。

从 2019 年下半年开始我们米联客团队计划研发全新的 2020 版本教程。经过一年半时间的更新, 2020 版本教程研发进度进入了关键攻坚时期。

2020 版本教程是适应当前 FPGA 技术发展, SOC 技术发展, 新形势下, 米联客做出的战略决策。2020 版本教程需要解决以下几个问题:

- 1)、FPGA 基础课程, 需要解决长期以来没有认真解决好的课程内容, 包括: FPGA 的构架、FPGA 常用 IP 使用、硬件编程经验、通信接口应用、代码规范、时序分析
- 2)、新版本 vitis 软件的使用技巧
- 3)、更多讲解高速通信的基础知识和应用解决方案
- 4)、ZYNQ-SOC 到 ZYNQ-MPSOC 教程统一部署, 达到 90%以上 demo 使用方法一样。不管是学习 ZYNQ-SOC 还是 ZYNQ-MPSOC, 已经学习的 demo 具有 2 个平台之间的互通性。
- 5)、ZYNQ 和 MPSOCLinux 课程做到适合初级入门, 并提供丰富的应用 demo
- 6)、适应未来发展趋势, 增加 FPGA 高层次加速算法编程领域的课程内容
- 7)、教程的构架能够支持 7 系列 FPGA、UltraScale 系列 FPGA、UltraScale+系列 FPGA 和 MPSOC

不管有多麻烦, 不管困难多大, 面对挑战我们坚信胜利!

米联客团队
2021 年 4 月 26 日

目录

米联客 2020 版 AXI4 总线专题	1
01AXI4 总线专题之 axi-lite-slave	2
1.1 概述	2
1.2AXI 总线协议介绍	2
1.2.1AXI 总线概述	2
1.2.2AXI-4 总线信号功能	2
1.2.3 数据有效的情况	4
1.2.4 突发式读写	5
1.3 创建 AXI4-Lite 总线接口 IP	6
1.4 程序分析	9
1:axi-lite-slave 的 axi_awready	10
2:axi-lite-slave 的 axi_awaddr	10
3:axi-lite-slave 的写数据寄存器	11
4:axi-lite-slave 的 axi_bvalid 信号	12
5:axi-lite-slave 的 axi_arready	13
6:axi-lite-slave 的 axi_araddr	13
7:axi-lite-slave 的读数据寄存器	14
1.5 实验结果	16
附录常见问题 1:	错误！未定义书签。

01AXI4 总线 axi-lite-slave

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1.1 概述

使用 XILINX 的软件工具 VIVADO 以及 XILINX 的 7 代以上的 FPGA 或者 SOC 掌握 AXI-4 总线结束, 并且可以灵活使用 AXI-4 总线技术完成数据的交换, 可以让我们在构建强大的 FPGA 内部总线数据互联通信方面取得高效、高速、标准化的优势。

本文实验目的:

- 1:学习 AXI 总线协议包括 AXI-FULL、AXI-Lite
- 2:掌握基于 VIVADO 工具产生 AXI 协议模板
- 3:掌握通过 VIVADO 工具产生 AXI-lite-Slave 代码, 并且会修改寄存器
- 4:理解 AXI-lite-Slave 中自定义寄存器的地址分配
- 5:掌握通过 VIVADO 封装 AXI-LITE-SLAVE 图形化 IP
- 6:通过仿真验证 AXI-LITE IP 的工作是否正常。

1.2AXI 总线协议介绍

1.2.1AXI 总线概述

在 XILINX FPGA 的软件工具 vivado 以及相关 IP 中有支持三种 AXI 总线, 拥有三种 AXI 接口, 当然用的都是 AXI 协议。其中三种 AXI 总线分别为:

AXI4: (For high-performance memory-mapped requirements.) 主要面向高性能地址映射通信的需求, 是面向地址映射的接口, 允许最大 256 轮的数据突发传输;

AXI4-Lite: (For simple, low-throughput memory-mapped communication) 是一个轻量级的地址映射单次传输接口, 占用很少的逻辑单元。

AXI4-Stream: (For high-speed streaming data.) 面向高速流数据传输; 去掉了地址项, 允许无限制的数据突发传输规模。

由于 AXI4 和 AXI4-Lite 信号大部分一样, 以下只介绍 AXI4 信号.另外对于 AXI4-Stream 协议不再本文中接收, 后面有单独介绍的文章。

1.2.2AXI-4 总线信号功能

1:时钟和复位

信号	方向	描述
ACLK	时钟源	全局时钟信号
ARESETn	复位源	全局复位信号, 低有效

写地址通道信号:

信号	方向	描述
AWID	主机 to 从机	写地址 ID, 用来标志一组写信号
AWADDR	主机 to 从机	写地址, 给出一次写突发传输的写地址

AWLEN	主机 to 从机	AWLEN[7:0]决定写传输的突发长度。AXI3 只支持 1~16 次的突发传输 (Burst_length=AxLEN[3:0]+1)，AXI4 扩展突发长度支持 INCR 突发类型为 1~256 次传输，对于其他的传输类型依然保持 1~16 次突发传输 (Burst_Length=AxLEN[7:0]+1)。 burst 传输具有如下规则： wrapping burst ,burst 长度必须是 2,4,8,16 burst 不能跨 4KB 边界 不支持提前终止 burst 传输
AWSIZE	主机 to 从机	写突发大小，给出每次突发传输的字节数支持 1、2、4、8、16、32、64、128
AWBURST	主机 to 从机	突发类型： 2'b00 FIXED: 突发传输过程中地址固定，用于 FIFO 访问 2'b01 INCR : 增量突发，传输过程中，地址递增。增加量取决 AxSIZE 的值。 2'b10 WRAP: 回环突发，和增量突发类似，但会在特定高地址的边界处回到低地址处。回环突发的长度只能是 2,4,8,16 次传输，传输首地址和每次传输的大小对齐。最低的地址整个传输的数据大小对齐。回环边界等于 (AxSIZE*AxLEN) 2'b11 Reserved
AWLOCK	主机 to 从机	总线锁信号，可提供操作的原子性
AWCACHE	主机 to 从机	内存类型，表明一次传输是怎样通过系统的
AWPROT	主机 to 从机	保护类型，表明一次传输的特权级及安全等级
AWQOS	主机 to 从机	质量服务 QoS
AWREGION	主机 to 从机	区域标志，能实现单一物理接口对应的多个逻辑接口
AWUSER	主机 to 从机	用户自定义信号
AWVALID	主机 to 从机	有效信号，表明此通道的地址控制信号有效
AWREADY	从机 to 主机	表明“从”可以接收地址和对应的控制信号

2: 写数据通道信号:

信号名	方向	描述
WID	主机 to 从机	一次写传输的 ID tag
WDATA	主机 to 从机	写数据
WSTRB	主机 to 从机	WSTRB[n:0]对应于对应的写字节，WSTRB[n]对应 WDATA[8n+7:8n]。WVALID 为低时，WSTRB 可以为任意值，WVALID 为高时，WSTRB 为高的字节线必须指示有效的数据。
WLAST	主机 to 从机	表明此次传输是最后一个突发传输
WUSER	主机 to 从机	用户自定义信号
WVALID	主机 to 从机	写有效，表明此次写有效
WREADY	从机 to 主机	表明从机可以接收写数据

写响应信号:

信号名	方向	描述
BID	从机 to 主机	写响应 ID tag
BRESP	从机 to 主机	写响应，表明写传输的状态
BUSER	从机 to 主机	用户自定义
BVALID	从机 to 主机	写响应有效
BREADY	主机 to 从机	表明主机能够接收写响应

3: 读地址通道信号:

信号	方向	描述
ARID	主机 to 从机	读地址 ID，用来标志一组写信号

ARADDR	主机 to 从机	读地址，给出一次读突发传输的读地址
ARLEN	主机 to 从机	ARLEN[7:0]决定读传输的突发长度。AXI3 只支持 1~16 次的突发传输 (Burst_length=AxLEN[3:0]+1)，AXI4 扩展突发长度支持 INCR 突发类型为 1~256 次传输，对于其他的传输类型依然保持 1~16 次突发传输 (Burst_Length=AxLEN[7:0]+1)。burst 传输具有如下规则： wrapping burst ,burst 长度必须是 2,4,8,16 burst 不能跨 4KB 边界 不支持提前终止 burst 传输
ARSIZE	主机 to 从机	读突发大小，给出每次突发传输的字节数支持 1、2、4、8、16、32、64、128
ARBURST	主机 to 从机	突发类型： 2'b00 FIXED: 突发传输过程中地址固定，用于 FIFO 访问 2'b01 INCR : 增量突发，传输过程中，地址递增。增加量取决 AxSIZE 的值。 2'b10 WRAP: 回环突发，和增量突发类似，但会在特定高地址的边界处回到低地址处。回环突发的长度只能是 2,4,8,16 次传输，传输首地址和每次传输的大小对齐。最低的地址整个传输的数据大小对齐。回环边界等于 (AxSIZE*AxLEN) 2'b11 Reserved
ARLOCK	主机 to 从机	总线锁信号，可提供操作的原子性
ARCACHE	主机 to 从机	内存类型，表明一次传输是怎样通过系统的
ARPROT	主机 to 从机	保护类型，表明一次传输的特权级及安全等级
ARQOS	主机 to 从机	质量服务 QoS
ARREGION	主机 to 从机	区域标志，能实现单一物理接口对应的多个逻辑接口
ARUSER	主机 to 从机	用户自定义信号
ARVALID	主机 to 从机	有效信号，表明此通道的地址控制信号有效
ARREADY	从机 to 主机	表明“从”可以接收地址和对应的控制信号

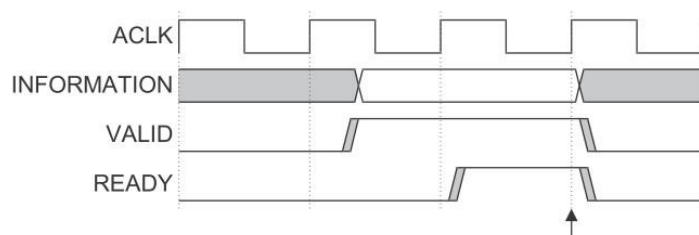
4: 读数据通道信号:

信号名	方向	描述
RID	从机 to 主机	一次读传输的 ID tag
RDATA	从机 to 主机	读数据
RRESP	从机 to 主机	读响应，表明读传输的状态
RLAST	从机 to 主机	表明此次传输是最后一个突发传输
RUSER	从机 to 主机	用户自定义信号
RVALID	从机 to 主机	写有效，表明此次写有效
RREADY	主机 to 从机	表明从机可以接收写数据

1.2.3 数据有效的情况

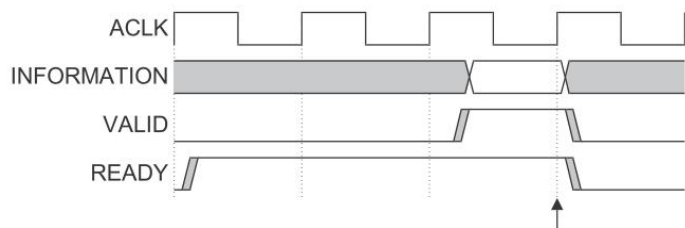
AXI4 所采用的是一种 READY, VALID 握手通信机制，简单来说主从双方进行数据通信前，有一个握手的过程。传输源产生 VLAID 信号来指明何时数据或控制信息有效。而目的地源产生 READY 信号来指明已经准备好接受数据或控制信息。传输发生在 VALID 和 READY 信号同时为高的时候。VALID 和 READY 信号的出现有三种关系。

(1) VALID 先变高 READY 后变高。时序图如下：



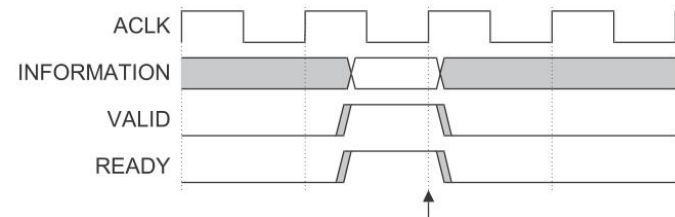
在箭头处信息传输发生。

(2) READY 先变高 VALID 后变高。时序图如下：



同样在箭头处信息传输发生。

(3) VALID 和 READY 信号同时变高。时序图如下：



在这种情况下，信息传输立马发生，如图箭头处指明信息传输发生。

1.2.4 突发式读写

1: 突发式写时序图

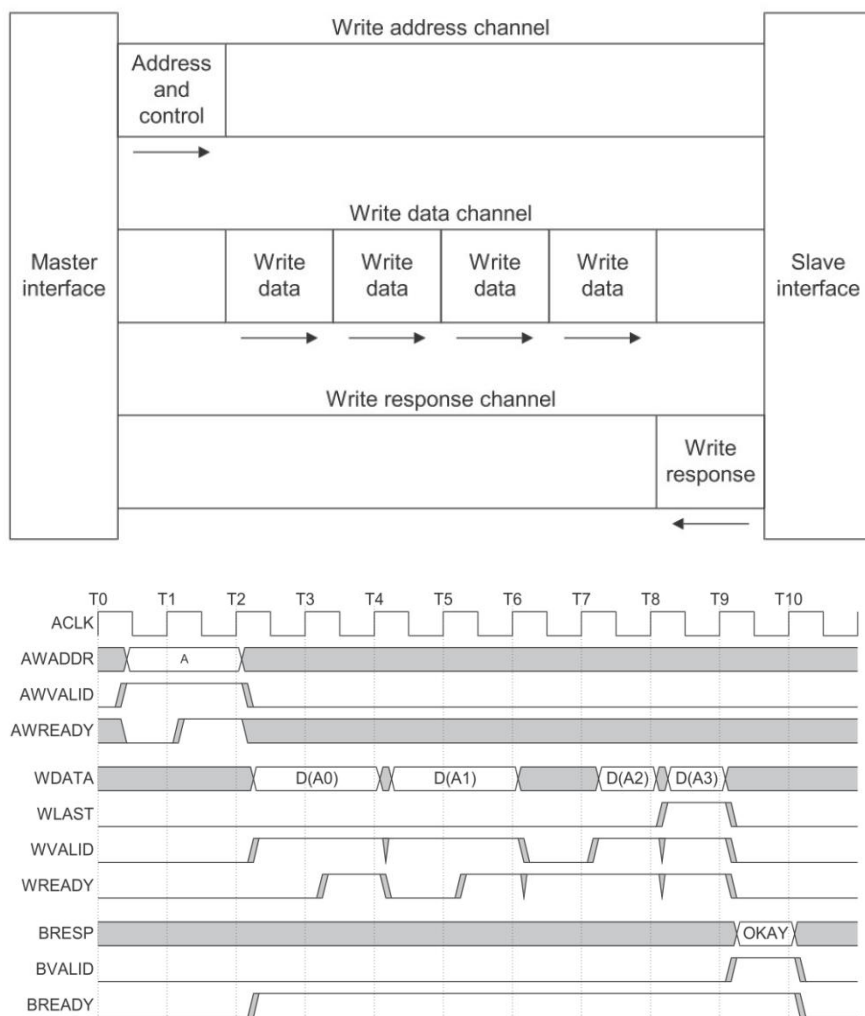


Figure 1-6 Write burst

这一过程的开始时，主机发送地址和控制信息到写地址通道中，然后主机发送每一个写数据到写数据通道中。当主机发送最后一个数据时，WLAST 信号就变为高。当设备接收完所有数据之后他将一个写响应发送回主机来表明写事务完成。

2:突发式读的时序图

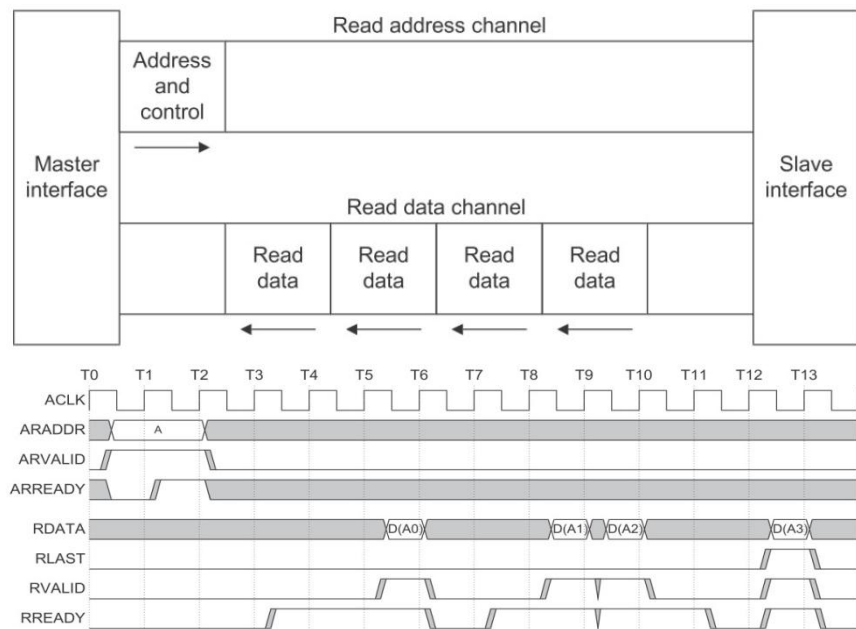


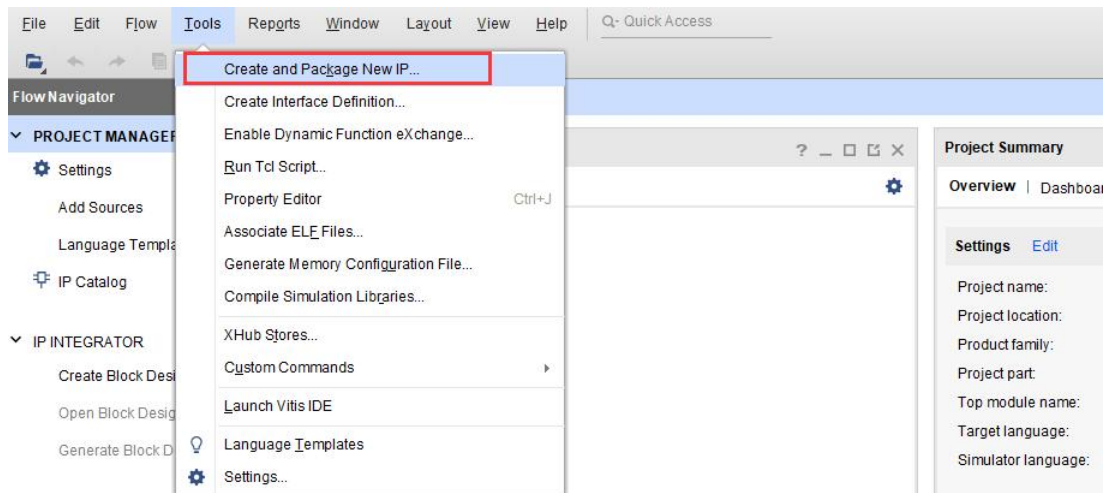
Figure 1-4 Read burst

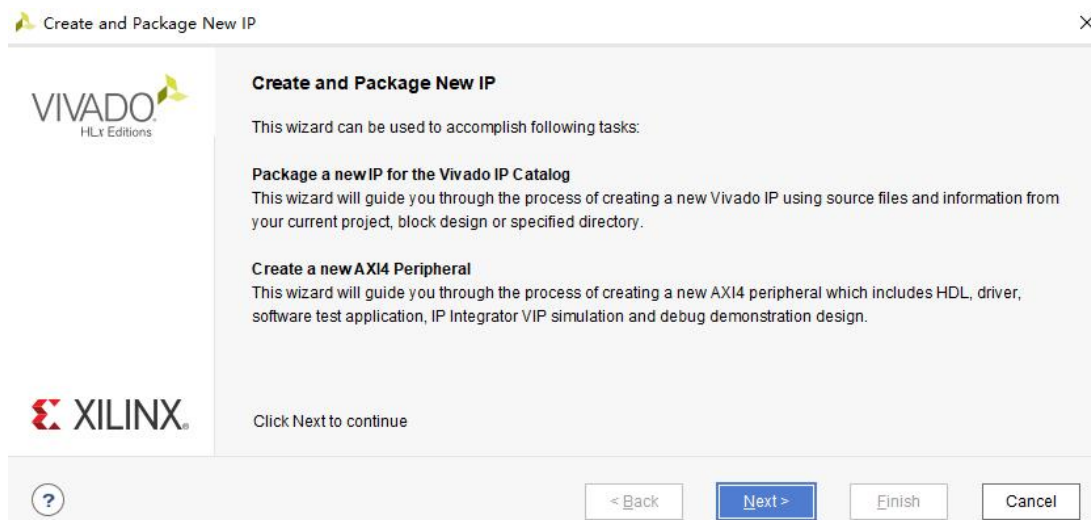
当地址出现在地址总线后，传输的数据将出现在读数据通道上。设备保持 VALID 为低直到读数据有效。为了表明一次突发式读写的完成，设备用 RLAST 信号来表示最后一个被传输的数据。

1.3 创建 axi4-lite-slave 总线接口 IP

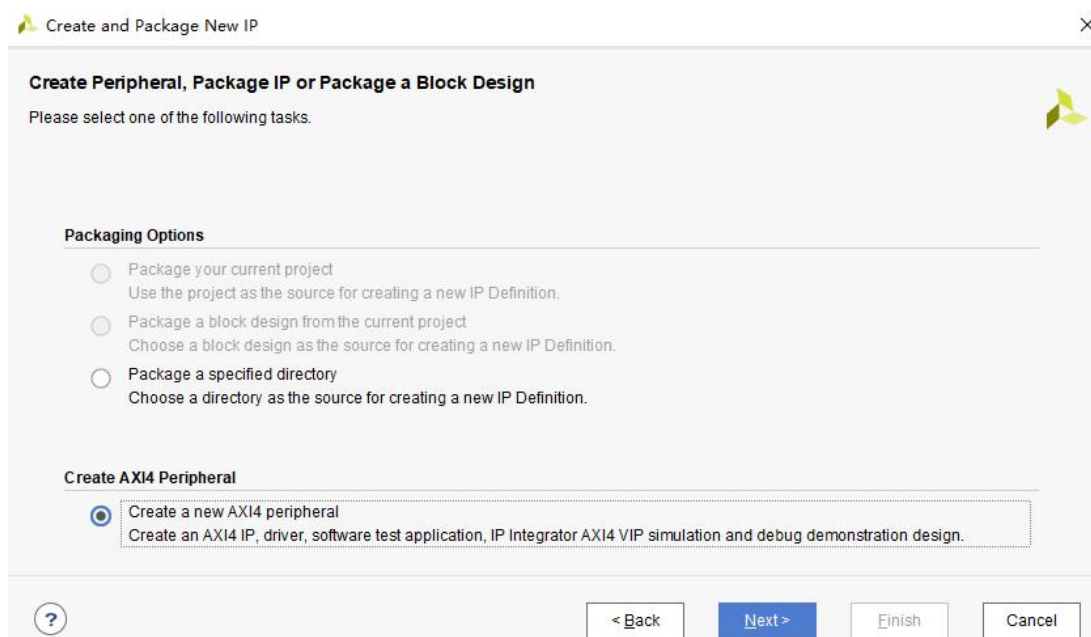
新建 fpga 工程，过程省略

新建完成工程后，单击菜单栏 Tools->Create and Package New IP,开始创建一个 AXI4-Lite 接口总线 IP

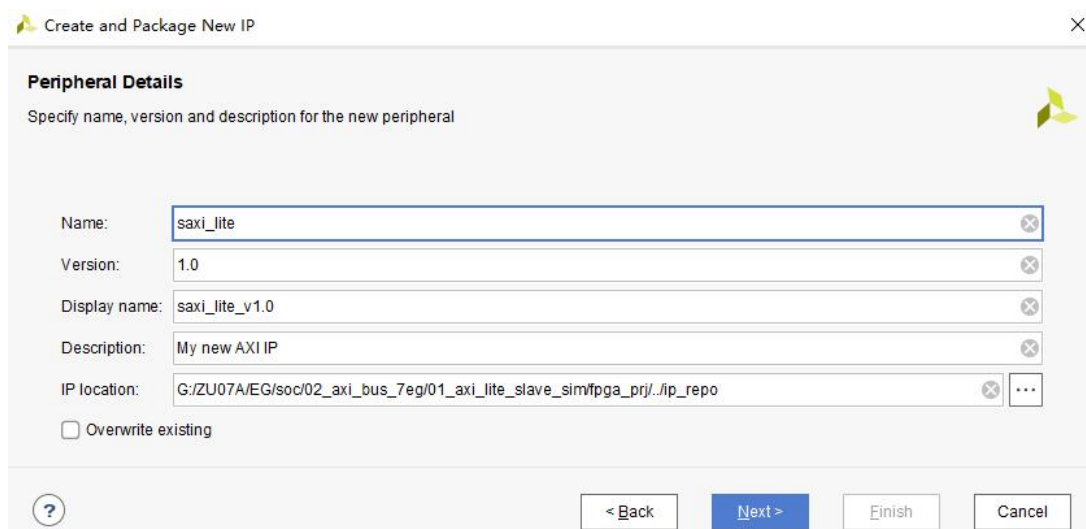




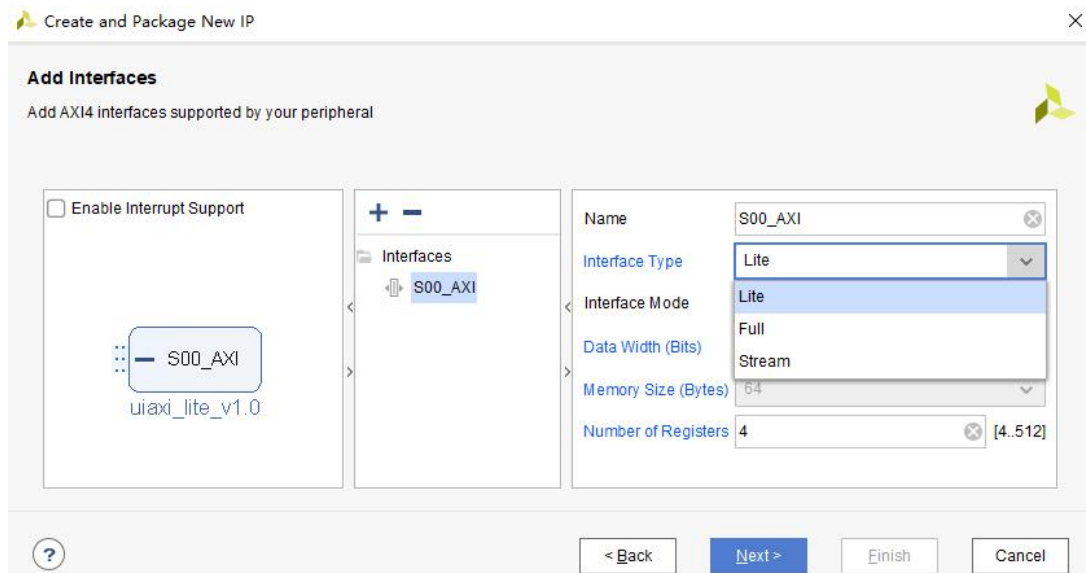
选择使用 vivado 自带的 AXI 总线模板创建一个 AXI4-Lite 接口 IP



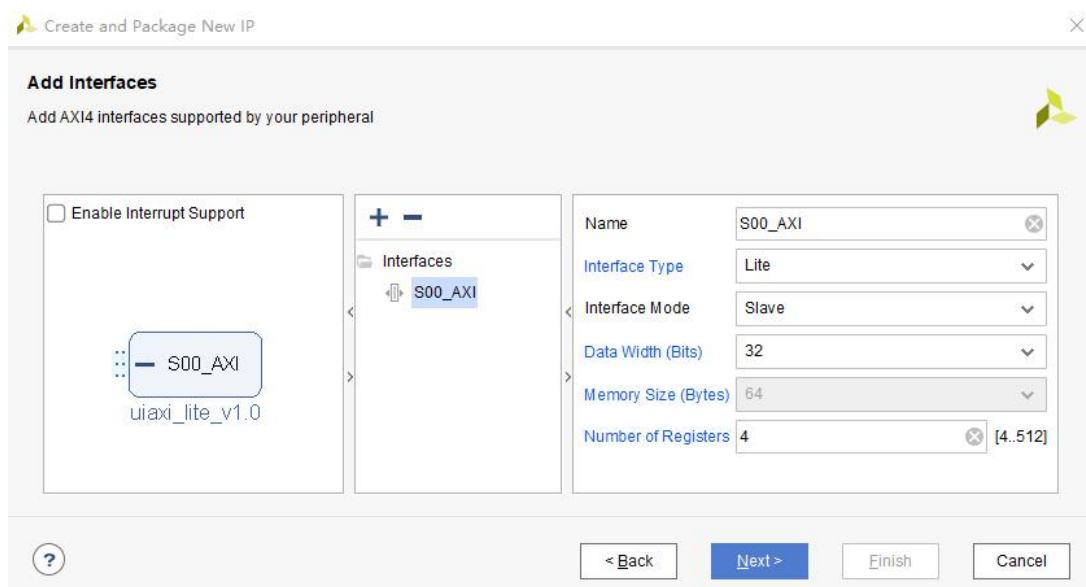
设置 IP 的名字为 saxi_lite



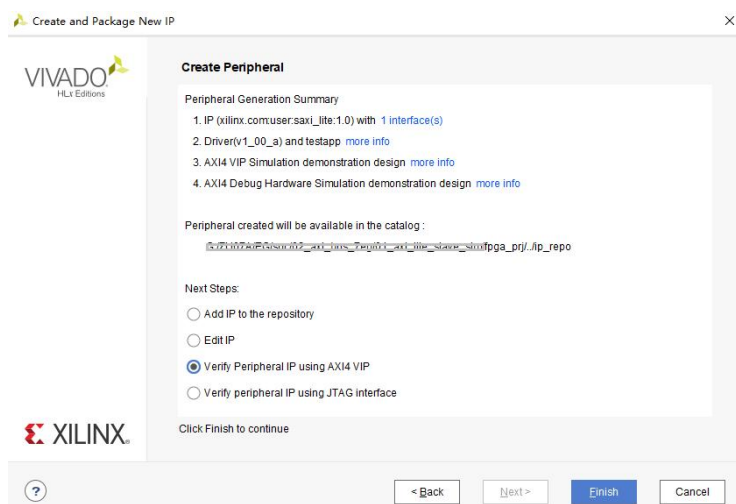
模板支持 3 中协议，分别是 AXI4-Full AXI4-Lite AXI4-Stream



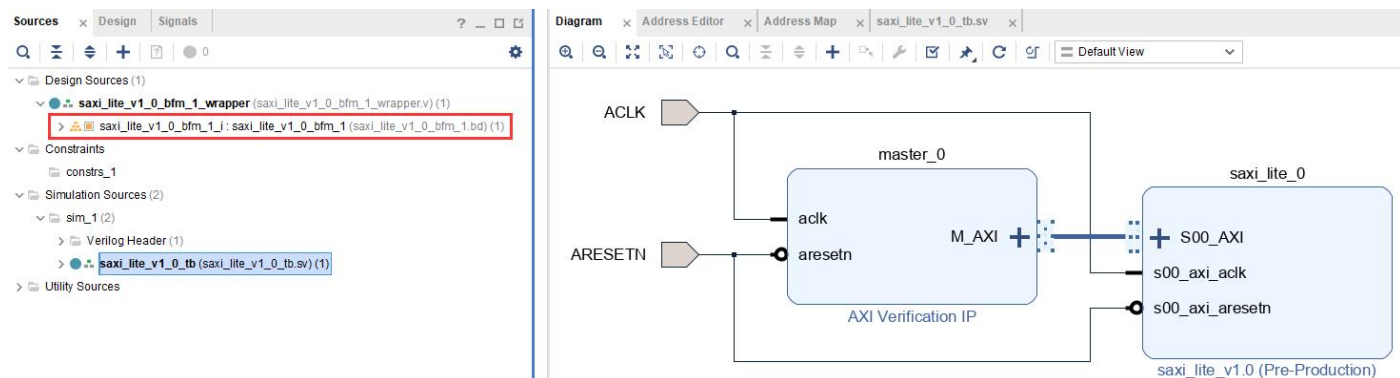
总线包括 Master 和 Slave 两种模式，这里选择 Slave 模式



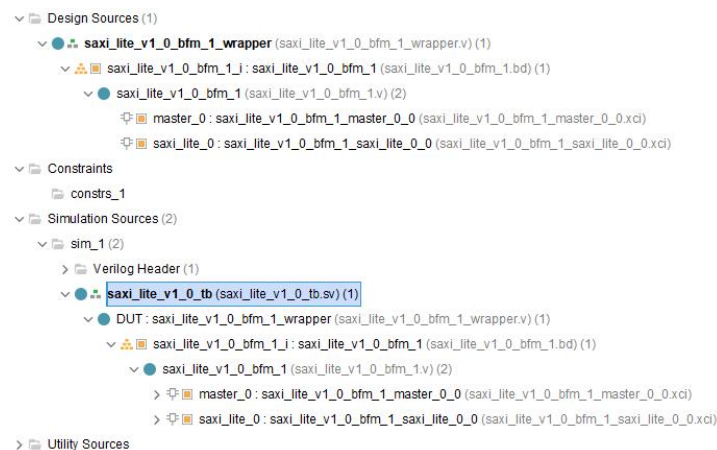
这里选择 Verify Peripheral IP using AXI4 VIP 可以对 AXI4-Lite 快速验证



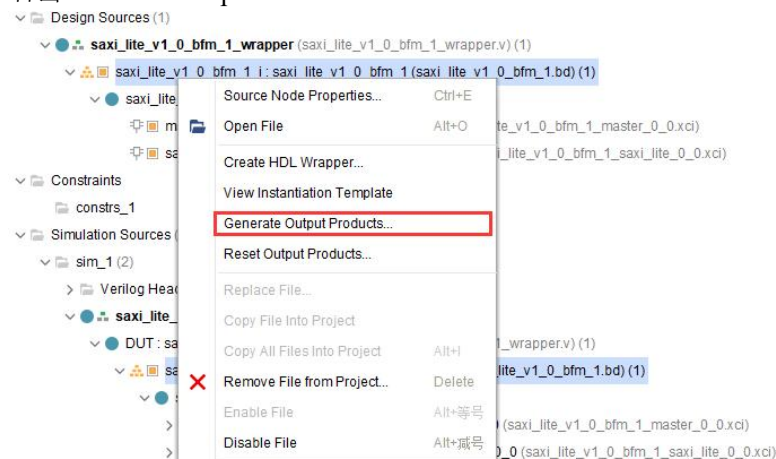
单击 Finish 后展开 VIVADO 自动产生的 demo，单击 Block Design 的工程，可以看到如下 2 个 IP。其中 saxi_lite_0 就是我们自定义的 IP，另外一个 master_0 是用来读写我们自定义的 saxi_lite_0，以此验证我们的 IP 正确性。



继续站看代码看看里面有什么东西



右击 Generate Output Products



路径 uisrc/03_ip/saxi_lite_1.0/hdl 路径下的 saxi_lite_v1_0_S00_AXI.v 就是我们的源码。另外一个 saxi_lite_v1_0.v 是软件产生了一个接口文件，如果我们自己定义 IP 可有可无。

fpga_prj > uisrc > 03_ip > saxi_lite_1.0 > hdl				
名称	修改日期	类型	大小	
saxi_lite_v1_0.v	2021/6/2 20:08	Verilog File	3 KB	
saxi_lite_v1_0_S00_AXI.v	2021/6/2 20:08	Verilog File	15 KB	

1.4 程序分析

axi 总线信号的关键无非是地址和数据，而写地址的有效取决于 AXI_AWVALID 和 AXI_AWREADY，写数据的有效取决于 S_AXI_WVALID 和 S_AXI_WREADY。同理，读地址的有效取决于 AXI_ARVALID 和 AXI_ARREADY，读数据的有效取决于 S_AXI_RVALID 和 S_AXI_RREADY。所以以下代码的阅读分析注意也是围绕以上 4 个信号的有效时序。

以下程序我们把关键信号的代码拆分阅读

1:axi-lite-slave 的 axi_awready

```
always @(posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
        begin
            axi_awready <= 1'b0;
            aw_en <= 1'b1;
        end
    else
        begin
            if (~axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en)
                begin
                    // slave is ready to accept write address when
                    // there is a valid write address and write data
                    // on the write address and data bus. This design
                    // expects no outstanding transactions.
                    axi_awready <= 1'b1;
                    aw_en <= 1'b0;
                end
            else if (S_AXI_BREADY && axi_bvalid)
                begin
                    aw_en <= 1'b1;
                    axi_awready <= 1'b0;
                end
            else
                begin
                    axi_awready <= 1'b0;
                end
            end
        end
    end
end
```

2:axi-lite-slave 的 axi_awaddr

```
always @(posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
        begin
            axi_awaddr <= 0;
        end
    else
        begin
            if (~axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en)
                begin
                    // Write Address latching
                    axi_awaddr <= S_AXI_AWADDR;
                end
            end
        end
    end
end
```

3:axi-lite-slave 的 axi_wready

当满足($\sim$ axi_wready && S_AXI_WVALID && S_AXI_AWVALID && aw_en)==1 条件, 设置 axi_wready 有效。

```
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_wready <= 1'b0;
    end
    else
    begin
        if (~axi_wready && S_AXI_WVALID && S_AXI_AWVALID && aw_en)
        begin
            // slave is ready to accept write data when
            // there is a valid write address and write data
            // on the write address and data bus. This design
            // expects no outstanding transactions.
            axi_wready <= 1'b1;
        end
        else
        begin
            axi_wready <= 1'b0;
        end
    end
end
```

4:axi-lite-slave 的写数据寄存器

axi-lite-slave 很重要一点功能就是配合 SOC 的处理器部分完成一些低速外设, 或者寄存器的控制。需要使用多寄存器或者外设, 一般在 ip 代码里面就已经设置好了。前面用 vivado 的模板产生自定义 ip 的时候, 我们选择了 4 个 32bits 寄存器, 以下的模板中 slv_reg0~slv_reg3 共计 4 个 32bits 寄存器。

```
assign slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;

always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        slv_reg0 <= 0;
        slv_reg1 <= 0;
        slv_reg2 <= 0;
        slv_reg3 <= 0;
    end
    else begin
        if (slv_reg_wren)
        begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
                2'h0:
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            // Respective byte enables are asserted as per write strobes
                        end
            endcase
        end
    end
end
```

```

        // Slave register 0

        slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];

    end

2'h1:

    for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
        if ( S_AXI_WSTRB[byte_index] == 1 ) begin

            // Respective byte enables are asserted as per write strobes

            // Slave register 1

            slv_reg1[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];

        end

2'h2:

    for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
        if ( S_AXI_WSTRB[byte_index] == 1 ) begin

            // Respective byte enables are asserted as per write strobes

            // Slave register 2

            slv_reg2[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];

        end

2'h3:

    for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
        if ( S_AXI_WSTRB[byte_index] == 1 ) begin

            // Respective byte enables are asserted as per write strobes

            // Slave register 3

            slv_reg3[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];

        end

    default : begin

        slv_reg0 <= slv_reg0;

        slv_reg1 <= slv_reg1;

        slv_reg2 <= slv_reg2;

        slv_reg3 <= slv_reg3;

    end

endcase

end

end

end

```

5:axi-lite-slave 的 axi_bvalid 信号

axi_bvalid 用于告知 axi master axi-slave 端已经完成数据接收了

```

always @( posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
        begin
            axi_bvalid <= 0;
            axi_bresp <= 2'b0;
        end
    else
        begin
            if (axi_awready && S_AXI_AWVALID && ~axi_bvalid && axi_wready && S_AXI_WVALID)
                begin
                    // indicates a valid write response is available

```

```
        axi_bvalid <= 1'b1;

        axi_bresp  <= 2'b0; // 'OKAY' response

    end                                // work error responses in future

else

    begin

        if (S_AXI_BREADY && axi_bvalid)

            //check if bready is asserted while bvalid is high)

            //(there is a possibility that bready is always asserted high)

            begin

                axi_bvalid <= 1'b0;

            end

        end

    end

end

end
```

6:axi-lite-slave 的 axi_arready

当条件满足($\sim$ axi_arready && S_AXI_ARVALID)==1 设置 axi_arready 有效, 并且寄存住总线上的地址 axi_araddr <= S_AXI_ARADDR

```
always @( posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
    begin
        axi_arready <= 1'b0;
        axi_araddr  <= 32'b0;
    end
    else
    begin
        if (~axi_arready && S_AXI_ARVALID)
        begin
            // indicates that the slave has accepted the valid read address

            axi_arready <= 1'b1;

            // Read address latching

            axi_araddr  <= S_AXI_ARADDR;

        end
        else
        begin
            axi_arready <= 1'b0;
        end
    end
end

end
```

7:axi-lite-slave 的 axi_araddr

```
always @( posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
    begin
        axi_arready <= 1'b0;
        axi_araddr  <= 32'b0;
```

```
end
else
begin
if (~axi_arready && S_AXI_ARVALID)
begin
// indicates that the slave has accepted the valid read address
axi_arready <= 1'b1;
// Read address latching
axi_araddr <= S_AXI_ARADDR;
end
else
begin
axi_arready <= 1'b0;
end
end
end
end
```

8:axi-lite-slave 的 axi_rvalid 信号

当条件满足(axi_arready && S_AXI_ARVALID && ~axi_rvalid)=1 的时候设置 axi_rvalid 有效，表示 axi-lite-slave 总线上的数据是有效的。

```
always @(posedge S_AXI_ACLK )
begin
if ( S_AXI_ARESETN == 1'b0 )
begin
axi_rvalid <= 0;
axi_rresp <= 0;
end
else
begin
if (axi_arready && S_AXI_ARVALID && ~axi_rvalid)
begin
// Valid read data is available at the read data bus
axi_rvalid <= 1'b1;
axi_rresp <= 2'b0; // 'OKAY' response
end
else if (axi_rvalid && S_AXI_RREADY)
begin
// Read data is accepted by the master
axi_rvalid <= 1'b0;
end
end
end
end

assign S_AXI_RVALID = axi_rvalid;
```

9:axi-lite-slave 的读数据寄存器

本文实验中，axi-master 写入 4 个寄存器数据，然后读出，通过查看数据是否一致可以确认 axi-lite-slave 工作是否正常。当 slv_reg_rden 有效的时候，数据被读入寄存器 axi_rdata，当 axi_rvalid 有效的时候，数据被锁存。

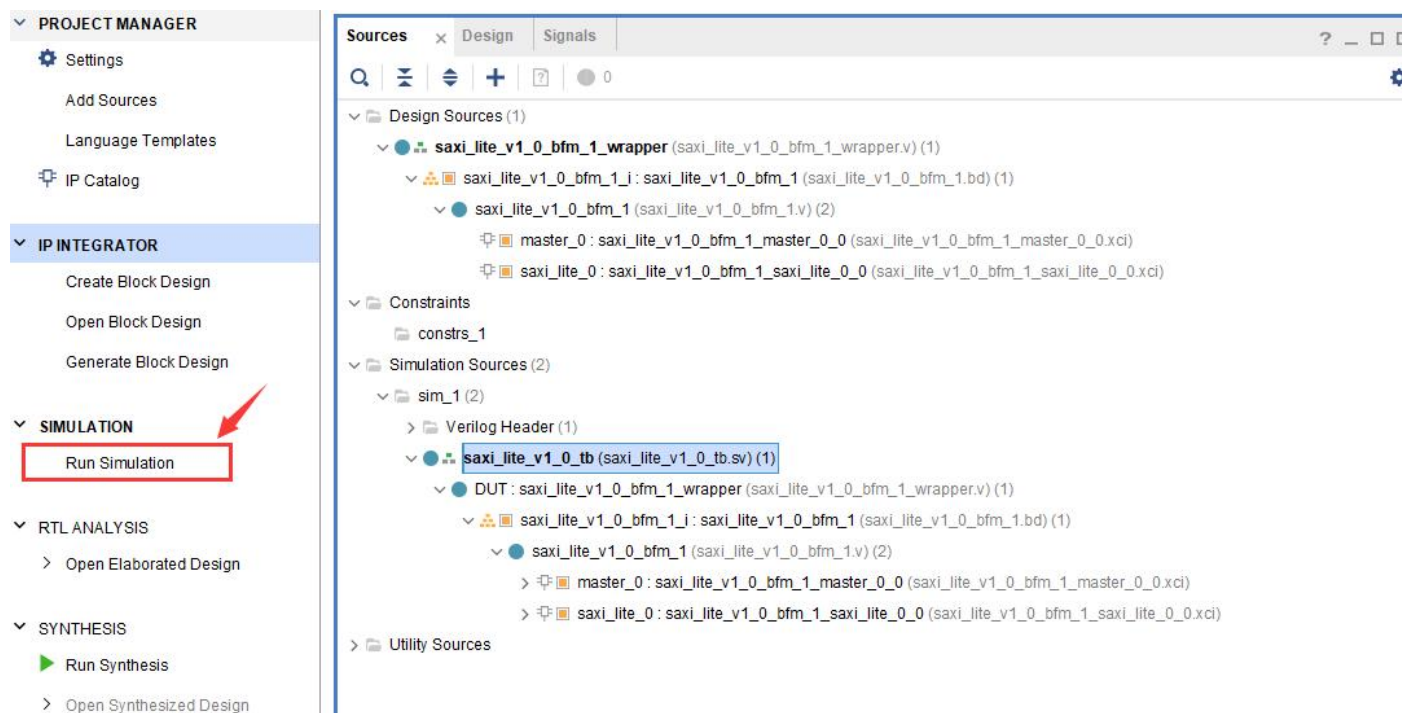
```
assign slv_reg_rden = axi_arready & S_AXI_ARVALID & ~axi_rvalid;

always @(*)
begin
    // Address decoding for reading registers
    case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
        2'h0 : reg_data_out <= slv_reg0;
        2'h1 : reg_data_out <= slv_reg1;
        2'h2 : reg_data_out <= slv_reg2;
        2'h3 : reg_data_out <= slv_reg3;
        default : reg_data_out <= 0;
    endcase
end

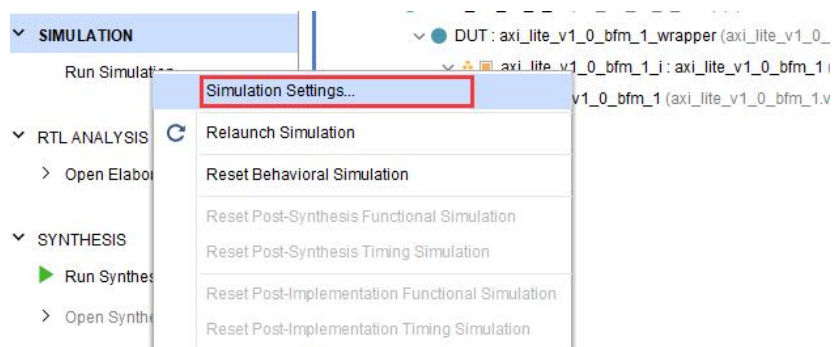
always @( posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
    begin
        axi_rdata <= 0;
    end
    else
    begin
        // When there is a valid read address (S_AXI_ARVALID) with
        // acceptance of read address by the slave (axi_arready),
        // output the read data
        if (slv_reg_rden)
        begin
            axi_rdata <= reg_data_out;    // register read data
        end
    end
end
end
```

当我们阅读后分析完以上代码后，可以发现，axi-lite-slave 的代码中没有突发长度的处理，每次只处理一个地址的一个数据。并且也没有 WLAST 和 RLAST 信号，说明 axi-lite-slave 适合一些低速的数据交互，但是可以节省一些 FPGA 的逻辑资源。

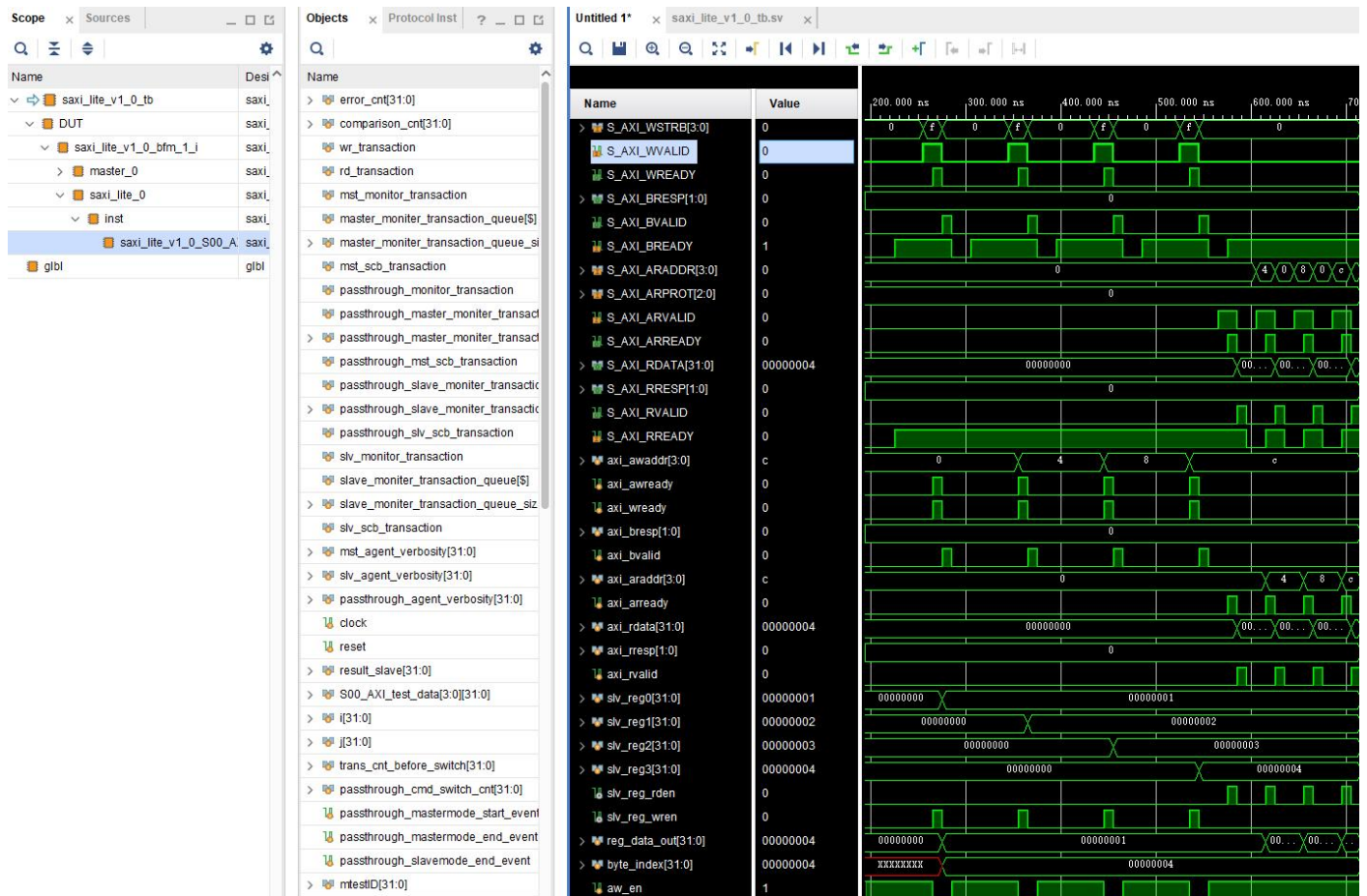
1.5 实验结果



单击仿真



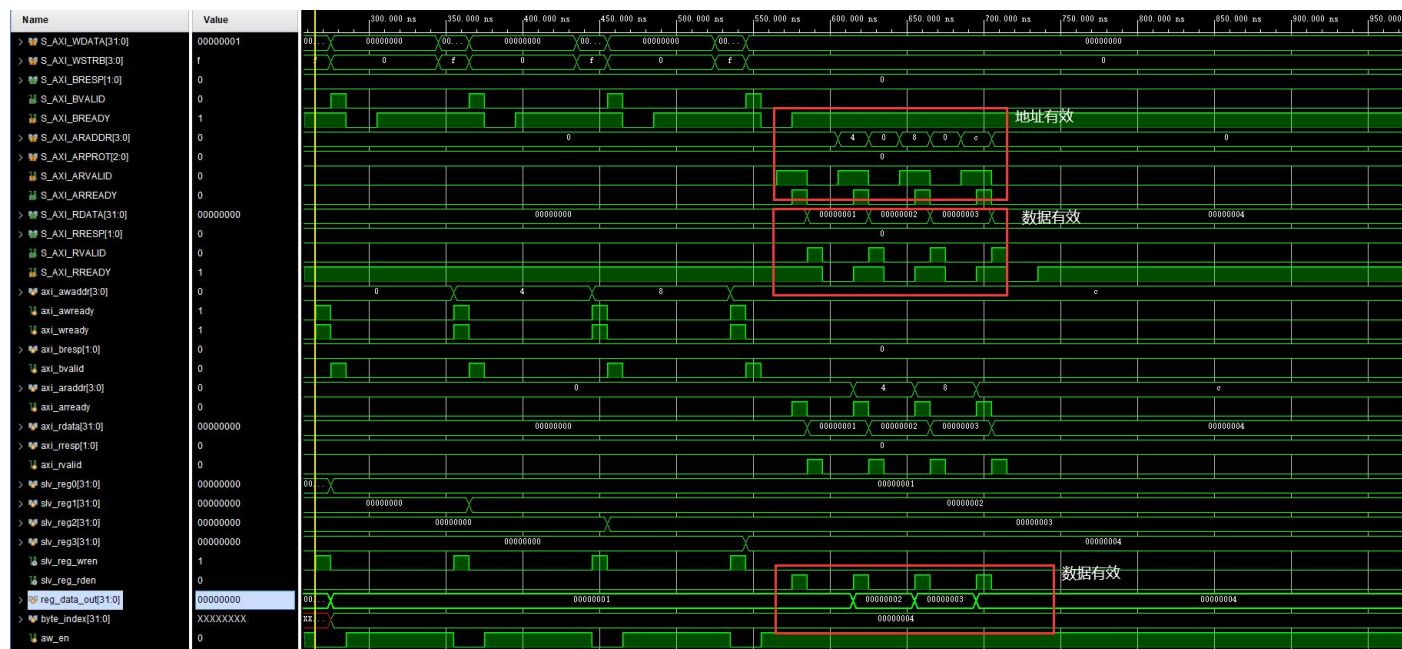
添加观察信号



AXI 总下依次写入 1 2 3 4, `slv_reg0~slv_reg3` 完成数据寄存



读数据



02AXI4 总线 axi-lite-master

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

2.1 概述

使用 XILINX 的软件工具 VIVADO 以及 XILINX 的 7 代以上的 FPGA 或者 SOC 掌握 AXI-4 总线结束, 并且可以灵活使用 AXI-4 总线技术完成数据的交换, 可以让我们在构建强大的 FPGA 内部总线数据互联通信方面取得高效、高速、标准化的优势。

关于 AXI4 总线协议的部分介绍请阅读“01AXI4 总线 axi-lite-slave”。

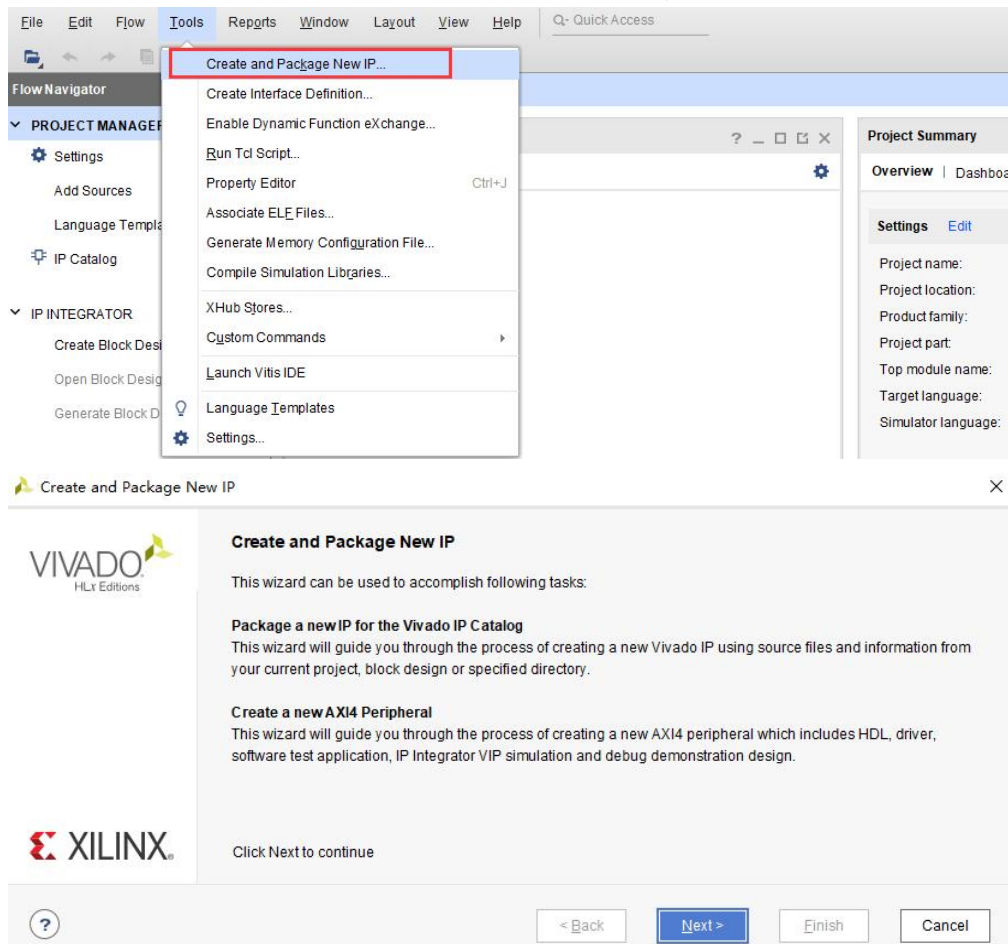
本文实验目的:

- 1:掌握基于 VIVADO 工具产生 AXI 协议模板
- 2:掌握通过 VIVADO 工具产生 AXI-lite-master 代码
- 3:理解 AXI-lite-master 中自定义寄存器的地址分配
- 4:掌握通过 VIVADO 封装 AXI-lite-master 图形化 IP
- 5:通过仿真验证 AXI-lite-master IP 的工作是否正常。

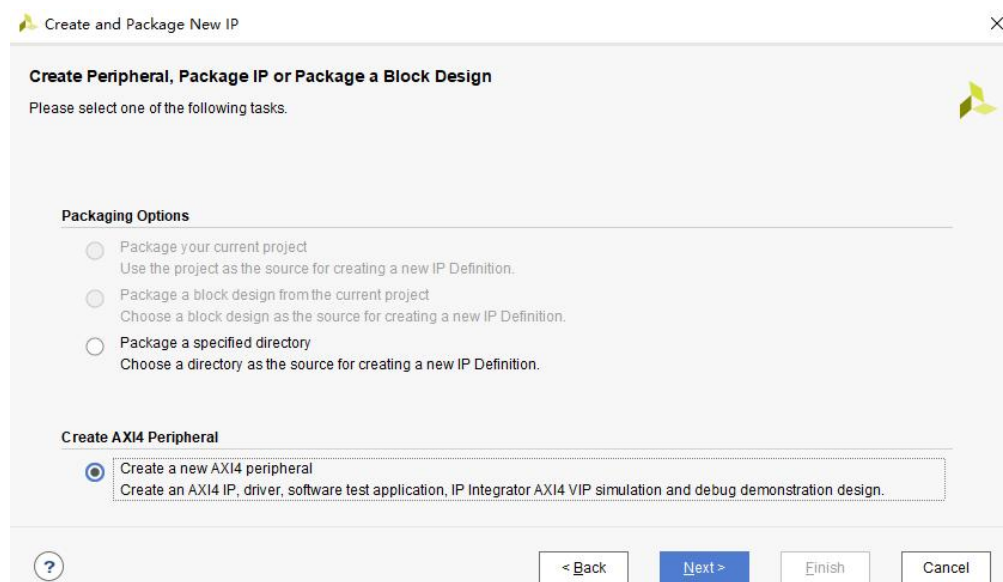
2.2 创建 axi4-lite-master 总线接口 IP

新建 fpga 工程, 过程省略

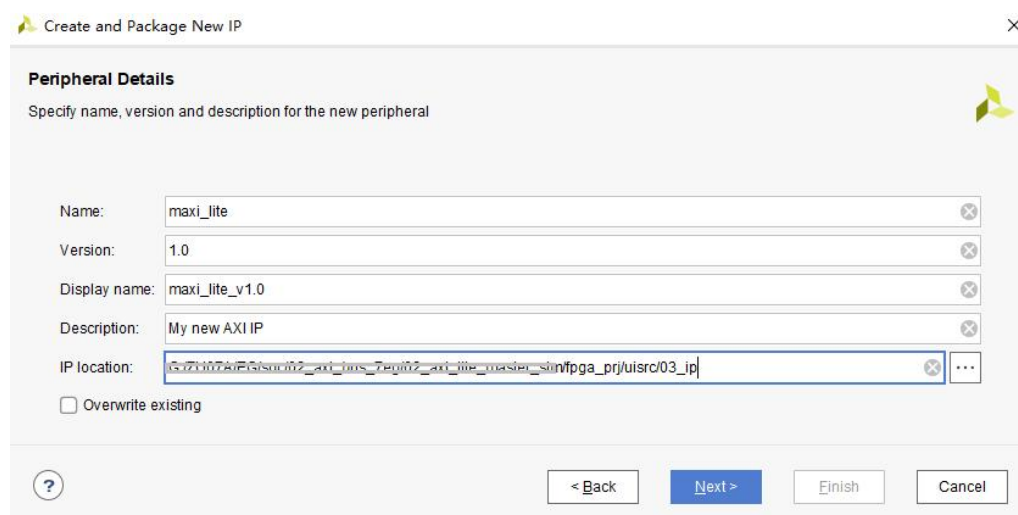
新建完成工程后, 单击菜单栏 Tools->Create and Package New IP,开始创建一个 AXI4-Lite 接口总线 IP



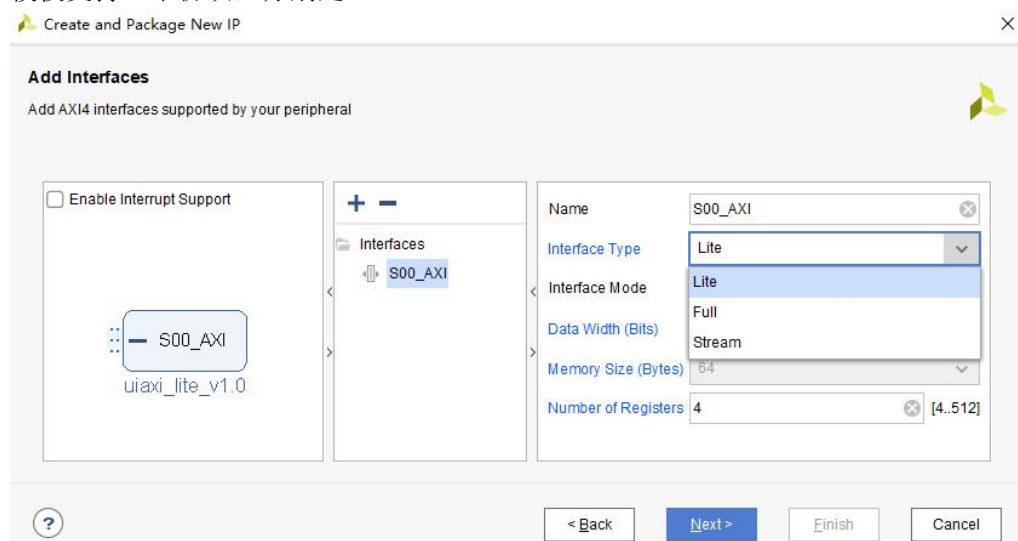
选择使用 vivado 自带的 AXI 总线模板创建一个 AXI4-Lite 接口 IP



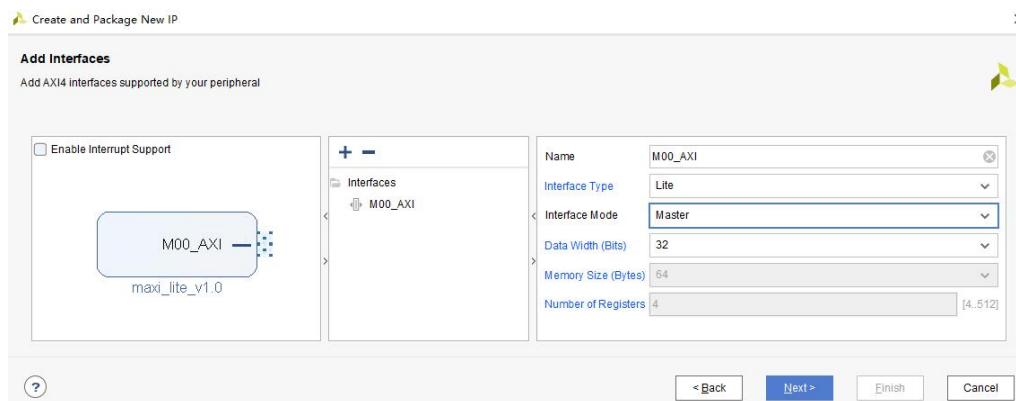
设置 IP 的名字为 maxi_lite



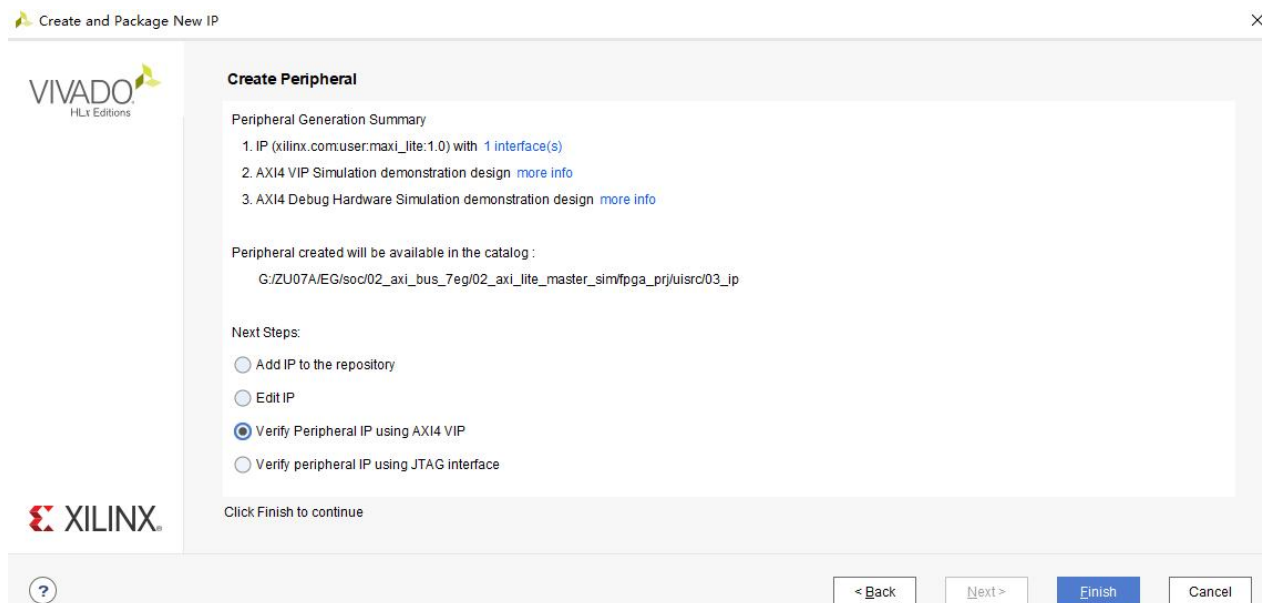
模板支持 3 中协议，分别是 AXI4-Full AXI4-Lite AXI4-Stream



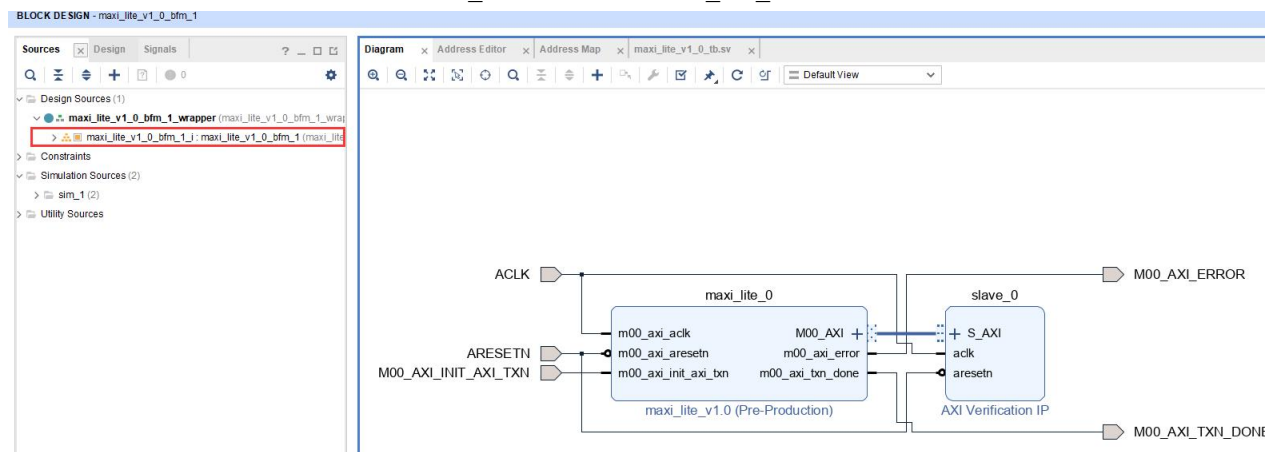
总线包括 Master 和 Slave 两种模式，这里选择 Slave 模式



这里选择 Verify Peripheral IP using AXI4 VIP 可以对 AXI4-Lite 快速验证



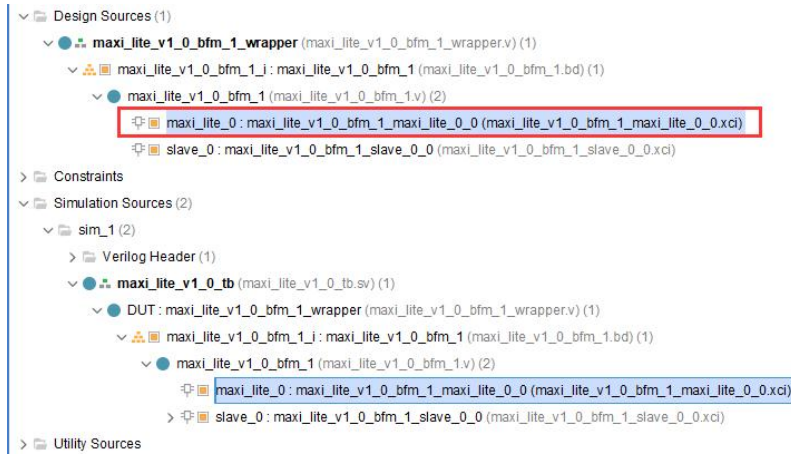
单击 Finish 后展开 VIVADO 自动产生的 demo，单击 Block Design 的工程，可以看到如下 2 个 IP。其中 maxi_lite_0 就是我们自定义的 IP，另外一个 slave_0 是用来验证 maxi_lite_0 正确性。



采用默认地址分配即可

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
maxi_lite_0					
/maxi_lite_0/M00_AXI (32 address bits: 4G)					
/slave_0/S_AXI	S_AXI	Reg	0x44A0_0000	64K	0x44A0_FFFF

继续站看代码看看里面有什么东西



路径 `uisrc/03_ip/ maxi_lite_1.0/hdl` 路径下的 `maxi_lite_v1_0_M00_AXI.v` 就是我们的源码。另外一个 `maxi_lite_v1_0.v` 是软件产生了一个接口文件，如果我们自己定义 IP 可有可无。

2.3 程序分析

axi 总线信号的关键无非是地址和数据，而写地址的有效取决于 AXI_AWVALID 和 AXI_AWREADY，写数据的有效取决于 S_AXI_WVALID 和 S_AXI_WREADY。同理，读地址的有效取决于 AXI_ARVALID 和 AXI_ARREADY，读数据的有效取决于 S_AXI_RVALID 和 S_AXI_RREADY。所以以下代码的阅读分析注意也是围绕以上 4 个信号的有效时序。

以下程序我们把关键信号的代码拆分阅读

1:产生初始化信号

```
always @(posedge M_AXI_ACLK)
begin
    // Initiates AXI transaction delay
    if (M_AXI_ARESETN == 0 )
        begin
            init_txn_ff <= 1'b0;
            init_txn_ff2 <= 1'b0;
        end
    else
        begin
            init_txn_ff <= INIT_AXI_TXN;
            init_txn_ff2 <= init_txn_ff;
        end
end
```

2:axi-lite-master 的 axi awvalid

当 start single write 有效，开始一次写传输，设置 axi awvalid 有效。

```
always @(posedge M_AXI_ACLK)

begin

    //Only VALID signals must be deasserted during reset per AXI spec

    //Consider inverting then registering active-low reset for higher fmax

    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)

        begin

            axi_awvalid <= 1'b0;

        end

    //Signal a new address/data command is available by user logic

else
```

```
begin
    if (start_single_write)
        begin
            axi_awvalid <= 1'b1;
        end
    //Address accepted by interconnect/slave (issue of M_AXI_AWREADY by slave)
    else if (M_AXI_AWREADY && axi_awvalid)
        begin
            axi_awvalid <= 1'b0;
        end
    end
end
```

3:axi-lite-master 的 axi_awaddr

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            axi_awaddr <= 0;
        end
    // Signals a new write address/ write data is
    // available by user logic
    else if (M_AXI_AWREADY && axi_awvalid)
        begin
            axi_awaddr <= axi_awaddr + 32'h00000004;
        end
    end
end
```

4:axi-lite-master 的 axi_wvalid

当 M_AXI_WREADY && axi_wvalid 同时有效的时候，数据才是有效的，对于 axi-lite_master 接口，M_AXI_WREADY && axi_wvalid 同时有效的时间窗口是一个时钟周期。

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            axi_wvalid <= 1'b0;
        end
    //Signal a new address/data command is available by user logic
    else if (start_single_write)
        begin
            axi_wvalid <= 1'b1;
        end
    //Data accepted by interconnect/slave (issue of M_AXI_WREADY by slave)
    else if (M_AXI_WREADY && axi_wvalid)
        begin
            axi_wvalid <= 1'b0;
        end
    end
end
```

```
end
```

5:axi-lite-master 的 axi_wdata

产生写测试用的测试数据

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            axi_wdata <= C_M_START_DATA_VALUE;
        end
        // Signals a new write address/ write data is
        // available by user logic
    else if (M_AXI_WREADY && axi_wvalid)
        begin
            axi_wdata <= C_M_START_DATA_VALUE + write_index;
        end
    end
end
```

6:写次数记录 write_index 计数器

这个 demo 中以 start_single_wirte 信号作为统计的，如果我们完全自定 axi-lite_master 代码可以自行优化，可以写出更好的代码。我们这里是用 vivado 模板产生的代码主要教会大家如何使用现有的手段和软件工具学习 axi4 总线。

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            write_index <= 0;
        end
        // Signals a new write address/ write data is
        // available by user logic
    else if (start_single_write)
        begin
            write_index <= write_index + 1;
        end
    end
end
```

7:axi-lite-master 的 axi_bready

当收到写通道的 axi-lite-slave 发回的 M_AXI_BVALIDI 应答信号，设置 axi_bready 为 1，BRESP 返回 AXI 写操作是否有错误。

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            axi_bready <= 1'b0;
        end
        // accept/acknowledge bresp with axi_bready by the master
        // when M_AXI_BVALID is asserted by slave
    else if (M_AXI_BVALID && ~axi_bready)
```

```
begin
    axi_bready <= 1'b1;
end

// deassert after one clock cycle
else if (axi_bready)
begin
    axi_bready <= 1'b0;
end

// retain the previous value
else
    axi_bready <= axi_bready;
end

//Flag write errors
assign write_resp_error = (axi_bready & M_AXI_BVALID & M_AXI_BRESP[1]);
```

8:axi-lite-slave 的 axi_arvalid

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
begin
    axi_arvalid <= 1'b0;
end

//Signal a new read address command is available by user logic
else if (start_single_read)
begin
    axi_arvalid <= 1'b1;
end

//RAddress accepted by interconnect/slave (issue of M_AXI_ARREADY by slave)
else if (M_AXI_ARREADY && axi_arvalid)
begin
    axi_arvalid <= 1'b0;
end

// retain the previous value
end
```

9:axi-lite-slave 的 axi_araddr

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
begin
    axi_araddr <= 0;
end

// Signals a new write address/ write data is
// available by user logic
else if (M_AXI_ARREADY && axi_arvalid)
begin
    axi_araddr <= axi_araddr + 32'h00000004;
```

```
end  
  
end
```

10:axi-lite-master 的 axi_rready

当 M_AXI_RVALID && axi_rready 同时有效的时候,数据才是有效的,对于 axi-lite_master 接口,M_AXI_RVALID && ~axi_rready==1 的时候设置 axi_rready=1, 当 axi_rready==1, 再设置 axi_rready=0

```
always @(posedge M_AXI_ACLK)  
begin  
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)  
        begin  
            axi_rready <= 1'b0;  
        end  
        // accept/acknowledge rdata/rresp with axi_rready by the master  
        // when M_AXI_RVALID is asserted by slave  
    else if (M_AXI_RVALID && ~axi_rready)  
        begin  
            axi_rready <= 1'b1;  
        end  
        // deassert after one clock cycle  
    else if (axi_rready)  
        begin  
            axi_rready <= 1'b0;  
        end  
        // retain the previous value  
    end  
  
    //Flag write errors  
    assign read_resp_error = (axi_rready & M_AXI_RVALID & M_AXI_RRESP[1]);
```

11:axi-lite-master 的 M_AXI_RDATA

当 M_AXI_RVALID && axi_rready 都有效的时候, 对读取的 M_AXI_RDATA 数据和 expected_rdata 数据进行比较。

```
always @(posedge M_AXI_ACLK)  
begin  
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)  
        read_mismatch <= 1'b0;  
  
        //The read data when available (on axi_rready) is compared with the expected data  
    else if ((M_AXI_RVALID && axi_rready) && (M_AXI_RDATA != expected_rdata))  
        read_mismatch <= 1'b1;  
    else  
        read_mismatch <= read_mismatch;  
    end
```

12:产生对比数据 expected_rdata

数据 expected_rdata 用于和读出的 M_AXI_RDATA 进行对比以此验证数据的正确性。

```
always @(posedge M_AXI_ACLK)  
begin  
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
```

```
begin
    expected_rdata <= C_M_START_DATA_VALUE;
end

// Signals a new write address/ write data is
// available by user logic
else if (M_AXI_RVALID && axi_rready)
begin
    expected_rdata <= C_M_START_DATA_VALUE + read_index;
end
end
```

13:读次数记录 read_index 计数器

这个 demo 中以 start_single_read 信号作为统计的，如果我们完全自定 axi-lite_master 代码可以自行优化，可以写出更好的代码。我们这里是用 vivado 模板产生的代码主要教会大家如何使用现有的手段和软件工具学习 axi4 总线。

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
begin
    read_index <= 0;
end
// Signals a new read address is
// available by user logic
else if (start_single_read)
begin
    read_index <= read_index + 1;
end
end
```

14:axi-lite-master 的状态机

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
begin
    expected_rdata <= C_M_START_DATA_VALUE;
end
// Signals a new write address/ write data is
// available by user logic
else if (M_AXI_RVALID && axi_rready)
begin
    expected_rdata <= C_M_START_DATA_VALUE + read_index;
end
end

//implement master command interface state machine
always @ (posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 1'b0)
begin
```

```
// reset condition

// All the signals are assigned default values under reset condition

mst_exec_state <= IDLE;
start_single_write <= 1'b0;
write_issued <= 1'b0;
start_single_read <= 1'b0;
read_issued <= 1'b0;
compare_done <= 1'b0;
ERROR <= 1'b0;

end

else

begin

// state transition

case (mst_exec_state)

IDLE:
// This state is responsible to initiate
// AXI transaction when init_txn_pulse is asserted
if ( init_txn_pulse == 1'b1 )
begin
mst_exec_state <= INIT_WRITE;
ERROR <= 1'b0;
compare_done <= 1'b0;
end
else
begin
mst_exec_state <= IDLE;
end

INIT_WRITE:
// This state is responsible to issue start_single_write pulse to
// initiate a write transaction. Write transactions will be
// issued until last_write signal is asserted.
// write controller
if (writes_done)
begin
mst_exec_state <= INIT_READ;//
end
else
begin
mst_exec_state <= INIT_WRITE;

if (~axi_awvalid && ~axi_wvalid && ~M_AXI_BVALID && ~last_write && ~start_single_write && ~write_issued)
begin
start_single_write <= 1'b1;
write_issued <= 1'b1;
end
else if (axi_bready)
begin
```

```
        write_issued <= 1'b0;

    end

else

    begin

        start_single_write <= 1'b0; //Negate to generate a pulse

    end

end

INIT_READ:

    // This state is responsible to issue start_single_read pulse to
    // initiate a read transaction. Read transactions will be
    // issued until last_read signal is asserted.
    // read controller
    if (reads_done)

        begin

            mst_exec_state <= INIT_COMPARE;

        end

    else

        begin

            mst_exec_state <= INIT_READ;

            if (~axi_arvalid && ~M_AXI_RVALID && ~last_read && ~start_single_read && ~read_issued)

                begin

                    start_single_read <= 1'b1;

                    read_issued <= 1'b1;

                end

            else if (axi_rready)

                begin

                    read_issued <= 1'b0;

                end

            else

                begin

                    start_single_read <= 1'b0; //Negate to generate a pulse

                end

            end

        end

    end

INIT_COMPARE:

    begin

        // This state is responsible to issue the state of comparison
        // of written data with the read data. If no error flags are set,
        // compare_done signal will be asserted to indicate success.

        ERROR <= error_reg;

        mst_exec_state <= IDLE;

        compare_done <= 1'b1;

    end

default :

    begin

        mst_exec_state <= IDLE;

    end

end
```

```

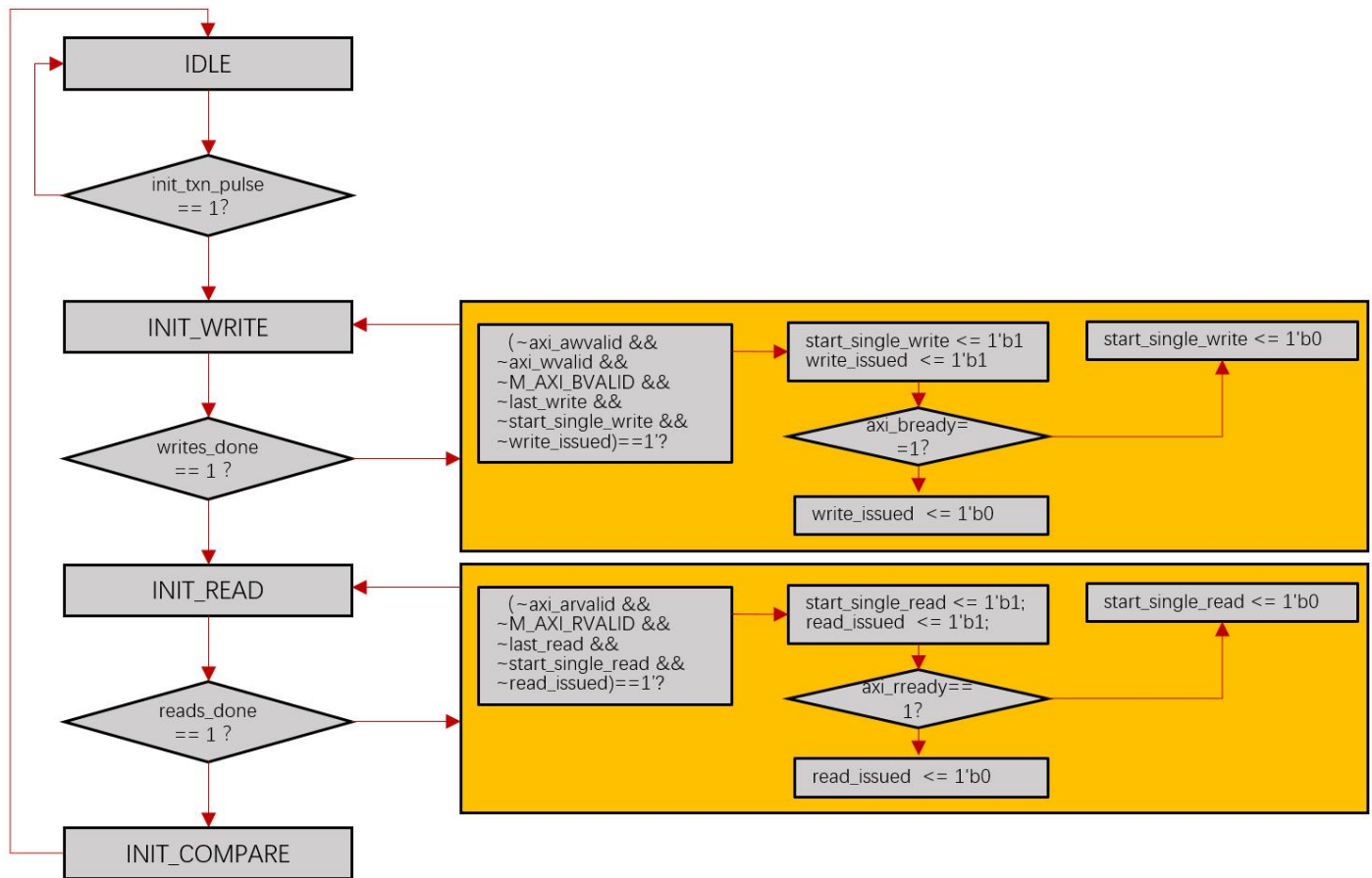
endcase

end

end //MASTER_EXECUTION_PROC

```

整理成流程图，更加容易理解：



15:最后一个写数据

```

always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        last_write <= 1'b0;

    //The last write should be associated with a write address ready response
    else if ((write_index == C_M_TRANSACTIONS_NUM) && M_AXI_AWREADY)
        last_write <= 1'b1;
    else
        last_write <= last_write;
    end

    //Check for last write completion.

    //This logic is to qualify the last write count with the final write
    //response. This demonstrates how to confirm that a write has been
    //committed.

    always @(posedge M_AXI_ACLK)

```

```
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        writes_done <= 1'b0;

        //The writes_done should be associated with a bready response
    else if (last_write && M_AXI_BVALID && axi_bready)
        writes_done <= 1'b1;
    else
        writes_done <= writes_done;
end
```

16:最后一个读数据

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        last_read <= 1'b0;

        //The last read should be associated with a read address ready response
    else if ((read_index == C_M_TRANSACTIONS_NUM) && (M_AXI_ARREADY))
        last_read <= 1'b1;
    else
        last_read <= last_read;
end

/*
Check for last read completion.

This logic is to qualify the last read count with the final read
response/data.
*/

always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        reads_done <= 1'b0;

        //The reads_done should be associated with a read ready response
    else if (last_read && M_AXI_RVALID && axi_rready)
        reads_done <= 1'b1;
    else
        reads_done <= reads_done;
end
```

2.4 实验结果

具体的仿真方法省略，以下是仿真结果。



03AXI4 总线 axi-full-slave

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

3.1 概述

使用 XILINX 的软件工具 VIVADO 以及 XILINX 的 7 代以上的 FPGA 或者 SOC 掌握 AXI-4 总线结束, 并且可以灵活使用 AXI-4 总线技术完成数据的交换, 可以让我们在构建强大的 FPGA 内部总线数据互联通信方面取得高效、高速、标准化的优势。

关于 AXI4 总线协议的部分介绍请阅读“01AXI4 总线 axi-lite-slave”。

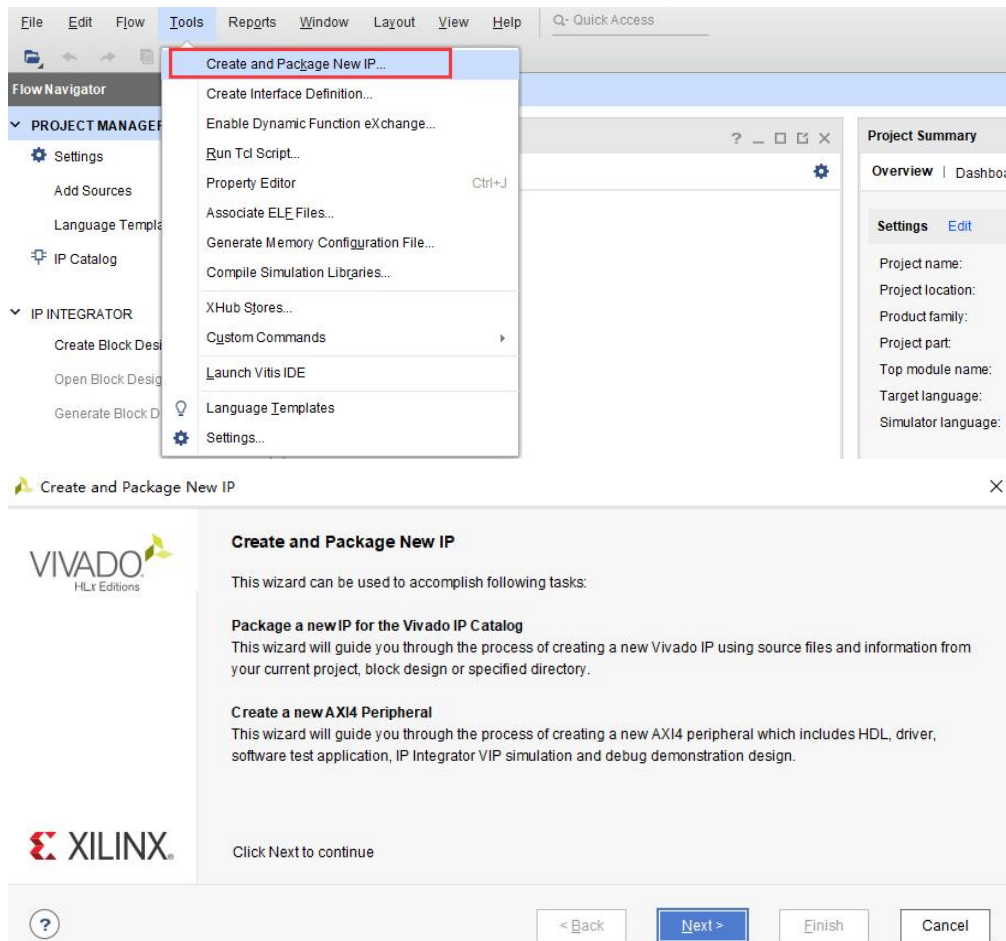
本文实验目的:

- 1:掌握基于 VIVADO 工具产生 AXI 协议模板
- 2:掌握通过 VIVADO 工具产生 AXI-full-slave 代码
- 3:理解 AXI-full-slave 中自定义寄存器的地址分配
- 4:掌握通过 VIVADO 封装 AXI-full-slave 图形化 IP
- 5:通过仿真验证 AXI-full-slave IP 的工作是否正常。

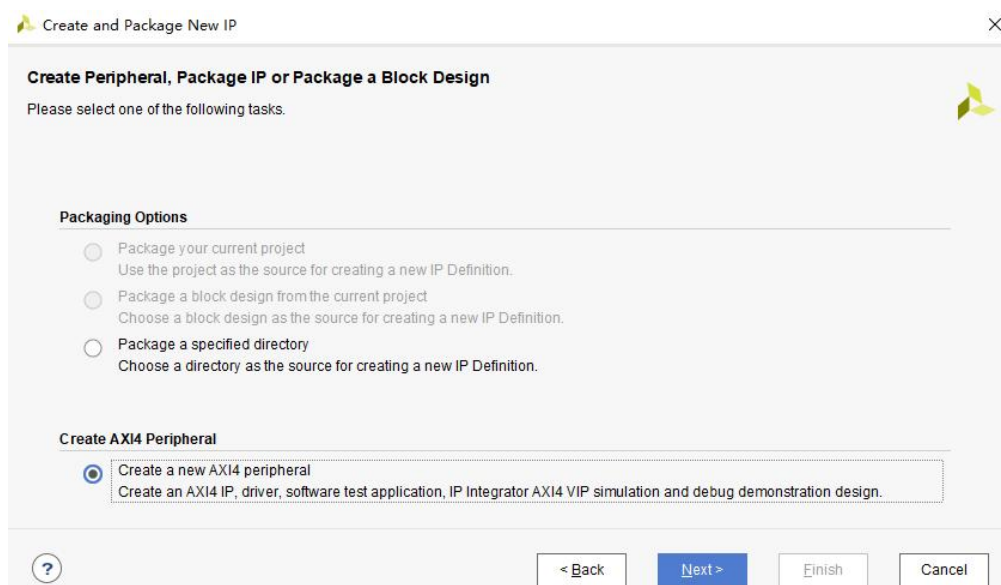
3.2 创建 axi4-full-slave 总线接口 IP

新建 fpga 工程, 过程省略

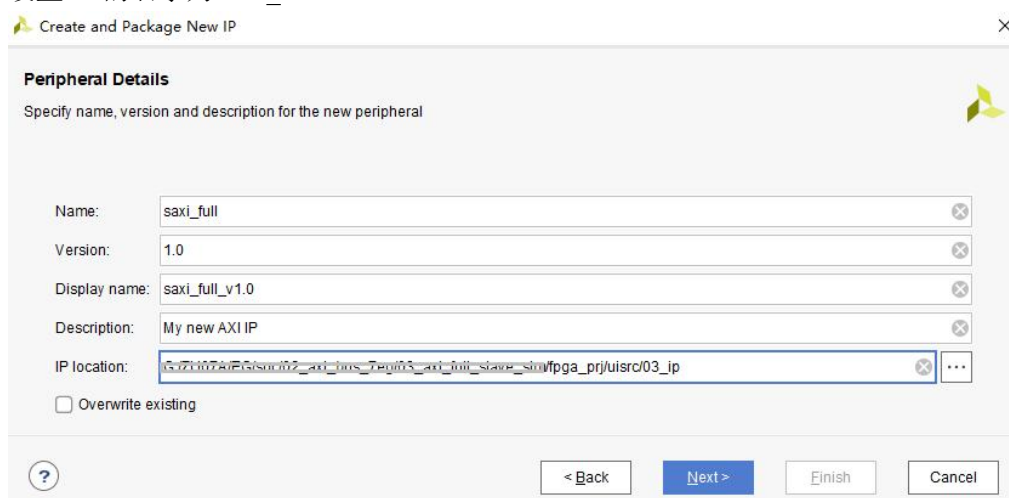
新建完成工程后, 单击菜单栏 Tools->Create and Package New IP,开始创建一个 AXI4-Full 接口总线 IP



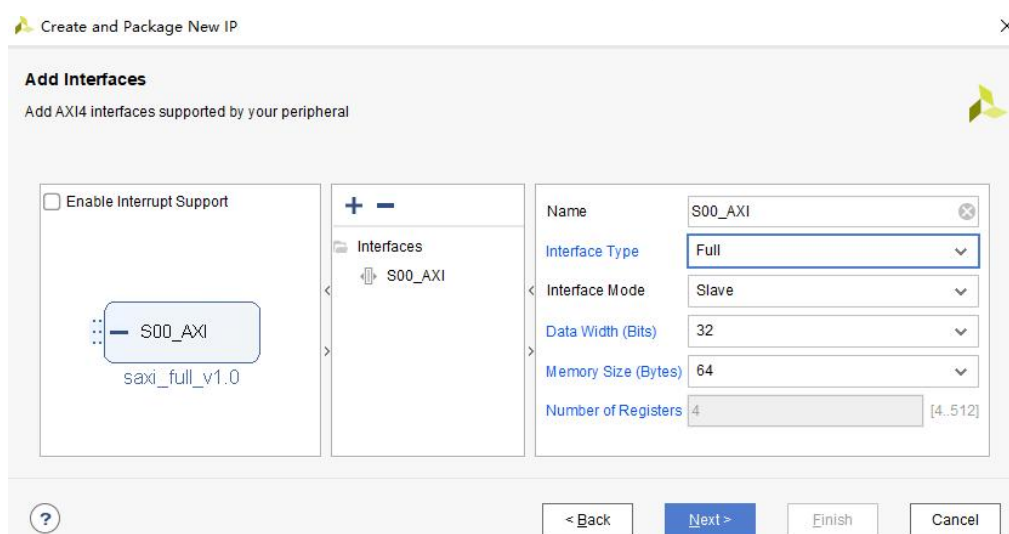
选择使用 vivado 自带的 AXI 总线模板创建一个 AXI4-FULL 接口 IP



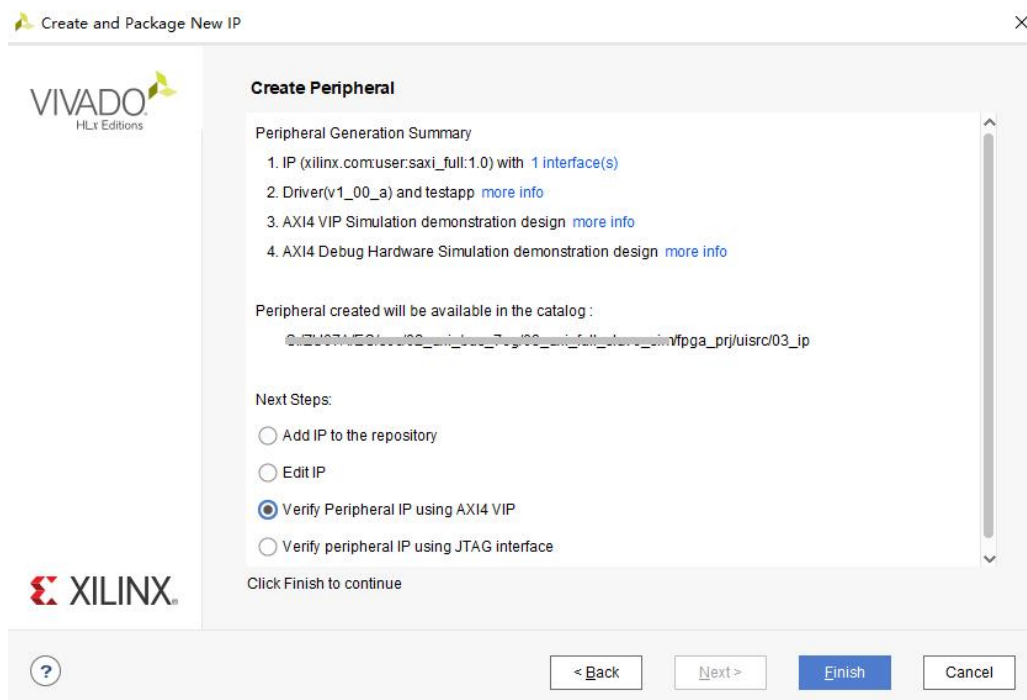
设置 IP 的名字为 saxi_full



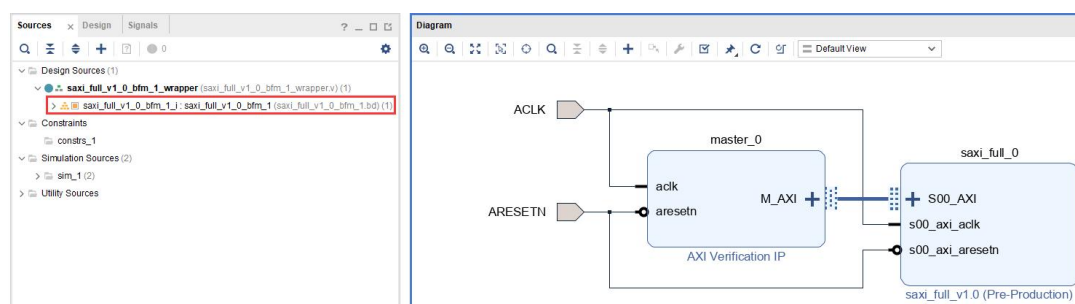
模板支持 3 中协议，分别是 AXI4-Full AXI4-Lite AXI4-Stream，这里选择 full;
总线包括 Master 和 Slave 两种模式，这里选择 Slave 模式



这里选择 Verify Peripheral IP using AXI4 VIP 可以对 AXI4-FULL 快速验证



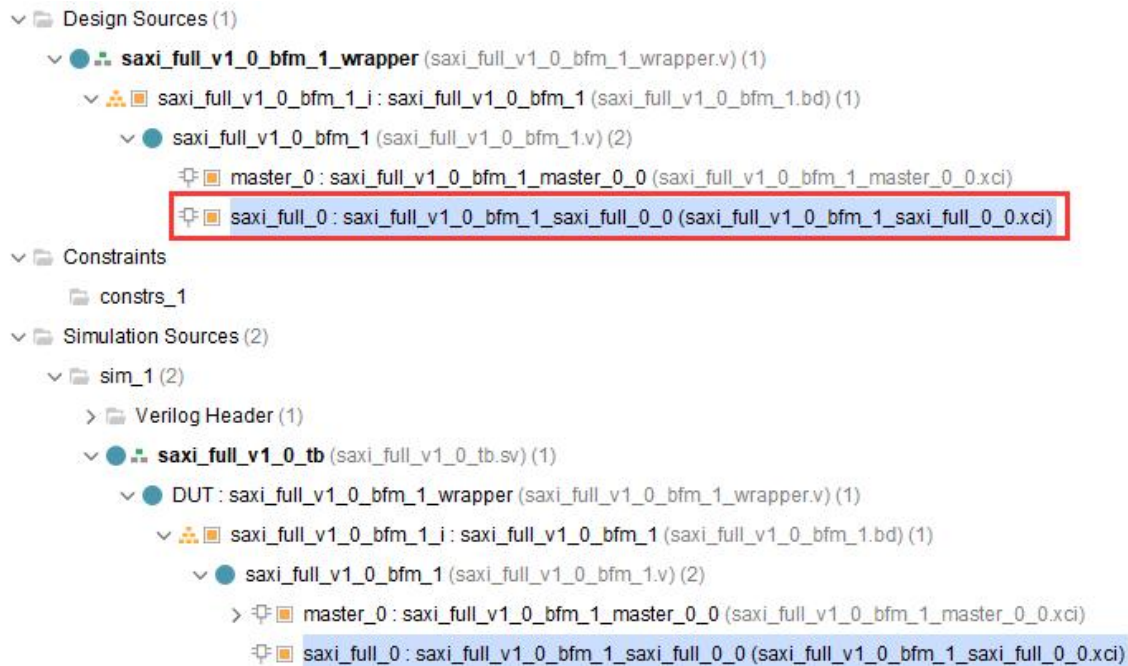
单击 Finish 后展开 VIVADO 自动产生的 demo，单击 Block Design 的工程，可以看到如下 2 个 IP。其中 saxi_full_0 就是我们自定义的 IP，另外一个 master_0 是用来读写我们自定义的 saxi_full_0，以此验证我们的 IP 正确性。



采用默认地址分配即可

Diagram x Address Editor x					
[Icons] [Assigned (1)] [Unassigned (0)] [Excluded (0)] [Hide All]					
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/master_0					
/master_0/Master_AXI (32 address bits : 4G)					
/saxi_full_0/S00_AXI	S00_AXI	S00_AXI_mem	0x7600_0000	64K	0x7600_FFFF

继续站看代码看看里面有什么东西



3.3 程序分析

1:axi-full-slave 的 axi_awready

当满足条件($\sim$ axi_awready && S_AXI_AWVALID && $\sim$ axi_awv_awr_flag && $\sim$ axi_arv_arr_flag)=1 的时候表示可以进行一次 AXI-FULL 的 burst 写操作了,这个时候 AXI-FULL-SLAVE 设置 axi_awready <= 1'b1 和 axi_awv_awr_flag <= 1'b1

```
// axi_awready is asserted for one S_AXI_ACLK clock cycle when both
// S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
// de-asserted when reset is low.
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_awready <= 1'b0;
        axi_awv_awr_flag <= 1'b0;
    end
    else
    begin
        if (~axi_awready && S_AXI_AWVALID && ~axi_awv_awr_flag && ~axi_arv_arr_flag)
        begin
            // slave is ready to accept an address and
            // associated control signals
            axi_awready <= 1'b1;
            axi_awv_awr_flag <= 1'b1;
            // used for generation of bresp() and bvalid
        end
        else if (S_AXI_WLAST && axi_wready)
        // preparing to accept next address after current write burst tx completion
        begin
            axi_awv_awr_flag <= 1'b0;
        end
        else
        begin
            axi_awready <= 1'b0;
        end
    end
end
```

2:axi-full-slave 的 axi_awaddr

AXI 的 burst 模式包括 3 种:

1:fixed burst 这种模式下地址都是相同的

2: incremental burst 这种模式下地址递增

3: Wrapping burst 这只模式下地址达到设置的地址边界后返回原来的地址。

本文 demo 种以下三种模式的具体代码如下:

```
// This process is used to latch the address when both
// S_AXI_AWVALID and S_AXI_WVALID are valid.
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_awaddr <= 0;
        axi_awlen_cntr <= 0;
        axi_awburst <= 0;
        axi_awlen <= 0;
    end
    else
    begin
        if (~axi_awready && S_AXI_AWVALID && ~axi_awv_awr_flag)
        begin
            // address latching
            axi_awaddr <= S_AXI_AWADDR[C_S_AXI_ADDR_WIDTH - 1:0];
            axi_awburst <= S_AXI_AWBURST;
            axi_awlen <= S_AXI_AWLEN;
            // start address of transfer
            axi_awlen_cntr <= 0;
        end
        else if((axi_awlen_cntr <= axi_awlen) && axi_wready && S_AXI_WVALID)
        begin

            axi_awlen_cntr <= axi_awlen_cntr + 1;

            case (axi_awburst)
                2'b00: // fixed burst
                    // The write address for all the beats in the transaction are fixed
                    begin
                        axi_awaddr <= axi_awaddr;
                        //for awsize = 4 bytes (010)
                    end
                2'b01: //incremental burst
                    // The write address for all the beats in the transaction are increments by awsize
                    begin
                        axi_awaddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] <= axi_awaddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] + 1;
                        //awaddr aligned to 4 byte boundary
                        axi_awaddr[ADDR_LSB-1:0] <= {ADDR_LSB{1'b0}};
                        //for awsize = 4 bytes (010)
                    end
                2'b10: //Wrapping burst
                    // The write address wraps when the address reaches wrap boundary
                    if (aw_wrap_en)
                    begin
                        axi_awaddr <= (axi_awaddr - aw_wrap_size);
                    end
                    else
                    begin
                        axi_awaddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] <= axi_awaddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] + 1;
                        axi_awaddr[ADDR_LSB-1:0] <= {ADDR_LSB{1'b0}};
                    end
            end
            default: //reserved (incremental burst for example)
```

```
begin
    axi_awaddr <= axi_awaddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] + 1;
    //for awsize = 4 bytes (010)
end
endcase
end
end
end
```

3:axi-full-slave 的 axi_wready

当满足条件(~axi_wready && S_AXI_WVALID && axi_awv_awr_flag)==1 设置 axi_wready 为 1.这里可以看出, S_AXI_WVALID 必须在一次 burst 种持续有效, 直到满足条件(S_AXI_WLAST && axi_wready), 否则 AXI-FULL-SLAVE 会出错, 这一点有别于 AXI-LITE-SLAVE 每次只读写一个数据。

```
// axi_wready is asserted for one S_AXI_ACLK clock cycle when both
// S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
// de-asserted when reset is low.
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_wready <= 1'b0;
    end
    else
    begin
        if (~axi_wready && S_AXI_WVALID && axi_awv_awr_flag)
        begin
            // slave can accept the write data
            axi_wready <= 1'b1;
        end
        //else if (~axi_awv_awr_flag)
        else if (S_AXI_WLAST && axi_wready)
        begin
            axi_wready <= 1'b0;
        end
    end
end
end
```

4:axi-full-slave 的 axi_bvalid 信号

axi_bvalid 用于告知 axi master axi-slave 端已经完成数据接收了

给出 ACK, 写操作 LAST 信号的下一个时钟, AXI-SLAVE 给出 ACK 信号

```
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_bvalid <= 0;
        axi_bresp <= 2'b0;
        axi_buser <= 0;
    end
    else
    begin
        if (axi_awv_awr_flag && axi_wready && S_AXI_WVALID && ~axi_bvalid && S_AXI_WLAST)
        begin
            axi_bvalid <= 1'b1;
            axi_bresp <= 2'b0;
            // 'OKAY' response
        end
    end
    else
    begin
        if (S_AXI_BREADY && axi_bvalid)
            //check if bready is asserted while bvalid is high)
        end
    end
end
```

```
        //(there is a possibility that bready is always asserted high)
        begin
            axi_bvalid <= 1'b0;
        end
    end
end
end
end
```

5:axi-full-slave 的 axi_arready 信号

当满足条件($\sim$ axi_arready && S_AXI_ARVALID && $\sim$ axi_awv_awr_flag && $\sim$ axi_arv_arr_flag)=1 的时候表示可以进行一次 AXI-FULL 的 burst 读操作了, 这个时候 AXI-FULL-SLAVE 设置 axi_arready <= 1'b1 和 axi_arv_arr_flag <= 1'b1

```
// axi_arready is asserted for one S_AXI_ACLK clock cycle when
// S_AXI_ARVALID is asserted. axi_arready is
// de-asserted when reset (active low) is asserted.
// The read address is also latched when S_AXI_ARVALID is
// asserted. axi_araddr is reset to zero on reset assertion.

always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_arready <= 1'b0;
        axi_arv_arr_flag <= 1'b0;
    end
    else
    begin
        if ( $\sim$ axi_arready && S_AXI_ARVALID &&  $\sim$ axi_awv_awr_flag &&  $\sim$ axi_arv_arr_flag)
        begin
            axi_arready <= 1'b1;
            axi_arv_arr_flag <= 1'b1;
        end
        else if (axi_rvalid && S_AXI_RREADY && axi_arlen_cnr == axi_arlen)
        // preparing to accept next address after current read completion
        begin
            axi_arv_arr_flag <= 1'b0;
        end
        else
        begin
            axi_arready <= 1'b0;
        end
    end
end
end
```

6:axi-full-slave 的 axi_araddr 信号

AXI-的读写操作几乎是相对的代码, AXI 的 burst 模式包括 3 种:

1:fixed burst 这种模式下地址都是相同的

2: incremental burst 这种模式下地址递增

3: Wrapping burst 这只模式下地址达到设置的最大地址边界后返回原来的地址。

本文 demo 种以下三种模式的具体代码如下:

```
//This process is used to latch the address when both
//S_AXI_ARVALID and S_AXI_RVALID are valid.
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_araddr <= 0;
        axi_arlen_cnr <= 0;
        axi_arburst <= 0;
        axi_arlen <= 0;
    end
end
```

```

    axi_rlast <= 1'b0;
    axi_ruser <= 0;
end
else
begin
    if (~axi_arready && S_AXI_ARVALID && ~axi_arv_arr_flag)
    begin
        // address latching
        axi_araddr <= S_AXI_ARADDR[C_S_AXI_ADDR_WIDTH - 1:0];
        axi_arburst <= S_AXI_ARBURST;
        axi_arlen <= S_AXI_ARLEN;
        // start address of transfer
        axi_arlen_cnr <= 0;
        axi_rlast <= 1'b0;
    end
    else if((axi_arlen_cnr <= axi_arlen) && axi_rvalid && S_AXI_RREADY)
    begin

        axi_arlen_cnr <= axi_arlen_cnr + 1;
        axi_rlast <= 1'b0;

        case (axi_arburst)
            2'b00: // fixed burst
                // The read address for all the beats in the transaction are fixed
                begin
                    axi_araddr <= axi_araddr;
                    //for arsize = 4 bytes (010)
                end
            2'b01: //incremental burst
                // The read address for all the beats in the transaction are increments by awsize
                begin
                    axi_araddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] <= axi_araddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] + 1;
                    //araddr aligned to 4 byte boundary
                    axi_araddr[ADDR_LSB-1:0] <= {ADDR_LSB{1'b0}};
                    //for awsize = 4 bytes (010)
                end
            2'b10: //Wrapping burst
                // The read address wraps when the address reaches wrap boundary
                if (ar_wrap_en)
                begin
                    axi_araddr <= (axi_araddr - ar_wrap_size);
                end
            else
                begin
                    axi_araddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] <= axi_araddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB] + 1;
                    //araddr aligned to 4 byte boundary
                    axi_araddr[ADDR_LSB-1:0] <= {ADDR_LSB{1'b0}};
                end
            default: //reserved (incremental burst for example)
                begin
                    axi_araddr <= axi_araddr[C_S_AXI_ADDR_WIDTH - 1:ADDR_LSB]+1;
                    //for arsize = 4 bytes (010)
                end
        endcase
    end
    else if((axi_arlen_cnr == axi_arlen) && ~axi_rlast && axi_arv_arr_flag )
    begin
        axi_rlast <= 1'b1;
    end
    else if(S_AXI_RREADY)
    begin
        axi_rlast <= 1'b0;
    end
end

```

```

end
end

```

7:axi-full-slave 的 axi_rvalid 信号

在用 VIVADO 模板产生的 demo 种, 读操作数据不是连续读的, 通过 axi_rvalid 设置 AXI-SLAVE FULL 读数据有效。

```

always @(posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
        begin
            axi_rvalid <= 0;
            axi_rresp  <= 0;
        end
    else
        begin
            if (axi_arv_arr_flag && ~axi_rvalid)
                begin
                    axi_rvalid <= 1'b1;
                    axi_rresp  <= 2'b0;
                    // 'OKAY' response
                end
            else if (axi_rvalid && S_AXI_RREADY)
                begin
                    axi_rvalid <= 1'b0;
                end
            end
        end
    end
end
end

```

8:数据保存到 block ram

以下是利用 block ram 完成数据的保存和回读

```

// implement Block RAM(s)
generate
    for(i=0; i<= USER_NUM_MEM-1; i=i+1)
        begin:BRAM_GEN
            wire mem_rden;
            wire mem_wren;

            assign mem_wren = axi_wready && S_AXI_WVALID ;

            assign mem_rden = axi_arv_arr_flag ; //& ~axi_rvalid

            for(mem_byte_index=0; mem_byte_index<= (C_S_AXI_DATA_WIDTH/8-1); mem_byte_index=mem_byte_index+1)
                begin:BYTE_BRAM_GEN
                    wire [8-1:0] data_in ;
                    wire [8-1:0] data_out;
                    reg  [8-1:0] byte_ram [0 : 15];
                    integer  j;

                    //assigning 8 bit data
                    assign data_in  = S_AXI_WDATA[(mem_byte_index*8+7) -: 8];
                    assign data_out = byte_ram[mem_address];

                    always @(posedge S_AXI_ACLK )
                        begin
                            if (mem_wren && S_AXI_WSTRB[mem_byte_index])
                                begin
                                    byte_ram[mem_address] <= data_in;
                                end
                            end
                        end

                    always @(posedge S_AXI_ACLK )

```


04AXI4 总线 axi-full-master

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

4.1 概述

使用 XILINX 的软件工具 VIVADO 以及 XILINX 的 7 代以上的 FPGA 或者 SOC 掌握 AXI-4 总线结束, 并且可以灵活使用 AXI-4 总线技术完成数据的交换, 可以让我们在构建强大的 FPGA 内部总线数据互联互通方面取得高效、高速、标准化的优势。

关于 AXI4 总线协议的部分介绍请阅读“01AXI4 总线 axi-lite-slave”。

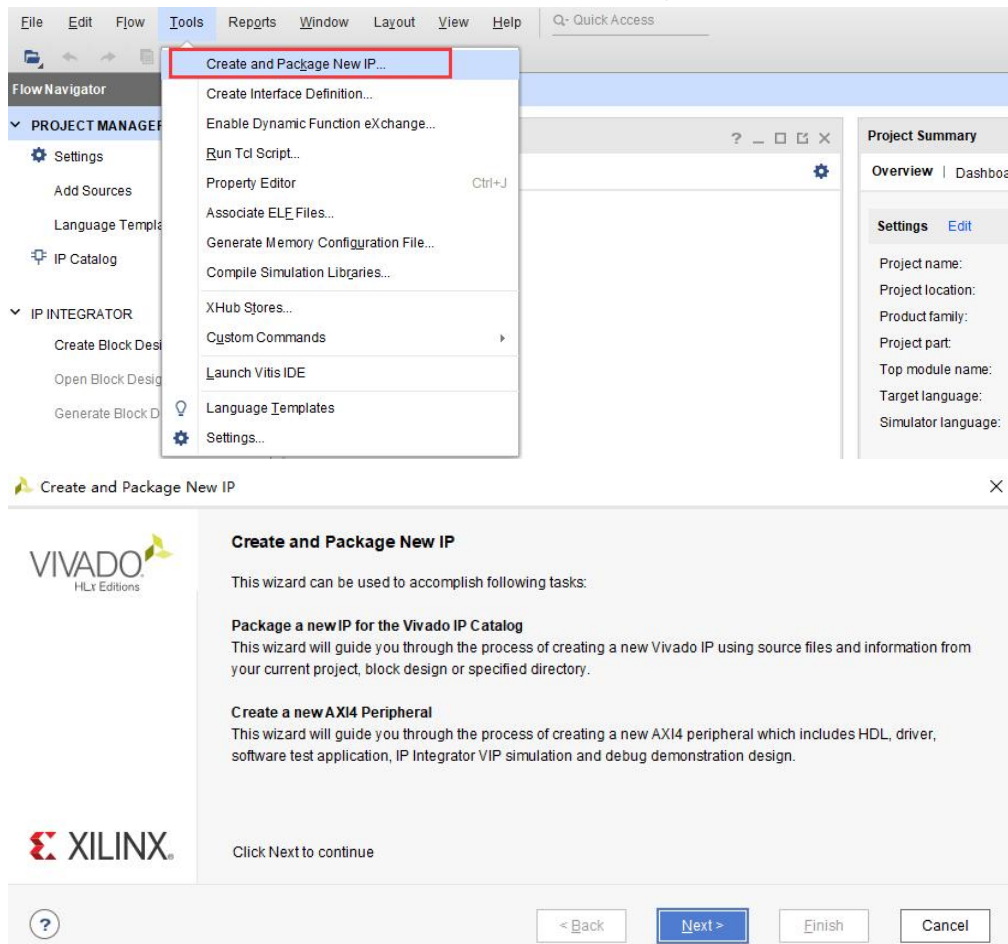
本文实验目的:

- 1:掌握基于 VIVADO 工具产生 AXI 协议模板
- 2:掌握通过 VIVADO 工具产生 AXI-full-master 代码
- 3:理解 AXI-full-master 中自定义寄存器的地址分配
- 4:掌握通过 VIVADO 封装 AXI-full-slave 图形化 IP
- 5:通过仿真验证 AXI-full-master IP 的工作是否正常。

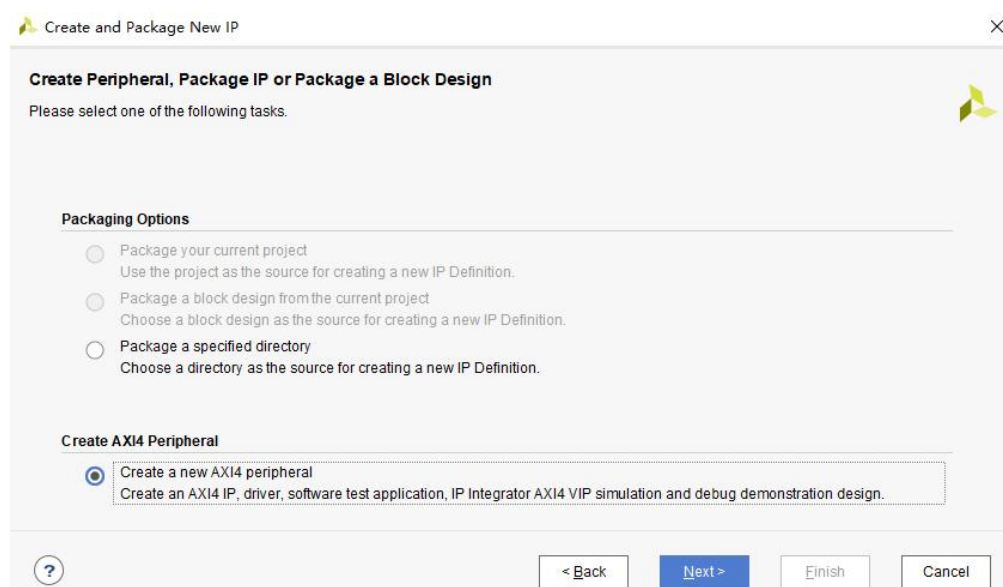
4.2 创建 axi4-full-master 总线接口 IP

新建 fpga 工程, 过程省略

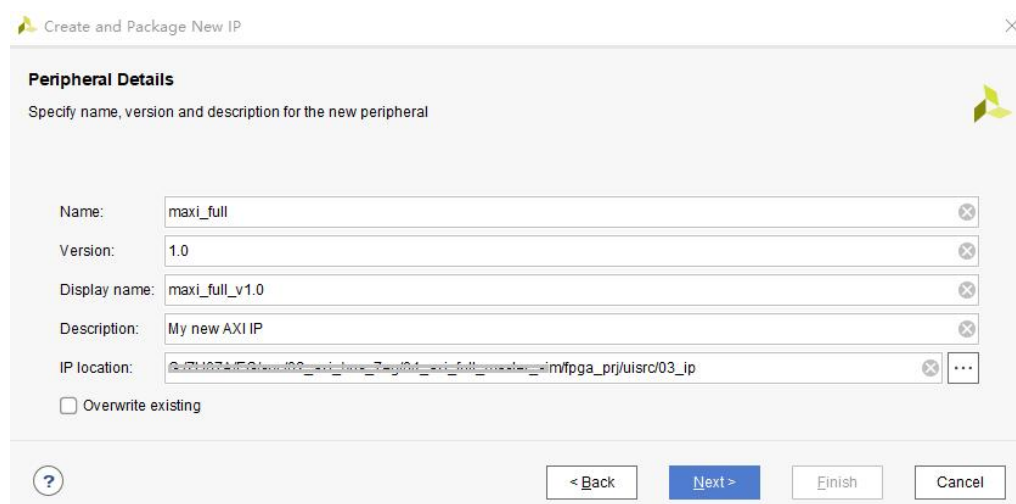
新建完成工程后, 单击菜单栏 Tools->Create and Package New IP,开始创建一个 AXI4-Full 接口总线 IP



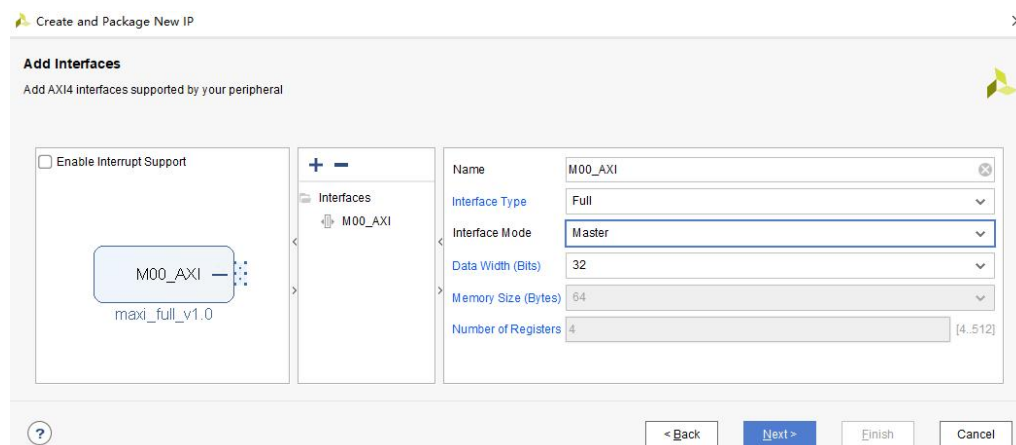
选择使用 vivado 自带的 AXI 总线模板创建一个 AXI4-FULL 接口 IP



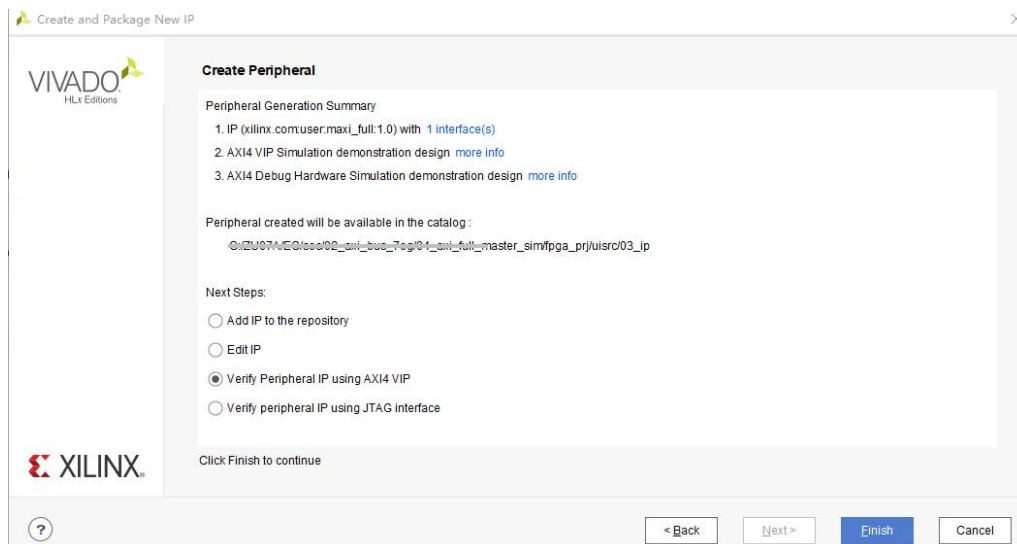
设置 IP 的名字为 maxi_full



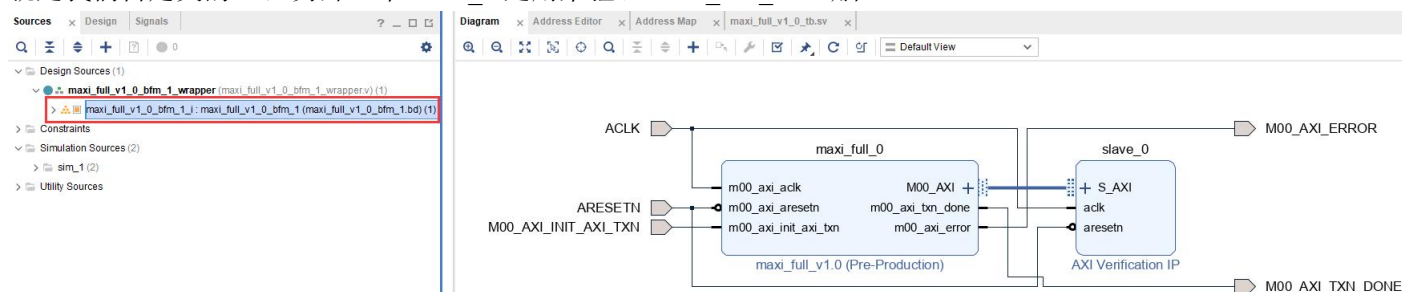
模板支持 3 中协议，分别是 AXI4-Full AXI4-Lite AXI4-Stream，这选择 Full;总线包括 Master 和 Slave 两种模式，这里选择 Master 模式



这里选择 Verify Peripheral IP using AXI4 VIP 可以对 AXI4-Lite 快速验证



单击 Finish 后展开 VIVADO 自动产生的 demo，单击 Block Design 的工程，可以看到如下 2 个 IP。其中 maxi_full_0 就是我们自定义的 IP，另外一个 slave_0 是用来验证 maxi_full_0 正确性。



采用默认地址分配即可

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
/maxi_full_0					
/maxi_full_0/M00_AXI (32 address bits : 4G)					
/slave_0/S_AXI	S_AXI	Reg	0x44A0_0000	64K	0x44A0_FFFF

继续站看代码看看里面有什么东西



路径 uisrc/03_ip/ maxi_full_1.0/hdl 路径下的 maxi_full_v1_0_M00_AXI.v 就是我们的源码。另外一个 maxi_full_v1_0.v 是软件产生了一个接口文件，如果我们自己定义 IP 可有可无。

4.3 程序分析

axi 总线信号的关键无非是地址和数据，而写地址的有效取决于 AXI_AWVALID 和 AXI_AWREADY，写数据的有效取决于 S_AXI_WVALID 和 S_AXI_WREADY。同理，读地址的有效取决于 AXI_ARVALID 和 AXI_ARREADY，读数据的有效取决于 S_AXI_RVALID 和 S_AXI_RREADY。所以下代码的阅读分析注意也是围绕以上 4 个信号的有效时序。

以下程序我们把关键信号的代码拆分阅读

1:产生初始化信号

```
//Generate a pulse to initiate AXI transaction.
always @(posedge M_AXI_ACLK)
begin
    // Initiates AXI transaction delay
    if (M_AXI_ARESETN == 0 )
    begin
        init_txn_ff <= 1'b0;
        init_txn_ff2 <= 1'b0;
    end
    else
    begin
        init_txn_ff <= INIT_AXI_TXN;
        init_txn_ff2 <= init_txn_ff;
    end
end
end
```

2:axi-full-master 的 axi_awvalid

当 (~axi_awvalid && start_single_burst_write) == 1 条件满足，开始一次写传输，设置 axi_awvalid 有效。

```
always @(posedge M_AXI_ACLK)
begin

    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
    begin
        axi_awvalid <= 1'b0;
    end
    // If previously not valid , start next transaction
    else if (~axi_awvalid && start_single_burst_write)
    begin
        axi_awvalid <= 1'b1;
    end
    /* Once asserted, VALIDs cannot be deasserted, so axi_awvalid
    must wait until transaction is accepted */
    else if (M_AXI_AWREADY && axi_awvalid)
    begin
        axi_awvalid <= 1'b0;
    end
    else
    axi_awvalid <= axi_awvalid;
end
end
```

3:axi-full-slave 的 axi_awaddr

写通道地址每当 M_AXI_AWREADY && axi_awvalid 地址加 1

```
// Next address after AWREADY indicates previous address acceptance
always @(posedge M_AXI_ACLK)
begin

    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
    begin
```

```
        axi_awaddr <= 'b0;

    end

    else if (M_AXI_AWREADY && axi_awvalid)

        begin

            axi_awaddr <= axi_awaddr + burst_size_bytes;

        end

    else

        axi_awaddr <= axi_awaddr;

    end
```

4:axi-full-master 的 axi_wvalid

设置 axi_wvalid <= 1'b1 开始写数据。wnext 信号有效代码 axi_full_master 写数据有效。

```
assign wnext = M_AXI_WREADY & axi_wvalid;

// WVALID logic, similar to the axi_awvalid always block above
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
        begin
            axi_wvalid <= 1'b0;
        end
    // If previously not valid, start next transaction
    else if (~axi_wvalid && start_single_burst_write)
        begin
            axi_wvalid <= 1'b1;
        end
    /* If WREADY and too many writes, throttle WVALID
    Once asserted, VALIDs cannot be deasserted, so WVALID
    must wait until burst is complete with WLAST */
    else if (wnext && axi_wlast)
        axi_wvalid <= 1'b0;
    else
        axi_wvalid <= axi_wvalid;
end
```

5:axi-full-master 的 axi_master_last

axi_master_last 信号, 当条件满足(((write_index == C_M_AXI_BURST_LEN-2 && C_M_AXI_BURST_LEN >= 2) && wnext) || (C_M_AXI_BURST_LEN == 1)) == 1 的时候, axi_wlast <= 1'b1。这是 VIVADO 自带的模板, 但是这里有个 bug, 那就是必须确保 slave 可以连续接收数据, 假设发送 last 的时候 wnext==0, 这样就不能把最后一个数据正确写入到 slave 中了。

```
//WLAST generation on the MSB of a counter underflow
// WVALID logic, similar to the axi_awvalid always block above
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
        begin
            axi_wlast <= 1'b0;
        end
    // axi_wlast is asserted when the write index
    // count reaches the penultimate count to synchronize
    // with the last write data when write_index is b1111
    // else if (&(write_index[C_TRANSACTIONS_NUM-1:1])&& ~write_index[0] && wnext)
    else if (((write_index == C_M_AXI_BURST_LEN-2 && C_M_AXI_BURST_LEN >= 2) && wnext) || (C_M_AXI_BURST_LEN == 1))
        begin
            axi_wlast <= 1'b1;
        end
    // Deassert axi_wlast when the last write data has been
    // accepted by the slave with a valid response
```

```

else if (wnext)
    axi_wlast <= 1'b0;
else if (axi_wlast && C_M_AXI_BURST_LEN == 1)
    axi_wlast <= 1'b0;
else
    axi_wlast <= axi_wlast;
end

```

删除以上代码，并且添加以下代码：

```

wire  wlast = (write_index == C_M_AXI_BURST_LEN-1) && wnext;
reg    wlast_r1 = 1'b0;
always @(posedge M_AXI_ACLK)
    wlast_r1 <= wlast;

assign  axi_wlast = (wlast_r1==1'b0)&&(wlast==1'b1);

```

另外需要修改 reg axi_wlast; 为 wire axi_wlast;
这样就可以确保发送 wlast 的时候数据肯定是有效的。

6:写次数记录 write_index 计数器

```

/* Burst length counter. Uses extra counter register bit to indicate terminal
count to reduce decode logic */
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 || start_single_burst_write == 1'b1)
        begin
            write_index <= 0;
        end
    else if (wnext && (write_index != C_M_AXI_BURST_LEN-1))
        begin
            write_index <= write_index + 1;
        end
    else
        write_index <= write_index;
end

```

7:axi-full-master 的 axi_wdata

axi_full_master 写数据计数写数据

```

/* Write Data Generator

Data pattern is only a simple incrementing count from 0 for each burst */
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        axi_wdata <= 'b1;
    //else if (wnext && axi_wlast)
    //    axi_wdata <= 'b0;
    else if (wnext)
        axi_wdata <= axi_wdata + 1;
    else
        axi_wdata <= axi_wdata;
end

```

8:axi-full-master 的 axi_bready

设置 axi_bready 信号

```

always @(posedge M_AXI_ACLK)

```

```
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
        begin
            axi_bready <= 1'b0;
        end
        // accept/acknowledge bresp with axi_bready by the master
        // when M_AXI_BVALID is asserted by slave
        else if (M_AXI_BVALID && ~axi_bready)
            begin
                axi_bready <= 1'b1;
            end
            // deassert after one clock cycle
        else if (axi_bready)
            begin
                axi_bready <= 1'b0;
            end
            // retain the previous value
        else
            axi_bready <= axi_bready;
        end
    end
```

9:axi-full-slave 的 axi_arvalid

Axi_full_master 读通道的分析非常类似，代码是对称的。

当 ($\sim$ axi_arvalid && start_single_burst_read) == 1 条件满足，开始一次写传输，设置 axi_arvalid 有效。

```
always @(posedge M_AXI_ACLK)
begin

    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
        begin
            axi_arvalid <= 1'b0;
        end
        // If previously not valid , start next transaction
        else if (~axi_arvalid && start_single_burst_read)
            begin
                axi_arvalid <= 1'b1;
            end
            else if (M_AXI_ARREADY && axi_arvalid)
                begin
                    axi_arvalid <= 1'b0;
                end
            else
                axi_arvalid <= axi_arvalid;
            end
    end
```

10:axi-full-slave 的 axi_araddr

读地址计算

```
// Next address after ARREADY indicates previous address acceptance
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            axi_araddr <= 'b0;
        end
        else if (M_AXI_ARREADY && axi_arvalid)
```

```
begin
    axi_araddr <= axi_araddr + burst_size_bytes;
end
else
    axi_araddr <= axi_araddr;
end
```

11:axi-full-master 的 axi_rready

读数据准备好

```
/*
The Read Data channel returns the results of the read request
In this example the data checker is always able to accept
more data, so no need to throttle the RREADY signal
*/
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 )
        begin
            axi_rready <= 1'b0;
        end
    // accept/acknowledge rdata/rresp with axi_rready by the master
    // when M_AXI_RVALID is asserted by slave
    else if (M_AXI_RVALID)
        begin
            if (M_AXI_RLAST && axi_rready)
                begin
                    axi_rready <= 1'b0;
                end
            else
                begin
                    axi_rready <= 1'b1;
                end
            end
        end
    // retain the previous value
end
```

12:读次数记录 read_index 计数器

读数据索引计数

```
assign rnext = M_AXI_RVALID && axi_rready;
// Burst length counter. Uses extra counter register bit to indicate
// terminal count to reduce decode logic
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1 || start_single_burst_read)
        begin
            read_index <= 0;
        end
    else if (rnext && (read_index != C_M_AXI_BURST_LEN-1))
        begin
            read_index <= read_index + 1;
        end
    else
        read_index <= read_index;
end
```

13:产生对比数据 expected_rdata

数据 expected_rdata 用于和读出的 M_AXI_RDATA 进行对比以此验证数据的正确性。

```
//Generate expected read data to check against actual read data

always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1) // M_AXI_RLAST)
        expected_rdata <= 'b1;
    else if (M_AXI_RVALID && axi_rready)
        expected_rdata <= expected_rdata + 1;
    else
        expected_rdata <= expected_rdata;
end
```

14:比对数据正确性

读写数据比较

```
//Check received read data against data generator

always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        begin
            read_mismatch <= 1'b0;
        end
    //Only check data when RVALID is active
    else if (rnext && (M_AXI_RDATA != expected_rdata))
        begin
            read_mismatch <= 1'b1;
        end
    else
        read_mismatch <= 1'b0;
end
```

15:读写状态机

读写状态机源码

```
//implement master command interface state machine

always @ ( posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 1'b0 )
        begin
            // reset condition
            // All the signals are assigned default values under reset condition

            mst_exec_state      <= IDLE;
            start_single_burst_write <= 1'b0;
            start_single_burst_read  <= 1'b0;
            compare_done         <= 1'b0;
            ERROR <= 1'b0;
        end
    else
```

```
begin

// state transition

case (mst_exec_state)

  IDLE:

    // This state is responsible to wait for user defined C_M_START_COUNT

    // number of clock cycles.

    if ( init_txn_pulse == 1'b1)

      begin

        mst_exec_state  <= INIT_WRITE;

        ERROR <= 1'b0;

        compare_done <= 1'b0;

      end

    else

      begin

        mst_exec_state  <= IDLE;

      end

  INIT_WRITE:

    // This state is responsible to issue start_single_write pulse to

    // initiate a write transaction. Write transactions will be

    // issued until burst_write_active signal is asserted.

    // write controller

    if (writes_done)

      begin

        mst_exec_state <= INIT_READ;//

      end

    else

      begin

        mst_exec_state  <= INIT_WRITE;

        if (~axi_awvalid && ~start_single_burst_write && ~burst_write_active)

          begin

            start_single_burst_write <= 1'b1;

          end

        else

          begin

            start_single_burst_write <= 1'b0; //Negate to generate a pulse

          end

        end

      end

  INIT_READ:

    // This state is responsible to issue start_single_read pulse to

    // initiate a read transaction. Read transactions will be

    // issued until burst_read_active signal is asserted.

    // read controller

    if (reads_done)

      begin

        mst_exec_state <= INIT_COMPARE;

      end

    else

      begin

        mst_exec_state  <= INIT_READ;
```

```

        if (~axi_arvalid && ~burst_read_active && ~start_single_burst_read)
            begin
                start_single_burst_read <= 1'b1;
            end
        else
            begin
                start_single_burst_read <= 1'b0; //Negate to generate a pulse
            end
        end

    end

INIT_COMPARE:

    // This state is responsible to issue the state of comparison
    // of written data with the read data. If no error flags are set,
    // compare_done signal will be asserted to indicate success.
    //if (~error_reg)

    begin
        ERROR <= error_reg;

        mst_exec_state <= IDLE;

        compare_done <= 1'b1;

    end

default :
    begin
        mst_exec_state <= IDLE;

    end

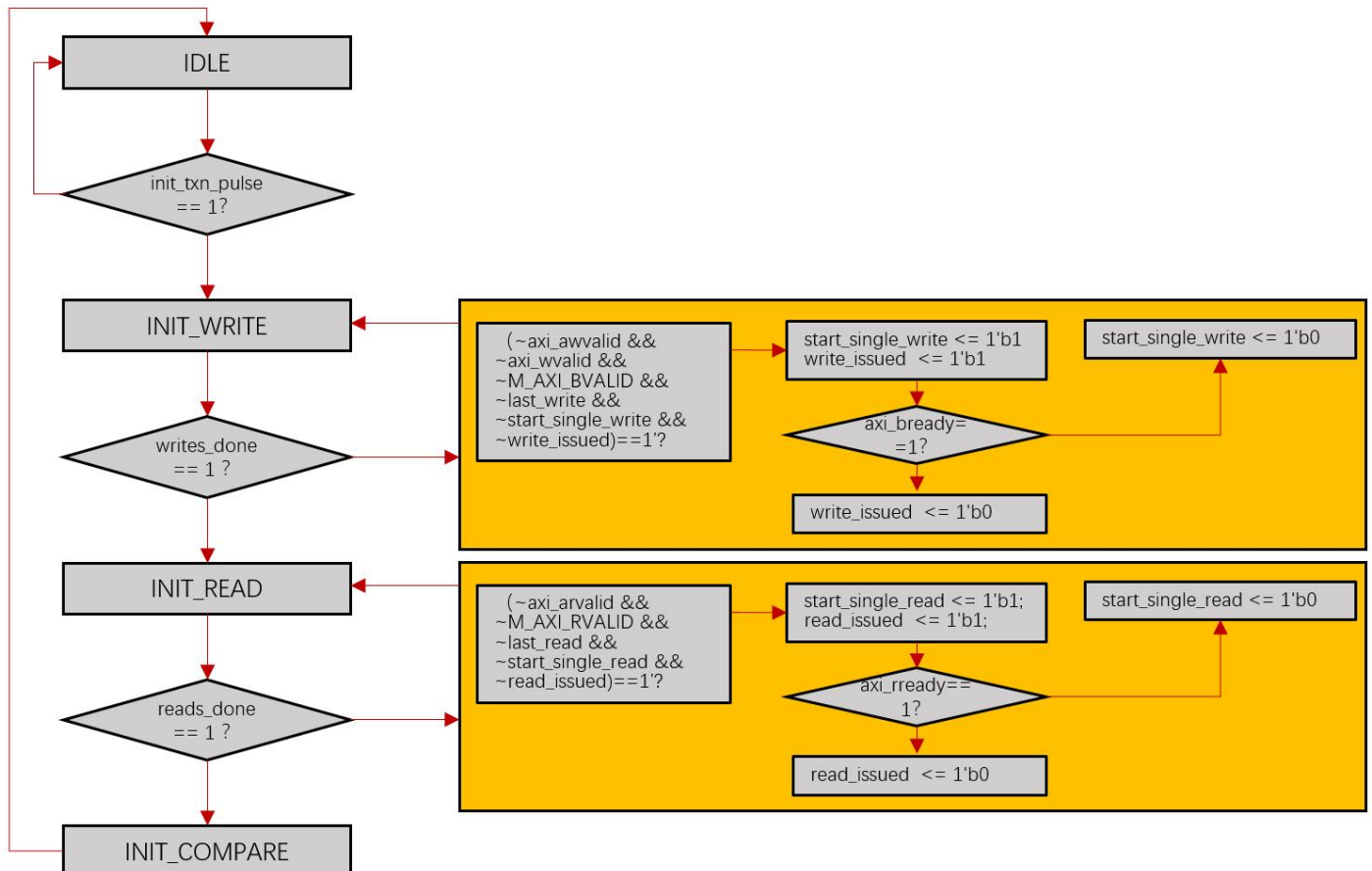
endcase

end

end //MASTER_EXECUTION_PROC

```

整理成流程图，更加容易理解：



16:正在写 burst_write_active

burst_write_active 代表正在写操作

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        burst_write_active <= 1'b0;

    //The burst_write_active is asserted when a write burst transaction is initiated
    else if (start_single_burst_write)
        burst_write_active <= 1'b1;
    else if (M_AXI_BVALID && axi_bready)
        burst_write_active <= 0;
end
```

17:写完成 writes_done

写数据完成 writes_done 信号

```
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        writes_done <= 1'b0;

    //The writes_done should be associated with a bready response
    //else if (M_AXI_BVALID && axi_bready && (write_burst_counter == {(C_NO_BURSTS_REQ-1){1}}) && axi_wlast)
    else if (M_AXI_BVALID && (write_burst_counter[C_NO_BURSTS_REQ]) && axi_bready)
        writes_done <= 1'b1;
    else
        writes_done <= writes_done;
end
```

18:正在读 burst_read_active

读 burst_read_active 代表正在读数据

```
// burst_read_active signal is asserted when there is a burst write transaction
// is initiated by the assertion of start_single_burst_write. start_single_burst_read
// signal remains asserted until the burst read is accepted by the master
always @(posedge M_AXI_ACLK)
begin
    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)
        burst_read_active <= 1'b0;

    //The burst_write_active is asserted when a write burst transaction is initiated
    else if (start_single_burst_read)
        burst_read_active <= 1'b1;
    else if (M_AXI_RVALID && axi_rready && M_AXI_RLAST)
        burst_read_active <= 0;
end
```

19:读完成 reads_done

读数据完成 reads_done 信号

```
always @(posedge M_AXI_ACLK)

begin

    if (M_AXI_ARESETN == 0 || init_txn_pulse == 1'b1)

        reads_done <= 1'b0;

//The reads_done should be associated with a ready response

//else if (M_AXI_BVALID && axi_bready && (write_burst_counter == {(C_NO_BURSTS_REQ-1){1}}) && axi_wlast)

else if (M_AXI_RVALID && axi_rready && (read_index == C_M_AXI_BURST_LEN-1) && (read_burst_counter[C_NO_BURSTS_REQ]))

    reads_done <= 1'b1;

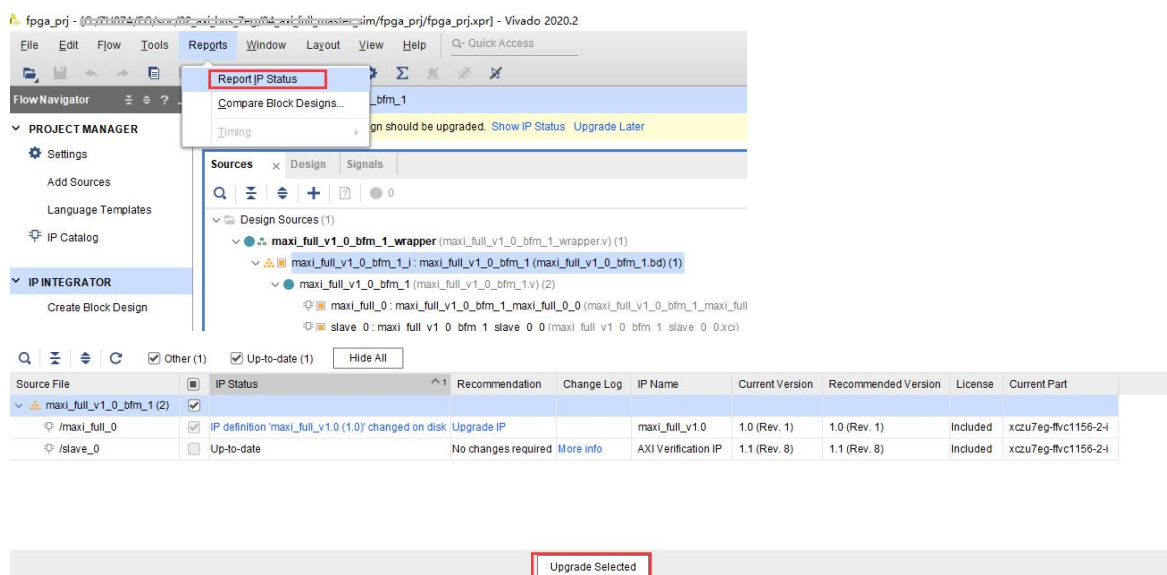
else

    reads_done <= reads_done;

end
```

20:IP 的更新

由于修改了代码，需要先更新 IP 状态，完成 IP 更新

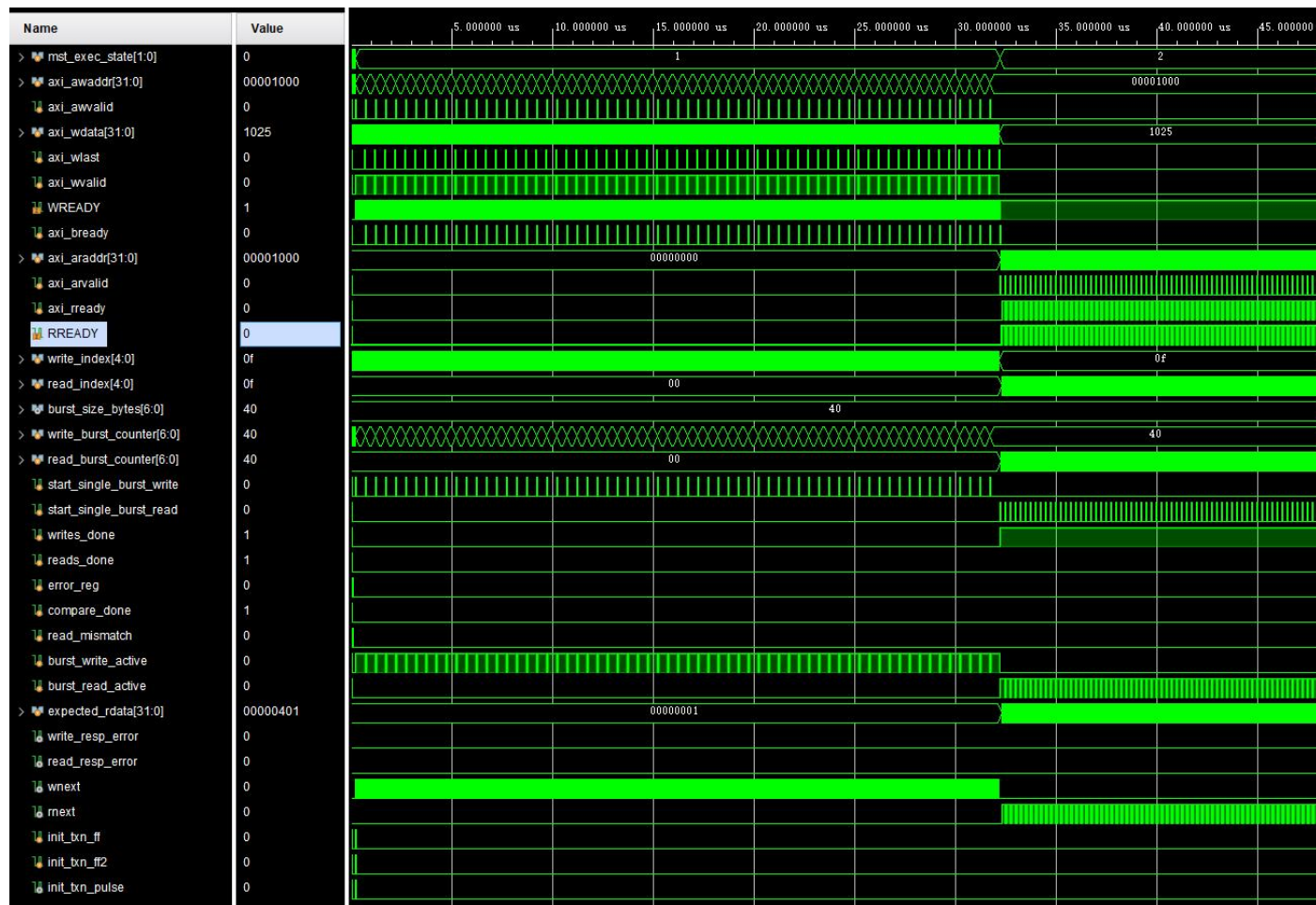


Source File	IP Status	Recommendation	Change Log	IP Name	Current Version	Recommended Version	License	Current Part
maxi_full_v1_0_bfm_1 (2)	☒	Upgrade IP		maxi_full_v1_0	1.0 (Rev. 1)	1.0 (Rev. 1)	Included	xczu7eg-flvc1156-2-i
/maxi_full_0	☒	IP definition 'maxi_full_v1_0 (1.0)' changed on disk	Upgrade IP	maxi_full_v1_0	1.0 (Rev. 1)	1.0 (Rev. 1)	Included	xczu7eg-flvc1156-2-i
/slave_0	☐	Up-to-date	No changes required	AXI Verification IP	1.1 (Rev. 8)	1.1 (Rev. 8)	Included	xczu7eg-flvc1156-2-i

Upgrade Selected

4.4 实验结果

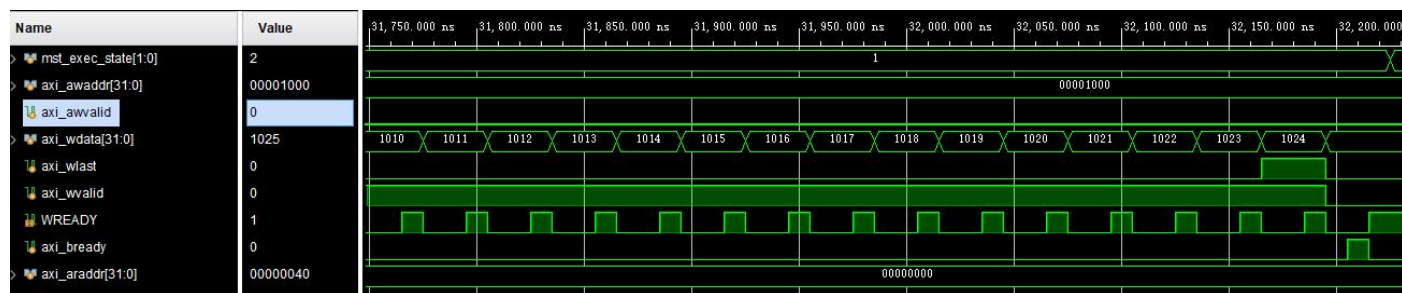
仿真结果



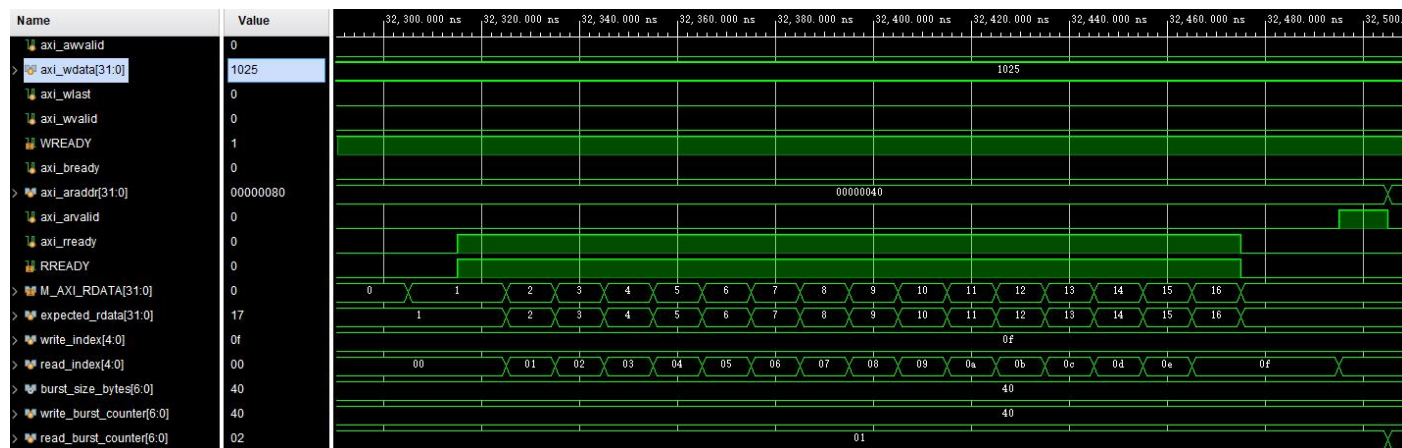
一次 axi4 写操作 burst lenh=16 如下图所示，由于 WREADY 信号不是连续的，所以可以传输效率不是最高的



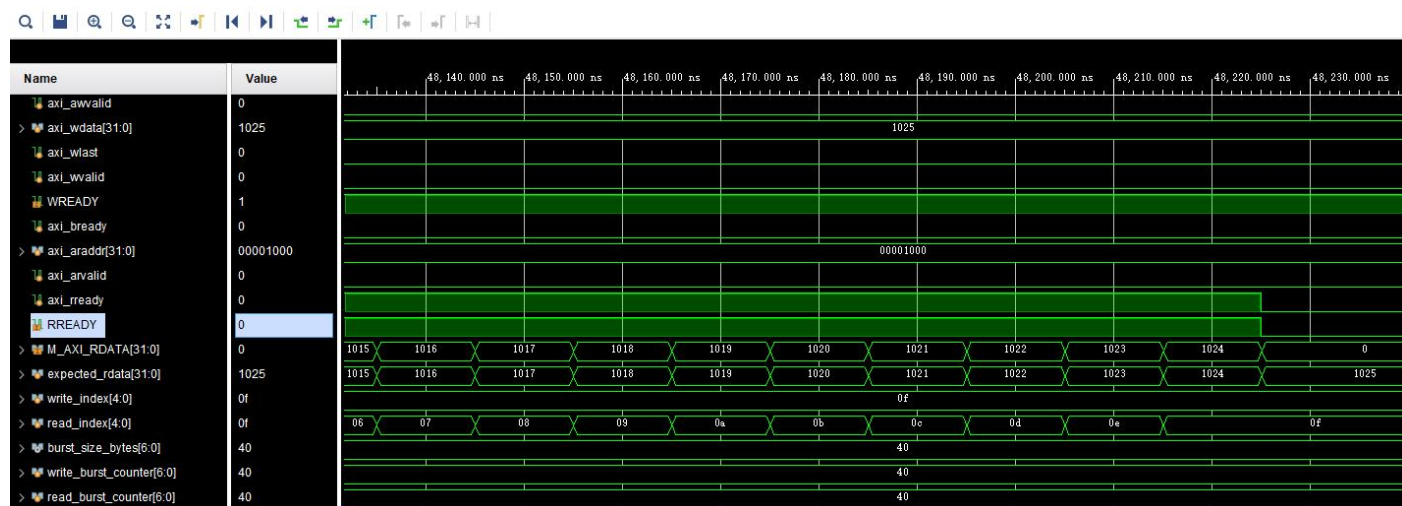
一共进行 64 次 burst 共计写了 1024 个 32bit 数据



一次读操作的 burst length 也是 16 如下图，但是可以看到读数据时连续的，所以效率最高



一共进行 64 次 burst 共计读了 1024 个 32bit 数据



可以看到读出的数据 RDATA 和 expected_rdata 一致，所以代码正确。

05AXI4 总线 axi-stream

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

5.1 概述

AXI4-Stream 去掉了地址, 允许无限制的数据突发传输规模, AXI4-Stream 接口在数据流传输中应用非常方便, 本来首先介绍了 AXI4-Stream 协议的型号定义, 并且给出了一些 Stream 接口的时序方案图。之后通过 VIVADO 自带的 AXI4 模板, 创建 axi-stream-master 和 axi-stream-slave ip。通过图形设计连线, 添加仿真激励完成验证。

本文实验目的:

- 1:掌握基于 VIVADO 工具产生 AXI 协议模板
- 2:掌握通过 VIVADO 工具产生 AXI-Stream 代码
- 3:掌握通过 VIVADO 封装 AXI-Stream 图形化 IP
- 4:通过仿真验证 AXI-Stream IP 的工作是否正常。

5.2AXI4-Stream 协议介绍

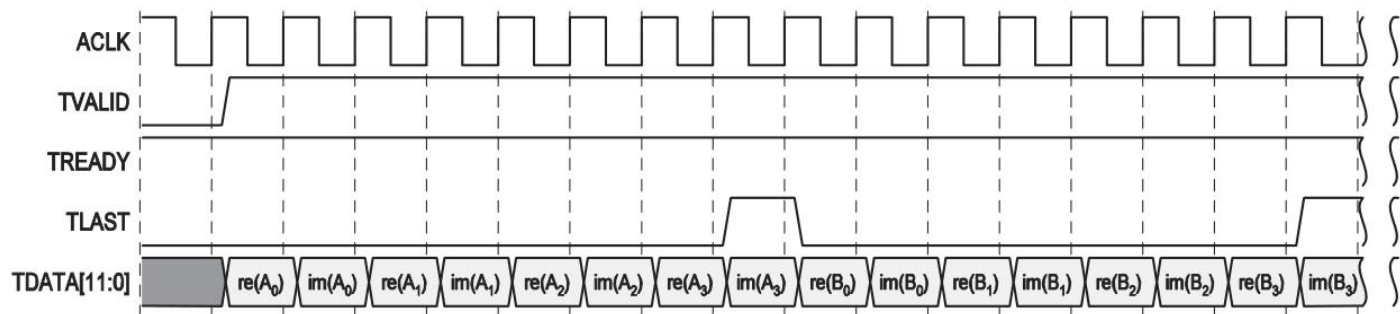
5.2.1 信号定义

信号	源	描述
ACLK	时钟源	信号在 ACLK 信号上升沿采样
ARESETn	复位源	复位信号,ARESETn 低电平有效
TVALID	master	TVALID 代表主设备数据有效, 当 TVALID 和 TREADY 同时有效完成数据收发
TREADY	slave	TREADY 表示从设备准备可以, 主设备可以发送数据, 当 TVALID 和 TREADY 同时有效完成数据收发
TDATA[(8n-1):0]	master	TDATA axi-stream 的数据
TSTRB[(n-1): 0]	master	TSTRB[n-1:0]对应的 bit 位为 1 代表对应的字节有效, 否则无效, 但是会占用这个数据位。
TKEEP[(n-1): 0]	master	KEEP[n-1:0]对应的 bit 位为 1 代表对应的字节有效, 否则为空, 可以丢掉。
TLAST	master	TLAST 代表最后一个数据。
TID[(i-1):0]	master	TID 是数据流的标识符, 用来表明不同的数据流。
TDEST[(d-1):0]	master	TDEST 为据流提供路由信息。
TUSER[(n-1):0]	master	TUSER 一般用于数据的同步, 代表 stream 数据的开始。

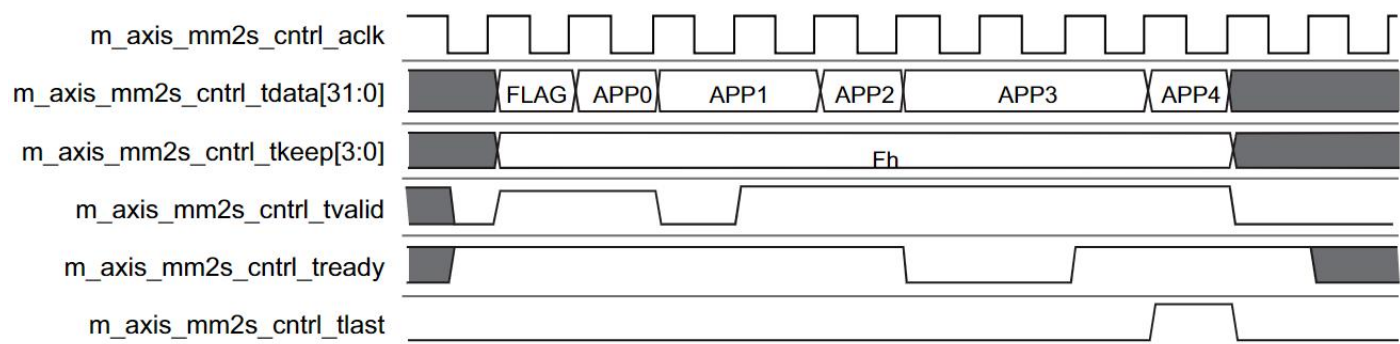
以上所有信号, 在 axi-stream 传输中, 不一定全部用到, 具体根据应用场景的情况而定。

5.2.1axi-stream 方案展示

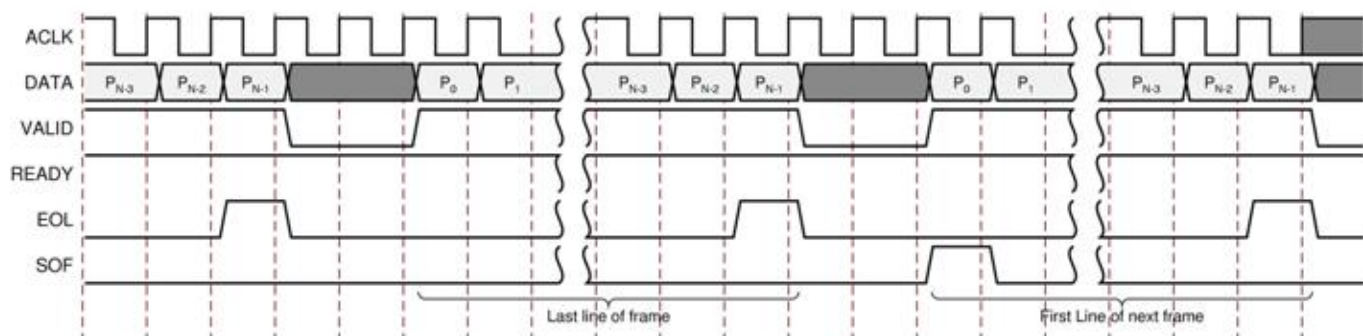
下图中除了 ACLK 外, axi-stream 的信号用到了, TVALID、TREADY、TLAST、TDATA。其中 TDATA 虽然是 12bit 但是实际上会占用 16bit 的物理总线。并且数据是循环发送, 用 TLAST 标识了一次循环的最后一个数据。



下图中截图来自 AXI-DMA mm2s 接口的时序图，除了 ACLK 外，axi-stream 的信号用到了，TVALID、TREADY、TLAST、TDATA、TKEEP。用 TLAST 标识了一次循环的最后一个数据。

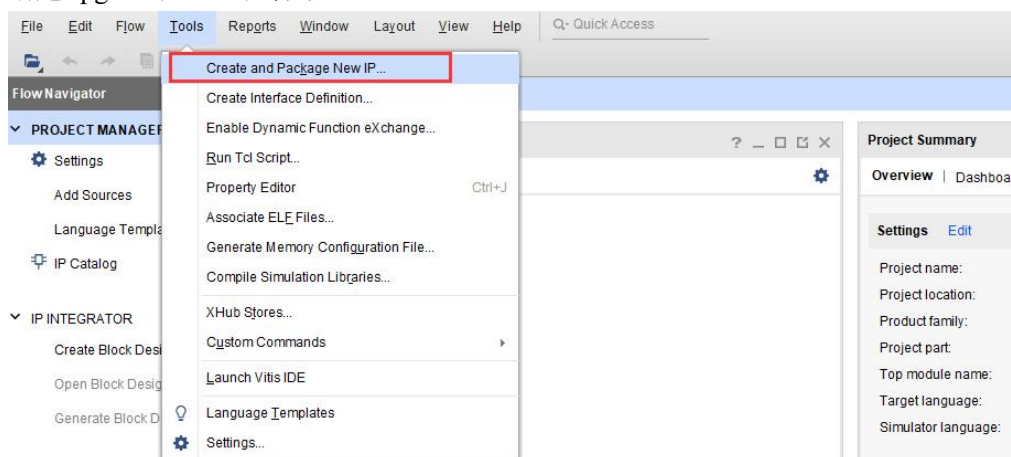


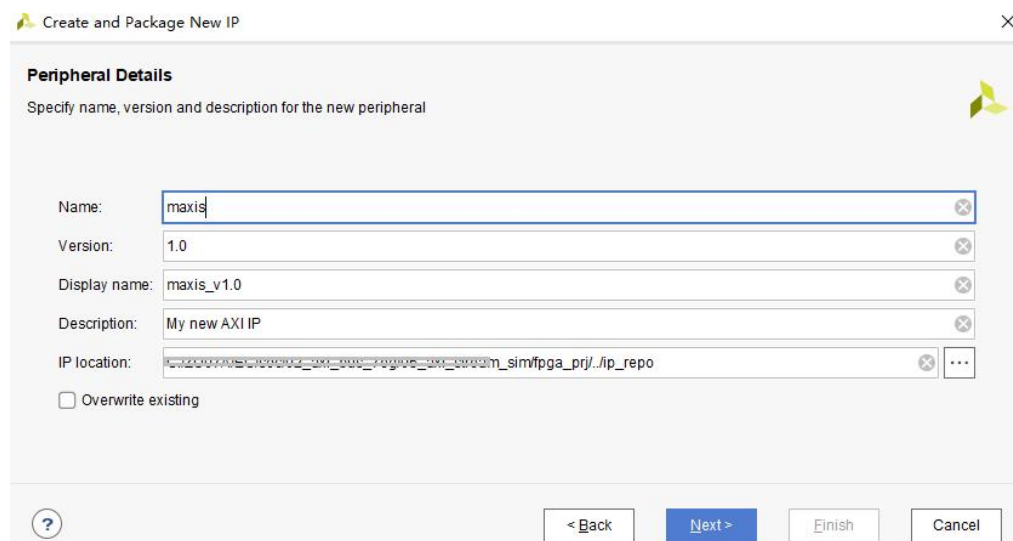
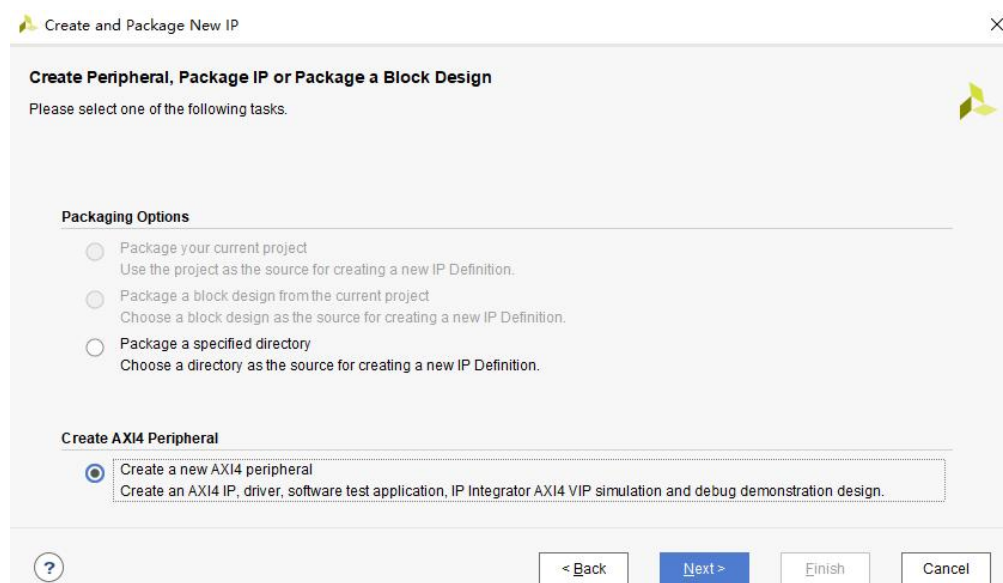
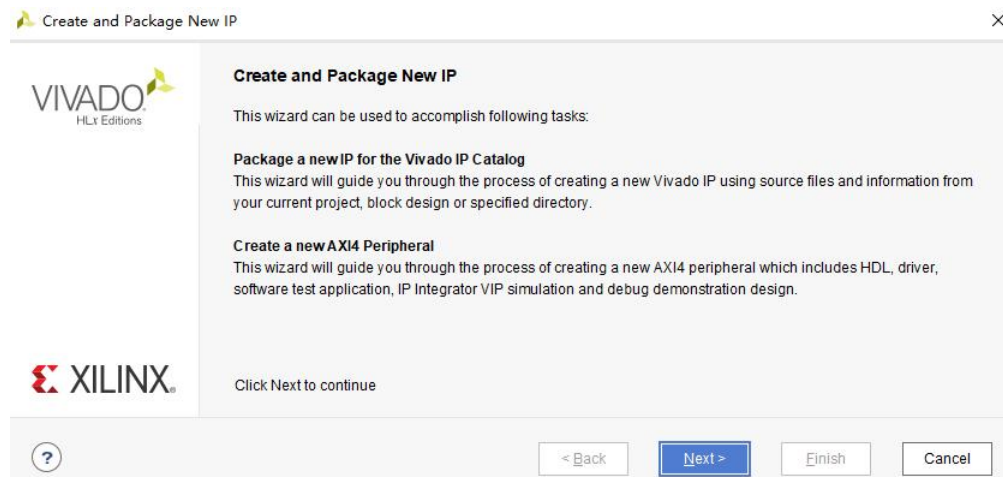
下图中是来自于 xilinx vivado 自带的 axis_vid_out ip 的视频输出时序。EOL 就是 tlast ,SOF 就是 tuser 初次外还包括了 VALID、READY、DATA 信号。

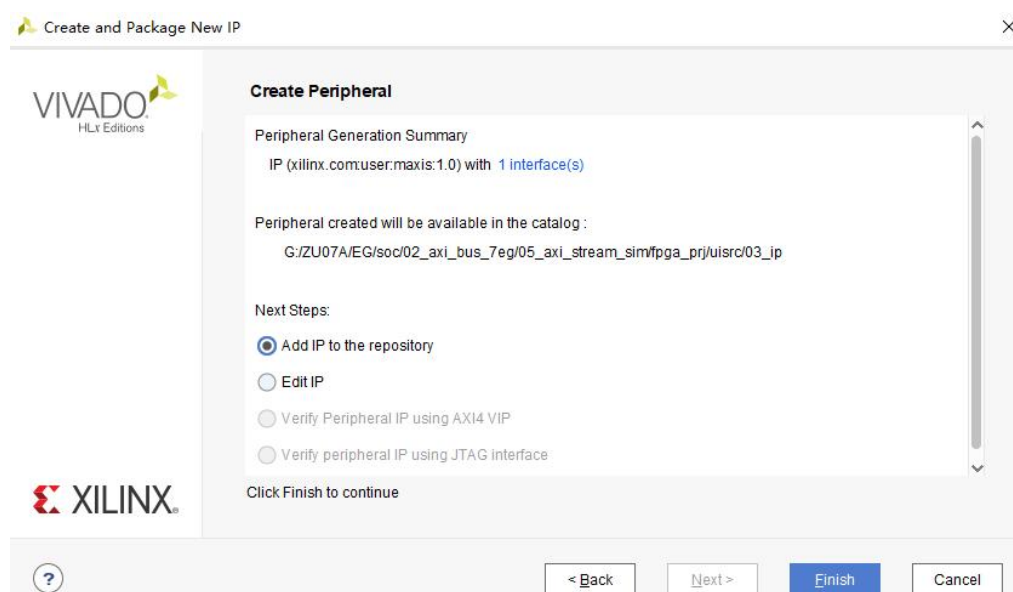
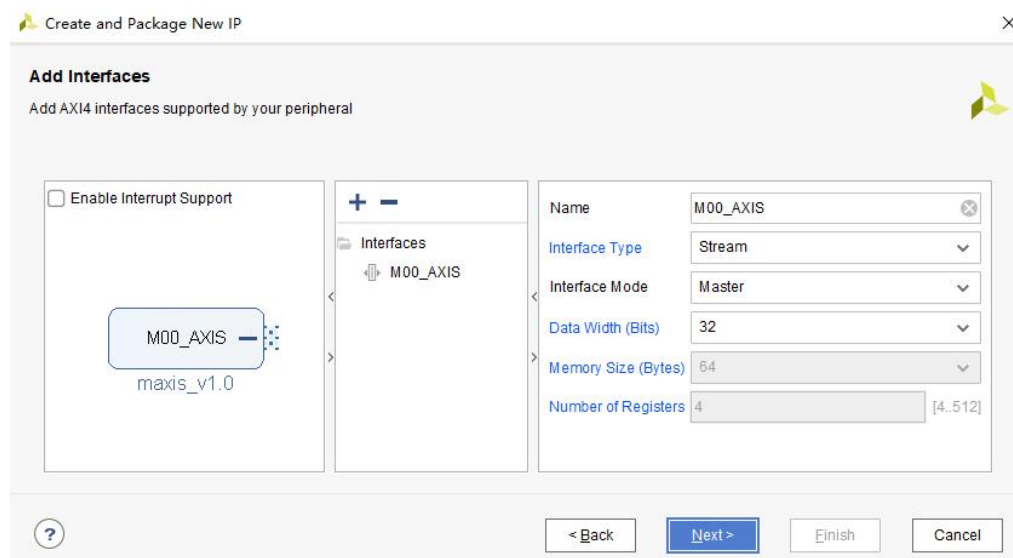


5.3 创建 axi-stream-slave 总线接口 IP

新建 fpga 工程，过程省略

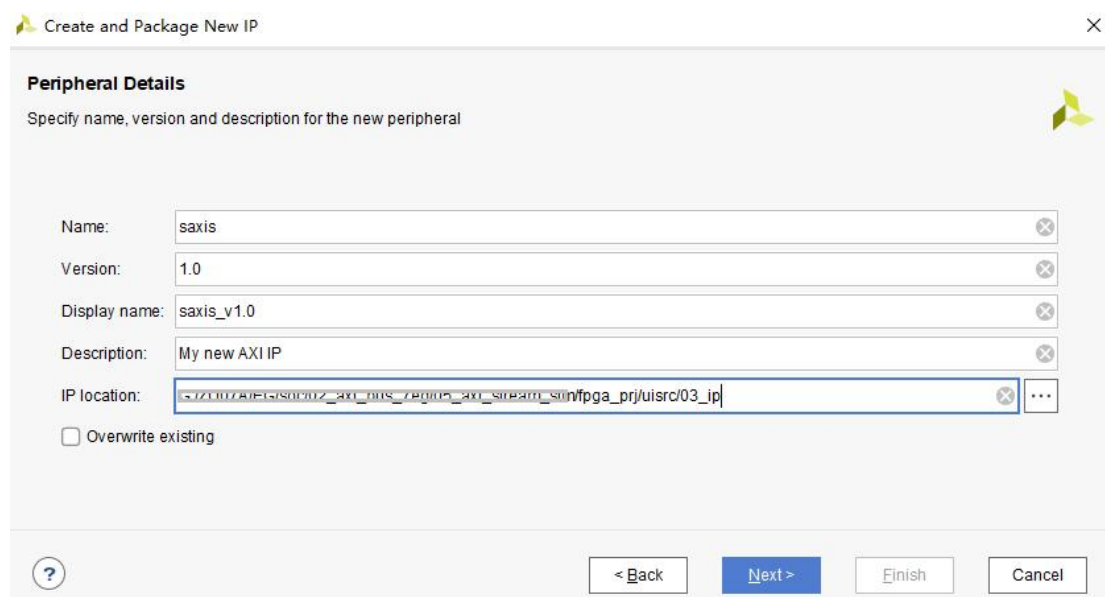


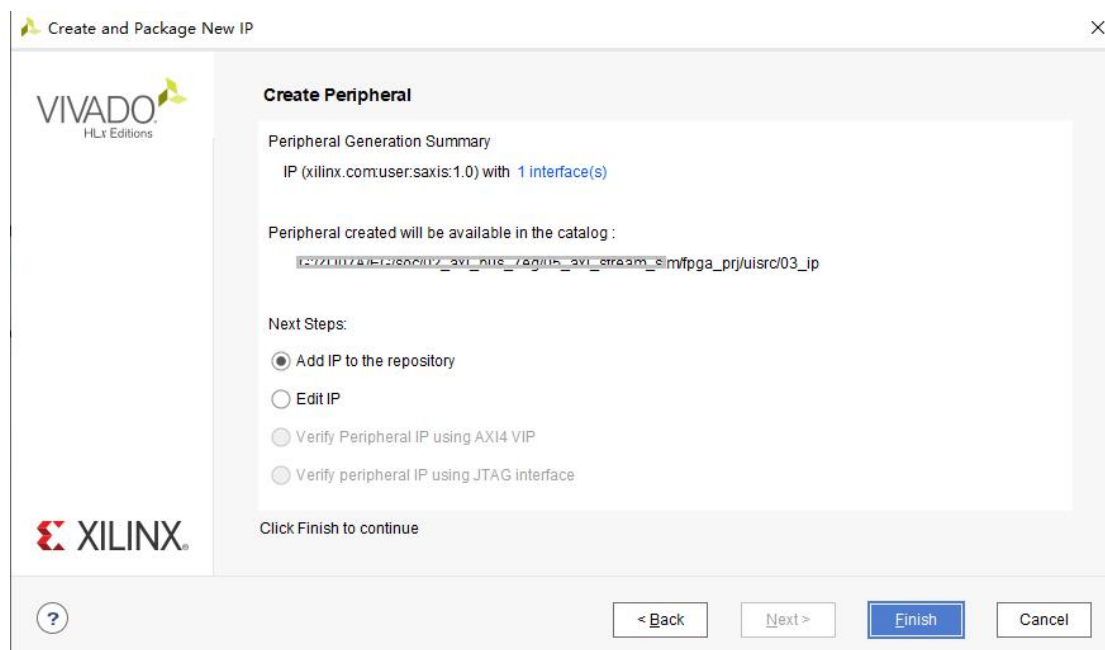
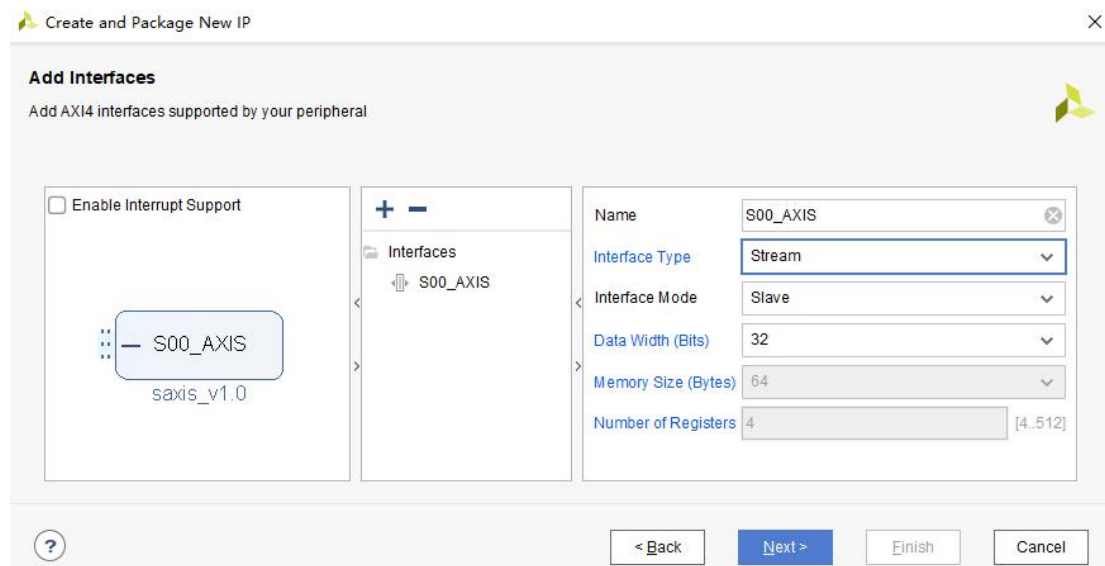




5.4 创建 axi-stream-master 总线接口 IP

未来完成 axi-stream 协议的验证, 采用以上方法, 我们再创建一个 saxis 的 IP

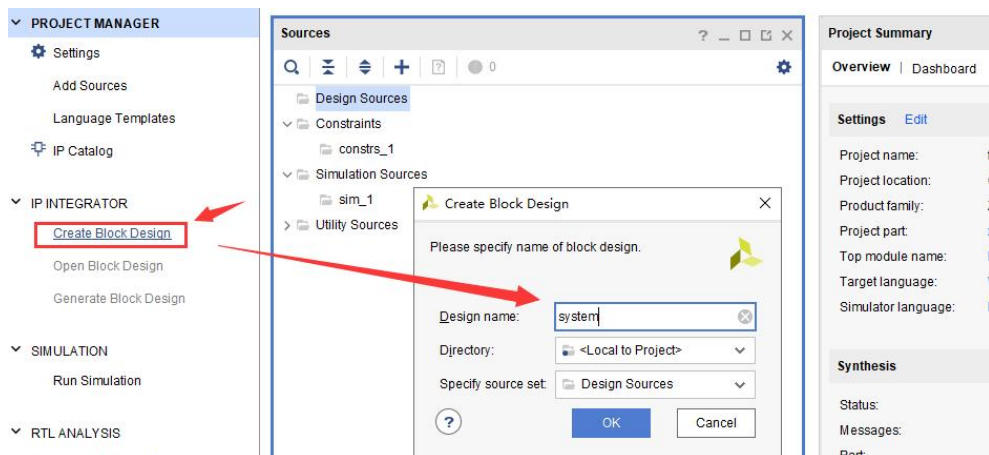




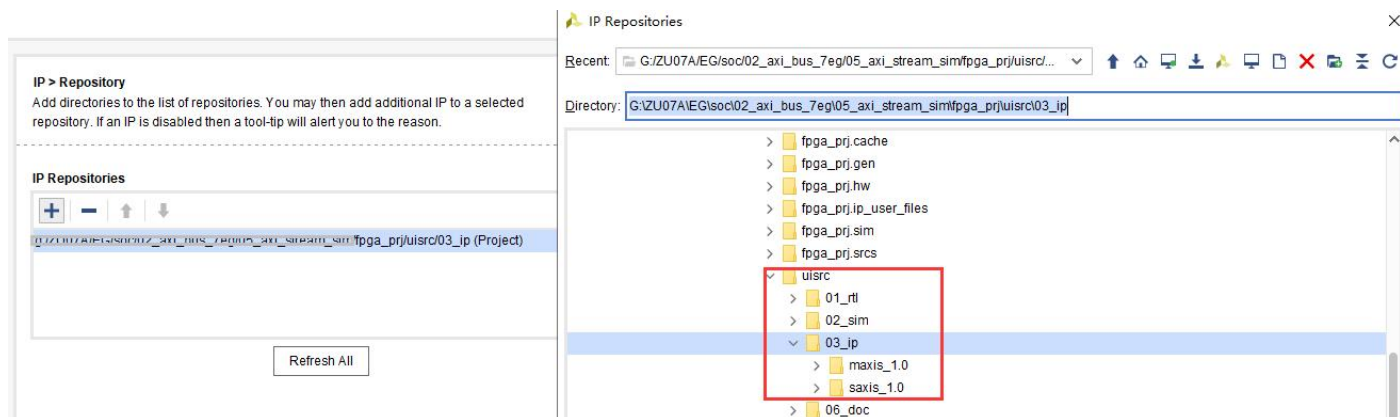
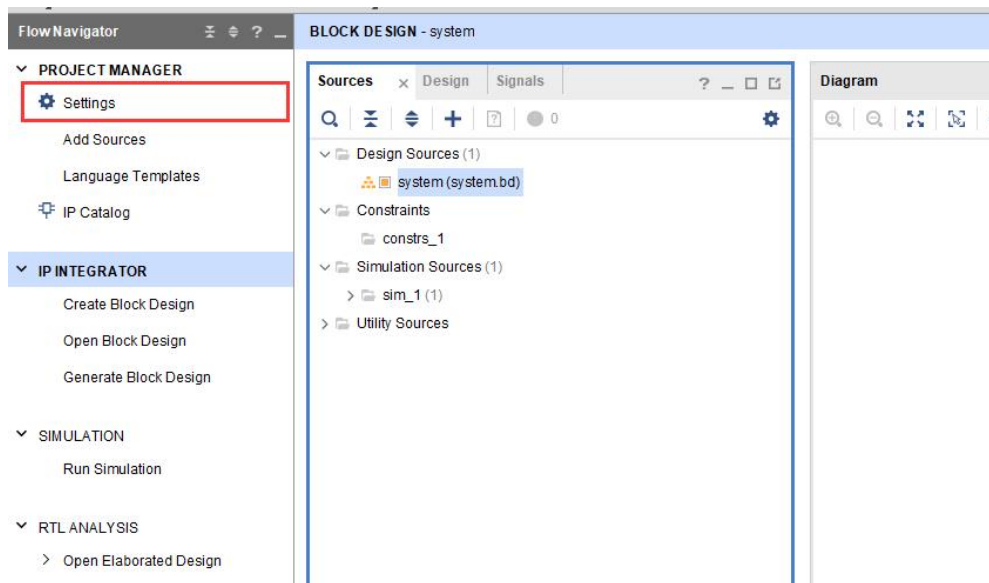
创建完成后如下图所示



5.5 创建 FPGA 图像化设计



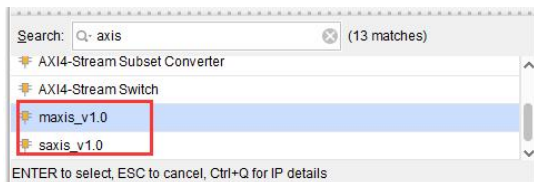
设置 IP 路径



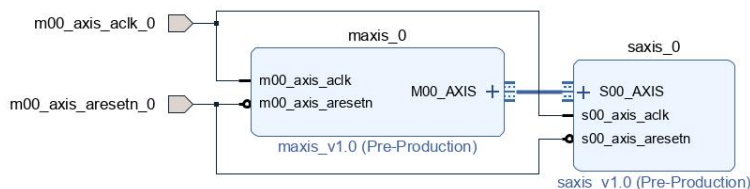
添加已经创建好的 IP



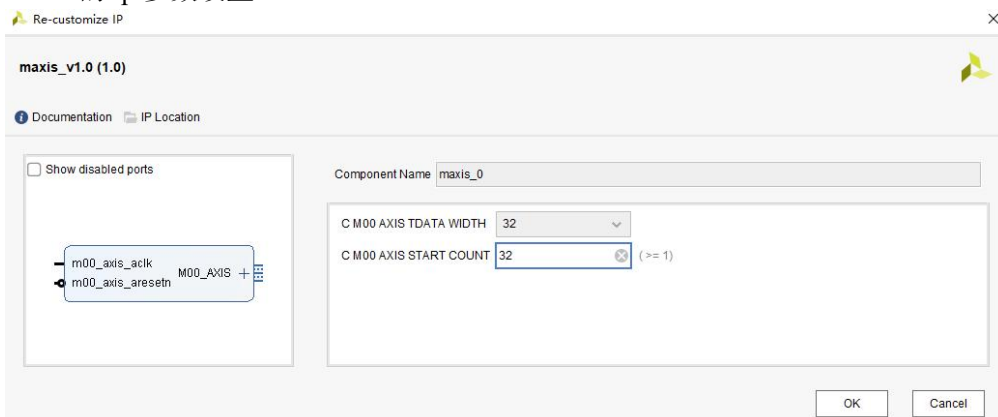
输入关键词 axis，在最后可以看到，双击添加 Ip



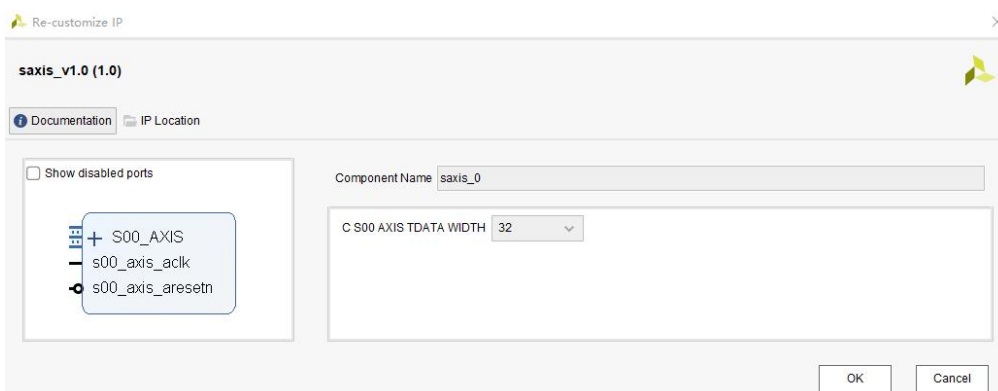
完成连线



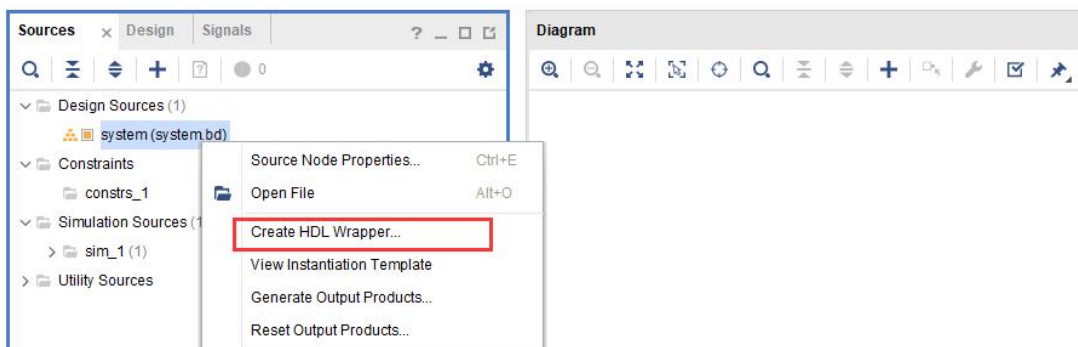
maxis 的 ip 参数设置

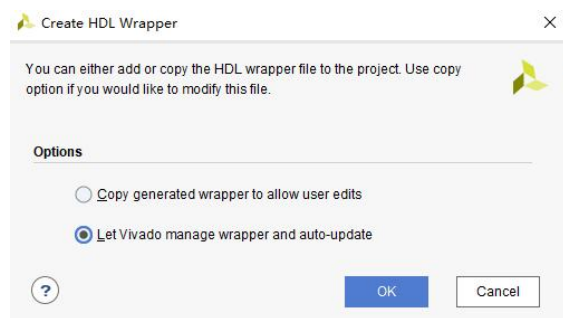


saxis 的 ip 参数设置

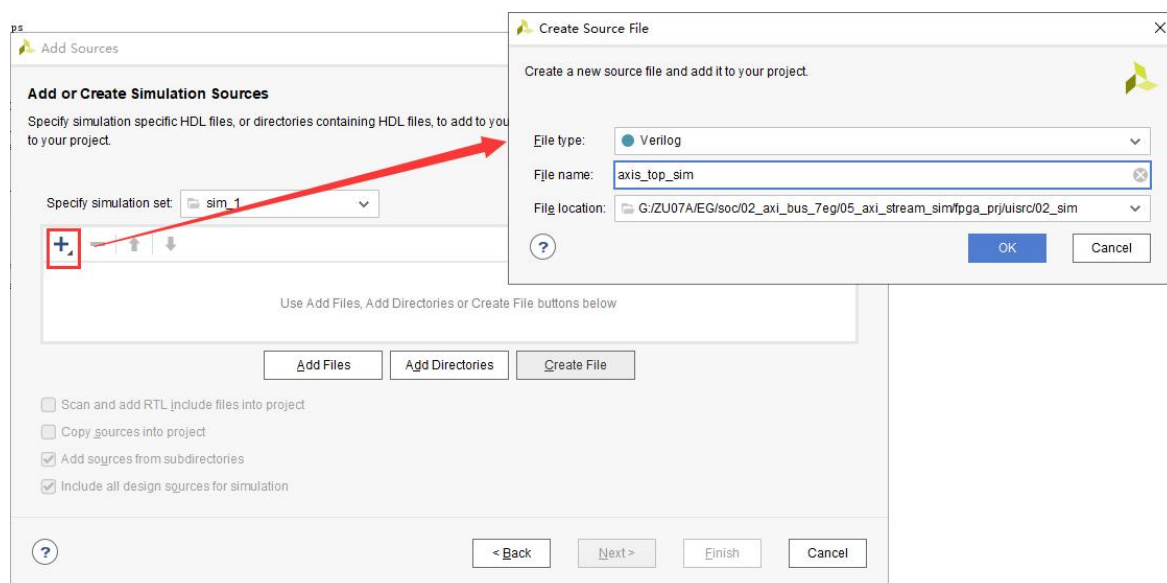
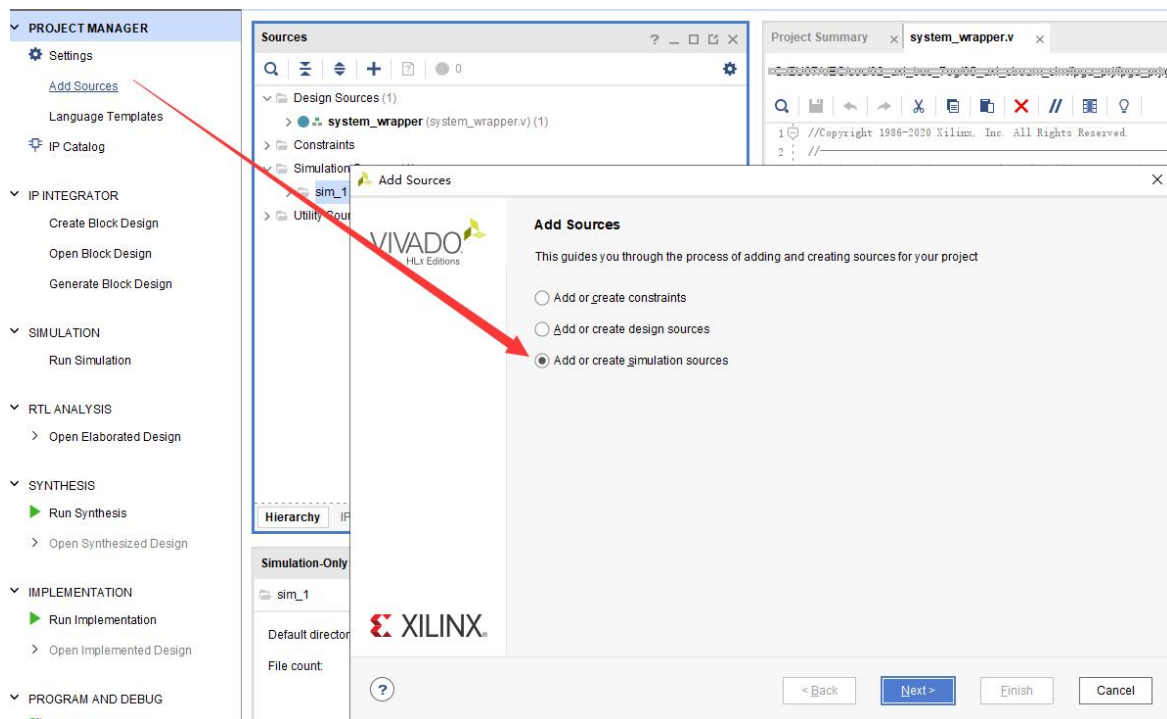


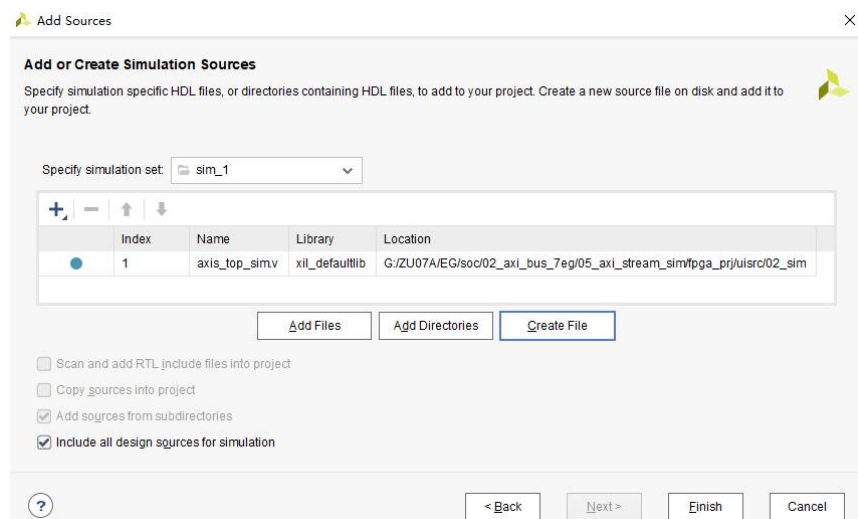
自动创建顶层文件





5.6 创建仿真文件





仿真程序非常简单，只要提供时钟和复位信号

```
`timescale 1ns / 1ns
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: 米联客(milianke)
Technical forum:uisrc.com
taobao1: milianke.taobao.com
taobao2: osrc.taobao.com
jd:milianke.jd.com
Create Date: 2019/12/17
Module Name: axis_top_sim
Description:
Copyright: Copyright (c) msxbo
Revision: 1.0
Signal description:
1) _i input
2) _o output
3) _n activ low
4) _dg debug signal
5) _r delay or register
6) _s state mechine
*/
////////////////////////////////////

module axis_top_sim();

reg m00_axis_aclk_0;
reg m00_axis_aresetn_0;

system_wrappertem system_wrapper_inst
(
    .m00_axis_aclk_0(m00_axis_aclk_0),
    .m00_axis_aresetn_0(m00_axis_aresetn_0)
);

initial begin
```

```
m00_axis_aclk_0 = 1'b0;

m00_axis_aresetn_0 = 1'b0;

#100;

m00_axis_aresetn_0 = 1'b1;

end

always

begin

    #5 m00_axis_aclk_0 = ~m00_axis_aclk_0;

end

endmodule
```

5.7 程序分析

默认的 maxis 模板才的代码有 bug，我们对其进行修改。

1:把以下代码替换默认的代码并且保存

```
`timescale 1 ns / 1 ps

module maxis_v1_0_M00_AXIS #
(
    // Width of S_AXIS address bus. The slave accepts the read and write addresses of width C_M_AXIS_TDATA_WIDTH.
    parameter integer C_M_AXIS_TDATA_WIDTH    = 32,
    // Start count is the number of clock cycles the master will wait before initiating/issuing any transaction.
    parameter integer C_M_START_COUNT        = 32
)
(
    // Global ports
    input wire  M_AXIS_ACLK,

    //
    input wire  M_AXIS_ARESETN,
    // Master Stream Ports. TVALID indicates that the master is driving a valid transfer, A transfer takes place when both TVALID and TREADY are asserted.
    output wire M_AXIS_TVALID,
    // TDATA is the primary payload that is used to provide the data that is passing across the interface from the master.
    output wire [C_M_AXIS_TDATA_WIDTH-1 : 0] M_AXIS_TDATA,
    // TSTRB is the byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte.
    output wire [(C_M_AXIS_TDATA_WIDTH/8)-1 : 0] M_AXIS_TSTRB,
    // TLAST indicates the boundary of a packet.
    output wire M_AXIS_TLAST,
    // TREADY indicates that the slave can accept a transfer in the current cycle.
    input wire  M_AXIS_TREADY
);

// Total number of output data
localparam NUMBER_OF_OUTPUT_WORDS = 8;

// function called clogb2 that returns an integer which has the
// value of the ceiling of the log base 2.
```

```

function integer clogb2 (input integer bit_depth);
    begin
        for(clogb2=0; bit_depth>0; clogb2=clogb2+1)
            bit_depth = bit_depth >> 1;
        end
    endfunction

// WAIT_COUNT_BITS is the width of the wait counter.
localparam integer WAIT_COUNT_BITS = clogb2(C_M_START_COUNT-1);

// bit_num gives the minimum number of bits needed to address 'depth' size of FIFO.
localparam bit_num = clogb2(NUMBER_OF_OUTPUT_WORDS);

// Define the states of state machine
// The control state machine oversees the writing of input streaming data to the FIFO,
// and outputs the streaming data from the FIFO
parameter [1:0] IDLE = 2'b00, // This is the initial/idle state

        INIT_COUNTER = 2'b01, // This state initializes the counter, once
        // the counter reaches C_M_START_COUNT count,
        // the state machine changes state to SEND_STREAM
        SEND_STREAM = 2'b10; // In this state the
        // stream data is output through M_AXIS_TDATA

// State variable
reg [1:0] mst_exec_state;
// Example design FIFO read pointer
reg [bit_num-1:0] read_pointer;

// AXI Stream internal signals
//wait counter. The master waits for the user defined number of clock cycles before initiating a transfer.
reg [WAIT_COUNT_BITS-1 : 0] count;
//streaming data valid
wire axis_tvalid;
//Last of the streaming data
wire axis_tlast;
wire tx_en;
//The master has issued all the streaming data stored in FIFO
wire tx_done;

// I/O Connections assignments
assign M_AXIS_TVALID = axis_tvalid;
assign M_AXIS_TDATA = read_pointer;
assign M_AXIS_TLAST = axis_tlast;
assign M_AXIS_TSTRB = {(C_M_AXIS_TDATA_WIDTH/8){1'b1}};

// Control state machine implementation
always @(posedge M_AXIS_ACLK)
begin
    if (!M_AXIS_ARESETN)

```

```
// Synchronous reset (active low)

begin

    mst_exec_state <= IDLE;

    count    <= 0;

end

else

    case (mst_exec_state)

        IDLE:

            mst_exec_state <= INIT_COUNTER;

            INIT_COUNTER:

                // The slave starts accepting tdata when

                // there tvalid is asserted to mark the

                // presence of valid streaming data

                if ( count == C_M_START_COUNT - 1 )

                    begin

                        mst_exec_state <= SEND_STREAM;

                    end

                else

                    begin

                        count <= count + 1;

                        mst_exec_state <= INIT_COUNTER;

                    end

                SEND_STREAM:

                    // The example design streaming master functionality starts

                    // when the master drives output tdata from the FIFO and the slave

                    // has finished storing the S_AXIS_TDATA

                    if (tx_done)

                        begin

                            mst_exec_state <= IDLE;

                        end

                    else

                        begin

                            mst_exec_state <= SEND_STREAM;

                        end

                    endcase

            end

        end

//tvalid generation

//axis_tvalid is asserted when the control state machine's state is SEND_STREAM and

//number of output streaming data is less than the NUMBER_OF_OUTPUT_WORDS.

assign axis_tvalid = ((mst_exec_state == SEND_STREAM) && (read_pointer < NUMBER_OF_OUTPUT_WORDS));

// AXI tlast generation

assign axis_tlast = (read_pointer == NUMBER_OF_OUTPUT_WORDS - 1'b1)&& tx_en;

assign tx_done = axis_tlast;

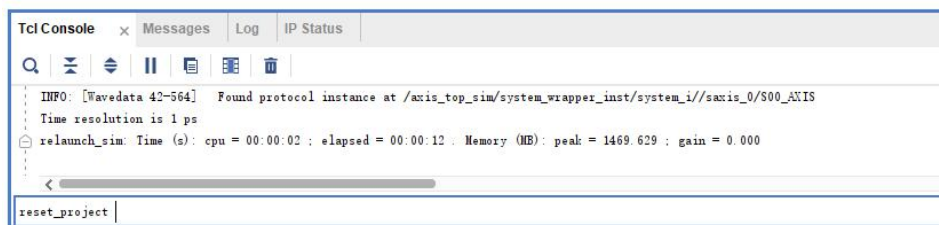
//FIFO read enable generation
```

```
assign tx_en = M_AXIS_TREADY && axis_tvalid;

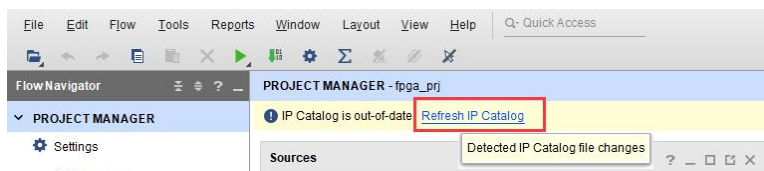
// Streaming output data is read from FIFO
always @(posedge M_AXIS_ACLK )
begin
    if(!M_AXIS_ARESETN)
        begin
            read_pointer <= 0;
        end
    else if (tx_en)
        begin
            read_pointer <= read_pointer + 32'b1;
        end
    end
end

endmodule
```

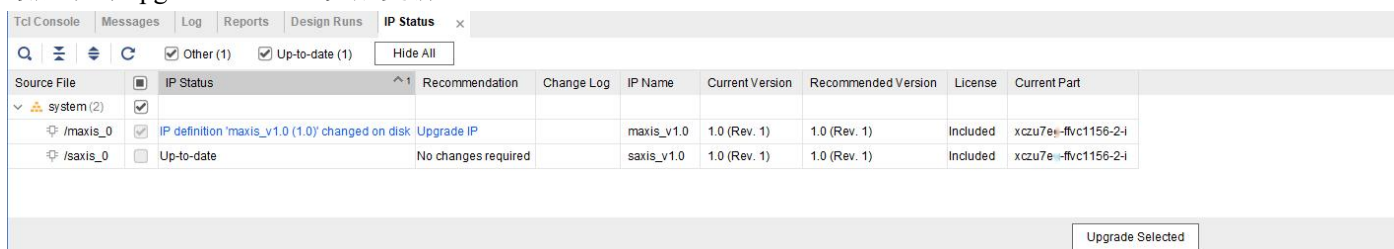
2:在 Tcl Console 中输入 reset_project 对工程 IP 复位



3:之后单击 Refresh IP Catalog



最后单击 upgrade Selected 完成更新



在看看看 saxis 的源码，这部分代码非常简单

首先是状态机,当 S_AXI_TVALID 有效, saxis 的状态机进入 WRITE_FIFO

```

always @(posedge S_AXIS_ACLK)
begin
  if (!S_AXIS_ARESETN)
    // Synchronous reset (active low)
    begin
      mst_exec_state <= IDLE;
    end
  else
    case (mst_exec_state)
      IDLE:
        // The sink starts accepting tdata when
        // there tvalid is asserted to mark the
        // presence of valid streaming data
        if (S_AXIS_TVALID)
          begin
            mst_exec_state <= WRITE_FIFO;
          end
        else
          begin
            mst_exec_state <= IDLE;
          end
      WRITE_FIFO:
        // When the sink has accepted all the streaming input data,
        // the interface switches functionality to a streaming master
        if (writes_done)
          begin
            mst_exec_state <= IDLE;
          end
        else
          begin
            // The sink accepts and stores tdata
            // into FIFO
            mst_exec_state <= WRITE_FIFO;
          end
    endcase
end

```

只要接收的数据小于(write_pointer <= NUMBER_OF_INPUT_WORDS-1)并且 saxis 状态机是 WRITE_FIFO 状态机, 表示 saxis 可以接收数据, 设置 axis_tready==1

如下: assign axis_tready = ((mst_exec_state == WRITE_FIFO) && (write_pointer <= NUMBER_OF_INPUT_WORDS-1));

write_pointer 用于写入了多少个数据。

```

always@(posedge S_AXIS_ACLK)
begin
  if(!S_AXIS_ARESETN)
    begin
      write_pointer <= 0;
      writes_done <= 1'b0;
    end
  else
    if (write_pointer <= NUMBER_OF_INPUT_WORDS-1)
      begin
        if (fifo_wren)
          begin
            // write pointer is incremented after every write to the FIFO
            // when FIFO write signal is enabled.
            write_pointer <= write_pointer + 1;
            writes_done <= 1'b0;
          end
        if ((write_pointer == NUMBER_OF_INPUT_WORDS-1) || S_AXIS_TLAST)
          begin
            // reads_done is asserted when NUMBER_OF_INPUT_WORDS numbers of streaming data
            // has been written to the FIFO which is also marked by S_AXIS_TLAST(kept for optional usage).
            writes_done <= 1'b1;
          end
      end
    end
end

```

写数据到 fifo 中

```

// FIFO write enable generation
assign fifo_wren = S_AXIS_TVALID && axis_tready;

// FIFO Implementation
generate
for(byte_index=0; byte_index<= (C_S_AXIS_TDATA_WIDTH/8-1); byte_index=byte_index+1)
begin:FIFO_GEN

  reg [(C_S_AXIS_TDATA_WIDTH/4)-1:0] stream_data_fifo [0 : NUMBER_OF_INPUT_WORDS-1];

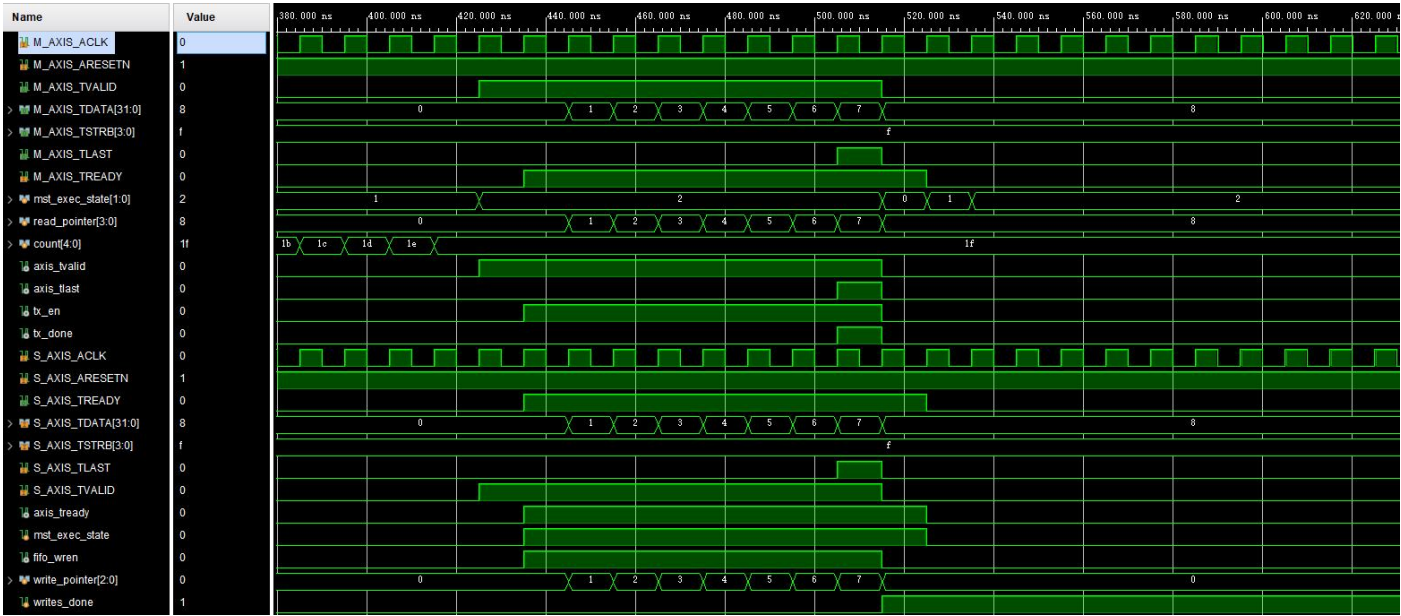
  // Streaming input data is stored in FIFO

  always @(posedge S_AXIS_ACLK )
  begin
    if (fifo_wren)// && S_AXIS_TSTRB[byte_index])
      begin
        stream_data_fifo[write_pointer] <= S_AXIS_TDATA[(byte_index*8+7) -: 8];
      end
    end
  end
endgenerate

```

5.8 实验结果

仿真结果



06AXI-Lite-Master 读写 AXI-Lite-Slave

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

6.1 概述

基于前面 5 篇文章中 5 个实验, 我们已经掌握了 AXI4 总线协议, 现在我们编写一个自定义的 AXI-Lite-Slave GPIO IP, 并且用编写的 AXI-Lite-Master IP 对齐进行仿真验证和上板验证。

本文实验目的:

- 1: 修改 VIVADO 产生的 saxi-lite- gpio 模板, 增加 GPIO 的定义
- 2: 修改 VIVADO 产生的 maxi-lite-gpio 模板, 增加对 saxi-lite- gpio 寄存器的读写操作。
- 3: 进一步掌握基于 vivado 实现的 ip 的封装

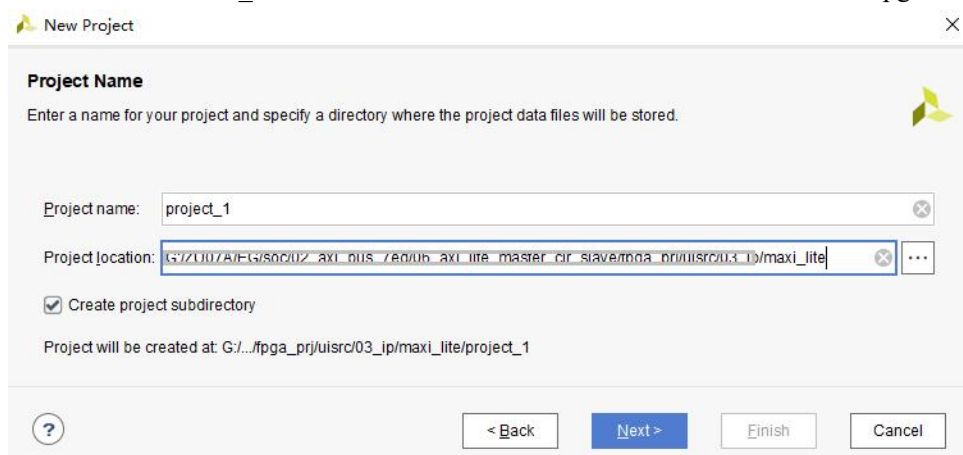
6.2 基于 VIVADO 的 IP 封装

6.2.1 封装 maxi_lite_gpio IP

这节课的源码文件已经编写好了, 我们直接拿来使用。新建 2 个文件, 把源码文件 maxi_lite_gpio.v 和 saxi_lite_gpio.v 并且方便放入 03_ip/maxi_lite 文件夹和 03_ip/saxi_lite 文件夹。



以创建自定义 maxi_lite 自定义为例, 先创建一个空的创建 IP 所需要的 fpga 工程



New Project

Project Type

Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☒ Do not specify sources at this time

☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.

☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

Select Device

Filter, search, and browse parts by their resources. The selection will be applied.

Parts | Boards

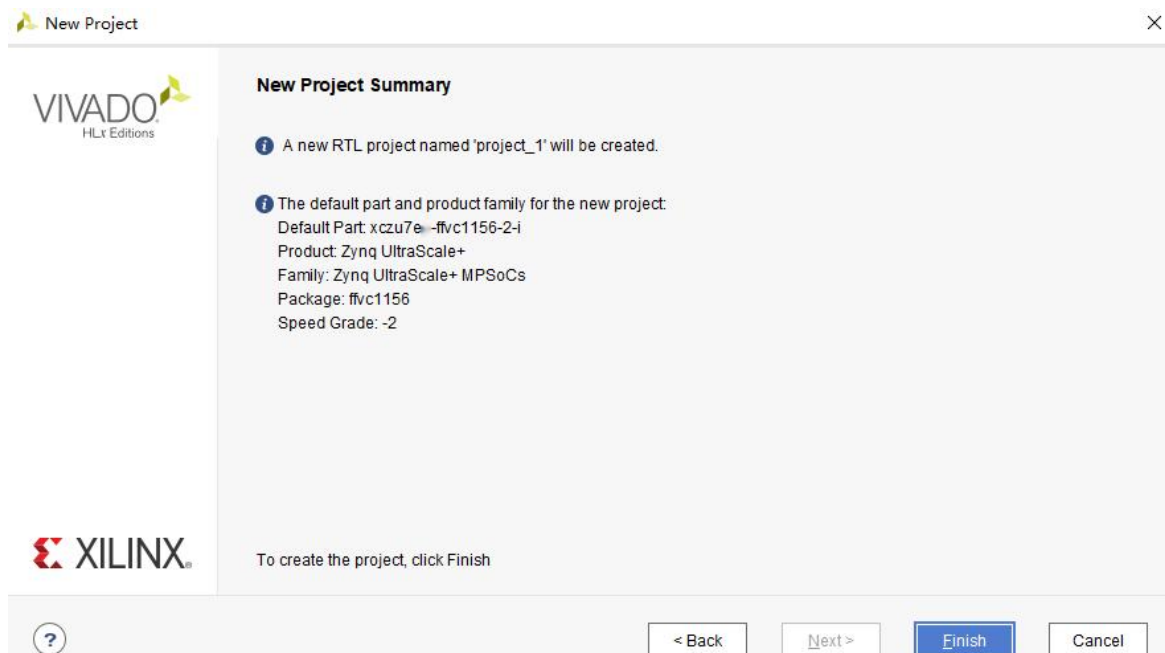
[Reset All Filters](#)

Category: All Package: fvc1156 Temperature: I

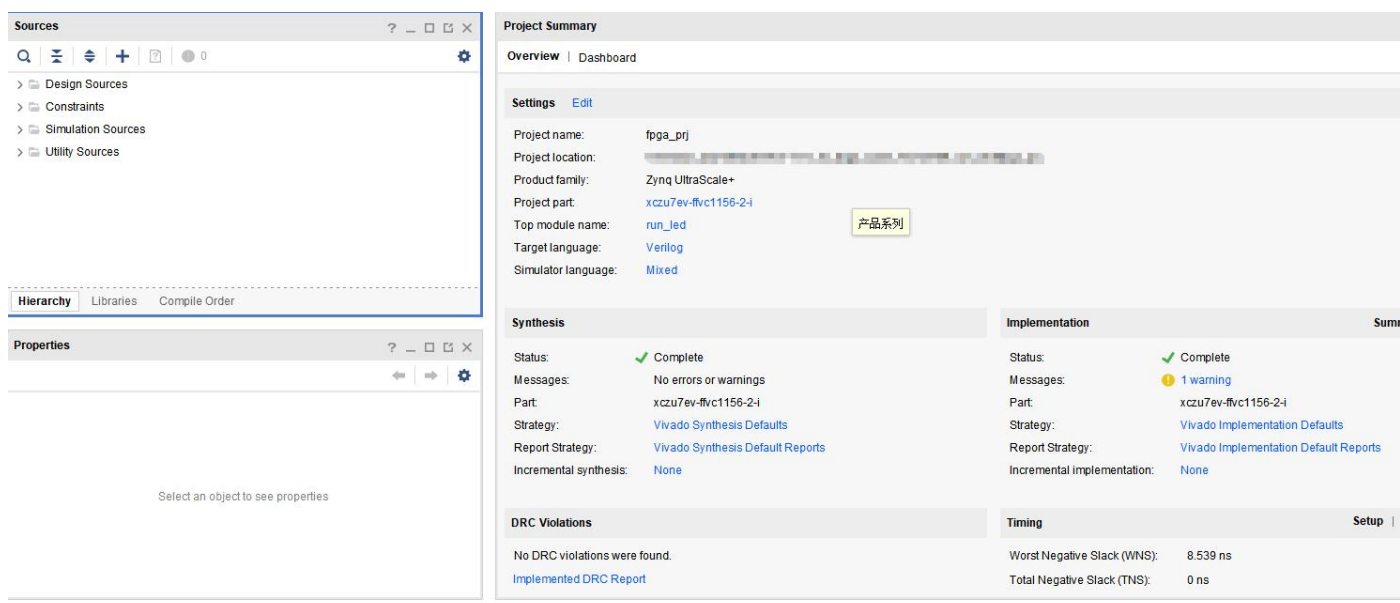
Family: Zynq UltraScale+ MPSoCs Speed: -2 Static power: All Remaining

Search: Q-

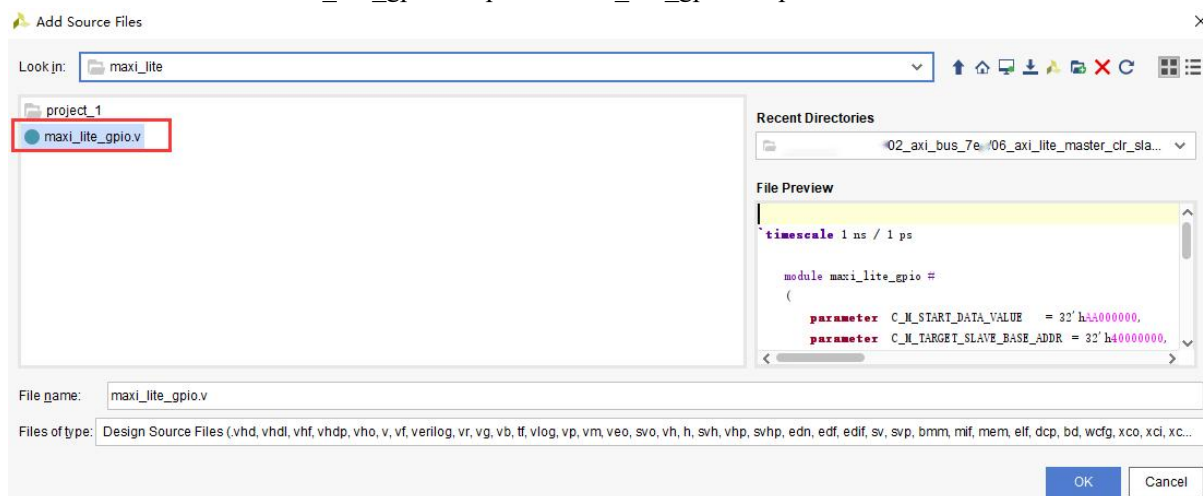
Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAM
xczu7cg-fvc1156-2-i	1156	360	230400	460800	312	96
xczu7eg-fvc1156-2-i	1156	360	230400	460800	312	96
xczu7ev-fvc1156-2-i	1156	360	230400	460800	312	96
xczu11eg-fvc1156-2-i	1156	360	298560	597120	600	80

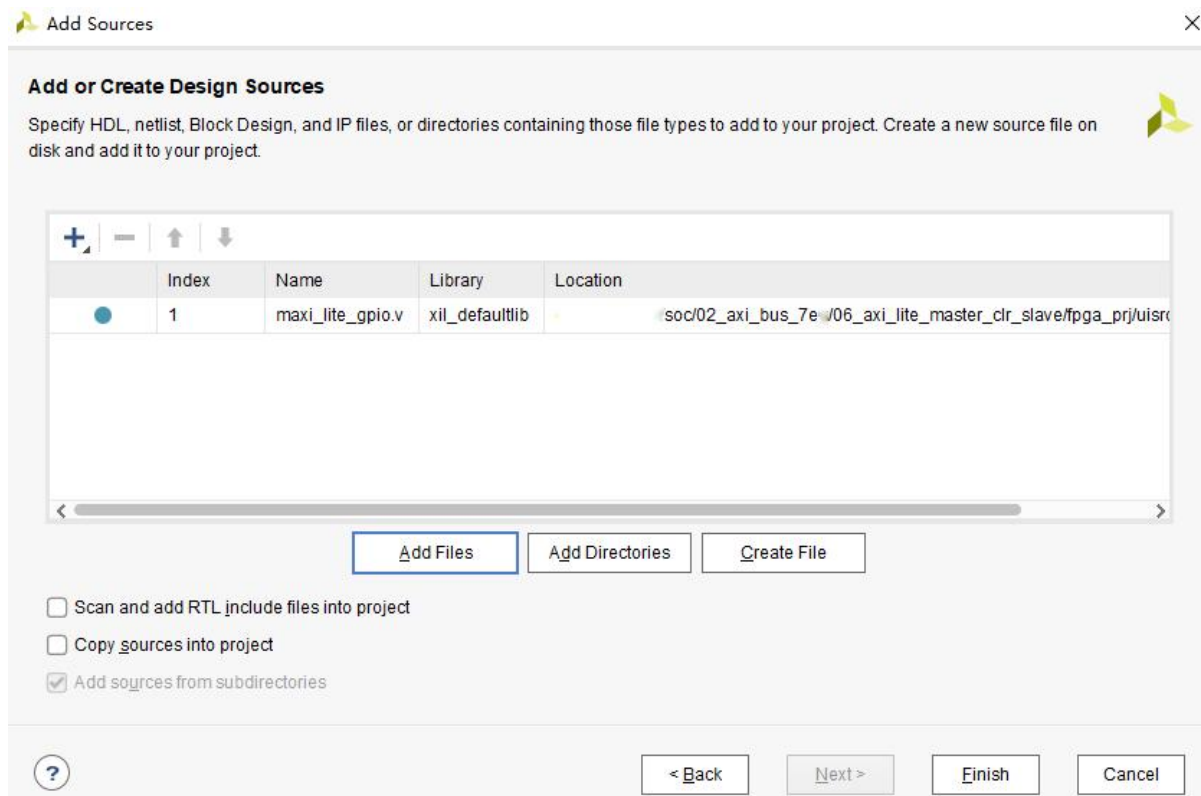


添加 ip 源码文件

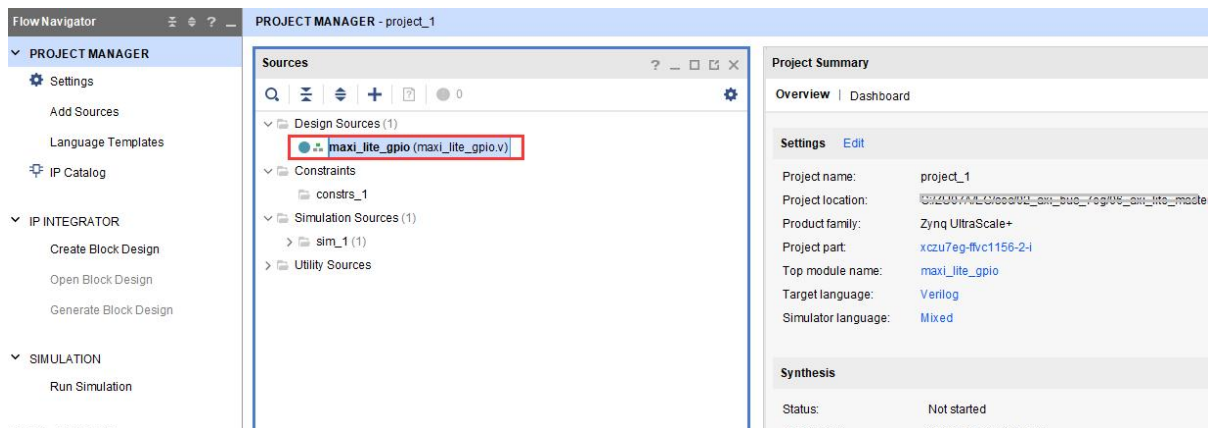


这里再次提醒是创建 maxi_lite_gpio 的 ip 至于 saxi_lite_gpio 的 ip 方法一样。

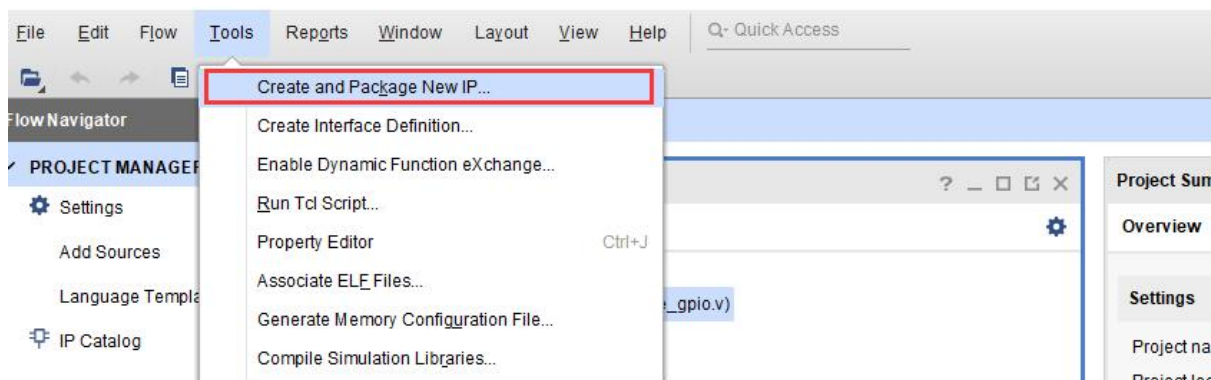


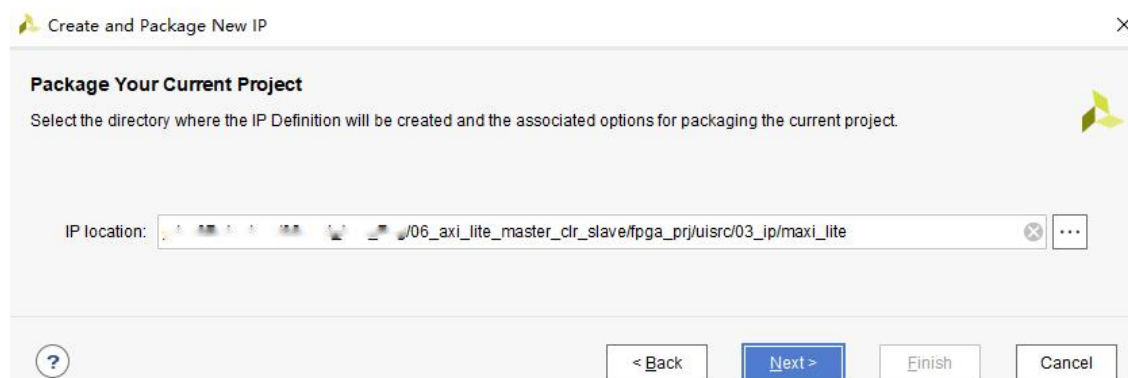
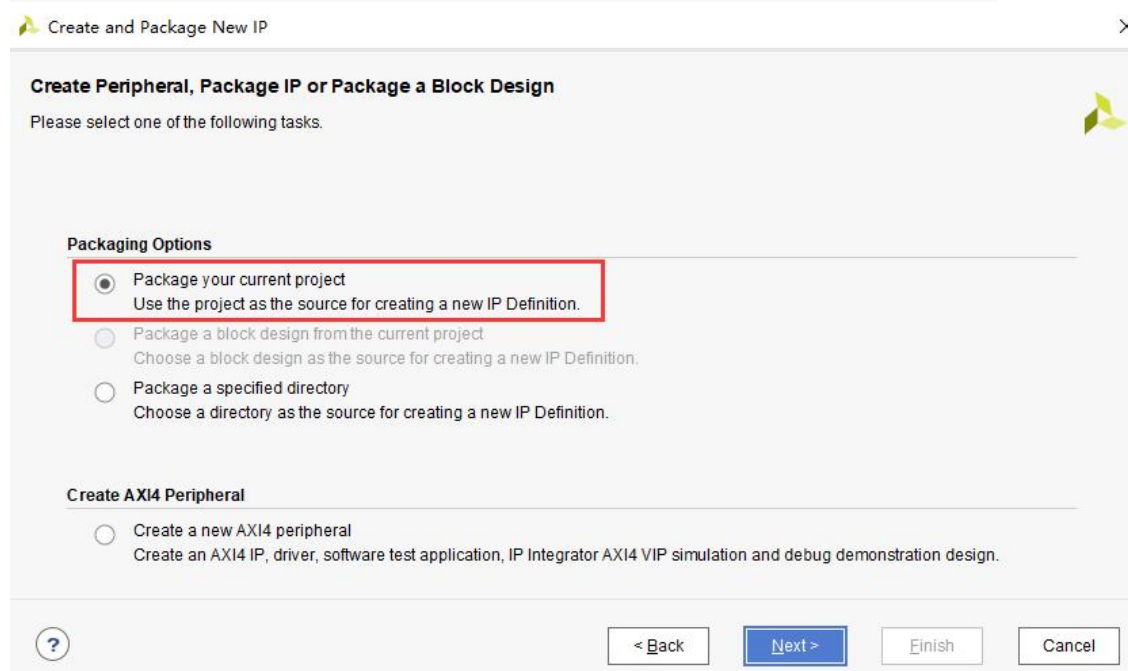
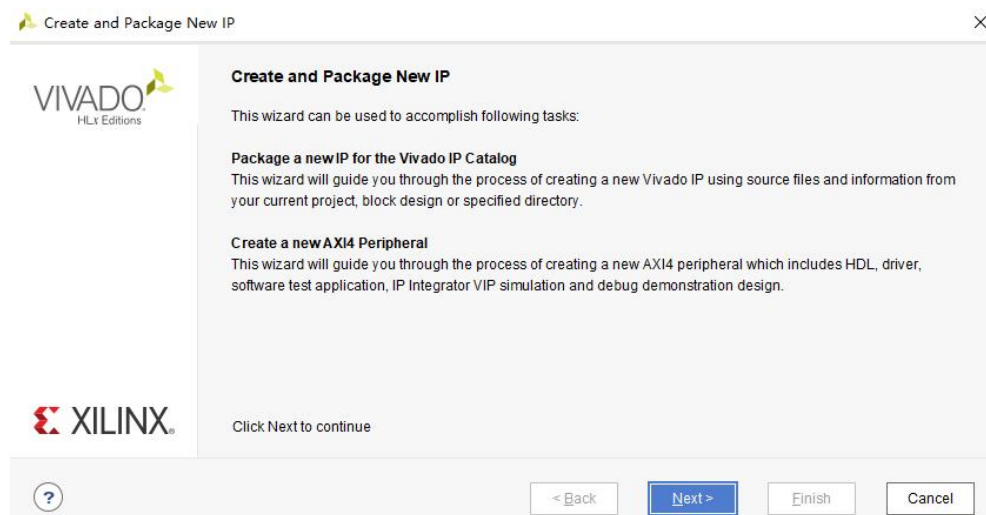


源码后面再分析，暂时我们不分析源码，先完成 IP 的创建

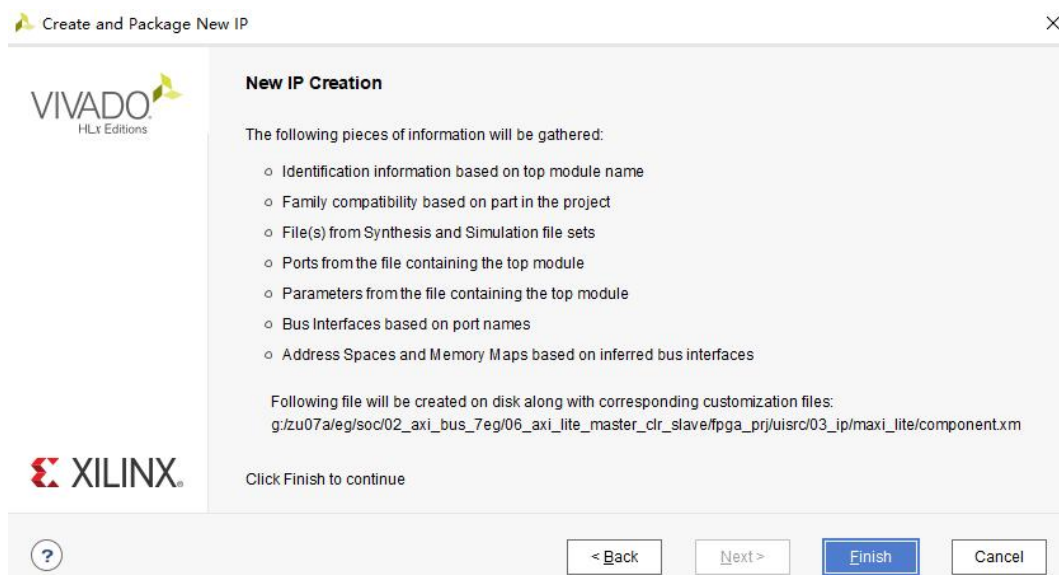
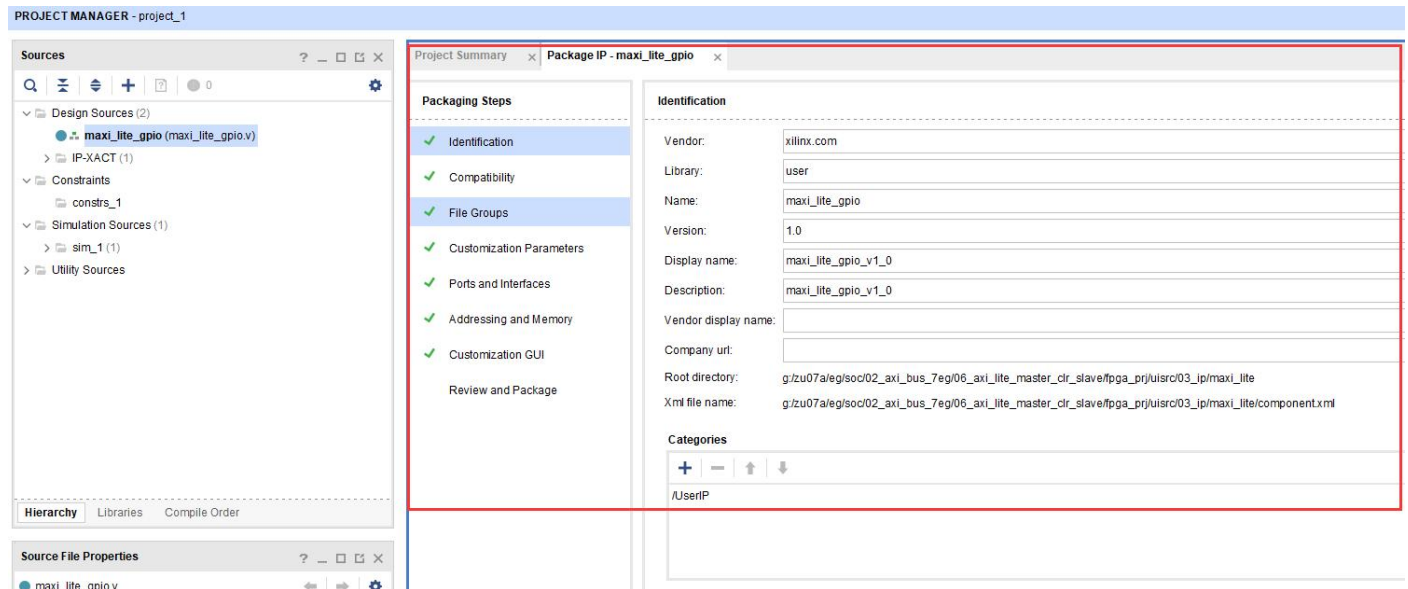


再 tools 菜单中找到 Create and Package New IP





新增 IP 编辑窗口



参数接口

Packaging Steps

✓ Identification

✓ Compatibility

✓ File Groups

Customization Parameters

✓ Ports and Interfaces

✓ Addressing and Memory

✓ Customization GUI

Review and Package

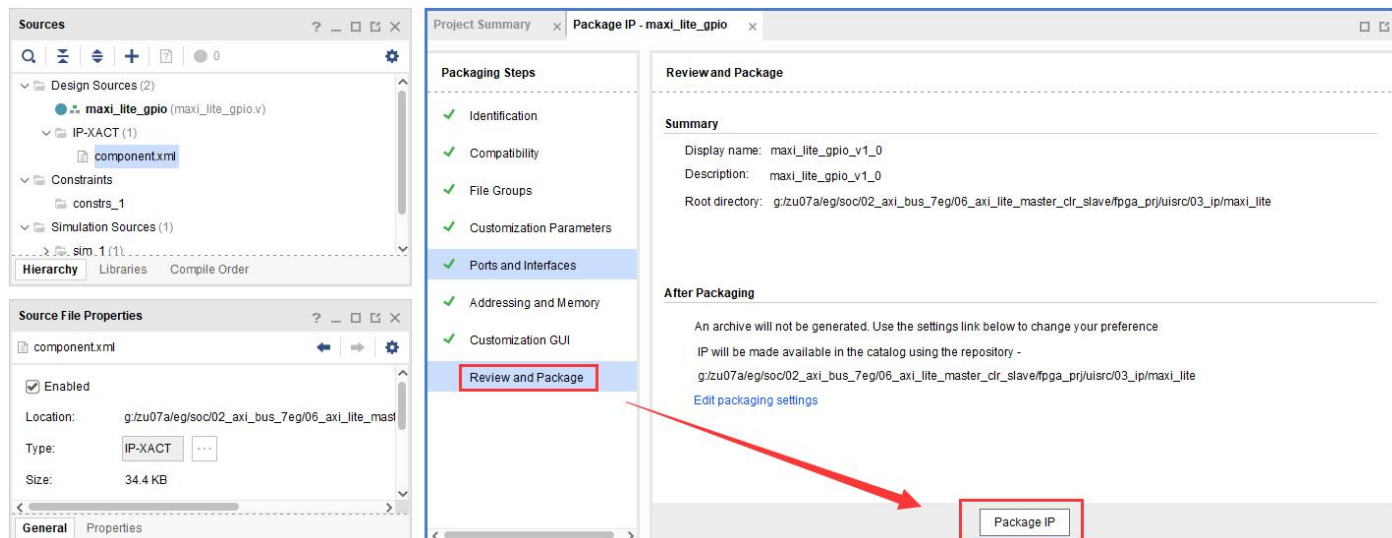
Customization Parameters

Name	Description	Display Name	Value	Value Bit String Length	Value Format	Value Source	Value Validation List	Value Validation Range Maximum	Value Validation Rang...	Parameter Types
Customization Parameters										
C_M_START_DATA_VALUE		C M Start Data Value	0xAA000000	32	bitString	default				
C_M_TARGET_SLAVE_BASE_ADDR		C M Target Slave Base Addr	0x40000000	32	bitString	default				
C_M_AXI_ADDR_WIDTH		C M Axi Addr Width	32	0	long	default				
C_M_AXI_DATA_WIDTH		C M Axi Data Width	32	0	long	default				
W_TRANSACTIONS_NUM		W Transactions Num	2	0	long	default				
R_TRANSACTIONS_NUM		R Transactions Num	1	0	long	default				

信号接口

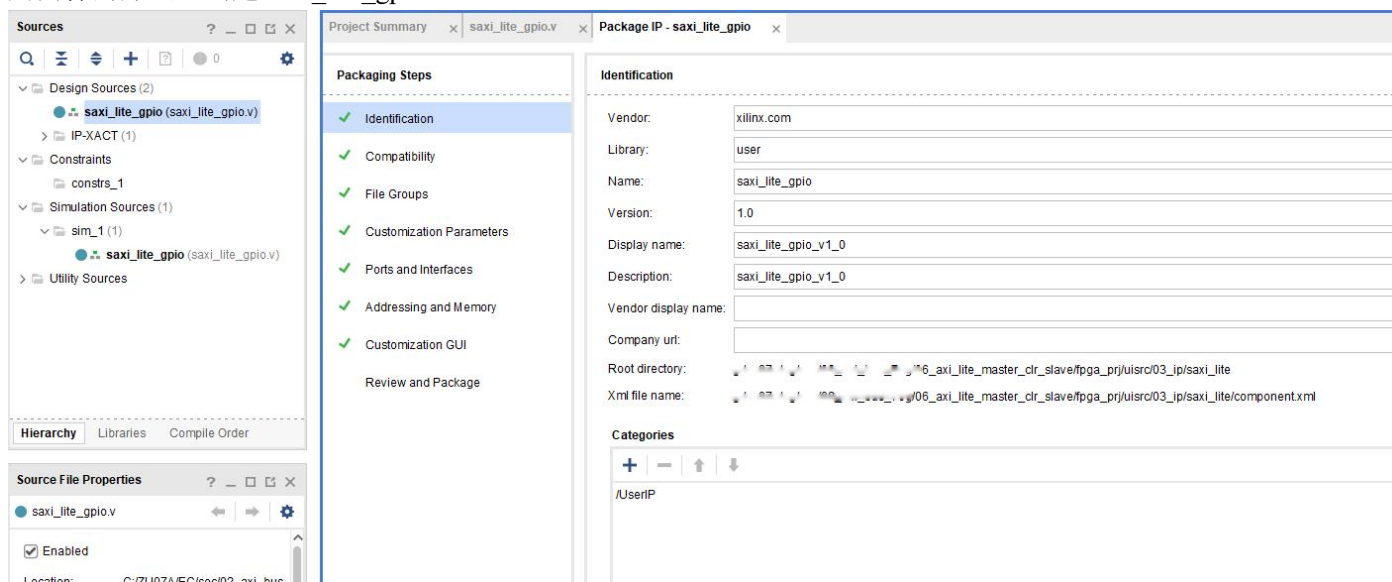
Packaging Steps		Ports and Interfaces									
✓ Identification		🔍 🔧 ⚙️ + 🗑️ 🔄									
✓ Compatibility											
✓ File Groups											
✓ Customization Parameters											
✓ Ports and Interfaces											
✓ Addressing and Memory											
✓ Customization GUI											
Review and Package											

完成 IP 打包



6.2.2 创建 saxi_lite_gpio ip

用同样的方法，创建 saxi_lite_gpio



Project Summary

saxi_lite_gpio.v

Package IP - saxi_lite_gpio

Packaging Steps

Identification

Compatibility

File Groups

Customization Parameters

Ports and Interfaces

Addressing and Memory

Customization GUI

Review and Package

Customization Parameters

Q

≡

⚙

⚙

+

↺

Name	Description	Display Name	Value	Value Bit String Length	Value Format	Value Source	Value Validation List	Value Validation Range Maximum	Value Validation Range Minimum	Parameter Types
Customization Parameters										
C_S_AXI_DATA_WIDTH		C S Axi Data Width	32	0	long	default				
C_S_AXI_ADDR_WIDTH		C S Axi Addr Width	4	0	long	default				

Packaging Steps

Identification

Compatibility

File Groups

Customization Parameters

Ports and Interfaces

Addressing and Memory

Customization GUI

Review and Package

Ports and Interfaces

Q

≡

⚙

+

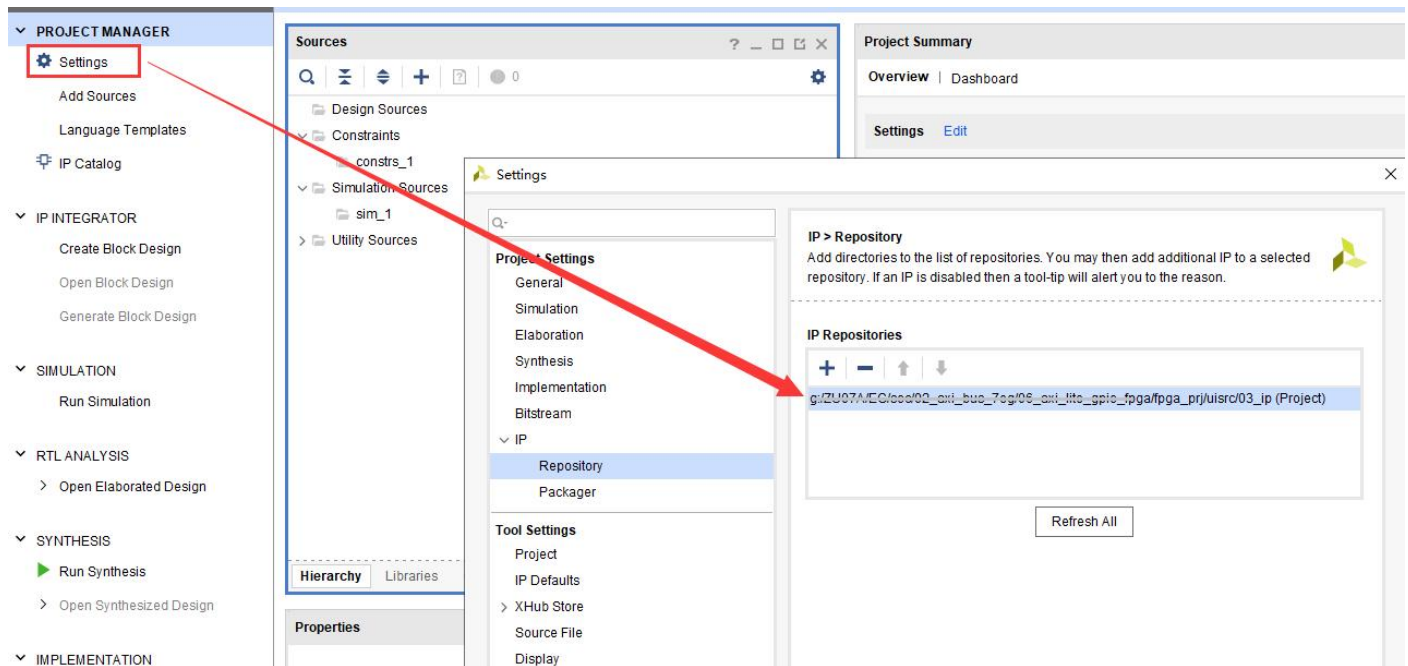
⚙

↺

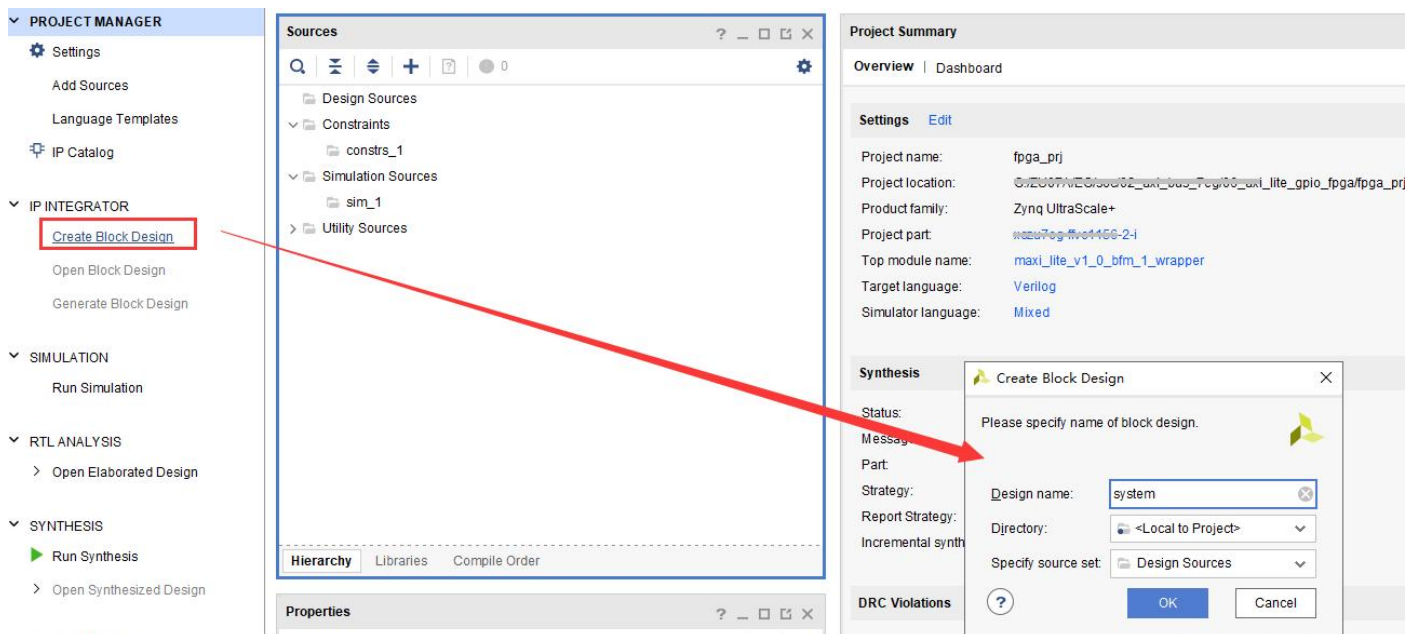
Name	Interface Mode	Enablement Dependency	Direction	Driver Value	Size Left	Size Right	Size Left Dependency	Size Right Dependency	Type Name
S_AXI	slave								
S_AXI_AWADDR			in	0	3	0	(C_S_AXI_DATA_WIDTH)		wire
S_AXI_AWPROT			in	0	2	0			wire
S_AXI_AWVALID			in	0					wire
S_AXI_AWREADY			out						wire
S_AXI_WDATA			in	0	31	0	(C_S_AXI_DATA_WIDTH)		wire
S_AXI_WSTRB			in	1	3	0	((C_S_AXI_DATA_WIDTH)/4)		wire
S_AXI_WVALID			in	0					wire
S_AXI_WREADY			out						wire
S_AXI_BRESP			out		1	0			wire
S_AXI_BVALID			out						wire
S_AXI_BREADY			in	0					wire
S_AXI_ARADDR			in	0	3	0	(C_S_AXI_DATA_WIDTH)		wire
S_AXI_ARPROT			in	0	2	0			wire
S_AXI_ARVALID			in	0					wire
S_AXI_ARREADY			out						wire
S_AXI_RDATA			out		31	0	(C_S_AXI_DATA_WIDTH)		wire
S_AXI_RRESP			out		1	0			wire
S_AXI_RVALID			out						wire
S_AXI_RREADY			in	0					wire
Clock and Reset Signals									
btn			in		1	0			wire
led			out		1	0			wire
cnt			out		31	0			wire

6.3FPGA 图形化编程

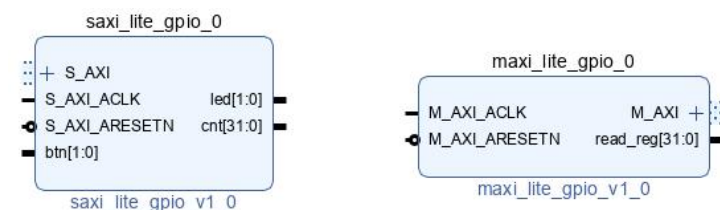
设置 IP 路径



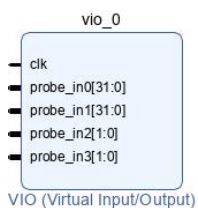
创建 BD 工程



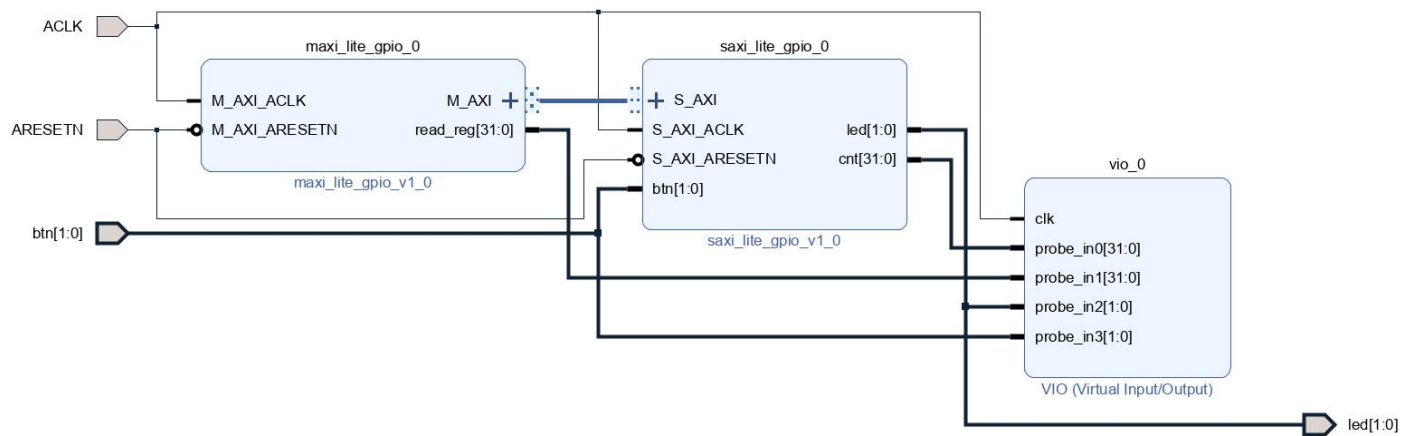
添加以上完成的 maxi_lite_gpio 和 saxi_lite_gpio 两个 IP



再添加虚拟 IO 用于观察数据



完成连线



编写顶层文件配套代码 uisrc/01_rtl 路径下有源码

```

module system_wrapper
(
input sysclk_p,
input sysclk_n,
input [1:0] btn,
output [1:0] led
);

wire ACLK;
wire ARESETN;

IBUFDS IBUFDS_inst (
.O(ACLK),
.I(sysclk_p),
.IB(sysclk_n)
);

reg [7:0] rstn_cnt=0;
always @(posedge ACLK)begin
if(rstn_cnt[7]==1'b0)
rstn_cnt <= rstn_cnt + 1'b1;
end

assign ARESETN = rstn_cnt[7];

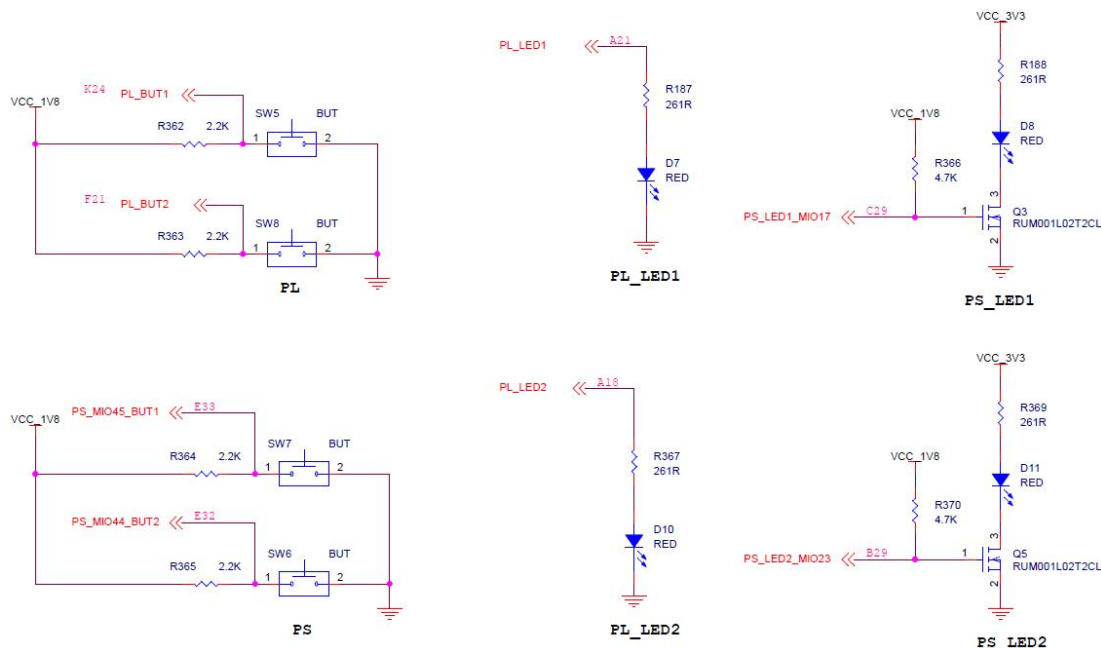
system system_i
(
.ACLK(ACLK),
.ARESETN(rstn_cnt[7]),
.btn(btn),
.led(led));
endmodule

```

6.3 硬件电路分析

PL 的 GPIO 可以定义为 EMIO,其中 PL 部分 FPGA 引脚 A21 和 A18 接到了 LED 上, K24、F21 接到了按键上。

6.3.1 原理图



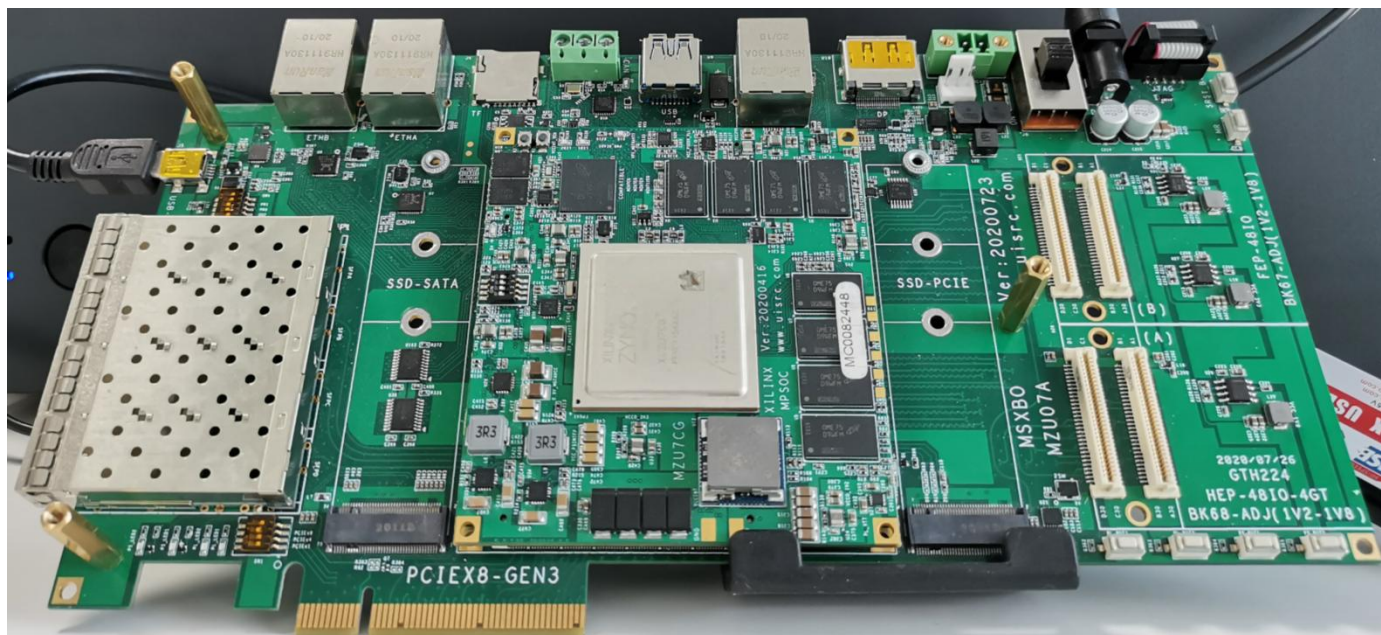
6.3.2 fpga_pin.xdc 中 IO 约束

最后的 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design] 是为了对 fpga 的 bit 进行压缩, 减少 bit 大小, 提高加载速度。

```
set_property PACKAGE_PIN A21 [get_ports {GPIO_LED[0]}]
set_property IOSTANDARD LVCMOS18 [get_ports {GPIO_LED[0]}]
set_property PACKAGE_PIN A18 [get_ports {GPIO_LED[1]}]
set_property IOSTANDARD LVCMOS18 [get_ports {GPIO_LED[1]}]
```

6.4 硬件接线

- 1: USB-232 USB 数据线 (我们已经知道开发板集成了 USB 转串口芯片) 接入电脑 USB 接口
- 2: 接通电源和 JTAG
- 3: 模式开关设置为 JTAG 模式, 模式开关全部切换到 ON



6.5 仿真文件

编写仿真文件，配套代码 uisrc/02_sim 路径下有源码

```
module axi_top_sim();

reg sysclk_p;

wire [1:0] btn = 2'b01;
wire [1:0] led;

system_wrapper system_wrapper_inst
(
    .sysclk_p(sysclk_p),
    .sysclk_n(~sysclk_p),
    .btn(btn),
    .led(led)
);

initial begin
    sysclk_p = 1'b0;
    #100;
end

always
begin
    #5 sysclk_p = ~sysclk_p;
end

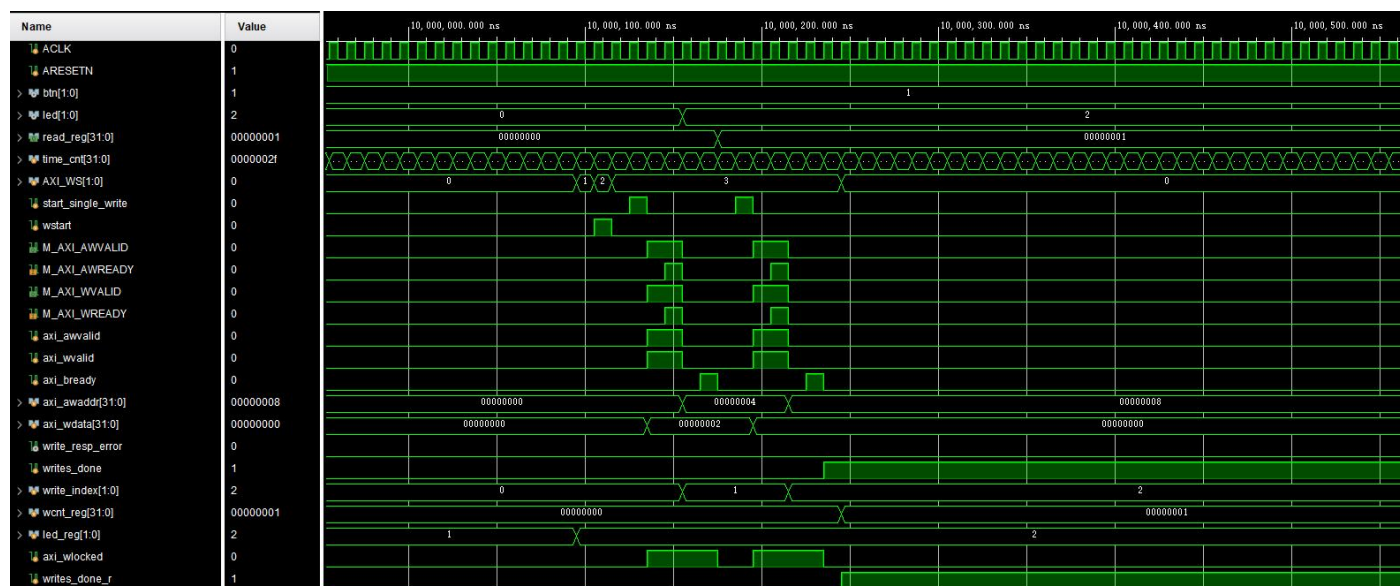
endmodule
```

6.6 实验结果

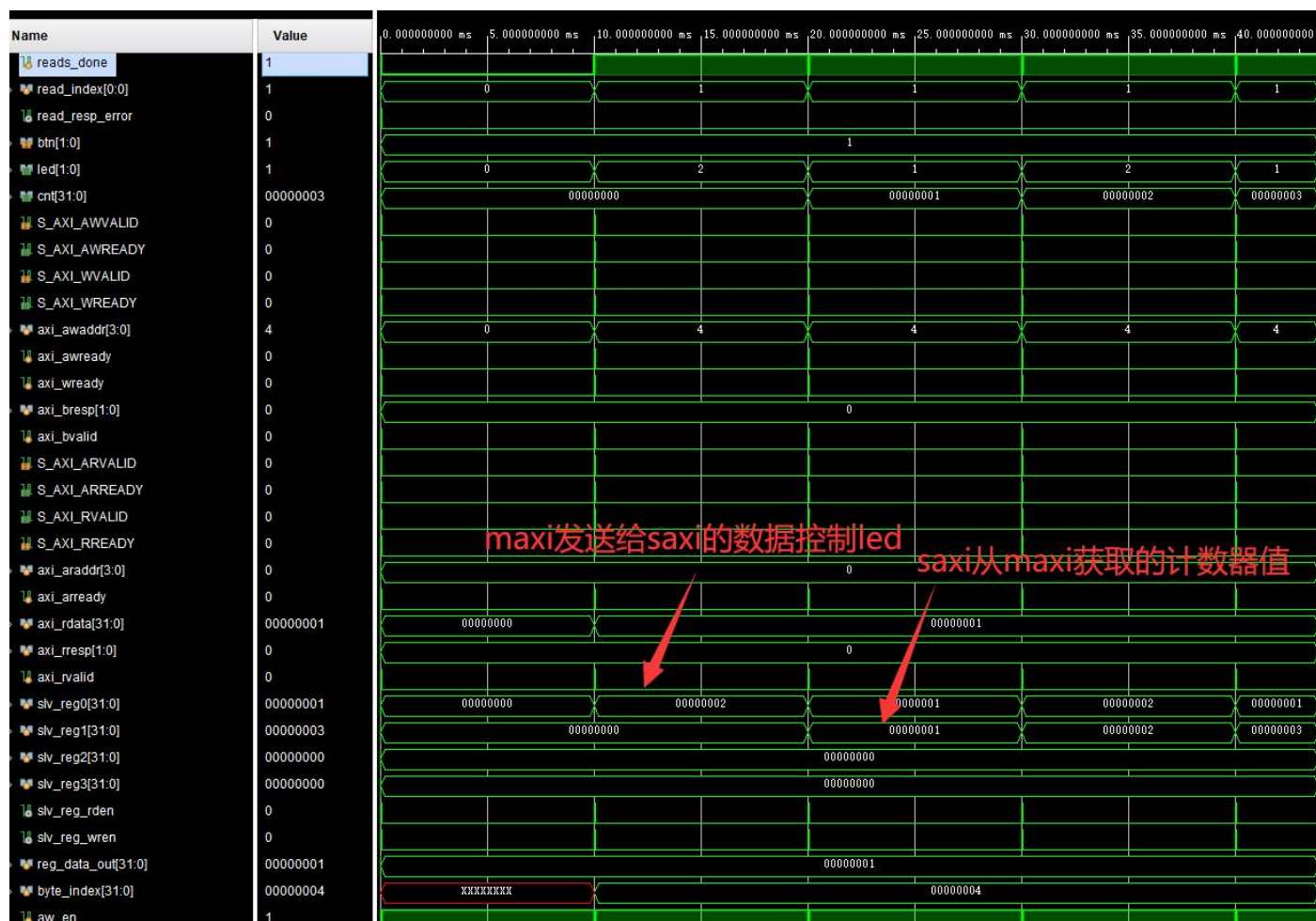
为了加快仿真，可以把 maxi_lite_gpio 的 IP 源码中读写的间隔时间减少，如下图所示：

```
maxi_lite_gpio.v
G: > ZU07A > EG > soc > 02_axi_bus_7eg > 06_axi_lite_gpio_fpga > fpga_prj > uisrc > 03_ip > maxi_lite > maxi_lite_gpio.v
23     input wire M_AXI_WREADY,
24     input wire [1 : 0] M_AXI_BRESP,
25     input wire M_AXI_BVALID,
26     output wire M_AXI_BREADY,
27     output wire [C_M_AXI_ADDR_WIDTH-1 : 0] M_AXI_ARADDR,
28     output wire [2 : 0] M_AXI_ARPROT,
29     output wire M_AXI_ARVALID,
30     input wire M_AXI_ARREADY,
31     input wire [C_M_AXI_DATA_WIDTH-1 : 0] M_AXI_RDATA,
32     input wire [1 : 0] M_AXI_RRESP,
33     input wire M_AXI_RVALID,
34     output wire M_AXI_RREADY,
35     output reg [31:0] read_reg
36 );
37
38     localparam TIME_SET = 999999;
39     localparam S_IDLE = 0;
40     localparam S_START1 = 1;
41     localparam S_START2 = 2;
42     localparam S_DATA = 3;
43     reg [31:0] time_cnt;
44
```

maxi 写入数据仿真



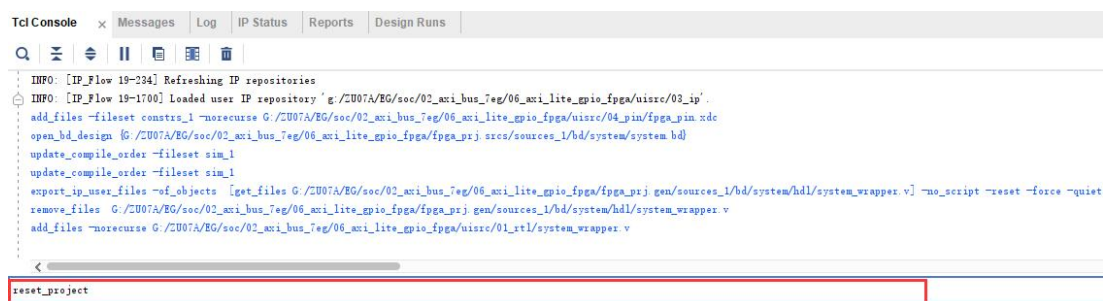
Name	Value	
AXI_RS[1:0]	0	
start_single_read	0	
rstart	0	
M_AXI_ARVALID	0	
M_AXI_ARREADY	0	
M_AXI_RVALID	0	
M_AXI_RREADY	0	
axi_arvalid	0	
axi_rlocked	0	
axi_araddr[31:0]	00000004	
axi_ready	0	
read_reg[31:0]	00000001	
reads_done	1	
read_index[0:0]	1	
read_resp_error	0	



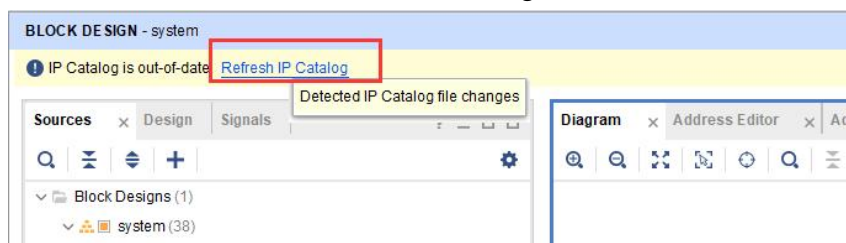
编译下载测试前，把 TIME_SET 改回到 TIME_SET = 99999999，这样对于 100M 时钟每间隔 500ms 读写一次。

```
maxi_lite_gpio.v
G: > ZU07A > EG > soc > 02_axi_lite_gpio_fpga > fpga_prj > uisrc > 03_ip > maxi_lite > maxi_lite_gpio.v
20      output wire [C_M_AXI_DATA_WIDTH-1 : 0] M_AXI_WDATA,
21      output wire [C_M_AXI_DATA_WIDTH/8-1 : 0] M_AXI_WSTRB,
22      output wire  M_AXI_WVALID,
23      input wire  M_AXI_WREADY,
24      input wire [1 : 0] M_AXI_BRESP,
25      input wire  M_AXI_BVALID,
26      output wire  M_AXI_BREADY,
27      output wire [C_M_AXI_ADDR_WIDTH-1 : 0] M_AXI_ARADDR,
28      output wire [2 : 0] M_AXI_ARPROT,
29      output wire  M_AXI_ARVALID,
30      input wire  M_AXI_ARREADY,
31      input wire [C_M_AXI_DATA_WIDTH-1 : 0] M_AXI_RDATA,
32      input wire [1 : 0] M_AXI_RRESP,
33      input wire  M_AXI_RVALID,
34      output wire  M_AXI_RREADY,
35      output reg  [31:0] read_reg
36  );
37
38      localparam TIME_SET = 99999999;
39      localparam S_IDLE   = 0;
40      localparam S_START1 = 1;
41      localparam S_START2 = 2;
42      localparam S_DATA   = 3;
43      reg [31:0] time_cnt;
44  **
```

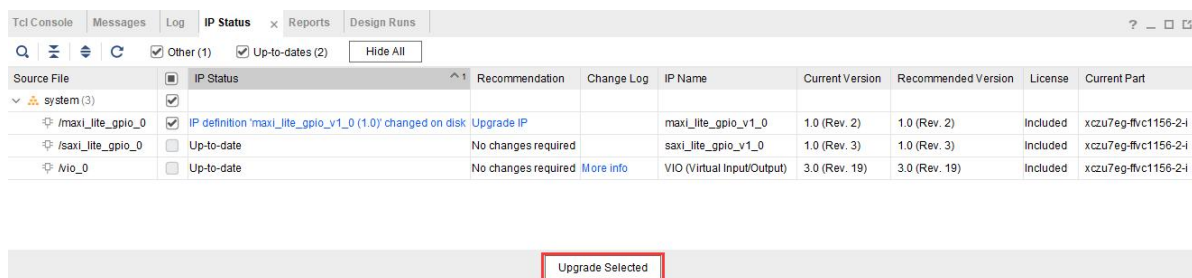
每次更改 ip 源码后，在 Tcl Console 中输入 reset_project 对 fpga 工程进行复位



然后根据提示单击下图中 Refresh IP Catalog 更新 IP 状态



再单击 Upgrade Selected



最后编译产生 bit 文件下载到开发板测试。

在路径..\uisrc\06_doc 路径有本实验的运行结果视频

07AXI4-FULL-MASTER IP FDMA 详解

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

7.1 概述

FDMA 是米联客的基于 AXI4 总线协议定制的一个 DMA 控制器。本文对 AXI4-FULL 总线接口进行了封装, 同时定义了简单的 APP 接口提供用户调用 AXI4 总线实现数据交互。这个 IP 我们命名为 FDMA(Fast Direct Memory Access)。

有了这个 IP 我们可以统一实现用 FPGA 代码直接读写 PL 的 DDR 或者 ZYNQ/ZYNQMP SOC PS 的 DDR 或者 BRAM。FDMA IP CORE 已经广泛应用于 ZYNQ SOC/Artix7/Kintex7 FPGA, 同样适用于 ultrascale/ultrascale+系列 FPGA/SOC。

如果用 ZYNQ/ZYNQMP 的都知道, 要直接操作 PS 的 DDR 通常是 DMA 或者 VDMA, 然而用过 XILINX 的 DMA IP 和 VDMA IP, 总有一种遗憾, 那就是不够灵活, 还需要对寄存器配置, 真是麻烦。XILINX 的总线接口是 AXI4 总线, 自定义 AXI4 IP 挂到总线上就能实现对内存地址空间的读写访问。因此, 我们只要掌握 AXI4 协议就能完成不管是 PS 还是 PL DDR 的读写操作。

米联客封装的 AXI4 总线协议命名位 uiFDMA, 自动 2018 年第一版本发布后, 就引起了很多 FPGA 工程师的兴趣, 并且得到了广大 FPGA 工程师的好评, 但是 FDMA1.0 版本还是有一些局限和 BUG, 再实际的应用中被 FPGA 工程师发现, 因此给了我们很多宝贵意见。借此 2020 版本教程更新发布之际, 我们也对 FDMA1.0 版本升级到 FDMA2.0 版本。目前最新的版本是 3.0 版本, 对 2.0 版本优化了自动 burst 的长度, 提高了效率。

uiFDMA2.0/3.0 新增特性:

- 1: 支持多个 FDMA IP 同时挂带 AXI-interconnect 总线, 同时工作
- 2: 支持自动计算没错 AXI-Burst 长度, 使用起来非常简单, 只需要给出需要 burst 的长度。

从本文开始, 我们从多个应用方案来演示 FDMA 的用途。

本文实验目的:

- 1: 分析 FDMA 源码, 掌握基于 FDMA 的 APP 接口实现 AXI4-FULL 总线接口的访问。
- 2: 掌握自定义总线接口封装方法
- 3: 自定义 AXI-FULL-Slave IP 用于验证 FDMA 的工作情况。

7.2 AXI 总线协议介绍

7.2.1 AXI 总线概述

在 XILINX FPGA 的软件工具 vivado 以及相关 IP 中有支持三种 AXI 总线, 拥有三种 AXI 接口, 当然用的都是 AXI 协议。其中三种 AXI 总线分别为:

AXI4: (For high-performance memory-mapped requirements.) 主要面向高性能地址映射通信的需求, 是面向地址映射的接口, 允许最大 256 轮的数据突发传输;

AXI4-Lite: (For simple, low-throughput memory-mapped communication) 是一个轻量级的地址映射单次传输接口, 占用很少的逻辑单元。

AXI4-Stream: (For high-speed streaming data.) 面向高速流数据传输; 去掉了地址项, 允许无限制的数据突发传输规模。

由于 AXI4 和 AXI4-Lite 信号大部分一样, 以下只介绍 AXI4 信号。另外对于 AXI4-Stream 协议不再本文中接收, 后面有单独介绍的文章。

7.2.2AXI-4 总线信号功能

1:时钟和复位

信号	方向	描述
ACLK	时钟源	全局时钟信号
ARESETn	复位源	全局复位信号，低有效

写地址通道信号：

信号	方向	描述
AWID	主机 to 从机	写地址 ID，用来标志一组写信号
AWADDR	主机 to 从机	写地址，给出一次写突发传输的写地址
AWLEN	主机 to 从机	AWLEN[7:0]决定写传输的突发长度。AXI3 只支持 1~16 次的突发传输（Burst_length=AxLEN[3:0]+1），AXI4 扩展突发长度支持 INCR 突发类型为 1~256 次传输，对于其他的传输类型依然保持 1~16 次突发传输（Burst_Length=AxLEN[7:0]+1）。burst 传输具有如下规则： wrapping burst ,burst 长度必须是 2,4,8,16 burst 不能跨 4KB 边界 不支持提前终止 burst 传输
AWSIZE	主机 to 从机	写突发大小，给出每次突发传输的字节数支持 1、2、4、8、16、32、64、128
AWBURST	主机 to 从机	突发类型： 2'b00 FIXED：突发传输过程中地址固定，用于 FIFO 访问 2'b01 INCR ：增量突发，传输过程中，地址递增。增加量取决 AxSIZE 的值。 2'b10 WRAP：回环突发，和增量突发类似，但会在特定高地址的边界处回到低地址处。回环突发的长度只能是 2,4,8,16 次传输，传输首地址和每次传输的大小对齐。最低的地址整个传输的数据大小对齐。回环边界等于（AxSIZE*AxLEN） 2'b11 Reserved
AWLOCK	主机 to 从机	总线锁信号，可提供操作的原子性
AWCACHE	主机 to 从机	内存类型，表明一次传输是怎样通过系统的
AWPROT	主机 to 从机	保护类型，表明一次传输的特权级及安全等级
AWQOS	主机 to 从机	质量服务 QoS
AWREGION	主机 to 从机	区域标志，能实现单一物理接口对应的多个逻辑接口
AWUSER	主机 to 从机	用户自定义信号
AWVALID	主机 to 从机	有效信号，表明此通道的地址控制信号有效
AWREADY	从机 to 主机	表明“从”可以接收地址和对应的控制信号

2:写数据通道信号：

信号名	方向	描述
WID	主机 to 从机	一次写传输的 ID tag
WDATA	主机 to 从机	写数据
WSTRB	主机 to 从机	WSTRB[n:0]对应于对应的写字节，WSTRB[n]对应 WDATA[8n+7:8n]。WVALID 为低时，WSTRB 可以为任意值，WVALID 为高时，WSTRB 为高的字节线必须指示有效的数据。
WLAST	主机 to 从机	表明此次传输是最后一个突发传输
WUSER	主机 to 从机	用户自定义信号
WVALID	主机 to 从机	写有效，表明此次写有效
WREADY	从机 to 主机	表明从机可以接收写数据

写响应信号：

信号名	方向	描述
BID	从机 to 主机	写响应 ID tag

BRESP	从机 to 主机	写响应, 表明写传输的状态
BUSER	从机 to 主机	用户自定义
BVALID	从机 to 主机	写响应有效
BREADY	主机 to 从机	表明主机能够接收写响应

3: 读地址通道信号:

信号	方向	描述
ARID	主机 to 从机	读地址 ID, 用来标志一组写信号
ARADDR	主机 to 从机	读地址, 给出一次读突发传输的读地址
ARLEN	主机 to 从机	ARLEN[7:0]决定读传输的突发长度。AXI3 只支持 1~16 次的突发传输 (Burst_length=AxLEN[3:0]+1), AXI4 扩展突发长度支持 INCR 突发类型为 1~256 次传输, 对于其他的传输类型依然保持 1~16 次突发传输 (Burst_Length=AxLEN[7:0]+1)。 burst 传输具有如下规则: wrapping burst ,burst 长度必须是 2,4,8,16 burst 不能跨 4KB 边界 不支持提前终止 burst 传输
ARSIZE	主机 to 从机	读突发大小, 给出每次突发传输的字节数支持 1、2、4、8、16、32、64、128
ARBURST	主机 to 从机	突发类型: 2'b00 FIXED: 突发传输过程中地址固定, 用于 FIFO 访问 2'b01 INCR : 增量突发, 传输过程中, 地址递增。增加量取决 AxSIZE 的值。 2'b10 WRAP: 回环突发, 和增量突发类似, 但会在特定高地址的边界处回到低地址处。回环突发的长度只能是 2,4,8,16 次传输, 传输首地址和每次传输的大小对齐。最低的地址整个传输的数据大小对齐。回环边界等于 (AxSIZE*AxLEN) 2'b11 Reserved
ARLOCK	主机 to 从机	总线锁信号, 可提供操作的原子性
ARCACHE	主机 to 从机	内存类型, 表明一次传输是怎样通过系统的
ARPROT	主机 to 从机	保护类型, 表明一次传输的特权级及安全等级
ARQOS	主机 to 从机	质量服务 QoS
ARREGION	主机 to 从机	区域标志, 能实现单一物理接口对应的多个逻辑接口
ARUSER	主机 to 从机	用户自定义信号
ARVALID	主机 to 从机	有效信号, 表明此通道的地址控制信号有效
ARREADY	从机 to 主机	表明“从”可以接收地址和对应的控制信号

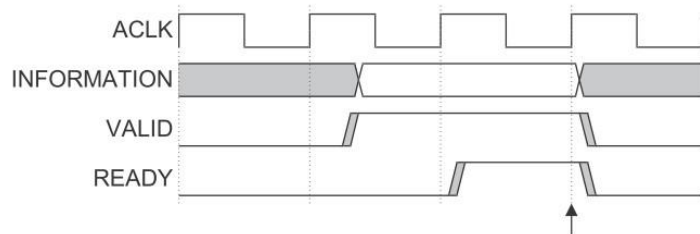
4: 读数据通道信号:

信号名	方向	描述
RID	从机 to 主机	一次读传输的 ID tag
RDATA	从机 to 主机	读数据
RRESP	从机 to 主机	读响应, 表明读传输的状态
RLAST	从机 to 主机	表明此次传输是最后一个突发传输
RUSER	从机 to 主机	用户自定义信号
RVALID	从机 to 主机	写有效, 表明此次写有效
RREADY	主机 to 从机	表明从机可以接收写数据

7.2.3 数据有效的情况

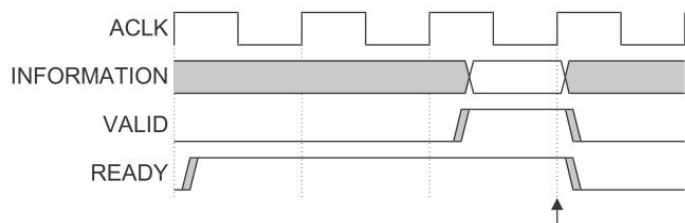
AXI4 所采用的是一种 READY, VALID 握手通信机制, 简单来说主从双方进行数据通信前, 有一个握手的过程。传输源产生 VLAID 信号来指明何时数据或控制信息有效。而目的地源产生 READY 信号来指明已经准备好接受数据或控制信息。传输发生在 VALID 和 READY 信号同时为高的时候。VALID 和 READY 信号的出现有三种关系。

(4) VALID 先变高 READY 后变高。时序图如下:



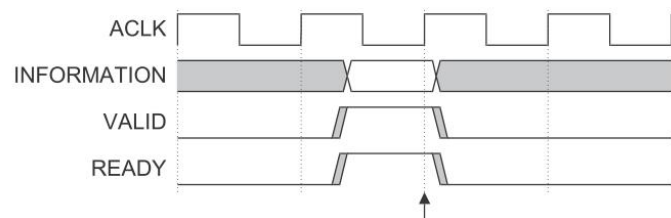
在箭头处信息传输发生。

(5) READY 先变高 VALID 后变高。时序图如下：



同样在箭头处信息传输发生。

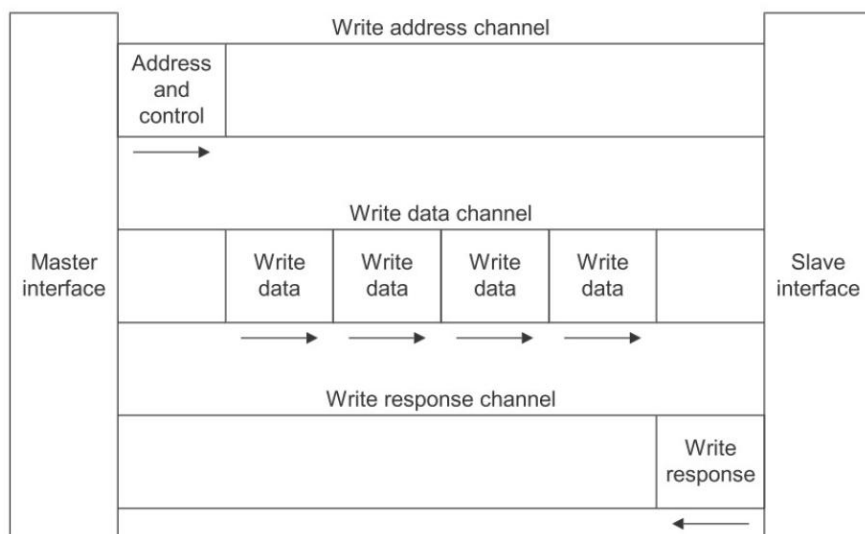
(6) VALID 和 READY 信号同时变高。时序图如下：



在这种情况下，信息传输立马发生，如图箭头处指明信息传输发生。

7.2.4 突发式读写

1: 突发式写时序图



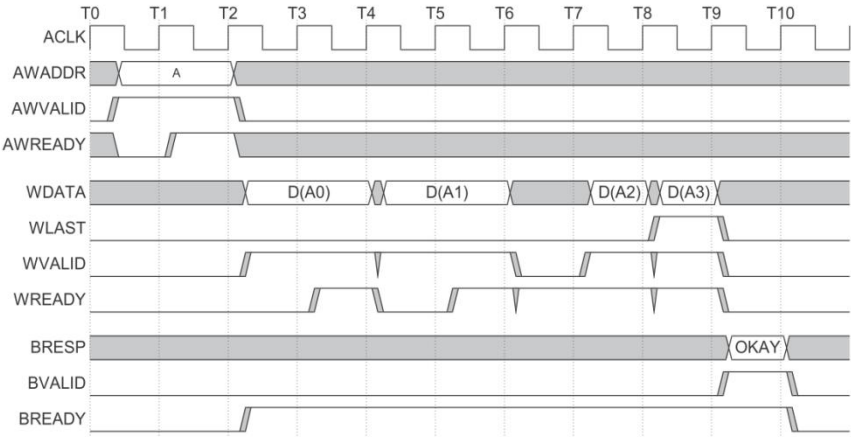


Figure 1-6 Write burst

这一过程的开始时，主机发送地址和控制信息到写地址通道中，然后主机发送每一个写数据到写数据通道中。当主机发送最后一个数据时，WLAST 信号就变为高。当设备接收完所有数据之后他将一个写响应发送回主机来表明写事务完成。

2:突发式读的时序图

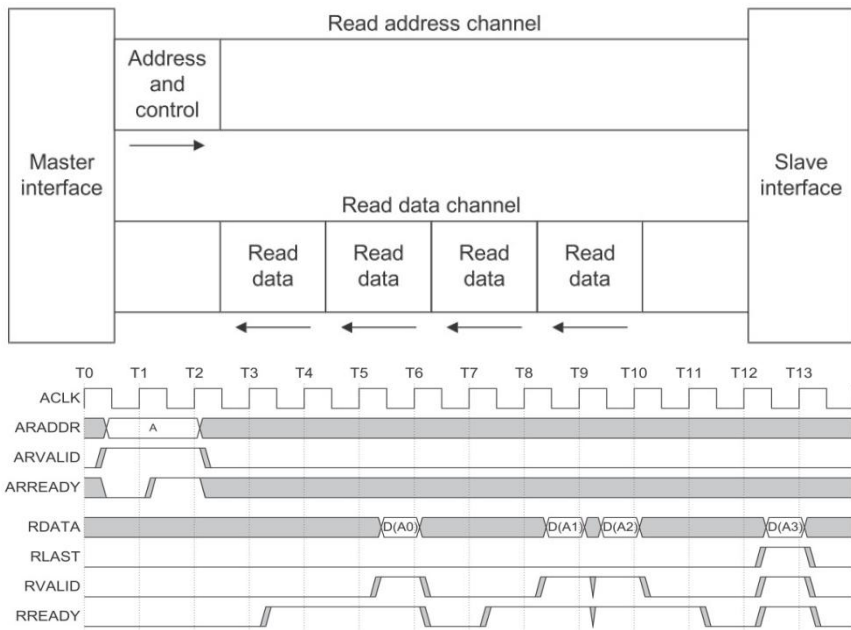


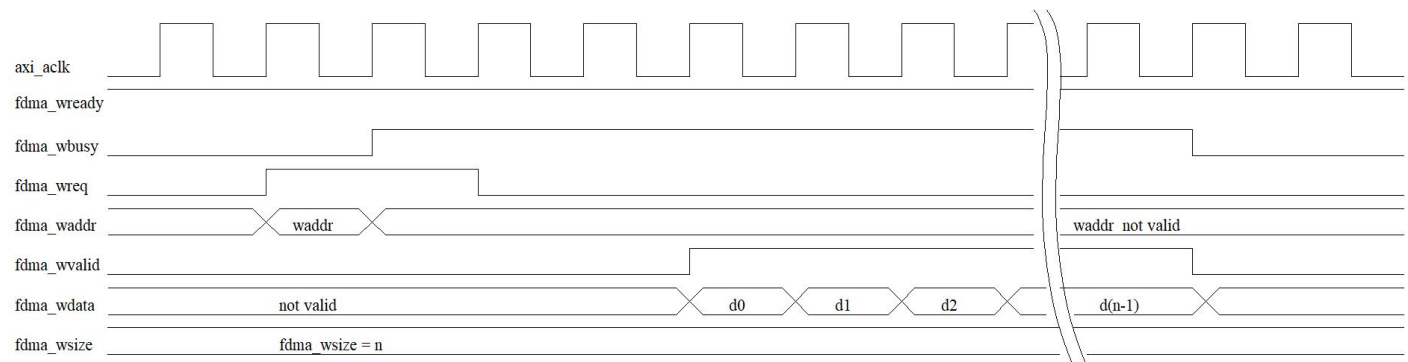
Figure 1-4 Read burst

当地址出现在地址总线后，传输的数据将出现在读数据通道上。设备保持 VALID 为低直到读数据有效。为了表明一次突发式读写的完成，设备用 RLAST 信号来表示最后一个被传输的数据。

7.3FDMA 源码分析

由于 AXI4 总线协议直接操作起来相对复杂一些,容易出错,因此我们封装一个简单的用户接口,间接操作 AXI4 总线会带来很多方便性。先看下我们计划设计一个怎么样的用户接口。

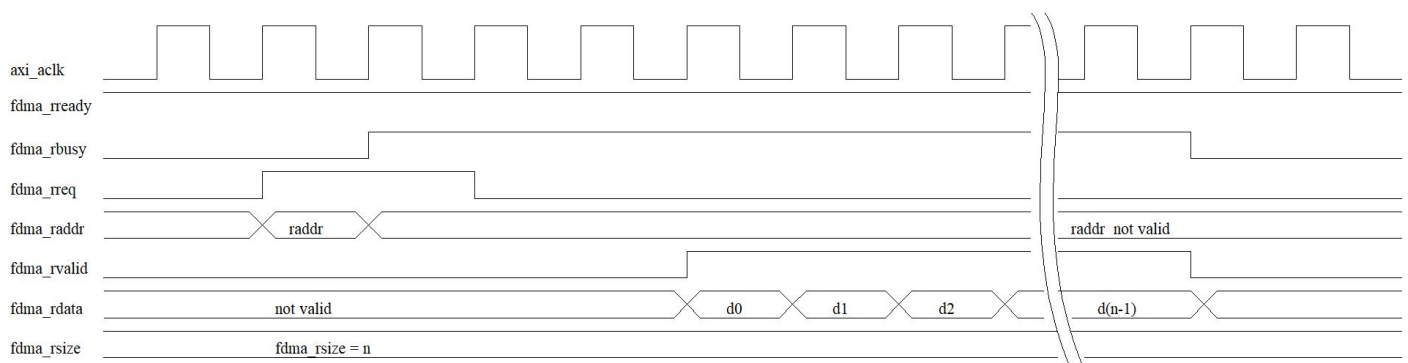
1:FDMA 的写时序



fdma_wready 设置为 1，当 fdma_wbusy=0 的时候代表 FDMA 的总线非忙，可以进行一次新的 FDMA 传输，这个时候可以设置 fdma_wreq=1，同时设置 fdma burst 的起始地址和 fdma_wsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_wvalid=1 的时候需要给出有效的数据，写入 AXI 总线。当最后一个数写完后，fdma_wvalid 和 fdma_wbusy 变为 0。

AXI4 总线最大的 burst length 是 256，而经过封装后，用户接口的 fdma_size 可以任意大小的，fdma ip 内部代码控制每次 AXI4 总线的 Burst 长度，这样极大简化了 AXI4 总线协议的使用。

2:FDMA 的读时序



fdma_rready 设置为 1，当 fdma_rbusy=0 的时候代表 FDMA 的总线非忙，可以进行一次新的 FDMA 传输，这个时候可以设置 fdma_rreq=1，同时设置 fdma burst 的起始地址和 fdma_rsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_rvalid=1 的时候需要给出有效的数据，写入 AXI 总线。当最后一个数写完后，fdma_rvalid 和 fdma_rbusy 变为 0。

同样对于 AXI4 总线的读操作，AXI4 总线最大的 burst length 是 256，而经过封装后，用户接口的 fdma_size 可以任意大小的，fdma ip 内部代码控制每次 AXI4 总线的 Burst 长度，这样极大简化了 AXI4 总线协议的使用。

3:FDMA 的 AXI4-Master 写操作

以下代码中我们给出 axi4-master 写操作的代码分析注释

```
//fdma axi write-----
reg  [M_AXI_ADDR_WIDTH-1 : 0]  axi_awaddr  =0; //AXI4 写地址
reg                                     axi_awvalid  = 1'b0; //AXI4 写地有效
wire  [M_AXI_DATA_WIDTH-1 : 0]  axi_wdata   ; //AXI4 写数据
wire                                     axi_wlast   ; //AXI4 写 LAST 信号
reg                                     axi_wvalid  = 1'b0; //AXI4 写数据有效
wire                                     w_next      = (M_AXI_WVALID & M_AXI_WREADY); //当 valid ready 信号都有效，代表 AXI4 数据传输有效
```

```

reg    [8:0]                wburst_len  = 1 ;//写传输的 axi burst 长度，代码会自动计算每次 axi 传输的 burst 长度
reg    [8:0]                wburst_cnt  = 0 ;//每次 axi burst 的计数器
reg    [15:0]               wfdma_cnt   = 0 ;//fdma 的写数据计数器
reg                                axi_wstart_locked =0; //axi 传输进行中，lock 住，用于时序控制
wire   [15:0] axi_wburst_size  =  wburst_len * AXI_BYTES;//axi 传输的地址长度计算

assign M_AXI_AWID            = M_AXI_ID; //写地址 ID，用来标志一组写信号, M_AXI_ID 是通过参数接口定义
assign M_AXI_AWADDR          = axi_awaddr;
assign M_AXI_AWLEN           = wburst_len - 1;//AXI4 burst 的长度
assign M_AXI_AWSIZE          = clogb2(AXI_BYTES-1);
assign M_AXI_AWBURST         = 2'b01;//AXI4 的 busr 类型 INCR 模式，地址递增
assign M_AXI_AWLOCK          = 1'b0;
assign M_AXI_AWCACHE         = 4'b0010;//不使用 cache,不使用 buffer
assign M_AXI_AWPROT          = 3'h0;
assign M_AXI_AWQOS           = 4'h0;
assign M_AXI_AWVALID         = axi_awvalid;
assign M_AXI_WDATA           = axi_wdata;
assign M_AXI_WSTRB            = {(AXI_BYTES){1'b1}};//设置所有的 WSTRB 为 1 代表传输的所有数据有效
assign M_AXI_WLAST           = axi_wlast;
assign M_AXI_WVALID          = axi_wvalid & fdma_wready;//写数据有效，这里必须设置 fdma_wready 有效
assign M_AXI_BREADY          = 1'b1;

//-----
//AXI4 FULL Write
assign  axi_wdata            = fdma_wdata;
assign  fdma_wvalid          = w_next;
reg      fdma_wstart_locked = 1'b0;
wire     fdma_wend;
wire     fdma_wstart;
assign   fdma_wbusy = fdma_wstart_locked ;
//在整个写过程中 fdma_wstart_locked 将保持有效，直到本次 FDMA 写结束
always @(posedge M_AXI_ACLK)
    if(M_AXI_ARESETN == 1'b0 || fdma_wend == 1'b1 )
        fdma_wstart_locked <= 1'b0;
    else if(fdma_wstart)
        fdma_wstart_locked <= 1'b1;
//产生 fdma_wstart 信号，整个信号保持 1 个 M_AXI_ACLK 时钟周期
assign fdma_wstart = (fdma_wstart_locked == 1'b0 && fdma_wareq == 1'b1);

//AXI4 write burst lenth busrt addr -----
//当 fdma_wstart 信号有效，代表一次新的 FDMA 传输，首先把地址本次 fdma 的 burst 地址寄存在 axi_awaddr 作为第一次 axi burst 的地址。如果 fdma 的数据
//长度大于 256，那么当 axi_wlast 有效的时候，自动计算下次 axi 的 burst 地址
always @(posedge M_AXI_ACLK)
    if(fdma_wstart)
        axi_awaddr <= fdma_waddr;
    else if(axi_wlast == 1'b1)
        axi_awaddr <= axi_awaddr + axi_wburst_size ;
//AXI4 write cycle -----
axi_wstart_locked_r1, axi_wstart_locked_r2 信号是用于时序同步
reg axi_wstart_locked_r1 = 1'b0, axi_wstart_locked_r2 = 1'b0;

```

```

always @(posedge M_AXI_ACLK)begin
    axi_wstart_locked_r1 <= axi_wstart_locked;
    axi_wstart_locked_r2 <= axi_wstart_locked_r1;
end

// axi_wstart_locked 的作用代表一次 axi 写 burst 操作正在进行中。
always @(posedge M_AXI_ACLK)
    if((fdma_wstart_locked == 1'b1) && axi_wstart_locked == 1'b0)
        axi_wstart_locked <= 1'b1;
    else if(axi_wlast == 1'b1 || fdma_wstart == 1'b1)
        axi_wstart_locked <= 1'b0;

//AXI4 addr valid and write addr-----
always @(posedge M_AXI_ACLK)
    if((axi_wstart_locked_r1 == 1'b1) && axi_wstart_locked_r2 == 1'b0)
        axi_awvalid <= 1'b1;
    else if((axi_wstart_locked == 1'b1 && M_AXI_AWREADY == 1'b1) || axi_wstart_locked == 1'b0)
        axi_awvalid <= 1'b0;

//AXI4 write data-----
always @(posedge M_AXI_ACLK)
    if((axi_wstart_locked_r1 == 1'b1) && axi_wstart_locked_r2 == 1'b0)
        axi_wvalid <= 1'b1;
    else if(axi_wlast == 1'b1 || axi_wstart_locked == 1'b0)
        axi_wvalid <= 1'b0;

//AXI4 write data burst len counter-----
always @(posedge M_AXI_ACLK)
    if(axi_wstart_locked == 1'b0)
        wburst_cnt <= 'd0;
    else if(w_next)
        wburst_cnt <= wburst_cnt + 1'b1;

assign axi_wlast = (w_next == 1'b1) && (wburst_cnt == M_AXI_AWLEN);

//fdma write data burst len counter-----
reg wburst_len_req = 1'b0;
reg [15:0] fdma_wleft_cnt = 16'd0;

// wburst_len_req 信号是自动管理每次 axi 需要 burst 的长度
always @(posedge M_AXI_ACLK)
    wburst_len_req <= fdma_wstart[axi_wlast];

// fdma_wleft_cnt 用于记录一次 FDMA 剩余需要传输的数据数量
always @(posedge M_AXI_ACLK)
    if( fdma_wstart )begin
        wfdma_cnt <= 1'd0;
        fdma_wleft_cnt <= fdma_wsize;
    end
    else if(w_next)begin
        wfdma_cnt <= wfdma_cnt + 1'b1;
        fdma_wleft_cnt <= (fdma_wsize - 1'b1) - wfdma_cnt;
    end
end

```

//当最后一个数据的时候, 产生 fdma_wend 信号代表本次 fdma 传输结束

```
assign fdma_wend = w_next && (fdma_wleft_cnt == 1);
```

//一次 axi 最大传输的长度是 256 因此当大于 256, 自动拆分多次传输

```
always @(posedge M_AXI_ACLK)begin
```

```
    if(wburst_len_req)begin
```

```
        if(fdma_wleft_cnt[15:8] > 0) wburst_len <= 256;
```

```
    else
```

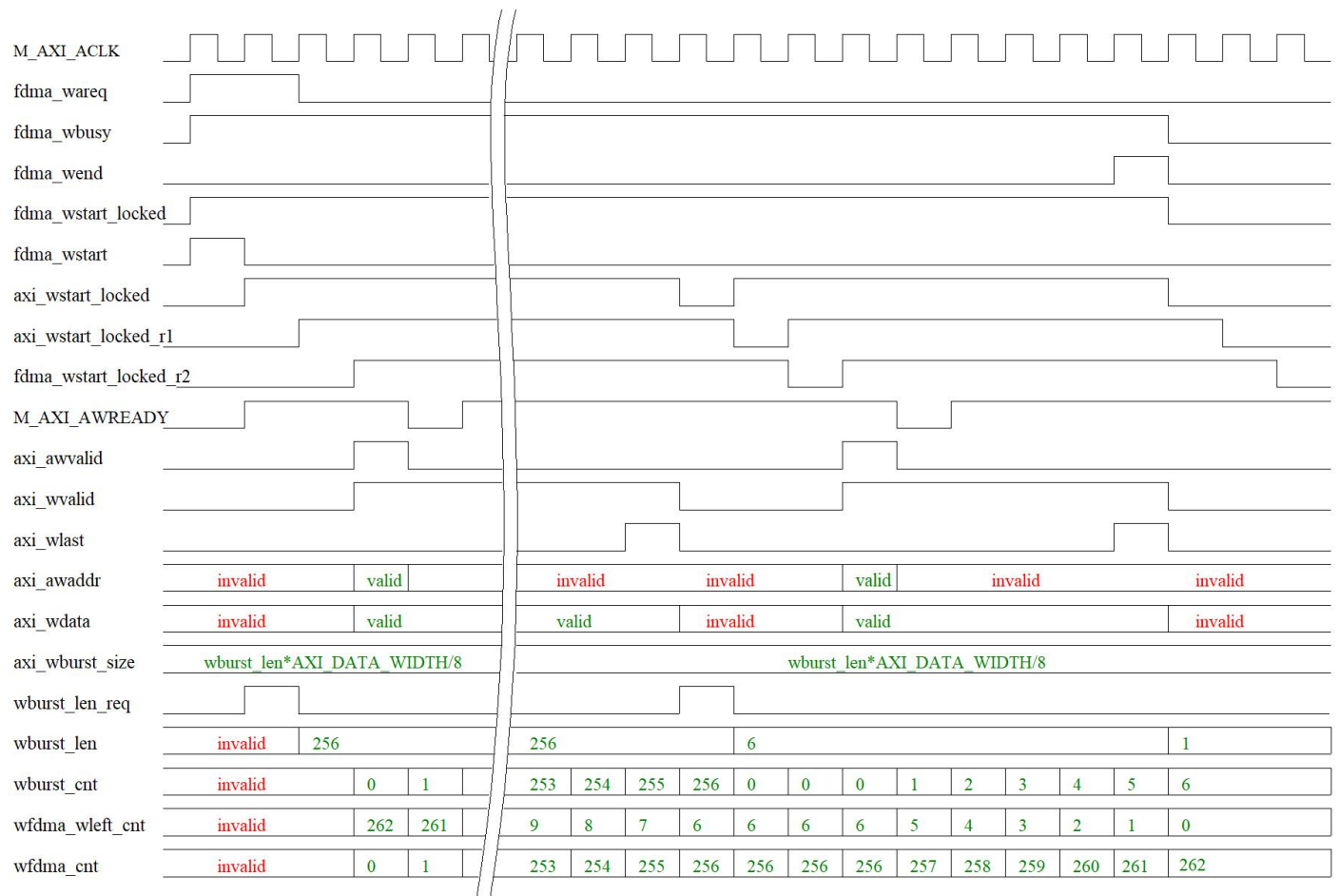
```
        wburst_len <= fdma_wleft_cnt[7:0];
```

```
    end
```

```
    else wburst_len <= wburst_len;
```

```
end
```

以上代码我们进行了详细的注释性分析。以下给出 FDMA 写操作源码部分的时序图。下图中一次传输以传输 262 个长度的数据为例, 需要 2 次 AXI4 BURST 才能完成, 第一次传输 256 个长度数据, 第二次传输 6 个长度的数据。



4:FDMA 的 AXI4-Master 读操作

以下代码中我们给出 axi4-master 读操作的代码分析注释

```
//fdma axi read-----
```

```
reg [M_AXI_ADDR_WIDTH-1:0] axi_araddr=0; //AXI4 读地址
```

```
reg axi_arvalid =1'b0; //AXI4 读地有效
```

```
wire axi_rlast ; //AXI4 读 LAST 信号
```

```
reg axi_rready =1'b0; //AXI4 读准备好
```

```
wire r_next = (M_AXI_RVALID && M_AXI_RREADY); // 当 valid ready 信号都有效, 代表 AXI4 数据传输有效
```

```
reg [8:0] rburst_len = 1 ; //读传输的 axi burst 长度, 代码会自动计算每次 axi 传输的 burst 长度
```

```
reg [8:0] rburst_cnt = 0 ; //每次 axi burst 的计数器
```

```

reg    [15:0]                rfdma_cnt    = 0    ;//fdma 的读数据计数器

reg                                axi_rstart_locked =0; //axi 传输进行中, lock 住, 用于时序控制

wire  [15:0] axi_rburst_size    =    rburst_len * AXI_BYTES; //axi 传输的地址长度计算


assign M_AXI_ARID                = M_AXI_ID; //读地址 ID, 用来标志一组写信号, M_AXI_ID 是通过参数接口定义
assign M_AXI_ARADDR              = axi_araddr;
assign M_AXI_ARLEN               = rburst_len - 1; //AXI4 burst 的长度
assign M_AXI_ARSIZE              = clogb2((AXI_BYTES)-1);
assign M_AXI_ARBURST = 2'b01; //AXI4 的 busr 类型 INCR 模式, 地址递增
assign M_AXI_ARLOCK              = 1'b0; //不使用 cache,不使用 buffer
assign M_AXI_ARCACHE= 4'b0010;
assign M_AXI_ARPROT              = 3'h0;
assign M_AXI_ARQOS               = 4'h0;
assign M_AXI_ARVALID = axi_arvalid;
assign M_AXI_RREADY              = axi_rready&&fdma_rready; //读数据准备好, 这里必须设置 fdma_rready 有效
assign fdma_rdata                = M_AXI_RDATA;
assign fdma_rvalid               = r_next;


//AXI4 FULL Read-----

reg    fdma_rstart_locked = 1'b0;
wire    fdma_rend;
wire    fdma_rstart;
assign    fdma_rbusy = fdma_rstart_locked ;

//在整个读过程中 fdma_rstart_locked 将保持有效, 直到本次 FDMA 写结束
always @(posedge M_AXI_ACLK)
    if(M_AXI_ARESETN == 1'b0 || fdma_rend == 1'b1)
        fdma_rstart_locked <= 1'b0;
    else if(fdma_rstart)
        fdma_rstart_locked <= 1'b1;

//产生 fdma_rstart 信号, 整个信号保持 1 个 M_AXI_ACLK 时钟周期
assign fdma_rstart = (fdma_rstart_locked == 1'b0 && fdma_rareq == 1'b1);

//AXI4 read burst lenth busr addr -----
//当 fdma_rstart 信号有效, 代表一次新的 FDMA 传输, 首先把地址本次 fdma 的 burst 地址寄存在 axi_araddr 作为第一次 axi burst 的地址。如果 fdma 的数据长度大于 256, 那么当 axi_rlast 有效的时候, 自动计算下次 axi 的 burst 地址
always @(posedge M_AXI_ACLK)
    if(fdma_rstart == 1'b1)
        axi_araddr <= fdma_raddr;
    else if(axi_rlast == 1'b1)
        axi_araddr <= axi_araddr + axi_rburst_size ;

//AXI4 r_cycle_flag-----
//axi_rstart_locked_r1, axi_rstart_locked_r2 信号是用于时序同步
reg axi_rstart_locked_r1 = 1'b0, axi_rstart_locked_r2 = 1'b0;
always @(posedge M_AXI_ACLK)begin
    axi_rstart_locked_r1 <= axi_rstart_locked;
    axi_rstart_locked_r2 <= axi_rstart_locked_r1;
end

// axi_rstart_locked 的作用代表一次 axi 读 burst 操作正在进行中。
always @(posedge M_AXI_ACLK)

```

```

    if(fdma_rstart_locked == 1'b1) && axi_rstart_locked == 1'b0)
        axi_rstart_locked <= 1'b1;
    else if(axi_rlast == 1'b1 || fdma_rstart == 1'b1)
        axi_rstart_locked <= 1'b0;

//AXI4 addr valid and read addr-----
always @(posedge M_AXI_ACLK)
    if(axi_rstart_locked_r1 == 1'b1) && axi_rstart_locked_r2 == 1'b0)
        axi_arvalid <= 1'b1;
    else if(axi_rstart_locked == 1'b1 && M_AXI_ARREADY == 1'b1) || axi_rstart_locked == 1'b0)
        axi_arvalid <= 1'b0;

//AXI4 read data-----
always @(posedge M_AXI_ACLK)
    if(axi_rstart_locked_r1 == 1'b1) && axi_rstart_locked_r2 == 1'b0)
        axi_rready <= 1'b1;
    else if(axi_rlast == 1'b1 || axi_rstart_locked == 1'b0)
        axi_rready <= 1'b0;

//AXI4 read data burst len counter-----
always @(posedge M_AXI_ACLK)
    if(axi_rstart_locked == 1'b0)
        rburst_cnt <= 'd0;
    else if(r_next)
        rburst_cnt <= rburst_cnt + 1'b1;
assign axi_rlast = (r_next == 1'b1) && (rburst_cnt == M_AXI_ARLEN);

//fdma read data burst len counter-----
reg rburst_len_req = 1'b0;
reg [15:0] fdma_rleft_cnt = 16'd0;
// rburst_len_req 信号是自动管理每次 axi 需要 burst 的长度
always @(posedge M_AXI_ACLK)
    rburst_len_req <= fdma_rstart | axi_rlast;
// fdma_rleft_cnt 用于记录一次 FDMA 剩余需要传输的数据数量
always @(posedge M_AXI_ACLK)
    if(fdma_rstart)begin
        rfdma_cnt <= 1'd0;
        fdma_rleft_cnt <= fdma_rsize;
    end
    else if(r_next)begin
        rfdma_cnt <= rfdma_cnt + 1'b1;
        fdma_rleft_cnt <= (fdma_rsize - 1'b1) - rfdma_cnt;
    end
end

//当最后一个数据的时候, 产生 fdma_rend 信号代表本次 fdma 传输结束
assign fdma_rend = r_next && (fdma_rleft_cnt == 1);

//axi auto burst len caculate-----
//一次 axi 最大传输的长度是 256 因此当大于 256, 自动拆分多次传输
always @(posedge M_AXI_ACLK)begin
    if(rburst_len_req)begin
        if(fdma_rleft_cnt[15:8] > 0)
            rburst_len <= 256;
    end
end

```

```

else
    rburst_len <= fdma_rleft_cnt[7:0];

end

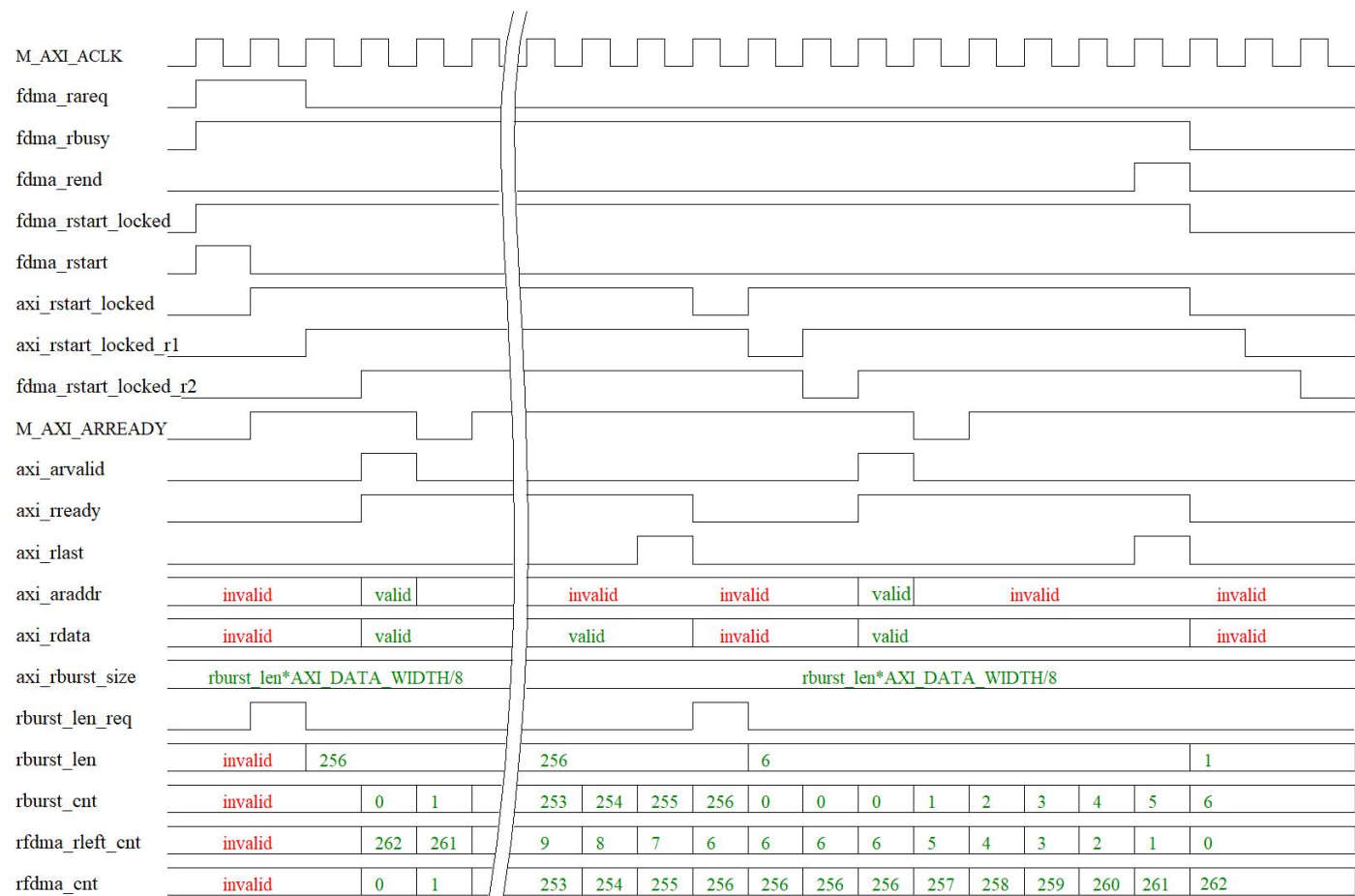
else rburst_len <= rburst_len;

end

```

以上代码我们进行了详细的注释性分析。FDMA 的读写代码高度对称，以上源码和以下波形图都和写操作类似，理解起来会提高很多效率。

以下给出 FDMA 写操作源码部分的时序图。下图中一次传输以传输 262 个长度的数据为例，需要 2 次 AXI4 BURST 才能完成，第一次传输 256 个长度数据，第二次传输 6 个长度的数据。



7.4 FDMA IP 的封装

我先讲解如何封装 FDMA IP,之后再分析源码。封装 IP 少不了源码，这里是利用已经编写好的 uiFDMA.v 进行封装。默认的源码路径在配套的工程 uisrc/uifdma 路径下

ZU07A > EC > 00 > 02_uifdma_7cg > 07_uifdma_full_fdma2axi_slave > uisrc > 03_ip > uifdma			
名称	修改日期	类型	大小
uiFDMA.v	2021/6/9 19:13	Verilog File	14 KB

创建一个新的空的 fpga 工程

 New Project ✕

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:

☐ Create project subdirectory

Project will be created at: G:/ZU07A/EG/soc/02_axi_bus_7eg/07_axi_full_fdma2axi_slave/uisrc/03_ip/uifdma



< Back

Next >

Finish

Cancel

 New Project ✕

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

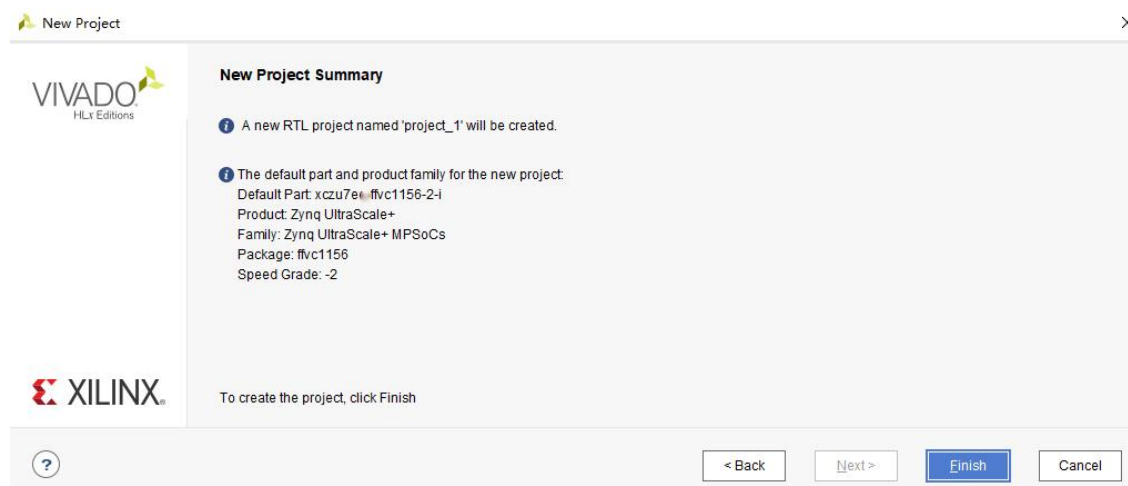
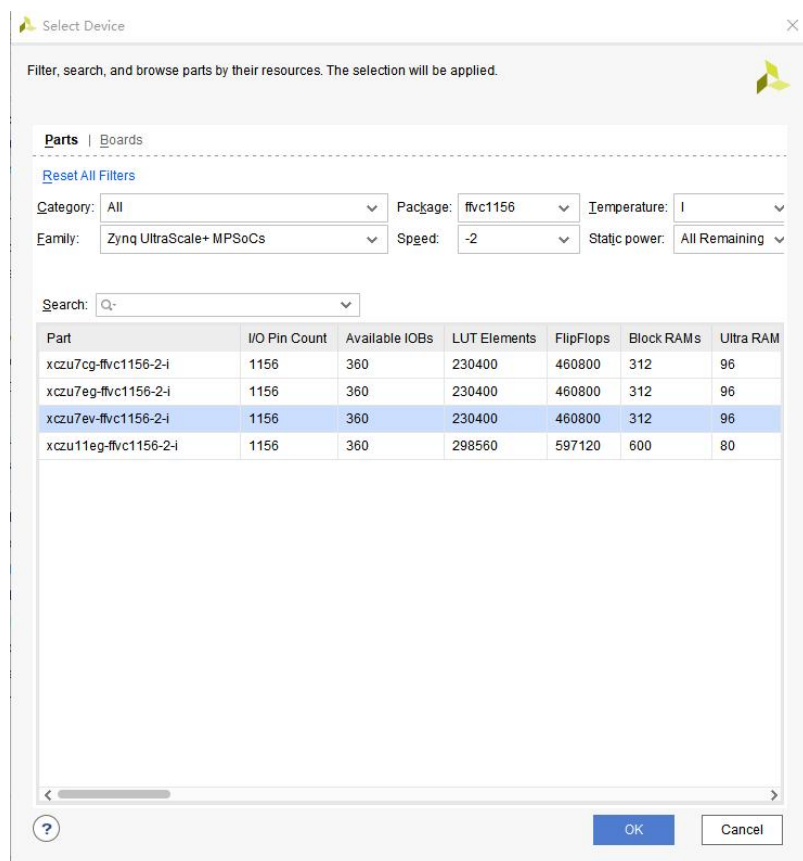


< Back

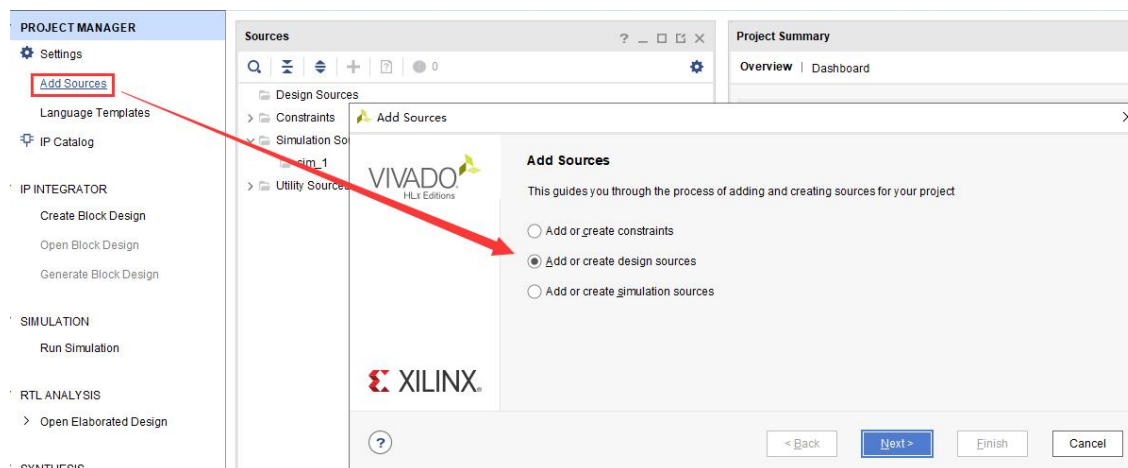
Next >

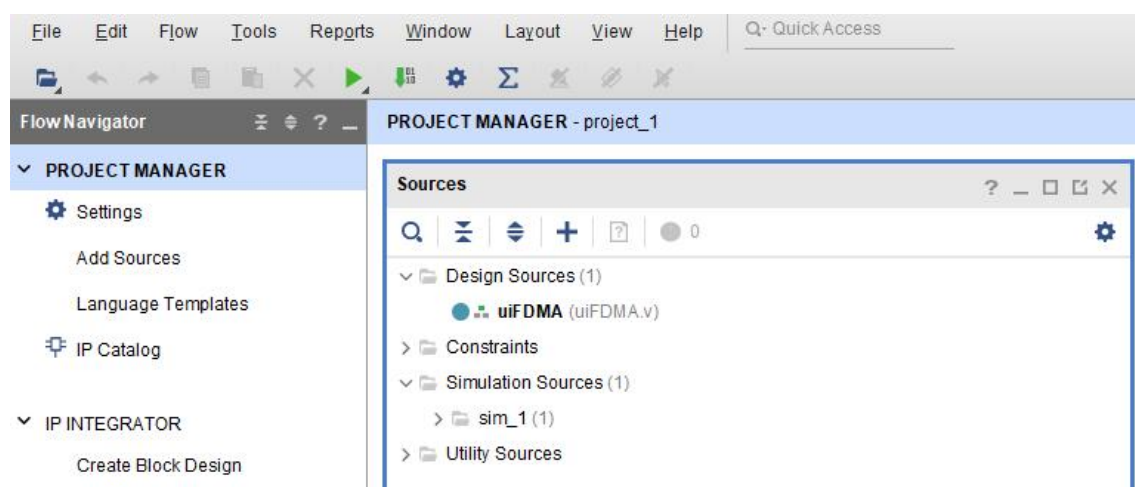
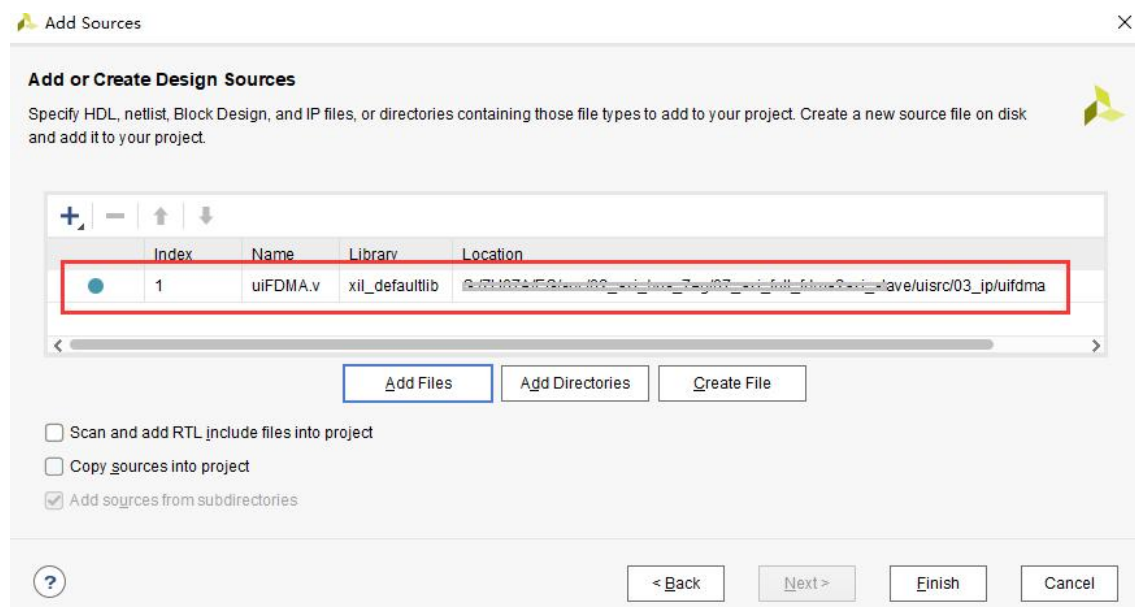
Finish

Cancel

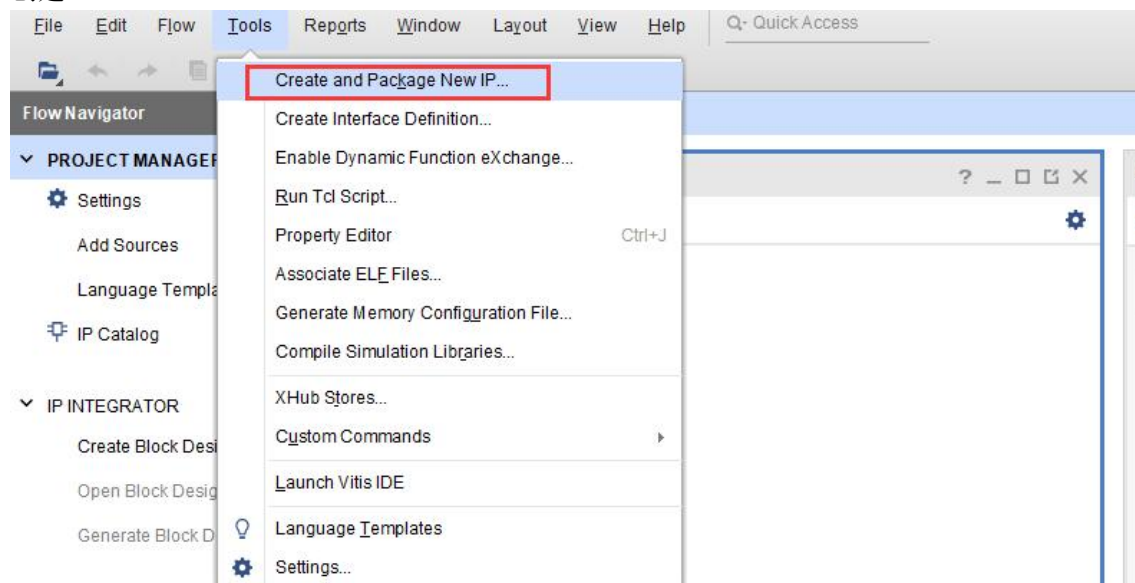


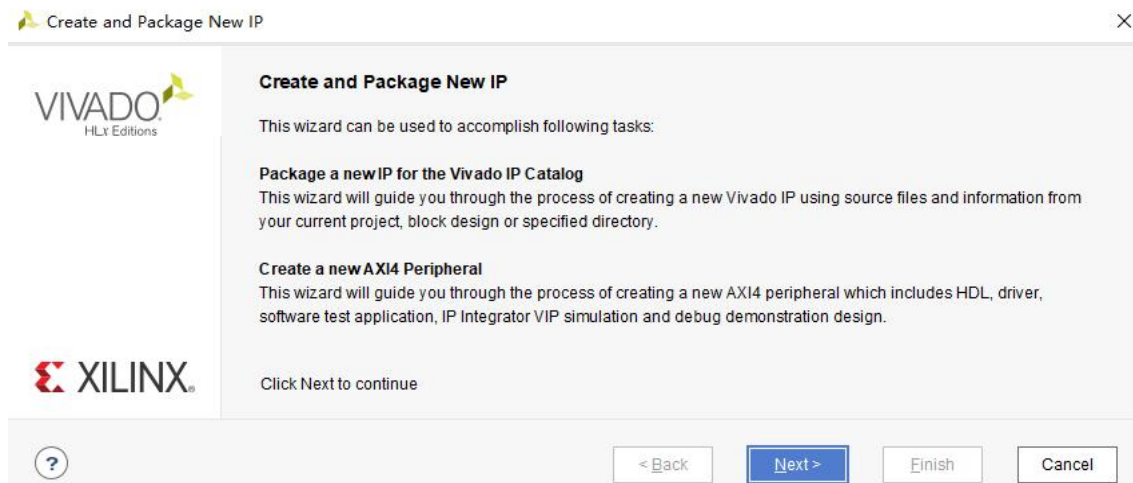
添加 uiFDMA.v 源码



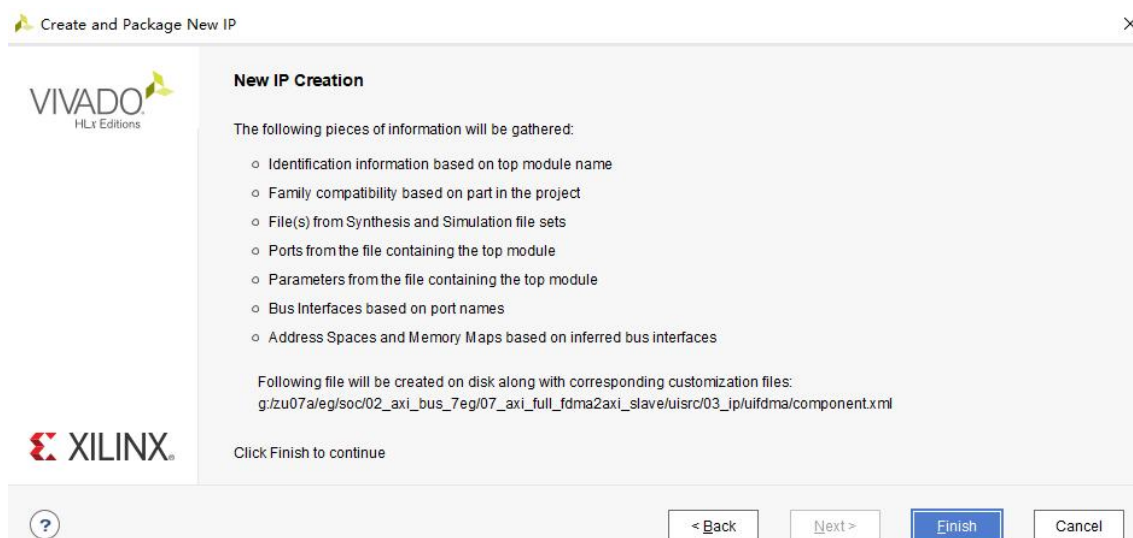
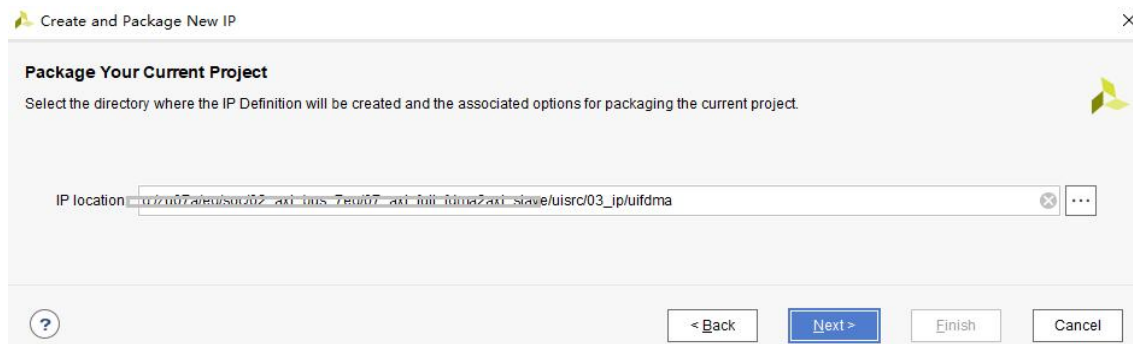
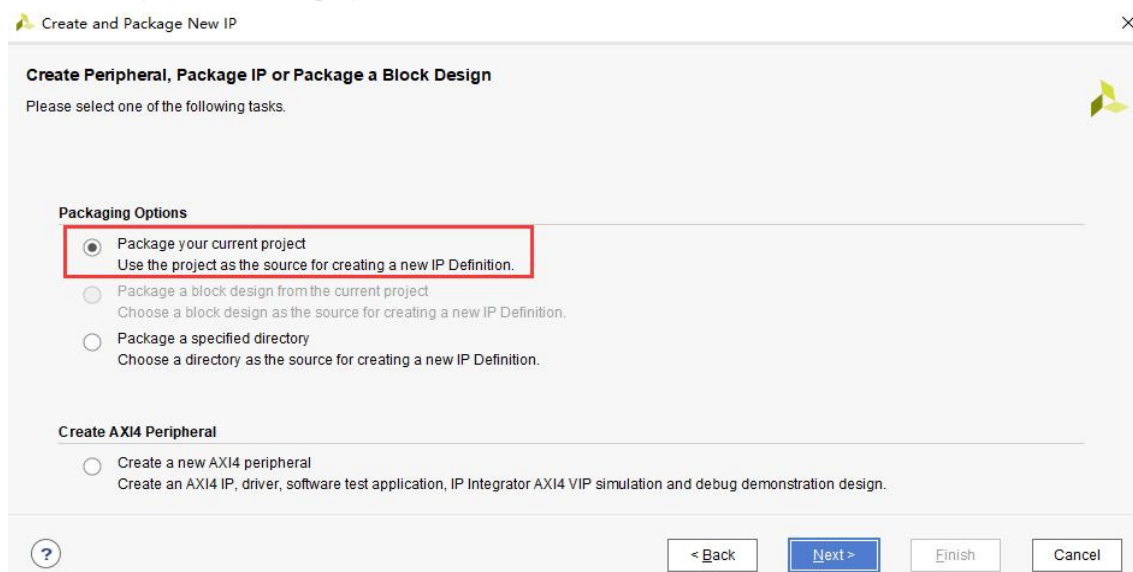


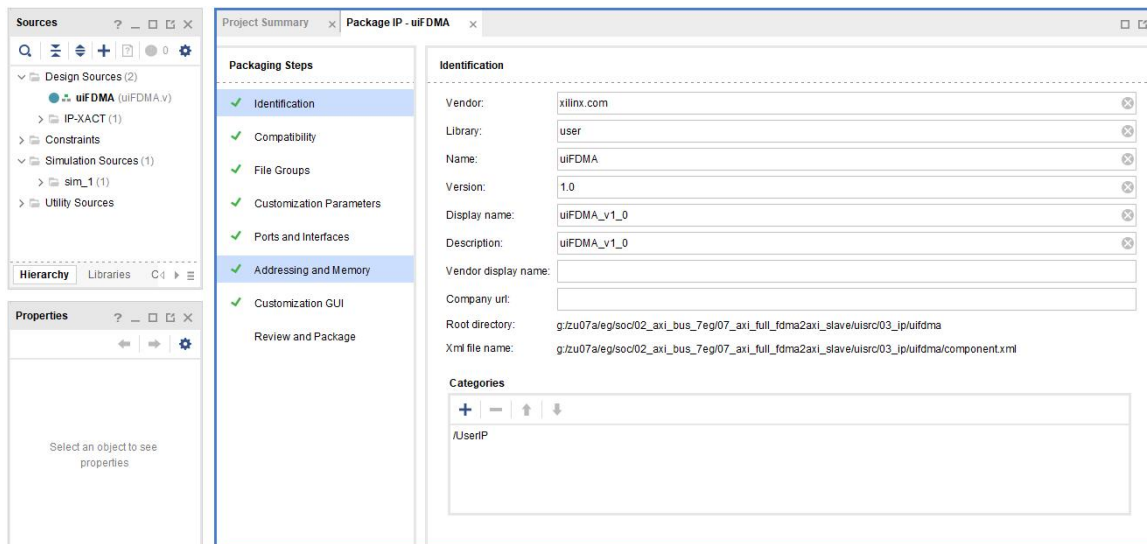
创建 IP



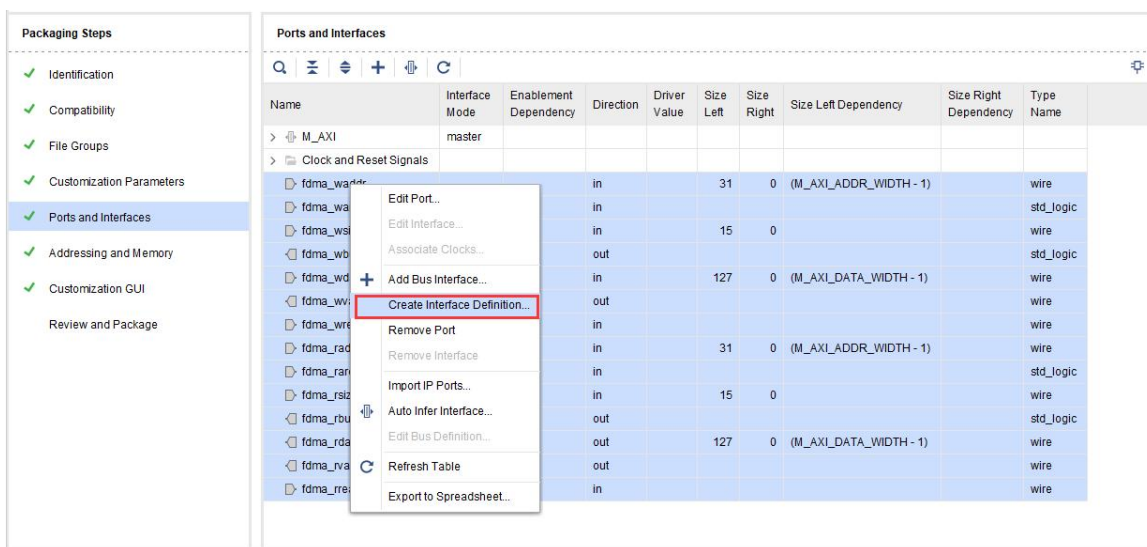


选择 Package your current project

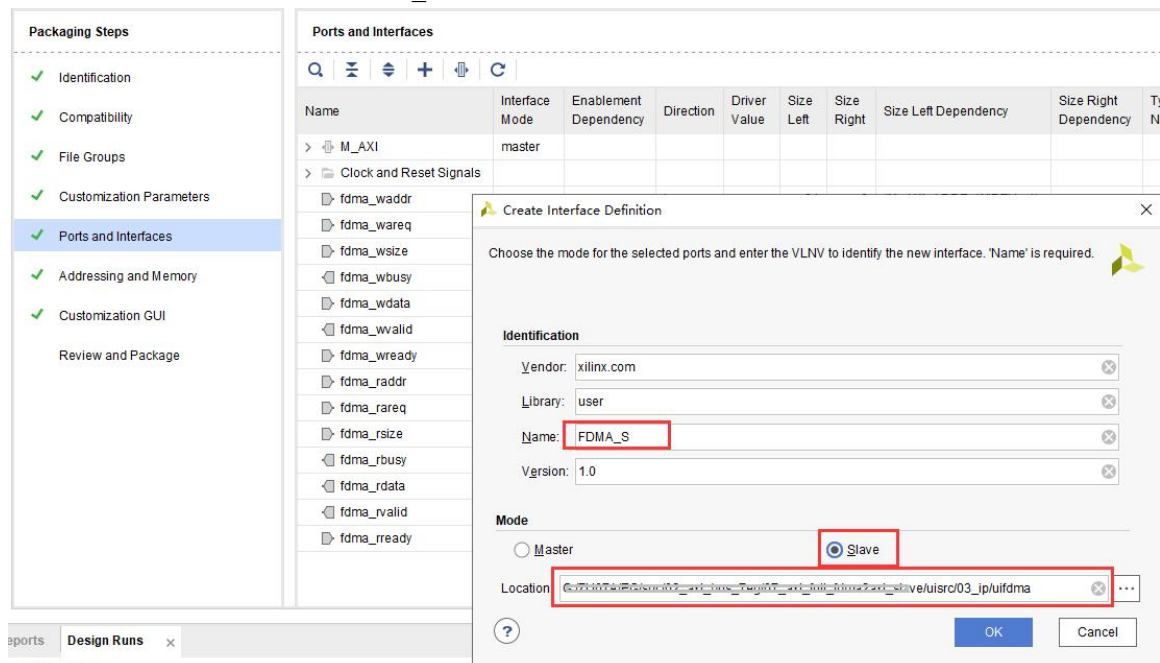




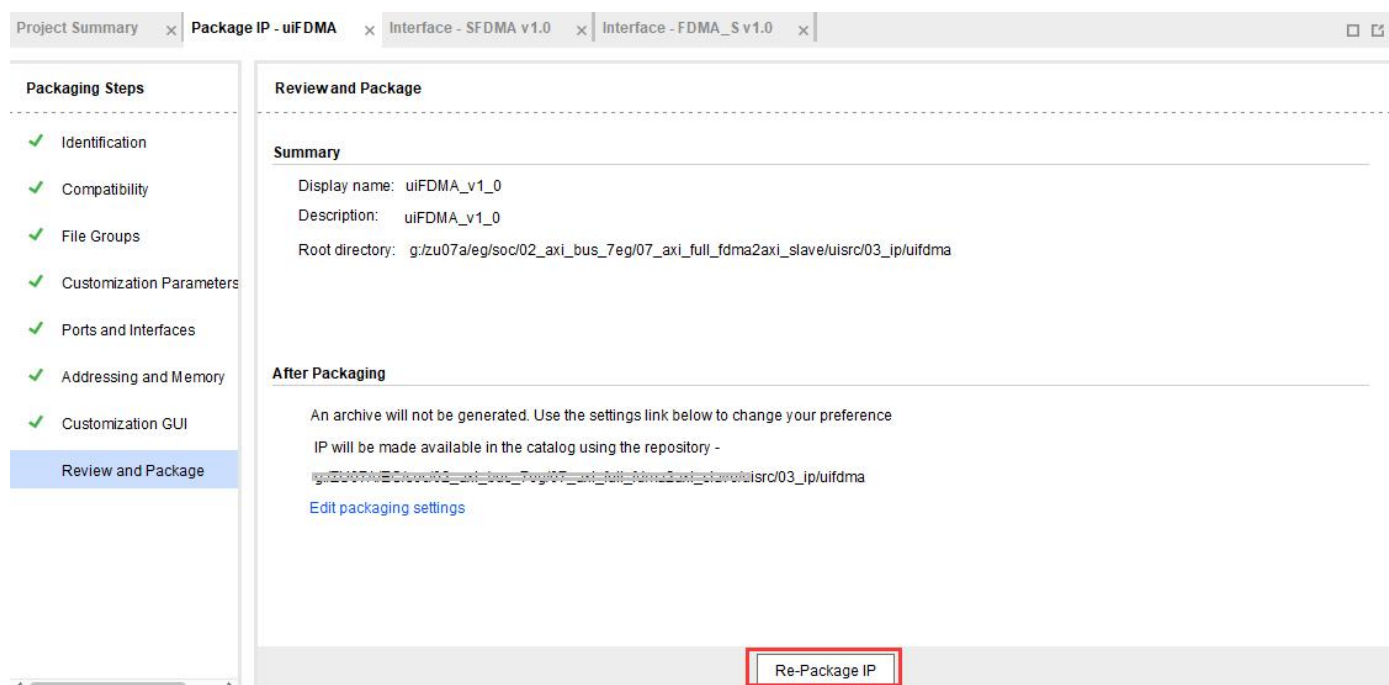
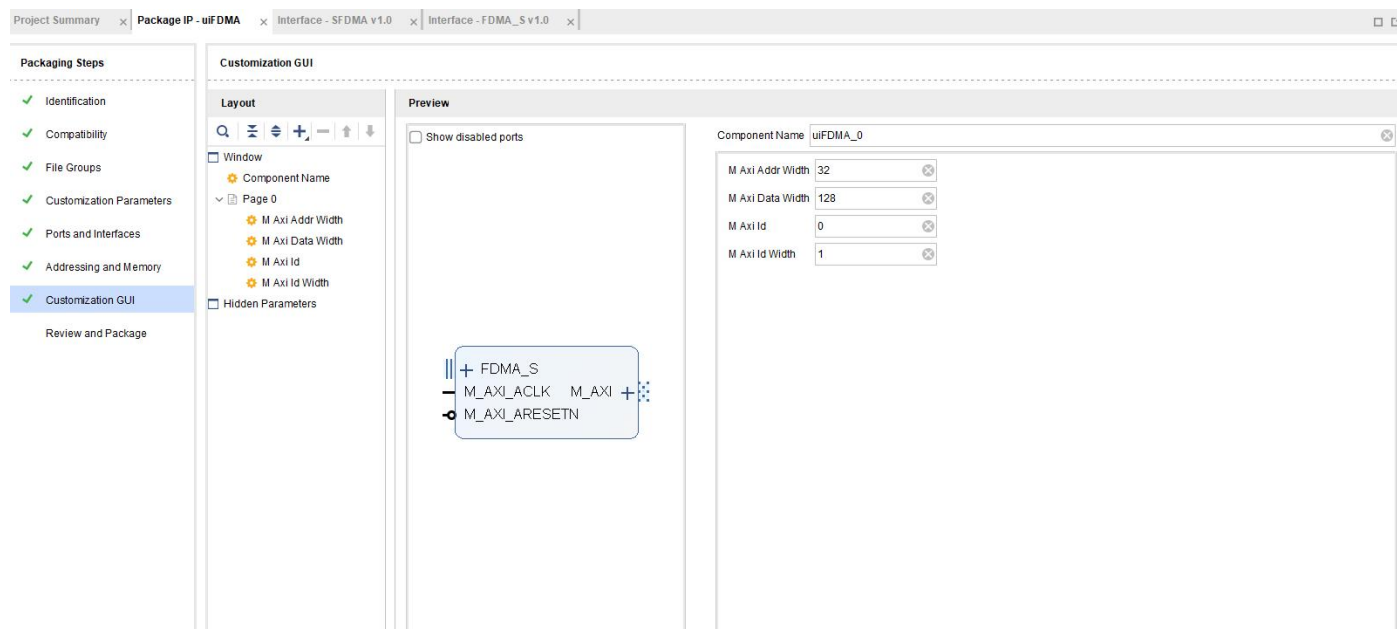
按住 shift 全选后，右击弹出菜单后选择 Create Interface Definition



接口定义为 slave，命名为 FDMA_S



设置完成，uisrc/03_ip/uifdma 路径下多出 2 个文件，这个两个文件就是定义了自定义的总线接口。



7.5 saxi_full_mem IP

这个 IP 的源码可以基于 axi-full-slave 的模板简单修改就可以实现。找到以下路径，中 saxi_full_v1_0_S00_AXI.v 文件，并且对齐修改。

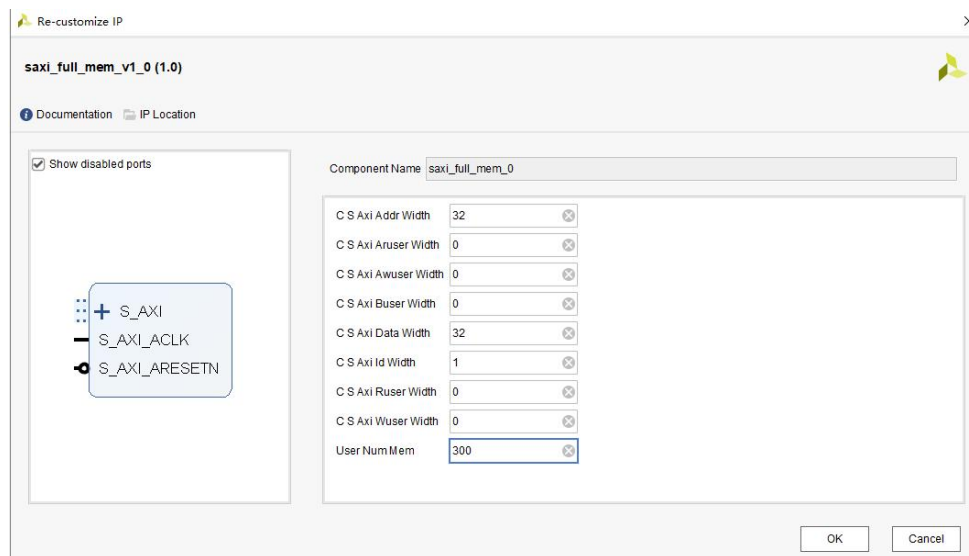


我们把修改后的代码命名为 saxi_full_mem.v 修改其中的部分代码，关键部分是 memory 部分定义。

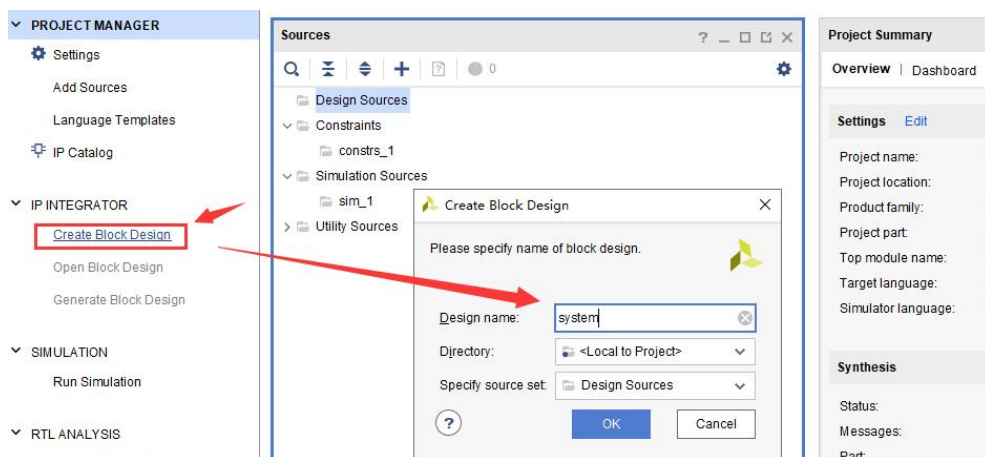
```
// -----  
// -- Example code to access user logic memory region  
// -----  
localparam MEM_LEN      = clogb2(USER_NUM_MEM)-1;  
  
reg [C_S_AXI_DATA_WIDTH-1:0] byte_ram [0 : USER_NUM_MEM -1];  
  
assign axi_rdata = axi_rvalid ? byte_ram[axi_araddr[ADDR_LSB+MEM_LEN:ADDR_LSB]] : 0;  
  
always @(posedge S_AXI_ACLK)begin  
    if (axi_wready && S_AXI_WVALID)begin  
        byte_ram[axi_awaddr[ADDR_LSB+MEM_LEN:ADDR_LSB]] <= S_AXI_WDATA;  
    end  
end  
end
```

修改的这部分代码支持 Memory 的任意长度设置(FPGA 内部 RAM 会消耗资源), 其中参数 USER_NUM_MEM 用于定义 RAM 的长度, 我们一次 FDMA 的 burst 长度应该小于等于 USER_NUM_MEM 这个参数。

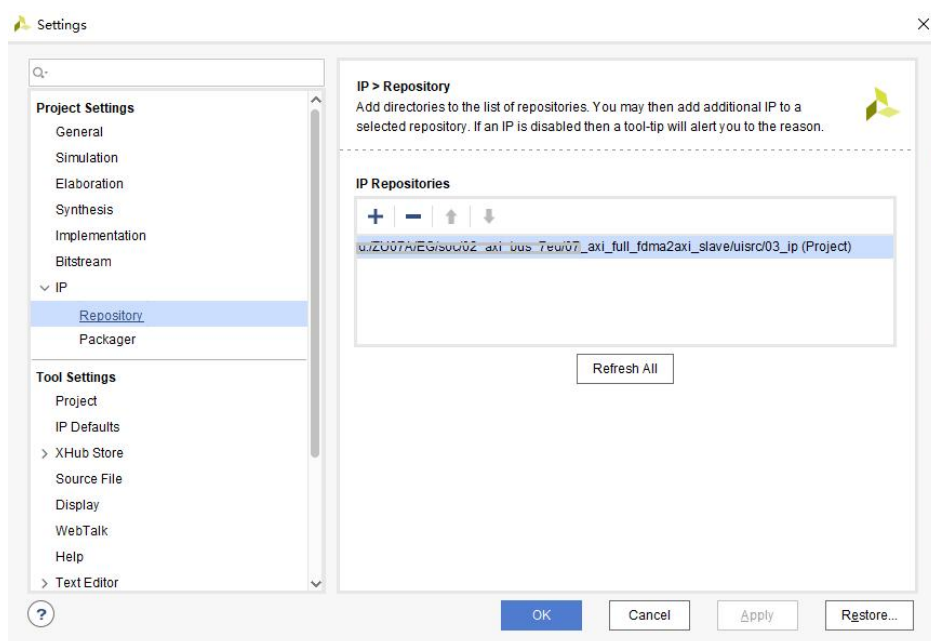
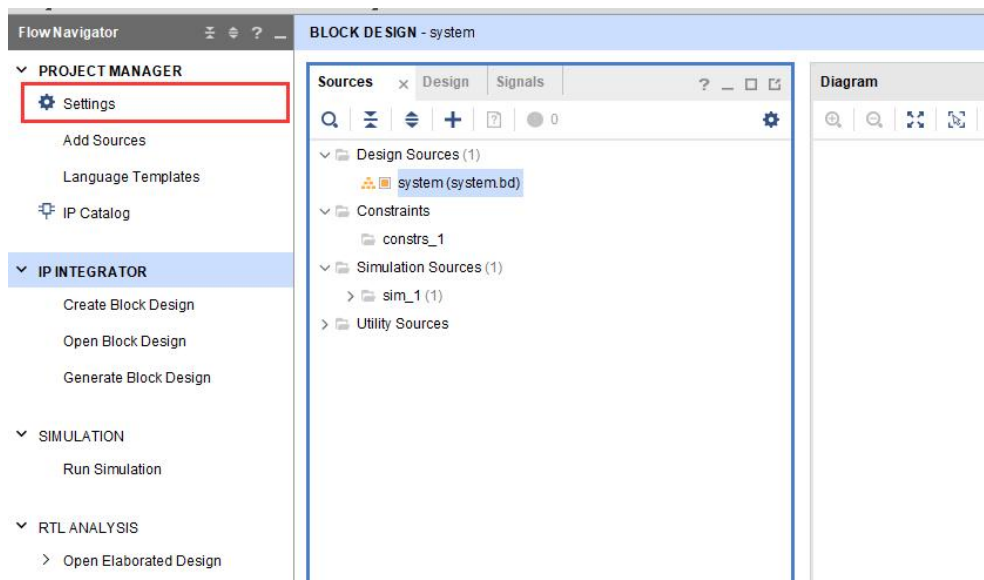
我们来看下 IP 的接口参数设置: 这里我们计划 FDMA 的读写长度是 262, 设置 USER_NUM_MEM=300 完全够用。



7.6 创建 FPGA 图像化设计



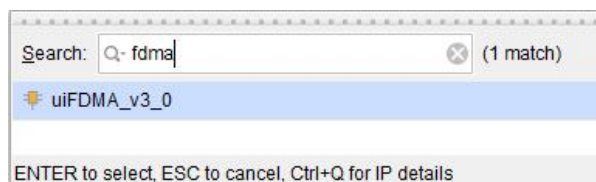
设置 IP 路径



添加已经创建好的 IP



输入关键词 fdma，在最后可以看到，双击添加 Ip

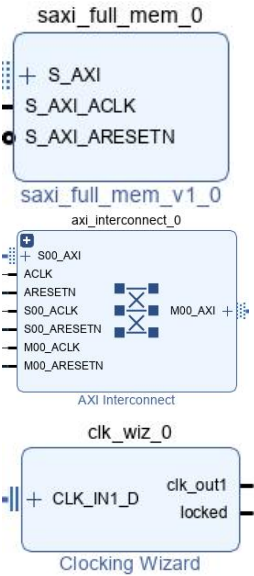


可以看到本文的 FDMA 版本升级到 3.0 版本，相比 2.0 而言，优化了 burst 传输效率



完成连线

继续添加剩余 IP



设置 IP 参数

Re-customize IP

uiFDMA_v3_0 (3.0)

Documentation IP Location

Show disabled ports

FDMA_S

M_AXI_ACLK

M_AXI_ARESETN

Component Name

uiFDMA_0

M_Axi Addr Width

32

M_Axi Data Width

32

M_Axi Id

0

M_Axi Id Width

1

OK

Cancel

Re-customize IP

Component Name

clk_wiz_0

Clocking Options

Output Clocks

MMCM Settings

Summary

Clock Monitor

Enable Clock Monitoring

Primitive

MMCM

PLL

Auto

Clocking Features

Frequency Synthesis

Minimize Power

Phase Alignment

Spread Spectrum

Dynamic Reconfig

Dynamic Phase Shift

Safe Clock Startup

Use CDDC

Jitter Optimization

Balanced

Minimize Output Jitter

Maximize Input Jitter filtering

Dynamic Reconfig Interface Options

AXI4Lite

DRP

Phase Duty Cycle Config

Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒	Primary	clk_in1	100.000	10.000 - 933.000	UI	0.010	Differential clock capable pin
☐	Secondary	clk_in2	100.000	66.667 - 133.333		0.010	Single ended clock capable pin

Re-customize IP

saxi_full_mem_v1_0 (1.0)

Documentation IP Location

Show disabled ports

S_AXI

S_AXI_ACLK

S_AXI_ARESETN

Component Name

saxi_full_mem_v1_0

C S Axi Addr Width

C S Axi Aruser W

C S Axi Awuser V

C S Axi Buser Wi

C S Axi Data Wid

C S Axi Id Width

C S Axi Ruser Wi

C S Axi Wuser W

User Num Mem

Component Name clk_wiz_0

Clocking OptionsOutput ClocksMMCM SettingsSummary

The phase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives	Matched Routing	Max Freq. of buffer
		Requested	Actual	Requested	Actual	Requested	Actual			
☒ clk_out1	clk_out1	100.000	100.00000	0.000	0.000	50.000	50.0	Buffer	☐	775.194
☐ clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	Block	☐	775.194
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	Block	☐	775.194

☐ USE CLOCK SEQUENCING

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Enable Optional Inputs / Outputs for MMCM/PLLReset TypePhase Shift Mode

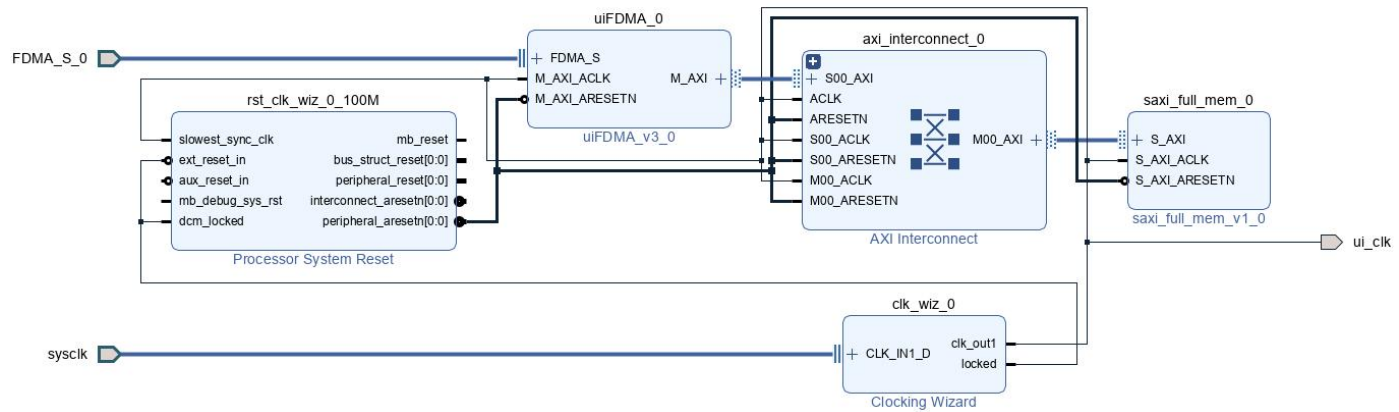
☐ reset☐ power_down☐ input_clk_stopped

☒ Active High☐ Active Low

☐ WAVEFORM☒ LATENCY

☒ locked☐ clkfbstopped

完成连线



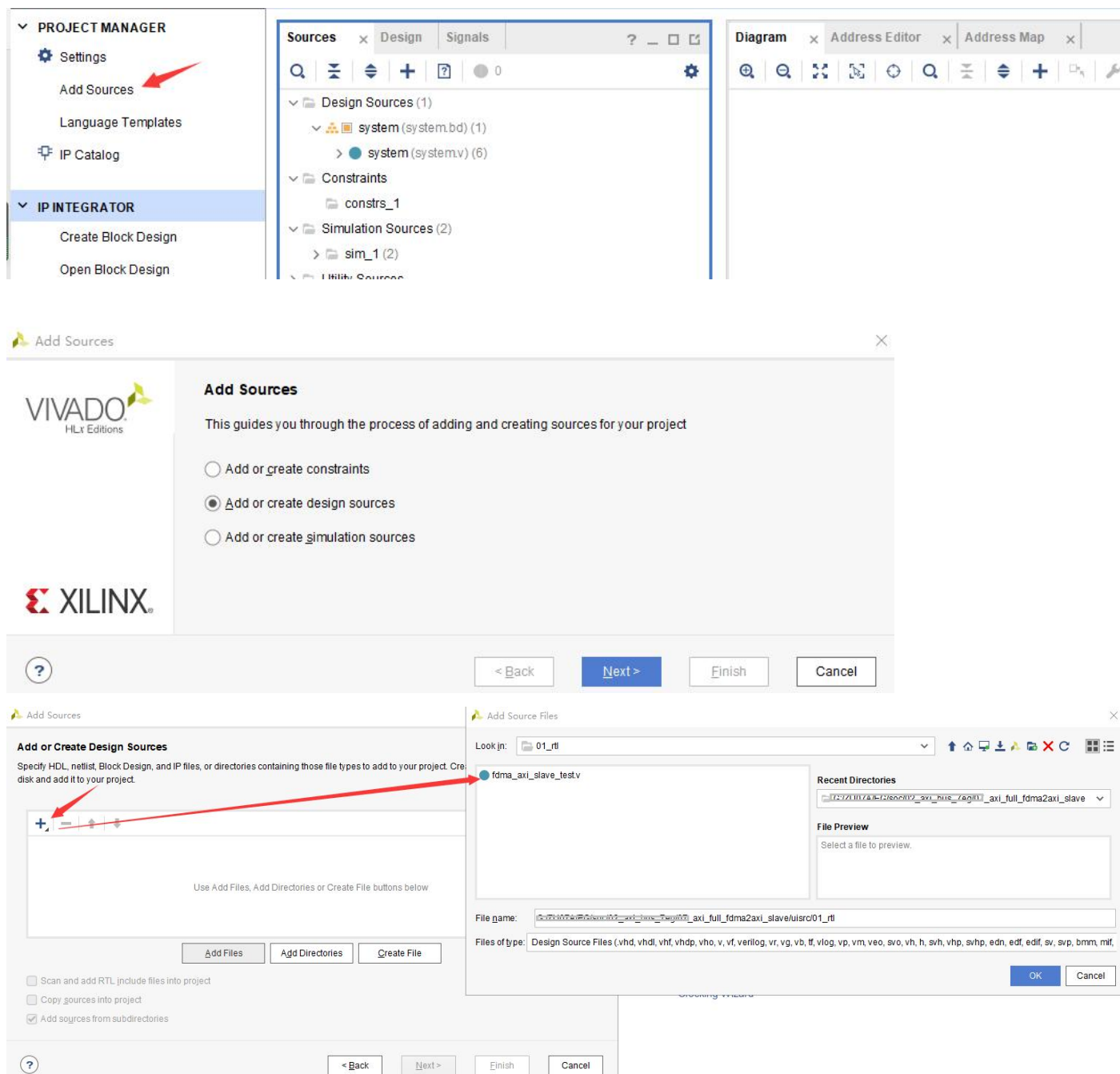
设置地址分配：

Diagram x Address Editor x Address Map x

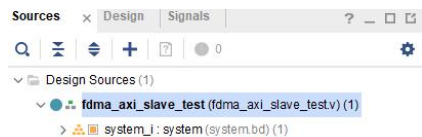
☒ Assigned (1) ☒ Unassigned (0) ☒ Excluded (0)

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/ uiFDMA_0					
/ uiFDMA_0/M_AXI (32 address bits : 4G)					
/saxi_full_mem_0/S_AXI	S_AXI	reg0	0x0000_0000	4G	0xFFFF_FFFF

7.7 添加 FDMA 接口控制代码



添加完成后如下图：



fdma_axi_slave_test.v 源码如下

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: (milianke)
Technical forum: uisrc.com
taobao: https://milianke.taobao.com   https://osrc.taobao.com
jd: https://milianke.jd.com
```

Create Date: 2021/04/25

Module Name: fdma_axi_slave_test

Description:

Copyright: Copyright (c) milianke

Revision: 1.0

Signal description:

1) _i input

2) _o output

3) _n activ lowpai

4) _dg debug signal

5) _r delay or register

6) _s state mechine

*/

////////////////////////////////////

```
module fdma_axi_slave_test(
```

```
    input sysclk_p,
```

```
    input sysclk_n
```

```
);
```

```
wire [31:0]    fdma_raddr;
```

```
reg           fdma_rareq;
```

```
wire          fdma_rbusy;
```

```
wire [31:0]    fdma_rdata;
```

```
wire [15:0]    fdma_rsize;
```

```
wire          fdma_rvalid;
```

```
wire [31:0]    fdma_waddr;
```

```
reg           fdma_wareq;
```

```
wire          fdma_wbusy;
```

```
wire [31:0]    fdma_wdata;
```

```
wire [15:0]    fdma_wsize;
```

```
wire          fdma_wvalid;
```

```
wire          ui_clk;
```

```
parameter TEST_MEM_SIZE    = 32'd4*20;
```

```
parameter FDMA_BURST_LEN   = 16'd262;
```

```
parameter ADDR_MEM_OFFSET = 0;
```

```
parameter ADDR_INC = 0;
```

```
parameter WRITE1 = 0;
```

```
parameter WRITE2 = 1;
```

```
parameter WAIT    = 2;
```

```
parameter READ1   = 3;
```

```
parameter READ2   = 4;
```

```
reg [31:0] t_data;
```

```
reg [31:0] fdma_waddr_r;
```

```
reg [2 :0] T_S = 0;
```

```
assign fdma_waddr = fdma_waddr_r + ADDR_MEM_OFFSET;
```

```
assign fdma_raddr = fdma_waddr;
```

```
assign fdma_wsize = FDMA_BURST_LEN;
```

```
assign fdma_rsize = FDMA_BURST_LEN;
```

```
assign fdma_wdata = {t_data,t_data,t_data,t_data};
```

```
//delay reset
```

```
reg [8:0] rst_cnt = 0;
```

```
always @(posedge ui_clk)
```

```
    if(rst_cnt[8] == 1'b0)
```

```
        rst_cnt <= rst_cnt + 1'b1;
```

```
    else
```

```

rst_cnt <= rst_cnt;

always @(posedge ui_clk)begin
    if(rst_cnt[8] == 1'b0)begin
        T_S <=0;
        fdma_wareq <= 1'b0;
        fdma_rareq <= 1'b0;
        t_data<=0;
        fdma_waddr_r <=0;
    end
    else begin
        case(T_S)
            WRITE1:begin
                if(fdma_waddr_r==TEST_MEM_SIZE) fdma_waddr_r<=0;
                if(!fdma_wbusy)begin
                    fdma_wareq <= 1'b1;
                    t_data <= 0;
                end
                if(fdma_wareq&&fdma_wbusy)begin
                    fdma_wareq <= 1'b0;
                    T_S <= WRITE2;
                end
            end
            end
            WRITE2:begin
                if(!fdma_wbusy) begin
                    T_S <= WAIT;
                    t_data <= 32'd0;
                end
                else if(fdma_wvalid) begin
                    t_data <= t_data + 1'b1;
                end
            end
            WAIT:begin//not needed
                T_S <= READ1;
            end
            READ1:begin
                if(!fdma_rbusy)begin
                    fdma_rareq <= 1'b1;
                    t_data <= 0;
                end
                if(fdma_rareq&&fdma_rbusy)begin
                    fdma_rareq <= 1'b0;
                    T_S <= READ2;
                end
            end
            READ2:begin
                if(!fdma_rbusy) begin
                    T_S <= WRITE1;
                    t_data <= 32'd0;
                    fdma_waddr_r <= fdma_waddr_r + ADDR_INC;//128/8=16
                end
                else if(fdma_rvalid) begin
                    t_data <= t_data + 1'b1;
                end
            end
            default:
                T_S <= WRITE1;
            endcase
        end
    end
end

wire test_error = (fdma_rvalid && (t_data[15:0] != fdma_rdata[15:0]));

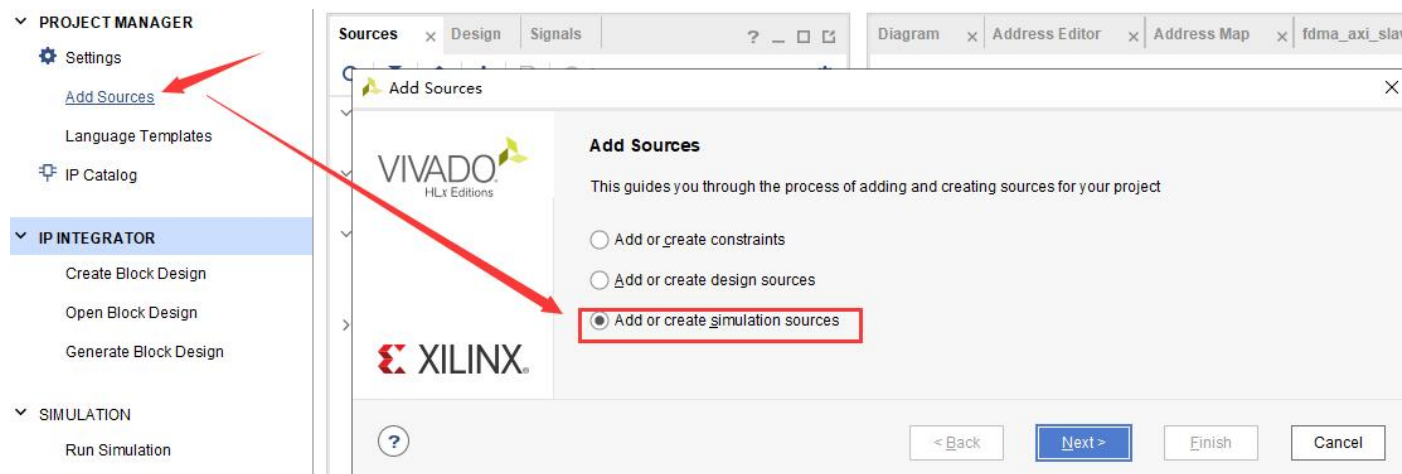
```

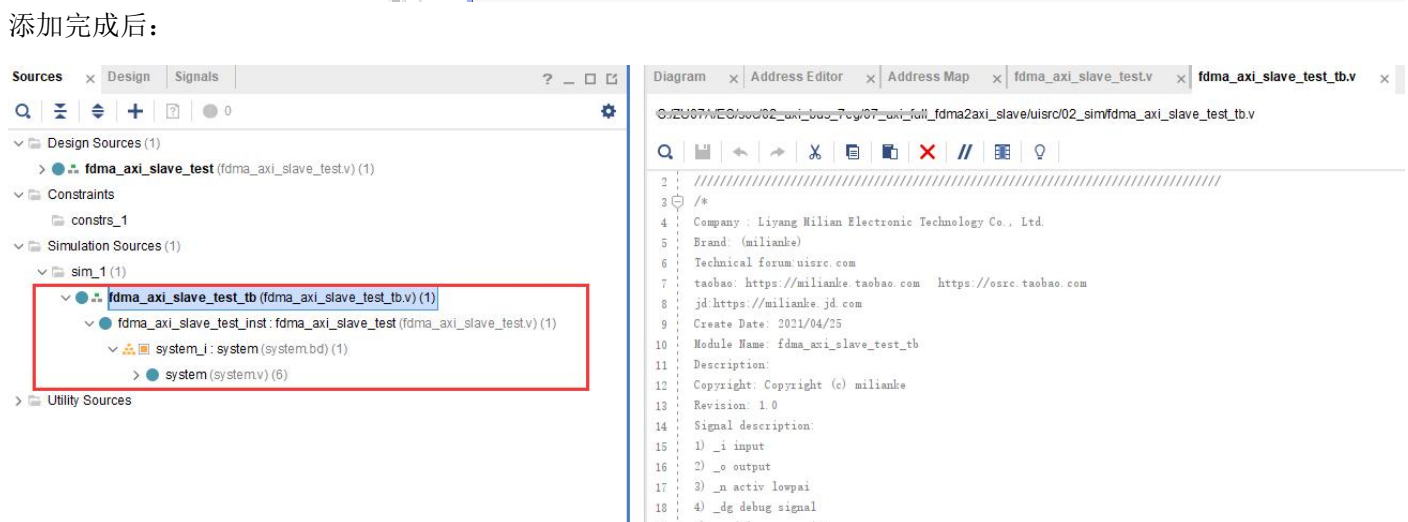
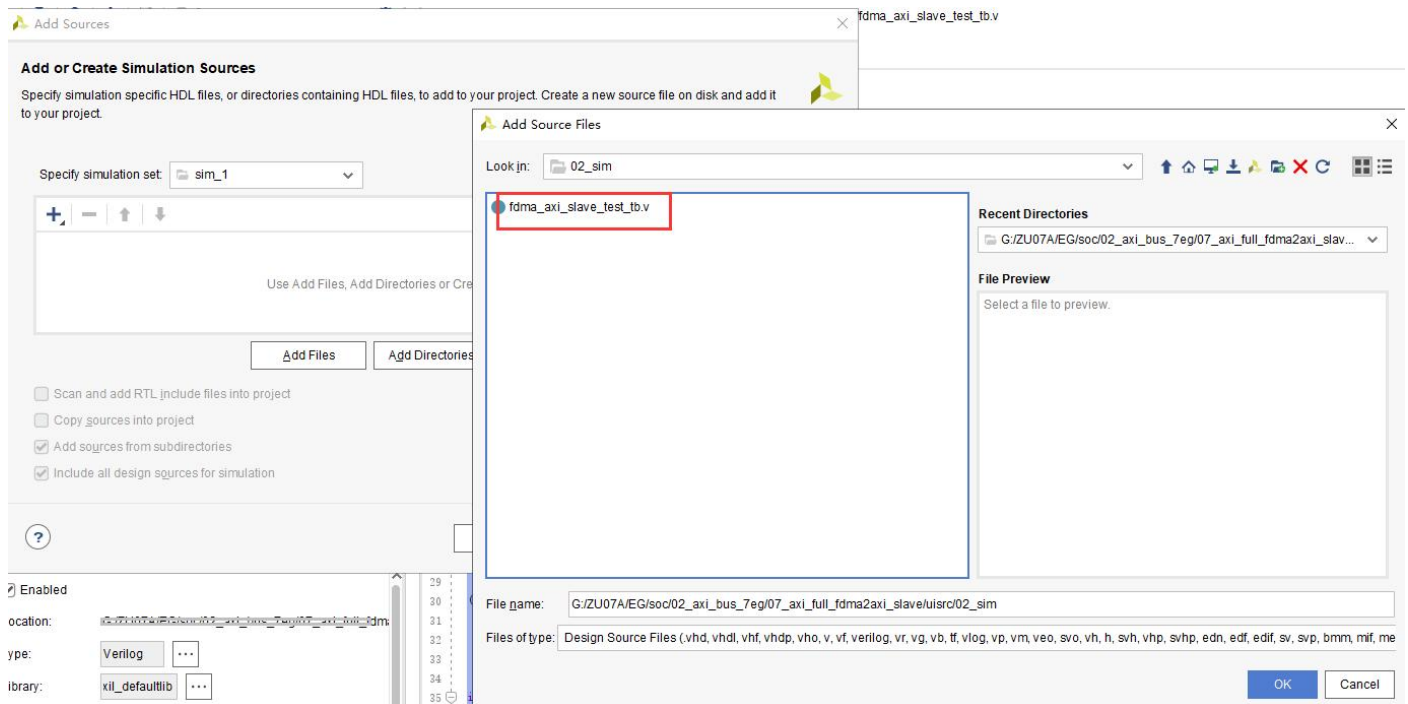
```
//ila_0_ila_dbg (  
//    .clk(ui_clk),  
//    .probe0({fdma_wdata[15:0],fdma_wareq,fdma_wvalid,fdma_wbusy}),  
//    .probe1({fdma_rdata[15:0],t_data[15:0],fdma_rvalid,fdma_rbusy,T_S,test_error})  
//);  
  
system system_i  
    (FDMA_S_0_fdma_raddr(fdma_raddr),  
    .FDMA_S_0_fdma_rareq(fdma_rareq),  
    .FDMA_S_0_fdma_rbusy(fdma_rbusy),  
    .FDMA_S_0_fdma_rdata(fdma_rdata),  
    .FDMA_S_0_fdma_rready(1'b1),  
    .FDMA_S_0_fdma_rsize(fdma_rsize),  
    .FDMA_S_0_fdma_rvalid(fdma_rvalid),  
    .FDMA_S_0_fdma_waddr(fdma_waddr),  
    .FDMA_S_0_fdma_wareq(fdma_wareq),  
    .FDMA_S_0_fdma_wbusy(fdma_wbusy),  
    .FDMA_S_0_fdma_wdata(fdma_wdata),  
    .FDMA_S_0_fdma_wready(1'b1),  
    .FDMA_S_0_fdma_wsize(fdma_wsize),  
    .FDMA_S_0_fdma_wvalid(fdma_wvalid),  
    .sysclk_clk_n(sysclk_n),  
    .sysclk_clk_p(sysclk_p),  
    .ui_clk(ui_clk)  
    );  
  
endmodule
```

以上代码中调用的 system.bd 的图形代码接口。在状态机中，每次写 262 个长度 32bit 的数据，再读出来判断数据是否正确。

7.8 仿真文件

添加仿真文件





仿真文件非常简单，只要提供时钟激励就可以。

```
module fdma_axi_slave_test_tb();
reg sysclk_p;

  fdma_axi_slave_test fdma_axi_slave_test_inst
  (
    .sysclk_p(sysclk_p),
    .sysclk_n(~sysclk_p)
  );

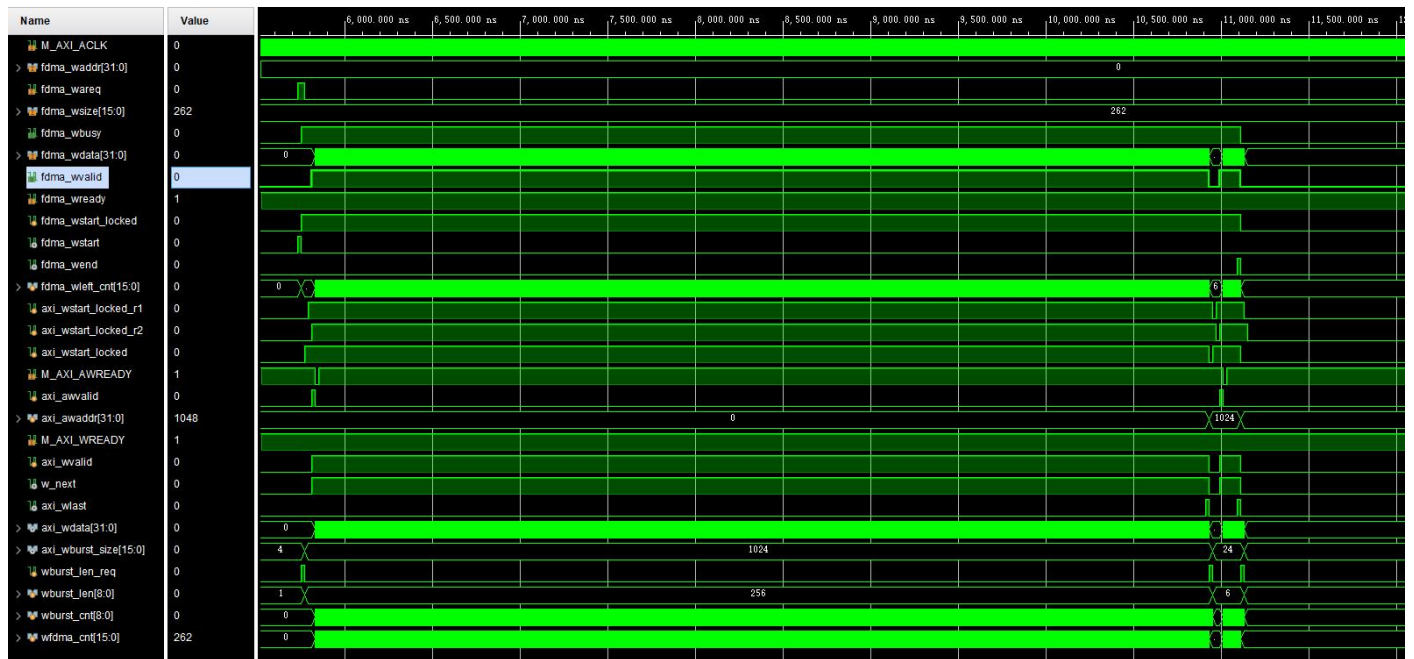
initial begin
  sysclk_p = 0;
  #100;
end

always #10 sysclk_p = ~sysclk_p;
endmodule
```

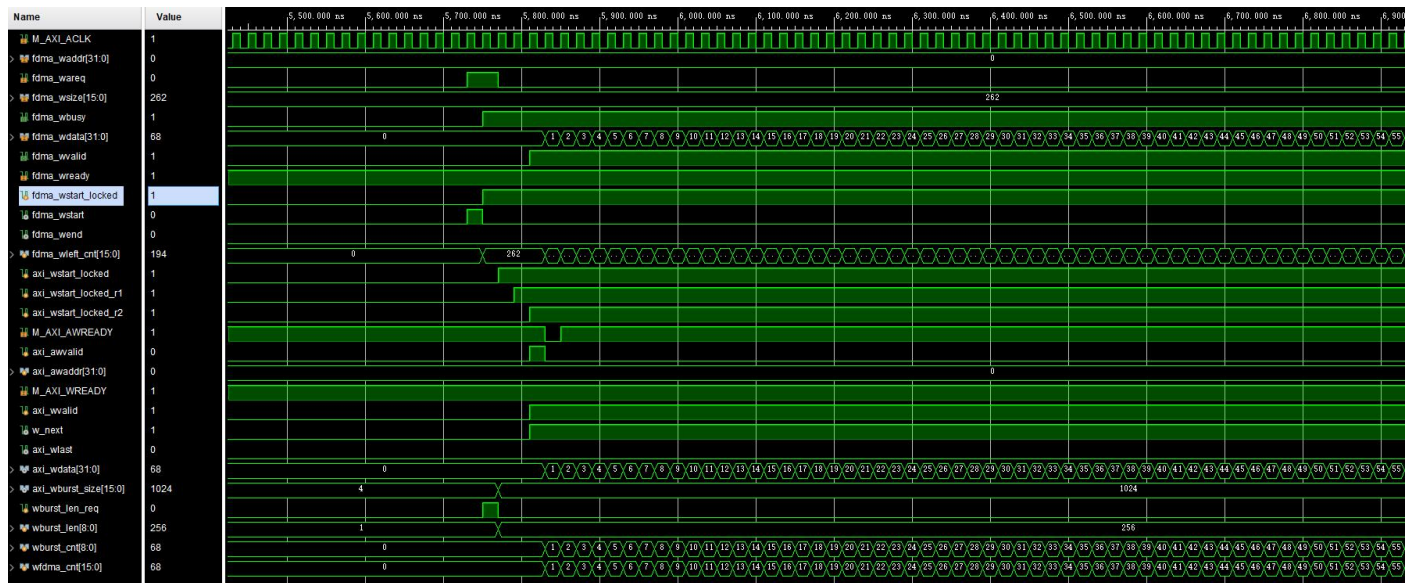
7.9 实验结果

FDMA 写操作仿真波形图，一次完成的 FDMA 写操作时序图如下：

这里一次 wburst_len_req 多产生一次，但是结果却不影响，大家可以思考下。如何设计出来和我们之前绘制的波形图一样。



一次 FDMA 写传输的起始时序

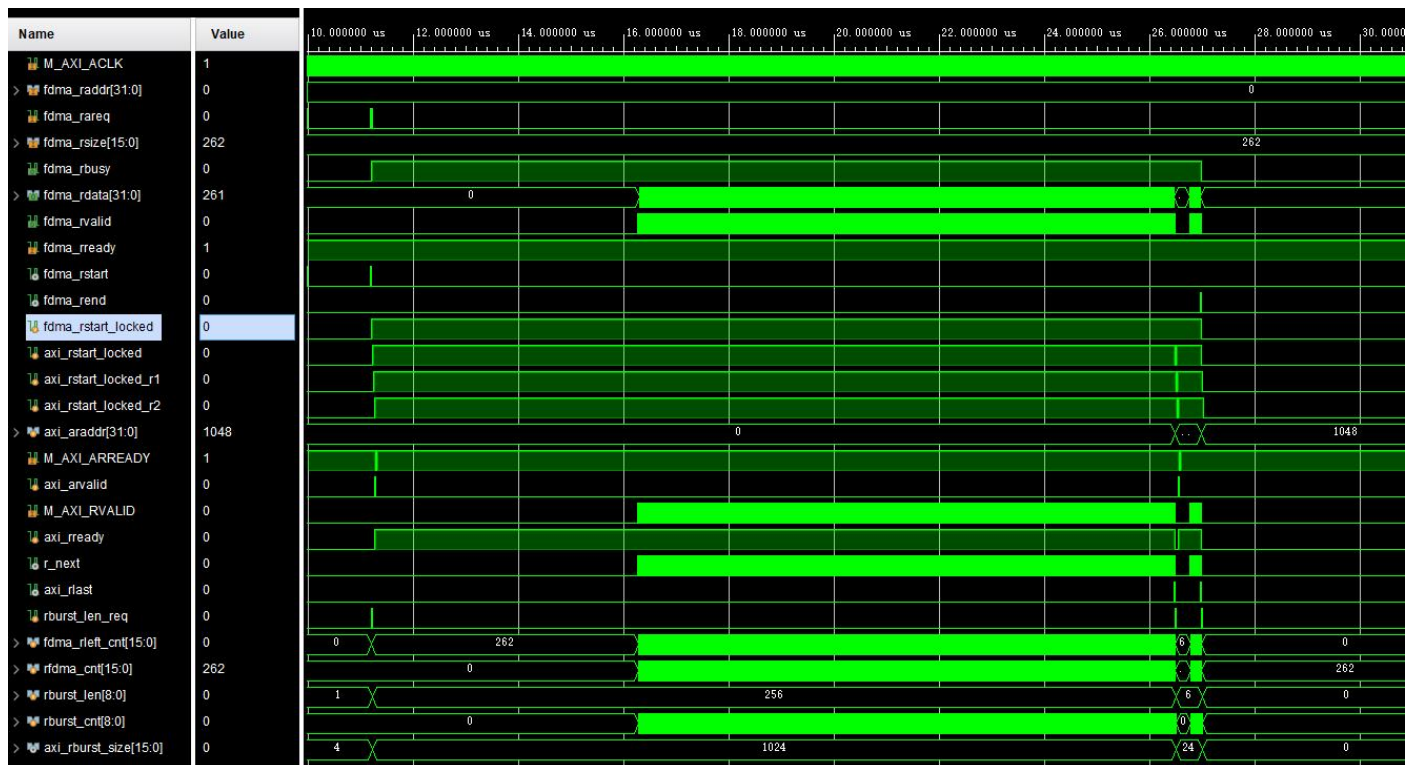


连续 burst,自动管理 burst 长度，以及一次 FDMA 写传输结束时序

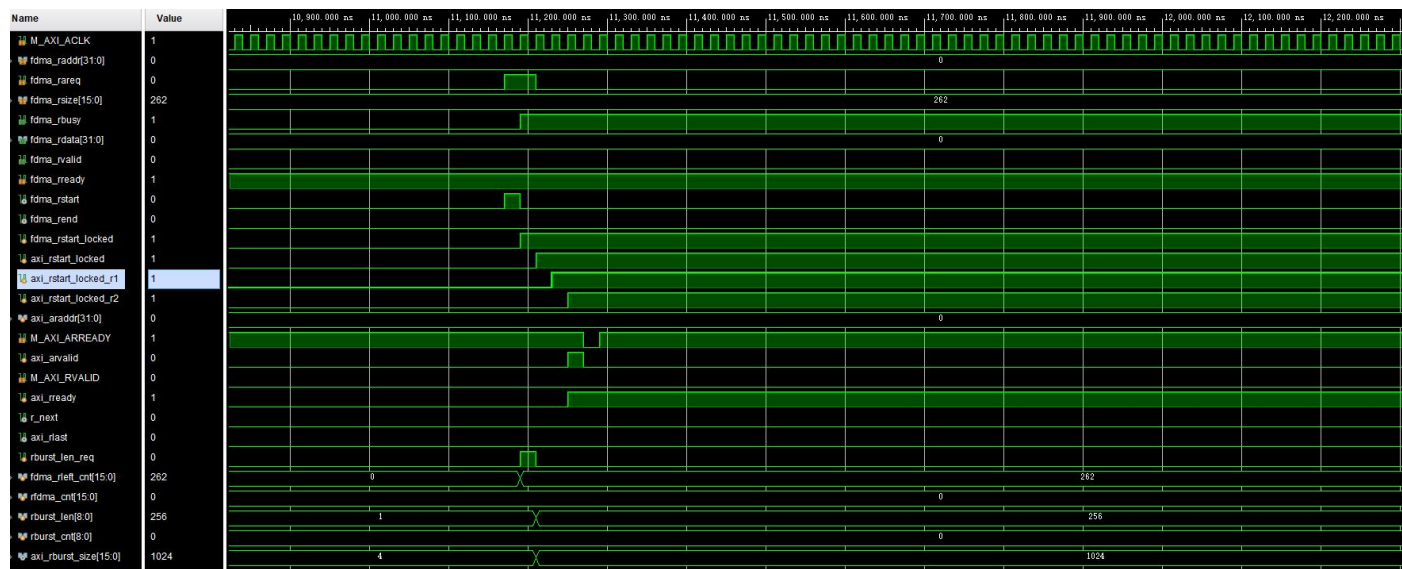


FDMA 读操作仿真波形图，一次完成的 FDMA 读操作时序图如下：

这里一次 `rburst_len_req` 多产生一次，但是结果却不影响，大家可以思考下。如何设计出来和我们之前绘制的波形图一样。和写操作不同，可以看到读操作的等待较长时间后才获取到数据。

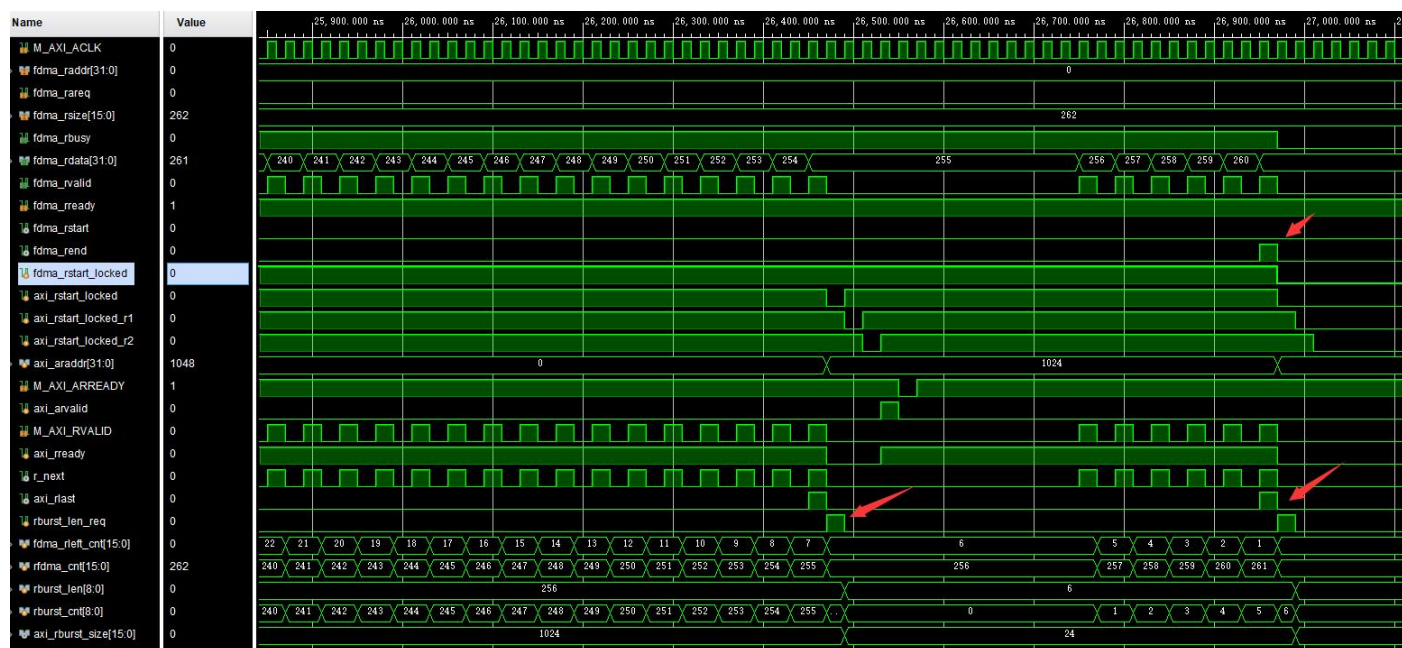


一次 FDMA 读传输的起始时序



连续 burst,自动管理 burst 长度, 以及一次 FDMA 读传输结束时序

另外放到后可以看到 rvalid 不是连续的, 这个读者也可以自己去优化 saxi_ful_mem ip 让这 IP 支持连续的 rvalid



08 使用 fdma 读写 axi-bram

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

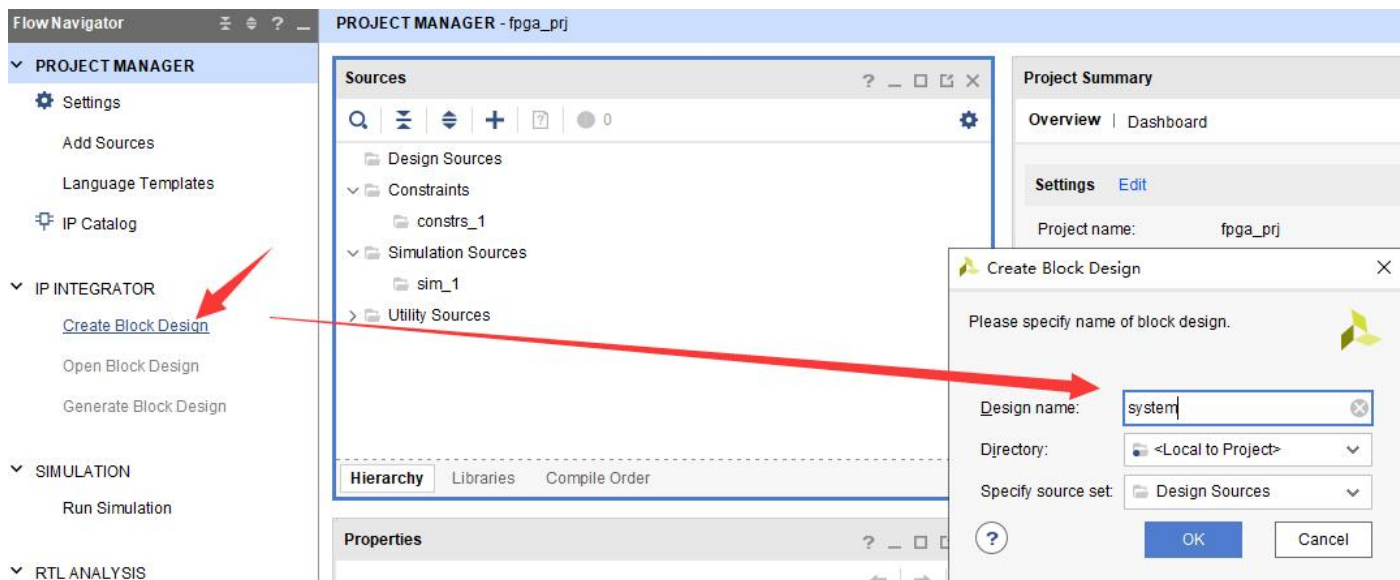
8.1 概述

基于 FDMA 可以完成很多数据读写存储类的应用, 本文将展示通过 FDMA 读写 AXI-BRAM
本文实验目的:

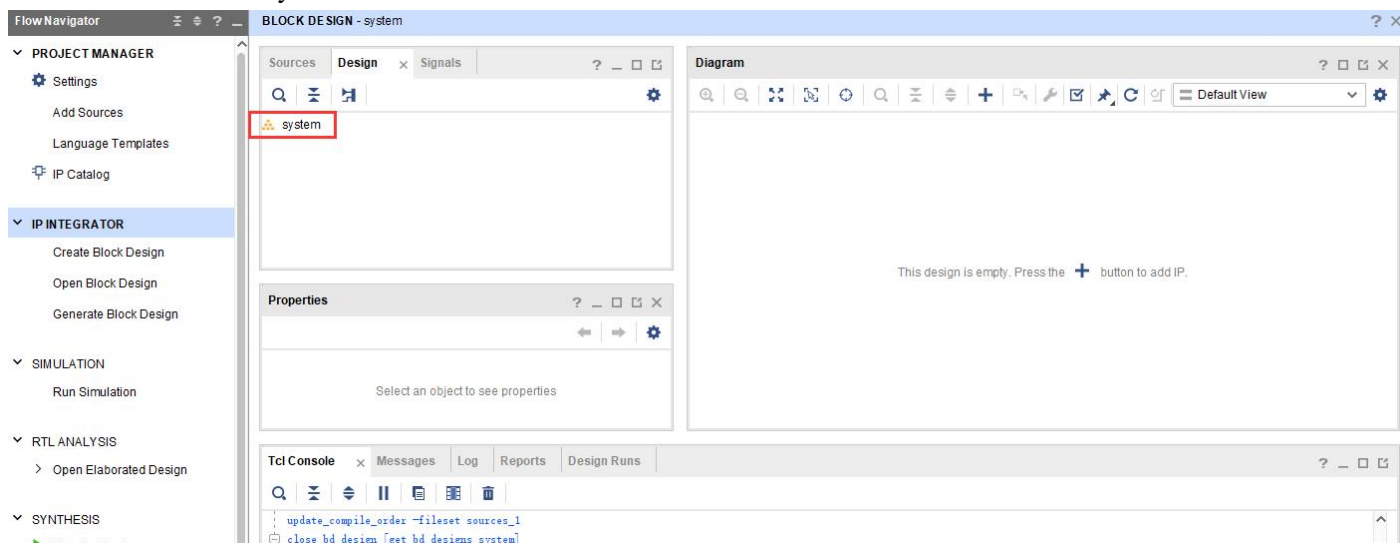
- 1:掌握基于 uiFDMA3.0 的 FPGA 工程设计
- 2:利用 uiFDMA3.0 提供的接口, 编写 BRAM 测试程序
- 3:对 AXI-BRAM 读写仿真和测试

8.2 搭建 FPGA 图形化工程

8.2.1 创建 Block Design 并且命名为 system

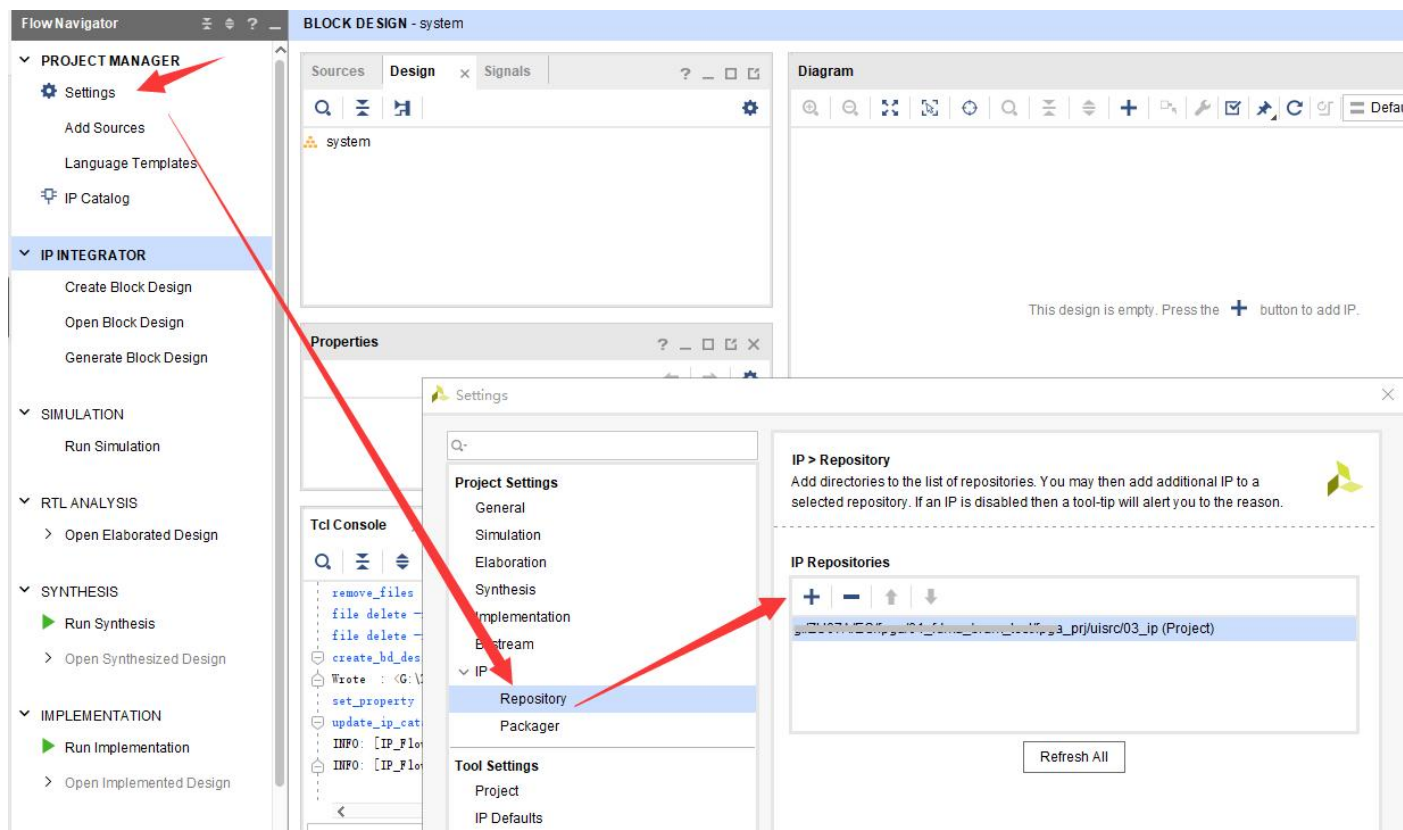


如下图所示, 图形化 system 就是一个代码容器, 接着我们要添加一些图像化的 IP

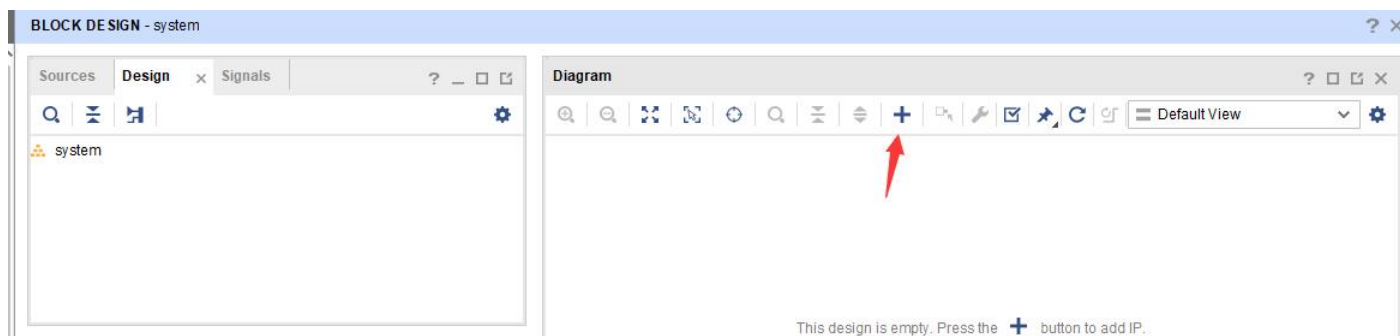


8.2.2 添加图形化 FPGA IP

首先设置自定义 IP 的路径，这里读者可以把我们配套工程根路径下的 uisrc 文件夹复制到目前的工程根路径,单击 Settings 在弹出的 Settings 窗口选择 IP->Repository 设置如下路径

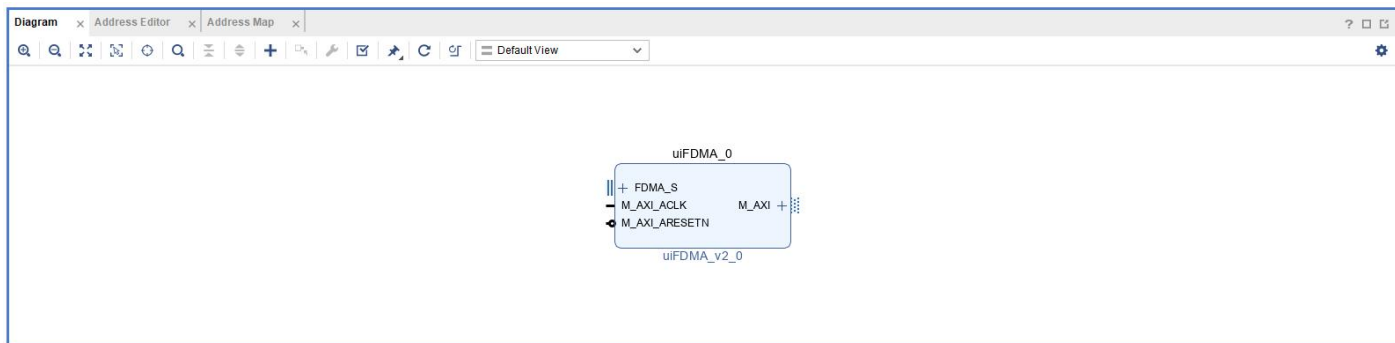


添加+号添加 IP

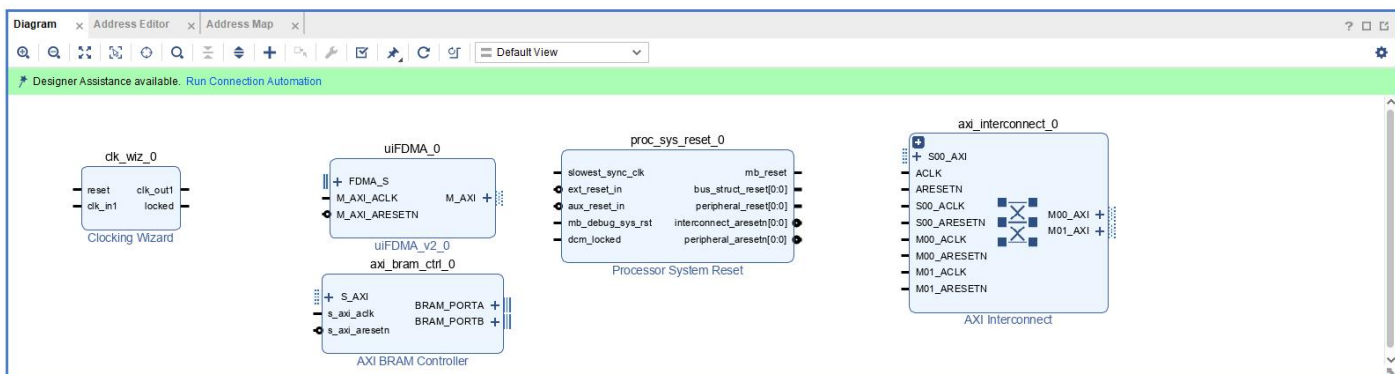


比如输入关键词 FDMA 就可以搜索到我们米联客 uiFDMA IP(注意最新版本是 3.0 版本)



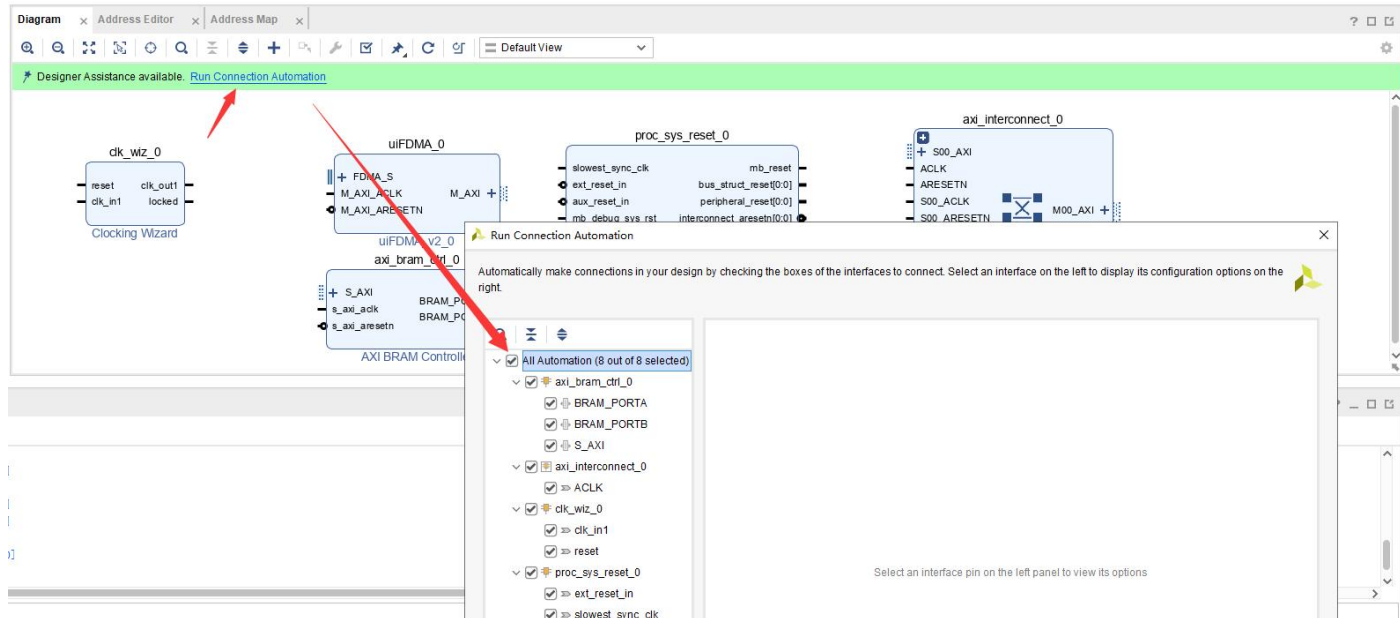


用类似的方法添加必要的 IP 如下图所示:

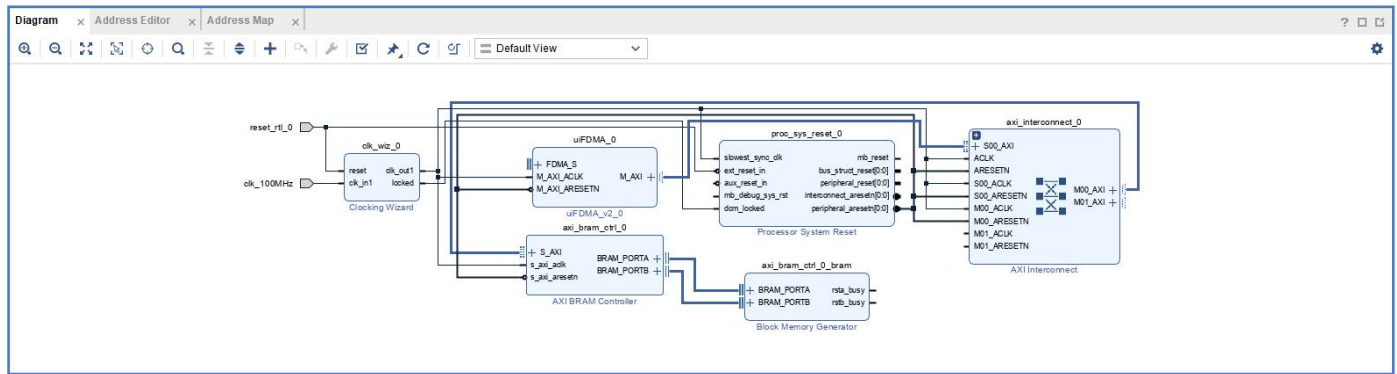


8.2.3 完成 IP 之间的信号自动

这种简单的工程可以先让软件先自动化线, 然后根据结果手动一些调整



可以看到连线结果



8.2.4 调整 IP 参数

1:BRAM 参数设置

首先把 IP 的配置参数修改下，双击需要设置的 IP 可以进行参数设置
FDMA 设置数据位宽 128bit 可以访问内存地址位宽 32bit 其他默认

☐ Show disabled ports

Component Name uFDMA_0

M Axi Addr Width32

M Axi Data Width128

M Axi Id0

M Axi Id Width1

BRAM 设置，使用 BRAM Controller 为真双口 RAM

IP SymbolPower Estimation

☐ Show disabled ports

Component Name axi_bram_ctrl_0_bram

BasicPort A OptionsPort B OptionsOther OptionsSummary

ModeBRAM Controller

Memory TypeTrue Dual Port RAM

Select the Primitive Type to be Used

ECC Options

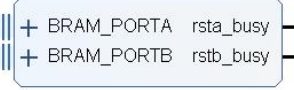
Write Enable

Algorithm Options

121

IP Symbol Power Estimation

☐ Show disabled ports



Component Name

Basic Port A Options Port B Options Other Options Summary

Memory Size (in words)

Write Width Range: 32 to 1024 (bits)

Read Width

Write Depth Range: 2 to 1048576

Read Depth

Operating Mode Enable Port Type

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☒ RSTA Pin (set/reset pin) Output Reset Value (Hex)

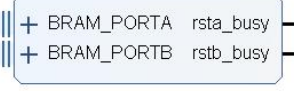
☐ Reset Memory Latch Reset Priority

READ Address Change A

☐ Read Address Change A

IP Symbol Power Estimation

☐ Show disabled ports



Component Name

Basic Port A Options Port B Options Other Options Summary

Memory Size (in words)

Write Width

Read Width

Write Depth : 8192

Read Depth : 8192

Operating Mode Enable Port Type

Port B Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Output Register ☐ REGCEB Pin ☐ Enable ECC PIPE

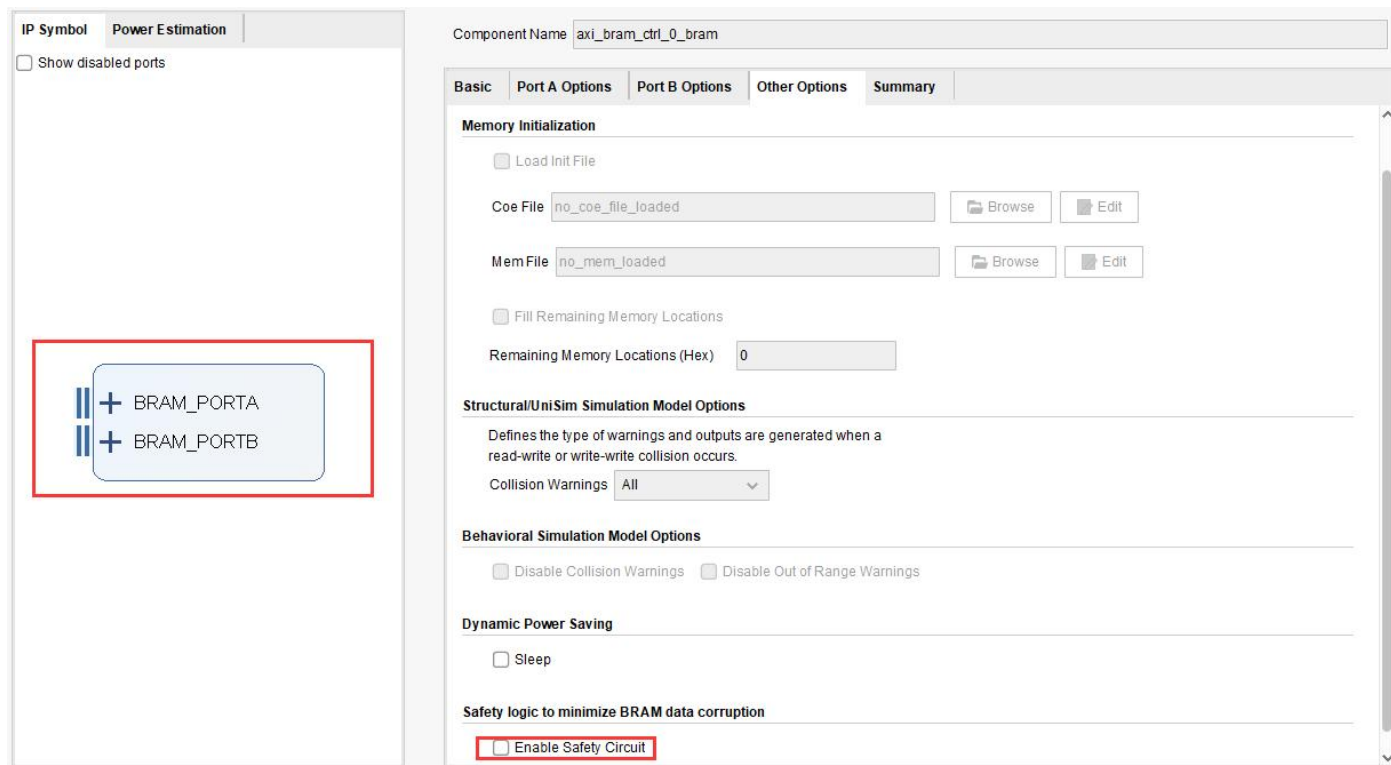
Port B Output Reset Options

☒ RSTB Pin (set/reset pin) Output Reset Value (Hex)

☐ Reset Memory Latch Reset Priority

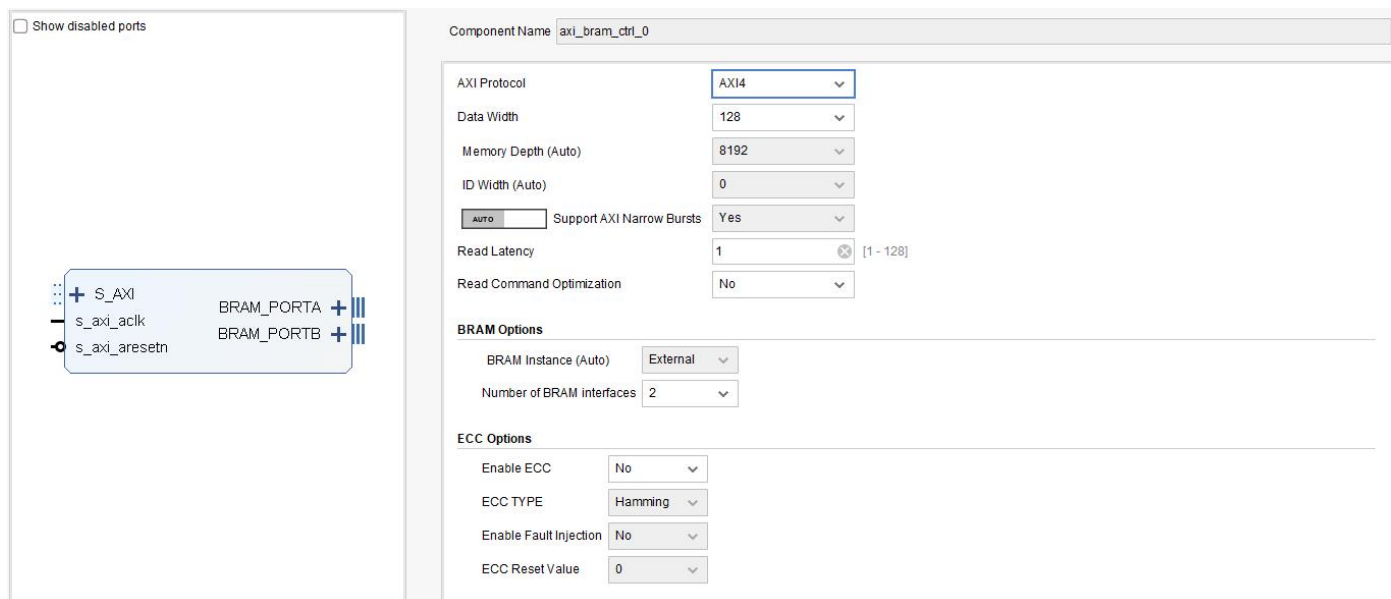
READ Address Change B

☐ Read Address Change B



2:BRAM Controller 参数设置

AXI BRAM Controller 设置 axi4 协议，数据位宽 128bit 读延迟 1 个时钟



3:Clocking Wizard 参数设置

IP SymbolResource

Show disabled ports

CLK_IN1_D

reset

clk_out1

locked

Component Nameclk_wiz_0

Clocking Options

Output Clocks

MMCM Settings

Summary

Clock Monitor

Enable Clock Monitoring

Primitive

MMCM

PLL

Auto

Clocking Features

Frequency Synthesis

Minimize Power

Phase Alignment

Spread Spectrum

Dynamic Reconfig

Dynamic Phase Shift

Safe Clock Startup

Use CDDC

Jitter Optimization

Balanced

Minimize Output Jitter

Maximize Input Jitter filtering

Dynamic Reconfig Interface Options

AXI4Lite

DRP

Phase Duty Cycle Config

Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒	Primary	clk_in1	AUTO100.000	10.000 - 933.000	UI	0.010	Differential clock capable pin
☐	Secondary	clk_in2	100.000	66.667 - 133.333		0.010	Single ended clock capable pin

IP SymbolResource

Show disabled ports

CLK_IN1_D

clk_out1

locked

Component Nameclk_wiz_0

Clocking Options

Output Clocks

MMCM Settings

Summary

The phase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives	Matched Routing	Max Freq. of buffer
		Requested	Actual	Requested	Actual	Requested	Actual			
☒ clk_out1	clk_out1	100.000	100.00000	0.000	0.000	50.000	50.0	Buffer	☐	775.194
☐ clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	Buffer	☐	775.194

USE CLOCK SEQUENCING

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Enable Optional Inputs / Outputs for MMCM/PLL

reset

power_down

input_clk_stopped

locked

clkfbstopped

Reset Type

Active High

Active Low

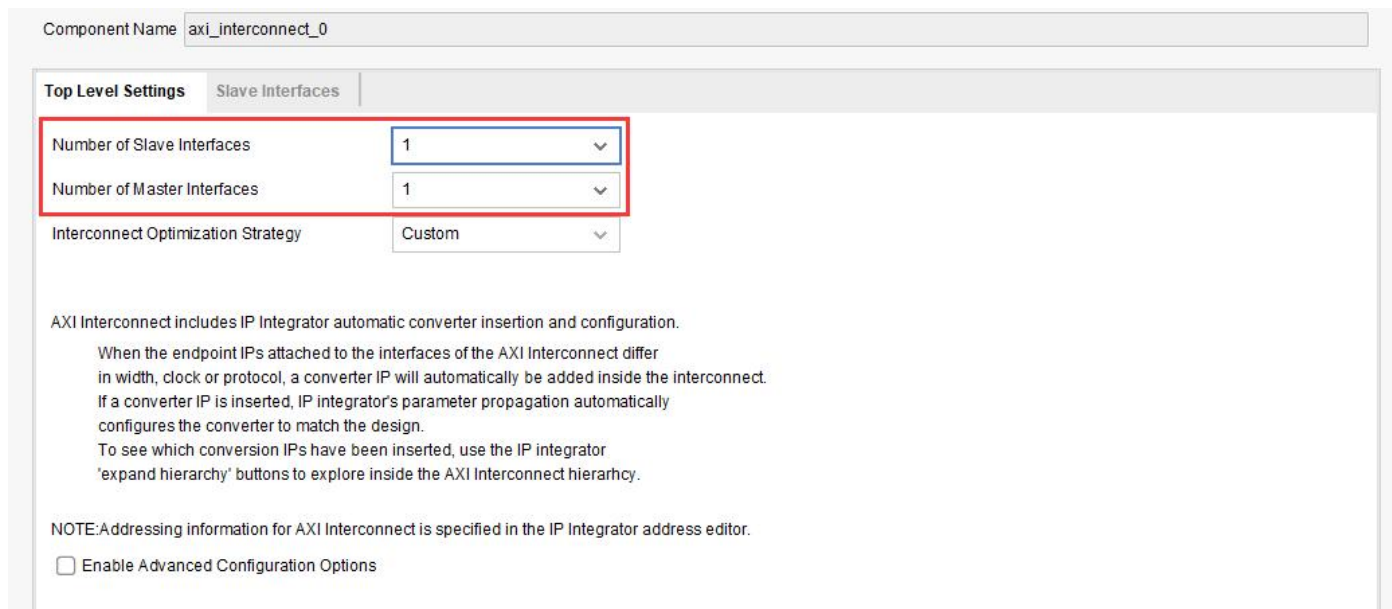
Phase Shift Mode

WAVEFORM

LATENCY

4:AXI Interconnect IP 设置

双击 AXI Interconnect IP 进行设置

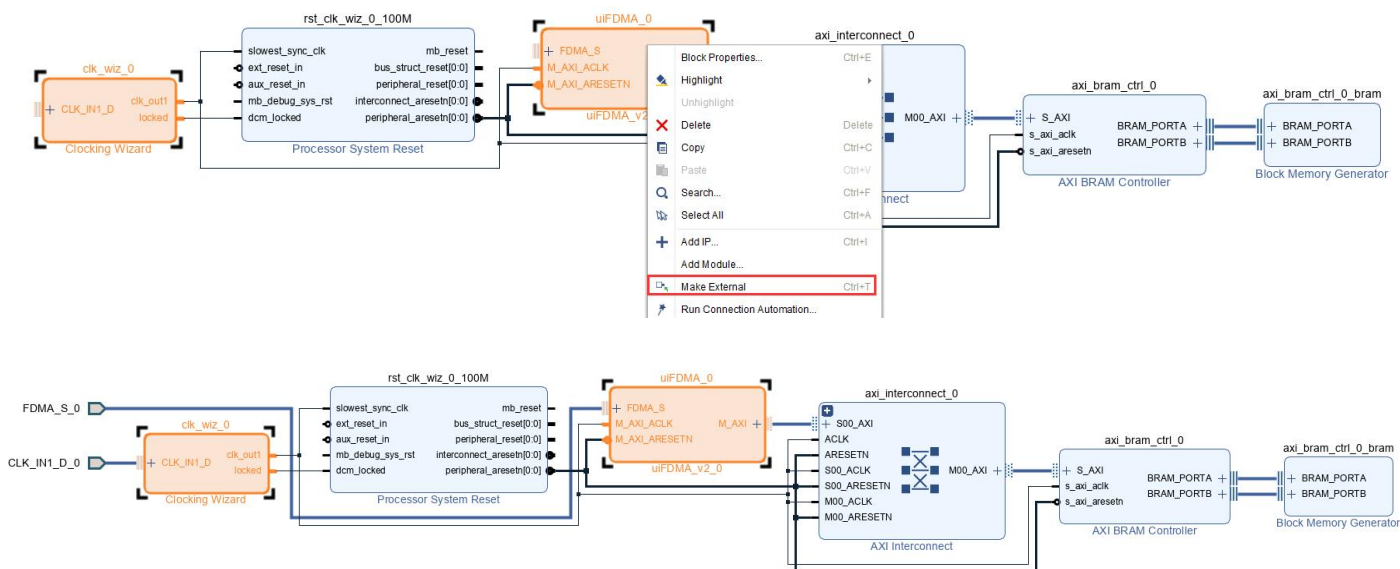


设置 AXI Interconnect IP 的性能参数，其中 Enable Register Slice 用于改善时序，Enable Data FIFO 用于增加 FIFO 大小，增加数据传输效率

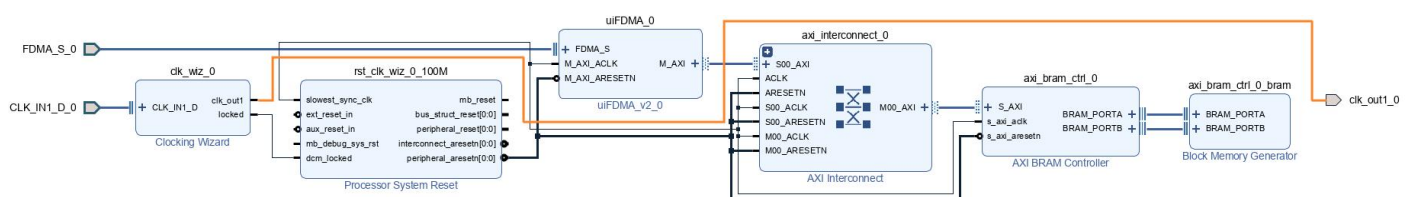
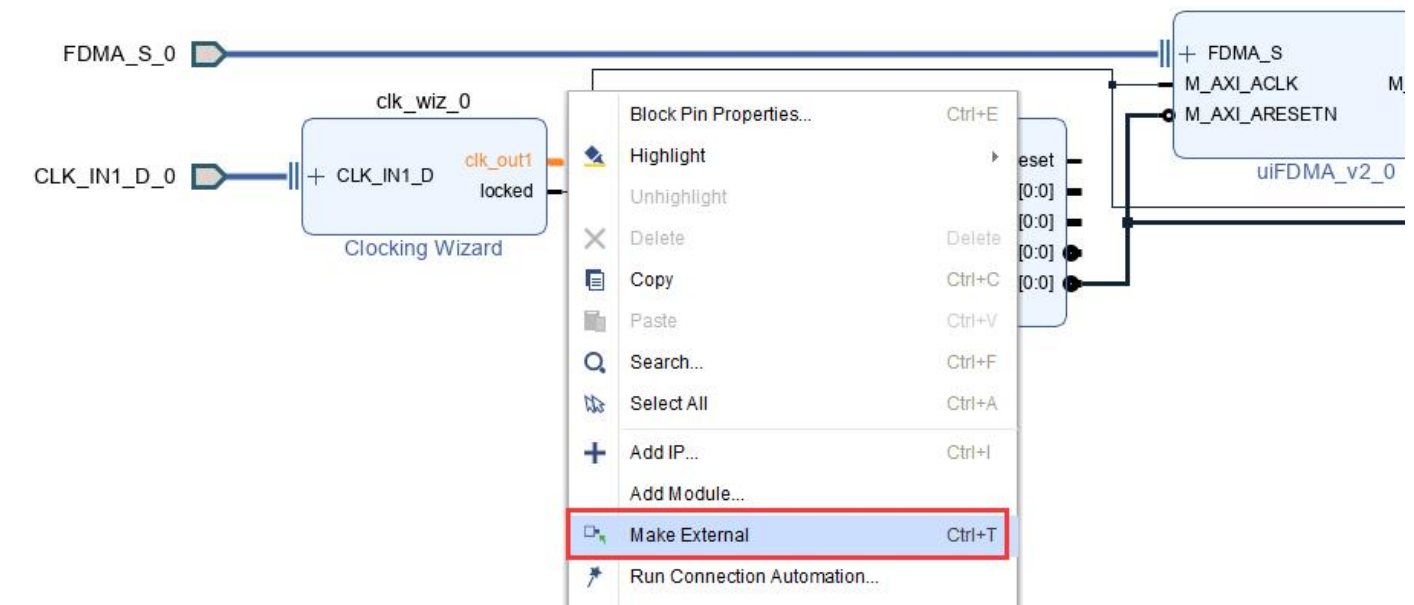
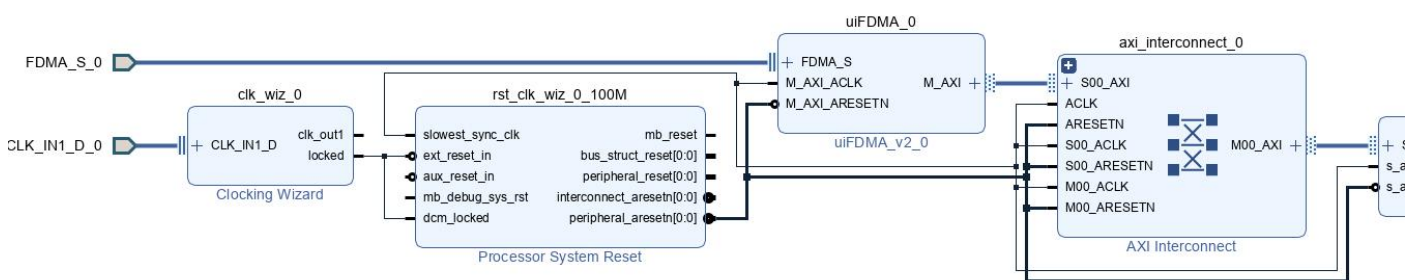
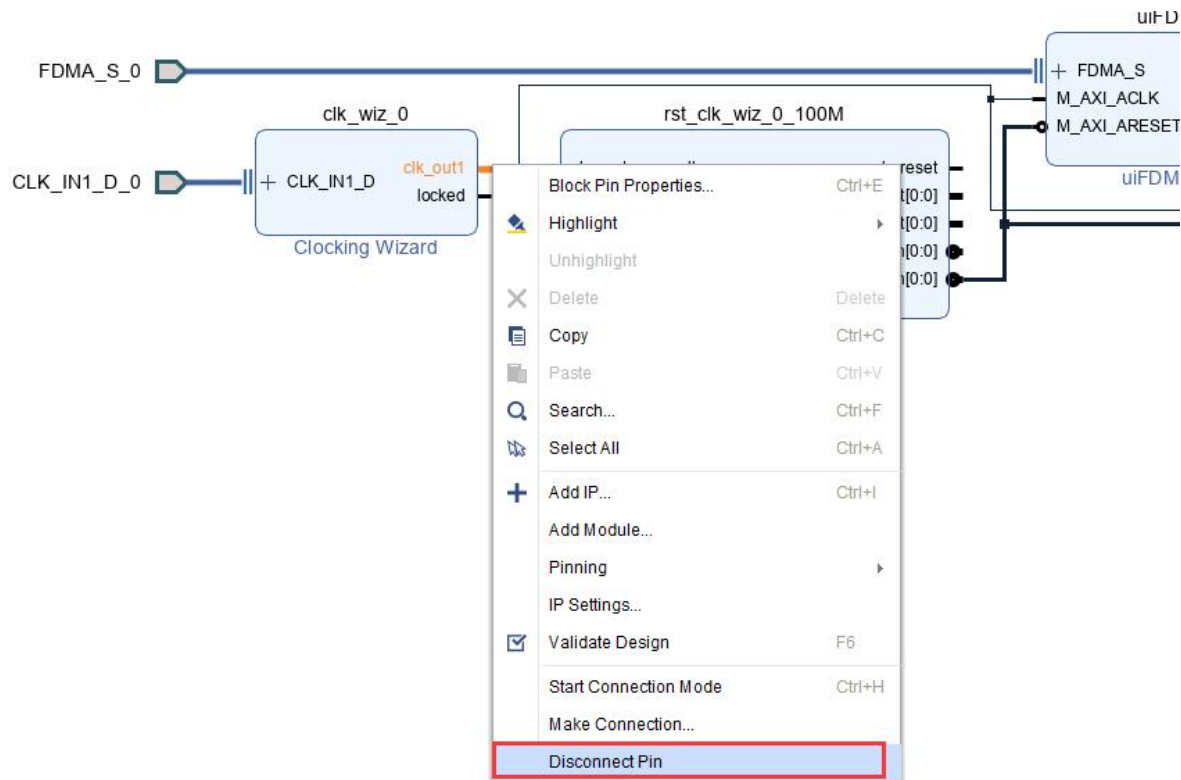


8.2.5 引出 FPGA 接口信号

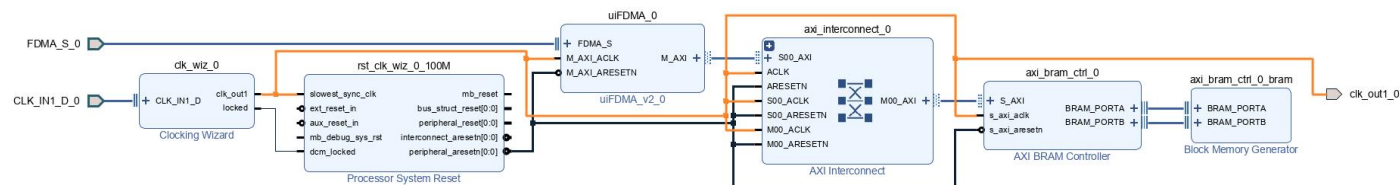
分别右击下图 2 个 IP,然后选择 Make External, 把需要引出到顶层的 FPGA 信号引出



为了引出时钟需先右击信号 PIN 脚断开连接，然后 Make External，之后重新连接

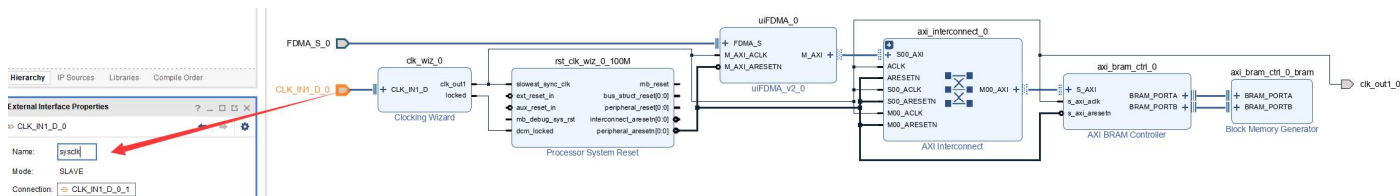


修改后重新连接时钟

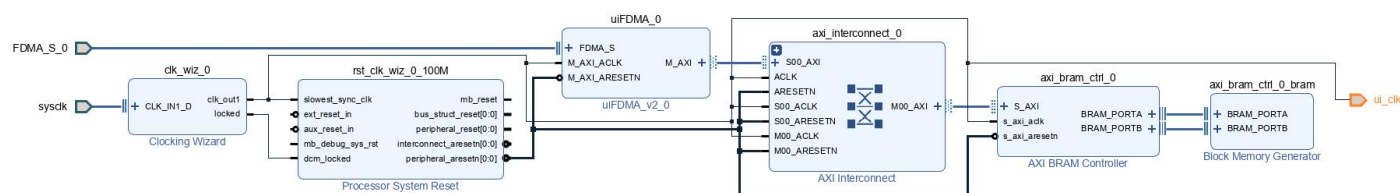


8.2.6 修改信号名字

默认的时钟名字不是很好，可以自己修改下



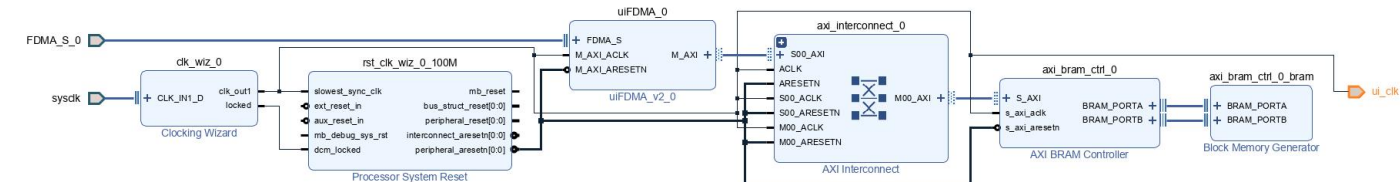
修改完成如下



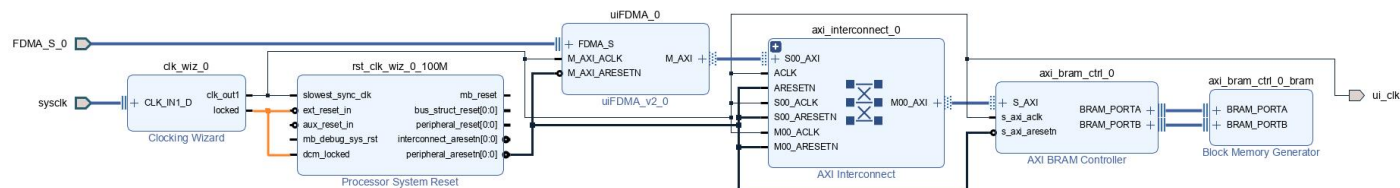
8.2.7 连线最后的遗漏

软件还在提示我们有线没有连接，这种提示是建立在 XILINX 自带的 IP，软件可以识别一部分比如复位时钟等，不是可以识别所有的信号，一般再最后我们如果看到还有这种提示说明还有未完成的连线

Designer Assistance available. Run Connection Automation

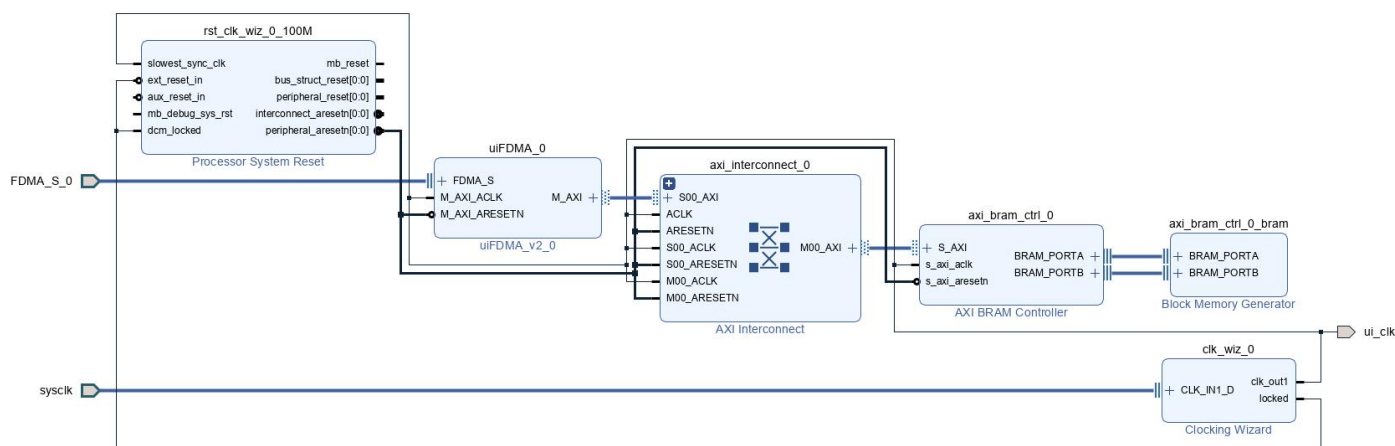
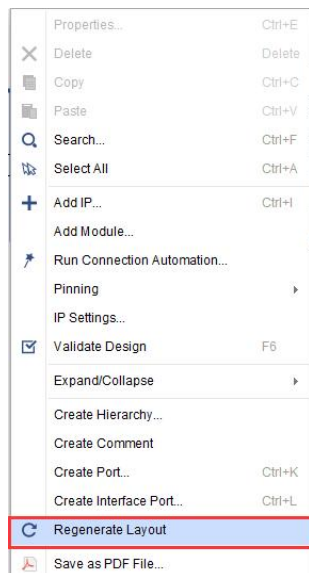


这里是 ext_reset_in 没有连接，也是接到 Clk_Wiz_0 的 Locked



8.2.8 视图优化

右击空白处，弹出菜单选择 Regenerate Layout 优化下视图



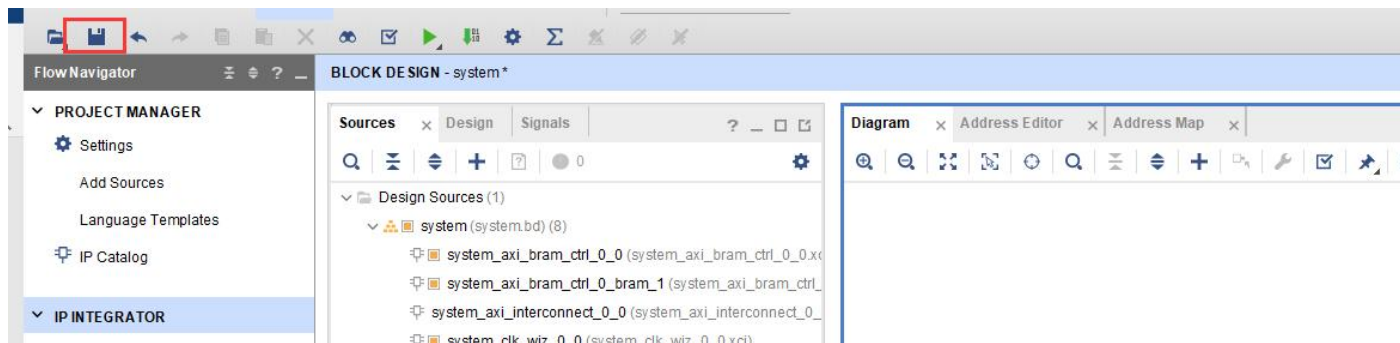
8.2.9 地址分配

AXI 总线必须分配地址，设置 uiFDMA 的地址空间分配，起始地址可以任意设置，我们设置从 0x0000_0000 开始，大小 64KB

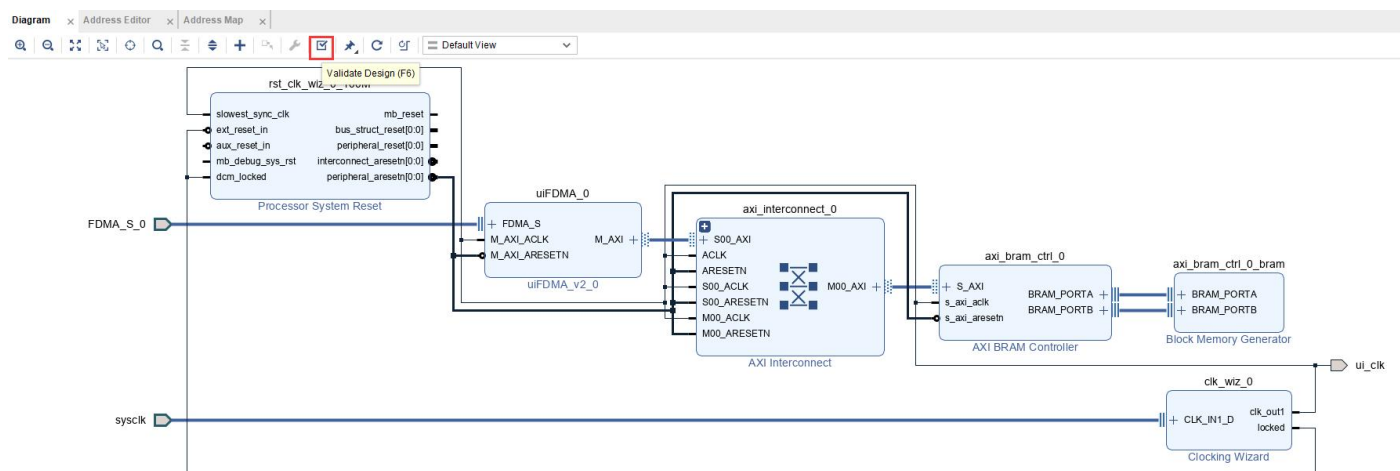
Diagram x Address Editor x Address Map x

8.2.10 自动校验

保存工程

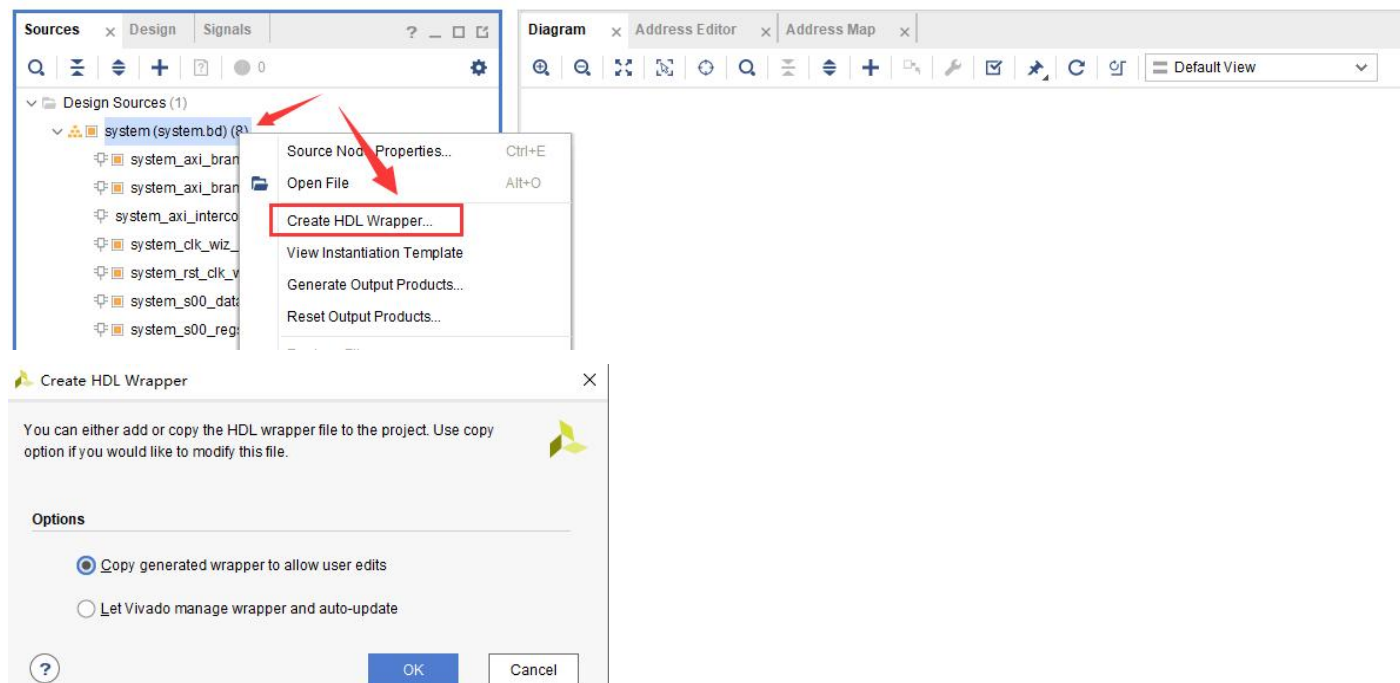


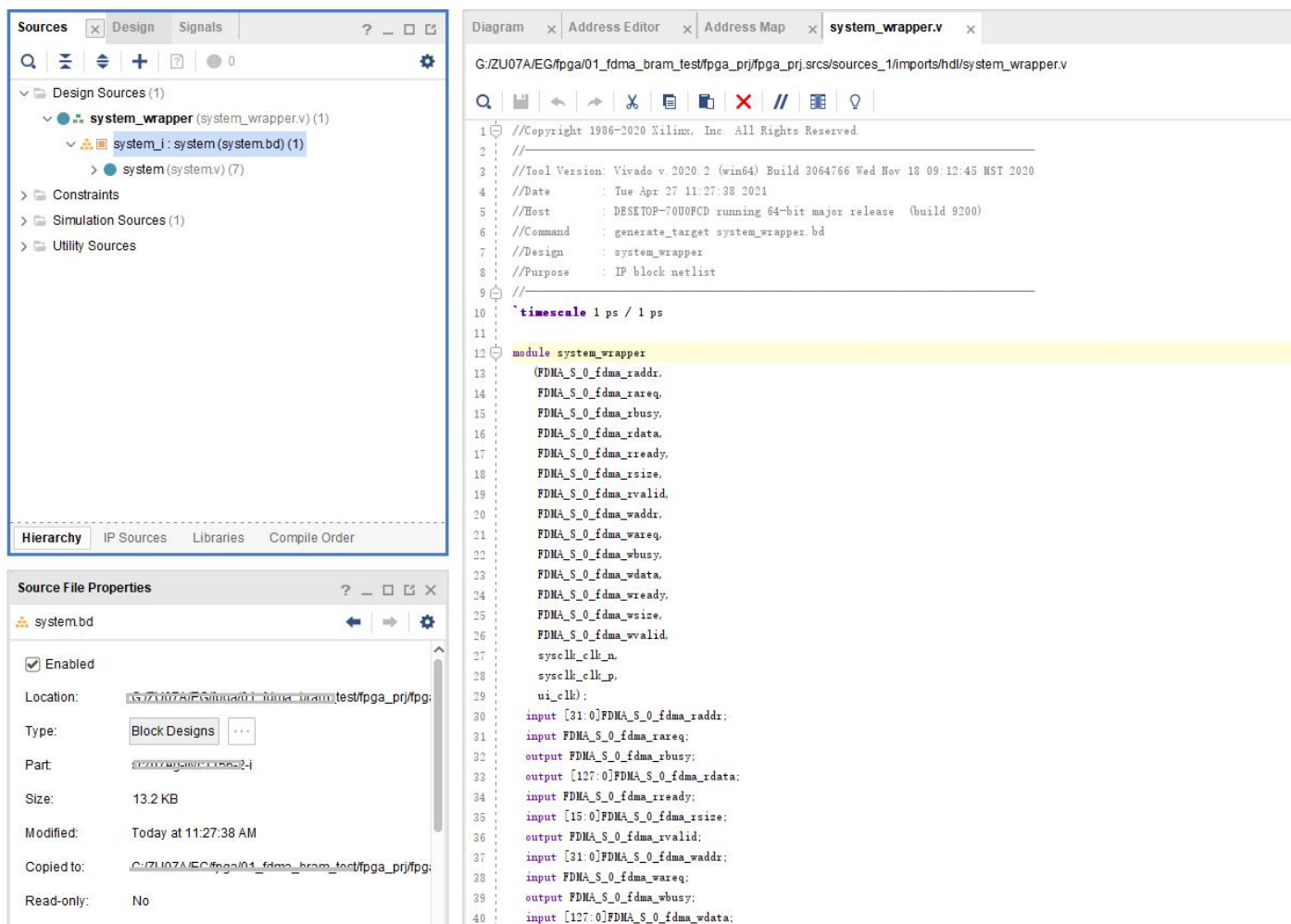
单击下图中的控件可以对图形化编程进行校验



8.2.11 自动产生顶层文件

右击 system,在弹出的菜单中选择 Create HDL Wrapper

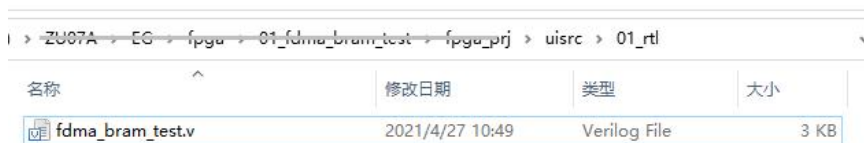




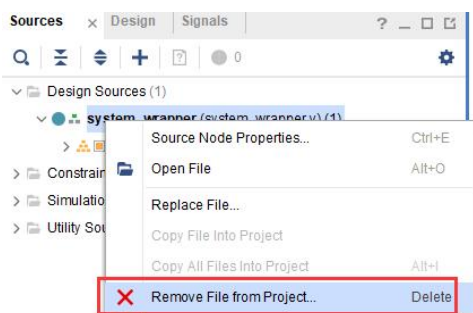
8.3 编写 FDMA 的 BRAM 测试代码

刚刚以上自动产生的顶层文件，只是引出的信号接口，并不能完成对 FDMA 的控制，所以我们需要自定义一个顶层文件，可以复制刚才产生的文件，修改，这样可以省一些编写调用接口的时间。

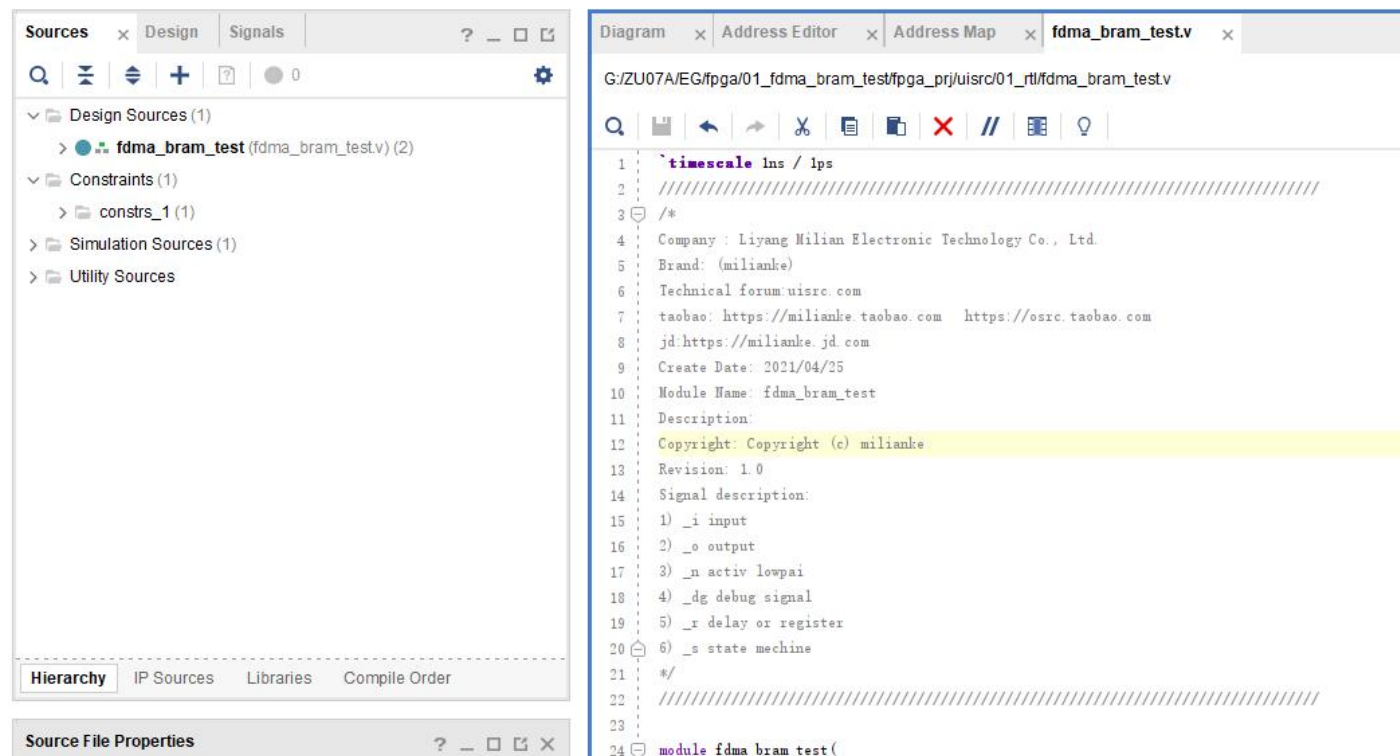
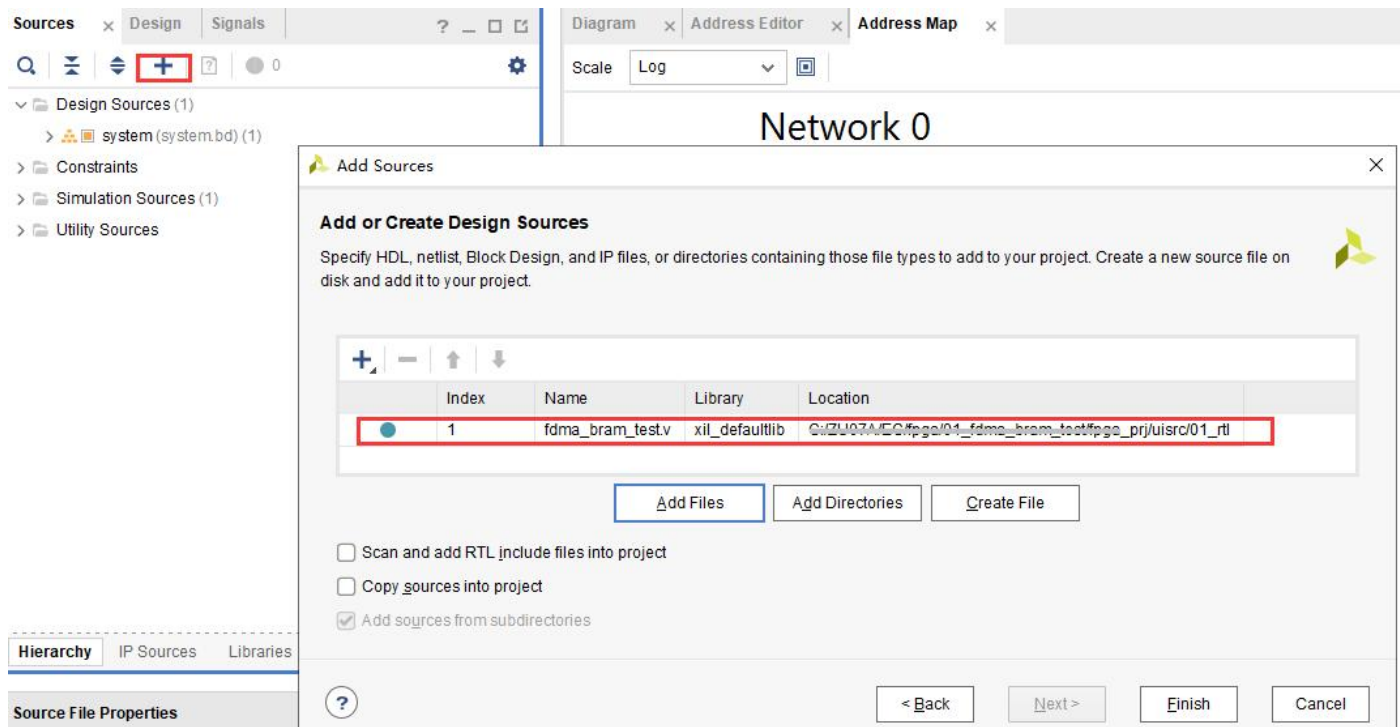
为了方便文件的管理，我们新建一个 fdma_bram_test.v 文件，并且复制以上代码，到这个文件夹。



右击删除刚刚自动产生的文件



添加 fdma_bram_test.v 文件



fdma_bram_test.v 文件

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: (milianke)
Technical forum:uisrc.com
taobao: https://milianke.taobao.com    https://osrc.taobao.com
jd:https://milianke.jd.com
Create Date: 2021/04/25
Module Name: fdma_bram_test
Description:
Copyright: Copyright (c) milianke
Revision: 1.0
```

Signal description:

- 1) _i input
- 2) _o output
- 3) _n activ lowpai
- 4) _dg debug signal
- 5) _r delay or register
- 6) _s state mechine

*/

////////////////////////////////////

module fdma_bram_test(

input sysclk_p,

input sysclk_n

);

wire [31:0] fdma_raddr;

reg fdma_rareq;

wire fdma_rbusy;

wire [127:0] fdma_rdata;

wire [15:0] fdma_rsize;

wire fdma_rvalid;

wire [31:0] fdma_waddr;

reg fdma_wareq;

wire fdma_wbusy;

wire [127:0] fdma_wdata;

wire [15:0] fdma_ysize;

wire fdma_wvalid;

wire [0:0] fdma_rstn;

wire ui_clk;

parameter TEST_MEM_SIZE = 32'd64*1024//64KB

parameter FDMA_BURST_LEN = 16'd512;

parameter ADDR_MEM_OFFSET = 0;

parameter ADDR_INC = FDMA_BURST_LEN * 16;

parameter WRITE1 = 0;

parameter WRITE2 = 1;

parameter WAIT = 2;

parameter READ1 = 3;

parameter READ2 = 4;

reg [31: 0] t_data;

reg [31: 0] fdma_waddr_r;

reg [2 :0] T_S = 0;

assign fdma_waddr = fdma_waddr_r + ADDR_MEM_OFFSET;

assign fdma_raddr = fdma_waddr;

assign fdma_ysize = FDMA_BURST_LEN;

assign fdma_rsize = FDMA_BURST_LEN;

assign fdma_wdata = {t_data,t_data,t_data,t_data};

//delay reset

reg [8:0] rst_cnt = 0;

always @(posedge ui_clk)

if(rst_cnt[8] == 1'b0)

rst_cnt <= rst_cnt + 1'b1;

else

rst_cnt <= rst_cnt;

always @(posedge ui_clk)begin

if(rst_cnt[8] == 1'b0)begin

```

    T_S <= 0;
    fdma_wareq <= 1'b0;
    fdma_rareq <= 1'b0;
    t_data <= 0;
    fdma_waddr_r <= 0;
end
else begin
    case(T_S)
    WRITE1:begin
        if(fdma_waddr_r==TEST_MEM_SIZE) fdma_waddr_r<=0;
        if(!fdma_wbusy)begin
            fdma_wareq <= 1'b1;
            t_data <= 0;
        end
        if(fdma_wareq&&fdma_wbusy)begin
            fdma_wareq <= 1'b0;
            T_S <= WRITE2;
        end
    end
    WRITE2:begin
        if(!fdma_wbusy) begin
            T_S <= WAIT;
            t_data <= 32'd0;
        end
        else if(fdma_wvalid) begin
            t_data <= t_data + 1'b1;
        end
    end
    WAIT:begin//not needed
        T_S <= READ1;
    end
    READ1:begin
        if(!fdma_rbusy)begin
            fdma_rareq <= 1'b1;
            t_data <= 0;
        end
        if(fdma_rareq&&fdma_rbusy)begin
            fdma_rareq <= 1'b0;
            T_S <= READ2;
        end
    end
    READ2:begin
        if(!fdma_rbusy) begin
            T_S <= WRITE1;
            t_data <= 32'd0;
            fdma_waddr_r <= fdma_waddr_r + ADDR_INC;//128/8=16
        end
        else if(fdma_rvalid) begin
            t_data <= t_data + 1'b1;
        end
    end
    default:
        T_S <= WRITE1;
    endcase
end
end

wire test_error = (fdma_rvalid && (t_data[15:0] != fdma_rdata[15:0]));

ila_0 ila_dbg (
    .clk(ui_clk),
    .probe0({fdma_wdata[15:0],fdma_wareq,fdma_wvalid,fdma_wbusy}),
    .probe1({fdma_rdata[15:0],t_data[15:0],fdma_rvalid,fdma_rbusy,T_S,test_error})

```

```
);

system system_i
(
    .FDMA_S_0_fdma_raddr(fdma_raddr),
    .FDMA_S_0_fdma_rareq(fdma_rareq),
    .FDMA_S_0_fdma_rbusy(fdma_rbusy),
    .FDMA_S_0_fdma_rdata(fdma_rdata),
    .FDMA_S_0_fdma_rready(1'b1),
    .FDMA_S_0_fdma_rsize(fdma_rsize),
    .FDMA_S_0_fdma_rvalid(fdma_rvalid),
    .FDMA_S_0_fdma_waddr(fdma_waddr),
    .FDMA_S_0_fdma_wareq(fdma_wareq),
    .FDMA_S_0_fdma_wbusy(fdma_wbusy),
    .FDMA_S_0_fdma_wdata(fdma_wdata),
    .FDMA_S_0_fdma_wready(1'b1),
    .FDMA_S_0_fdma_wsize(fdma_wsize),
    .FDMA_S_0_fdma_wvalid(fdma_wvalid),
    .sysclk_clk_n(sysclk_n),
    .sysclk_clk_p(sysclk_p),
    .ui_clk(ui_clk)
);

endmodule
```

8.4 RTL 仿真

8.4.1 仿真 tb 文件

编写仿真 tb 文件，fdma_bram_test_tb.v，非常简单，只需要给一个时钟

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: 米联客(milianke)
Technical forum: uisrc.com
taobao: osrc.taobao.com
Create Date: 2021/04/25
Module Name: fdma_bram_test_tb
Description:
Copyright: Copyright (c) msxbo
Revision: 1.0
Signal description:
1) _i input
2) _o output
3) _n activ low
4) _dg debug signal
5) _r delay or register
6) _s state mechine
*/
/////////////////////////////////////////////////////////////////

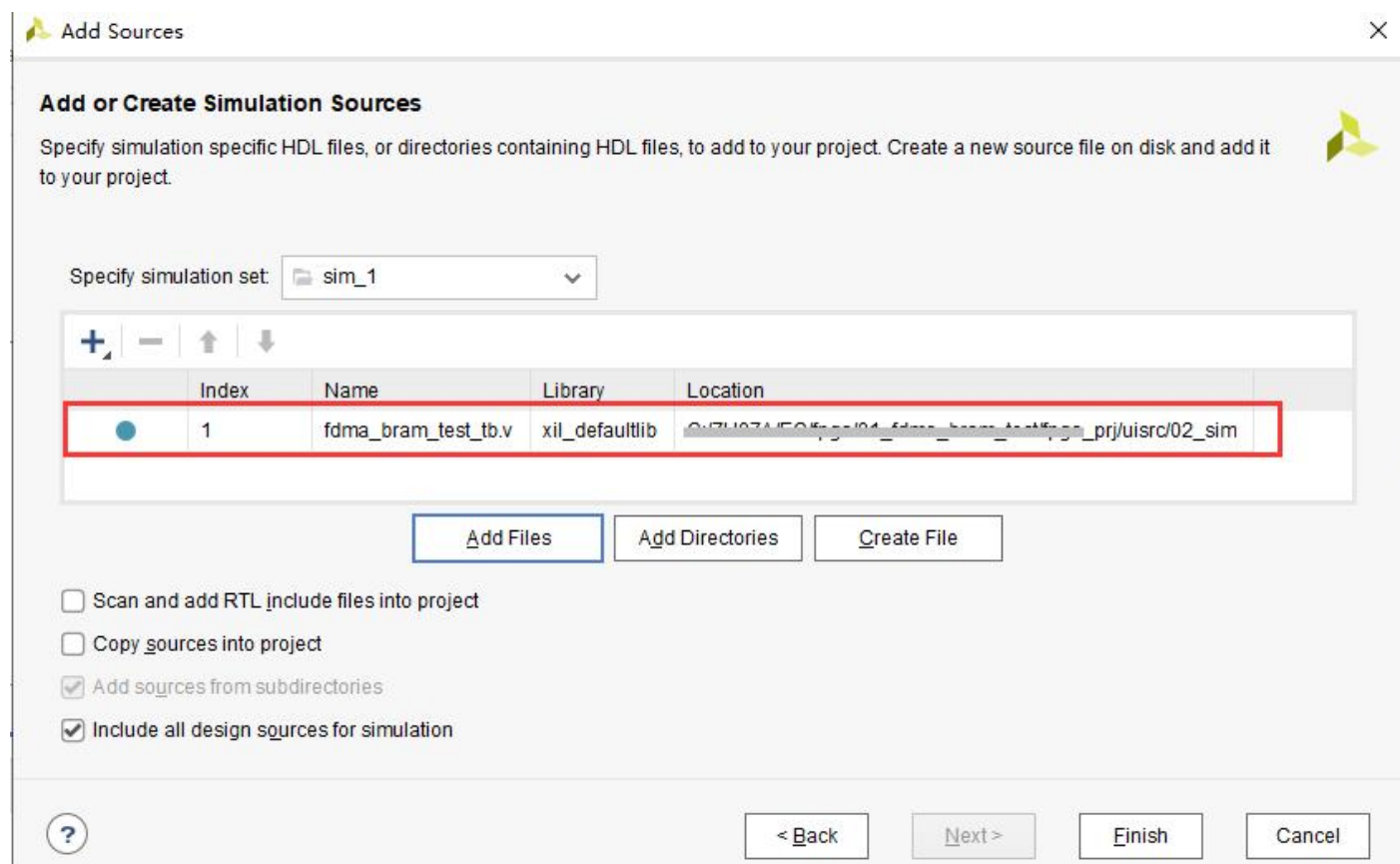
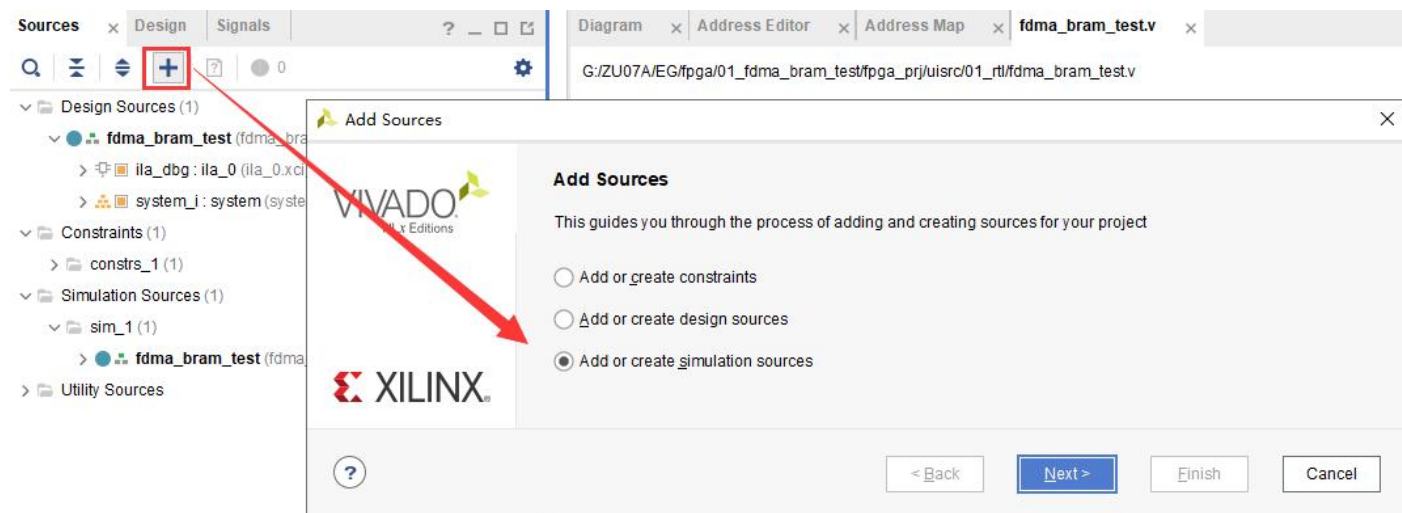
module fdma_bram_test_tb();

    reg sysclk_p;

    fdma_bram_test fdma_bram_test_inst
    (
        .sysclk_p(sysclk_p),
        .sysclk_n(~sysclk_p)
```

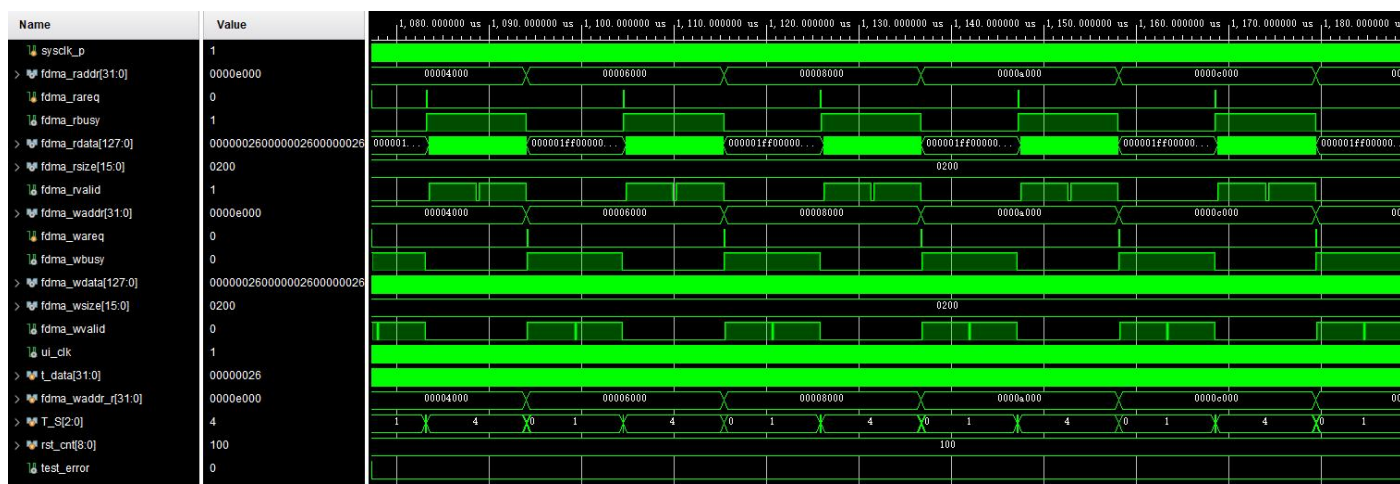
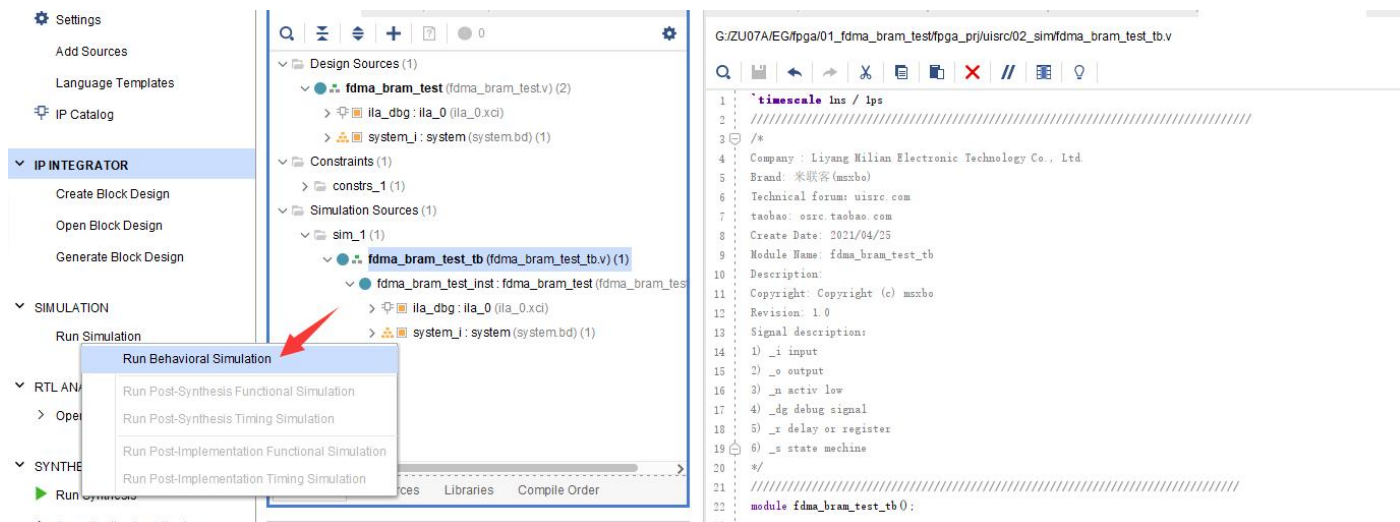
```
);  
  
initial begin  
    sysclk_p = 0;  
    #100;  
end  
  
always #10 sysclk_p = ~sysclk_p;  
  
endmodule
```

添加 fdma_bram_test_tb.v 到工程中

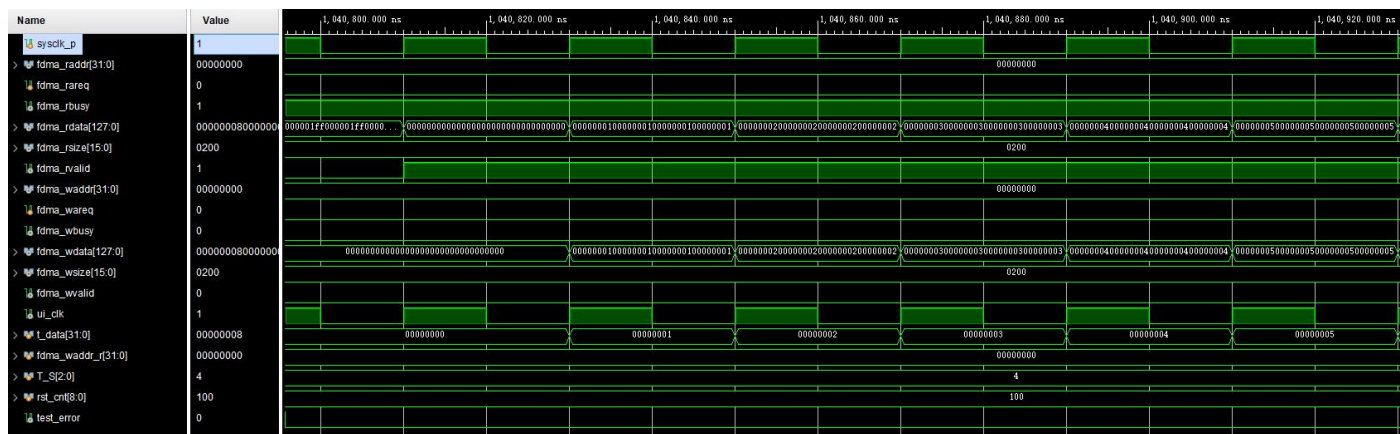


8.4.2 仿真测试

进行 RTL 行为仿真



放大后观察数据



我们也可以继续深入看 FDMA 源码里面的信号工作情况，这个读者可以自己区分析下，我们米联客计划出一份专门讲解 AXI 总线部分的教程，里面详细分析 FDMA 的设计过程，以及信号仿真。本章节主要是侧重 FDMA 的应用，只要会熟练应用可以了！

8.5 编译测试

添加 ila IP CORE 和 fpga pin 定义

The Sources window shows the design hierarchy. Under 'Design Sources (1)', 'fdma_bram_test (fdma_bram_test.v) (2)' is expanded, showing 'ila_dbg:ila_0 (ila_0.xci)' and 'system_i:system (system.bd) (8)'. Under 'Constraints (1)', 'constrs_1 (1)' is expanded, showing 'fpga_pin.xdc'. Under 'Simulation Sources (1)', 'sim_1 (1)' is expanded, showing 'fdma_bram_test_tb (fdma_bram_test_tb.v) (1)' and 'fdma_bram_test_inst:fdma_bram_test (fdma_bram_test.v) (2)'. The Project Summary window shows the Verilog code for 'fdma_bram_test.v'.

```
92      fdma_wareq <= '1'b0;  
93      I_S        <= WRITE2;  
94      end  
95      end  
96      WRITE2:begin  
97          if (!fdma_wbusy) begin  
98              I_S <= WAIT;  
99              t_data <= 32'd0;  
100          end  
101          else if (fdma_wvalid) begin  
102              t_data <= t_data + 1'b1;  
103          end  
104      end  
105      WAIT:begin//not needed  
106          I_S <= READ1;  
107      end  
108      READ1:begin  
109          if (!fdma_rbusy)begin  
110              fdma_xareq <= '1'b1;  
111              t_data <= 0;
```

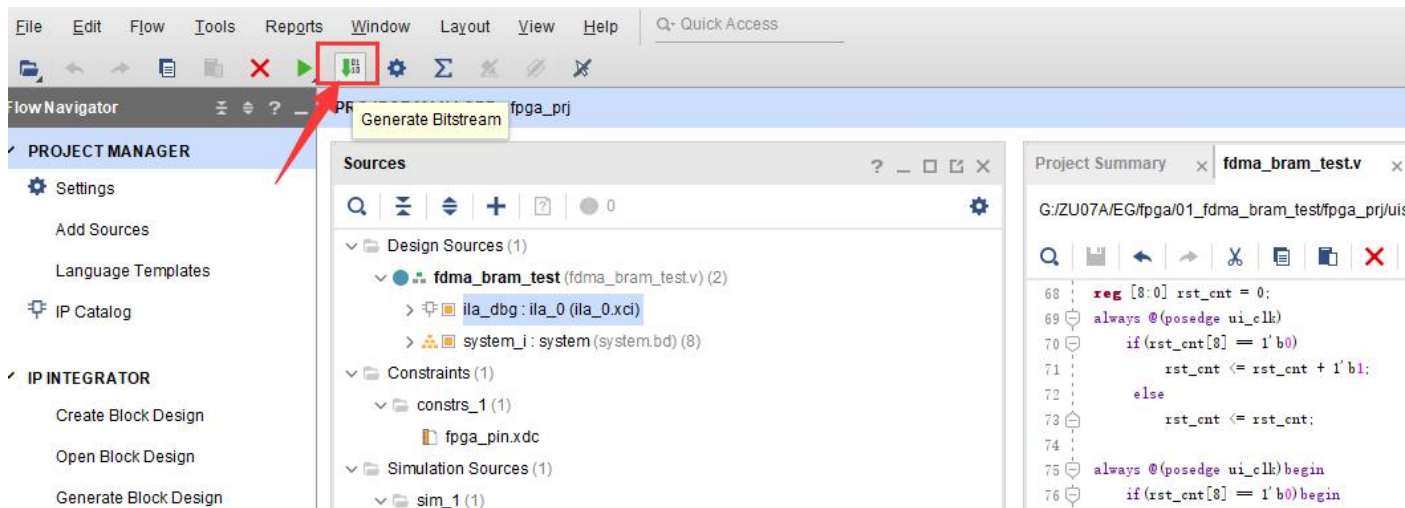
ila 的设置如下

The ILA configuration window shows the following settings:

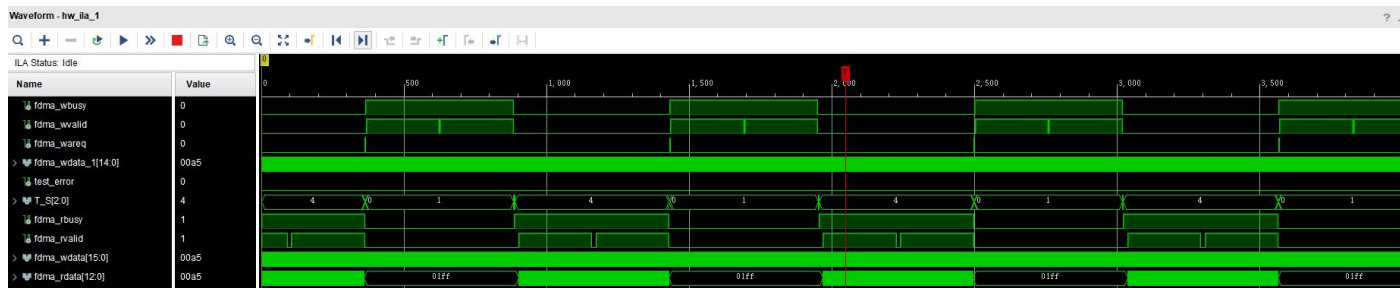
- Component Name: ila_0
- To configure more than 64 probe ports use Vivado Tcl Console
- General Options: Probe Ports(0..1)
- Monitor Type: Native
- Number of Probes: 2
- Sample Data Depth: 4096
- Same Number of Comparators for All Probe Ports: ☒
- Number of Comparators: 1
- Trigger Out Port: ☐
- Trigger In Port: ☐
- Input Pipe Stages: 0
- Trigger And Storage Settings: ☐ Capture Control, ☐ Advanced Trigger
- GUI configuration mode is limited to 64 probe ports.

Probe Port	Probe Width [1..4096]	Number of Comparators	Probe Trigger or Data
PROBE0	18	1	DATA AND TRIGGER
PROBE1	35	1	DATA AND TRIGGER

编译产生 bit

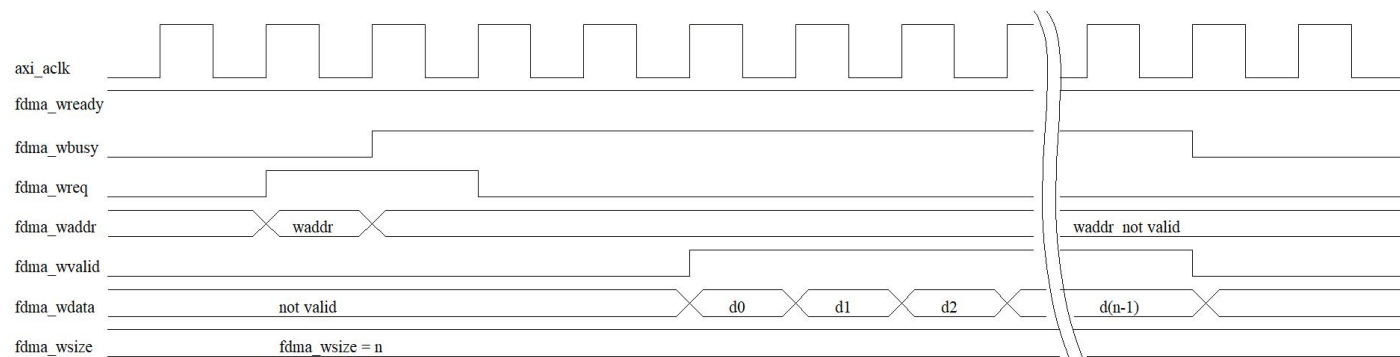


下载程序到开发板测试，通过在线逻辑分析仪观察



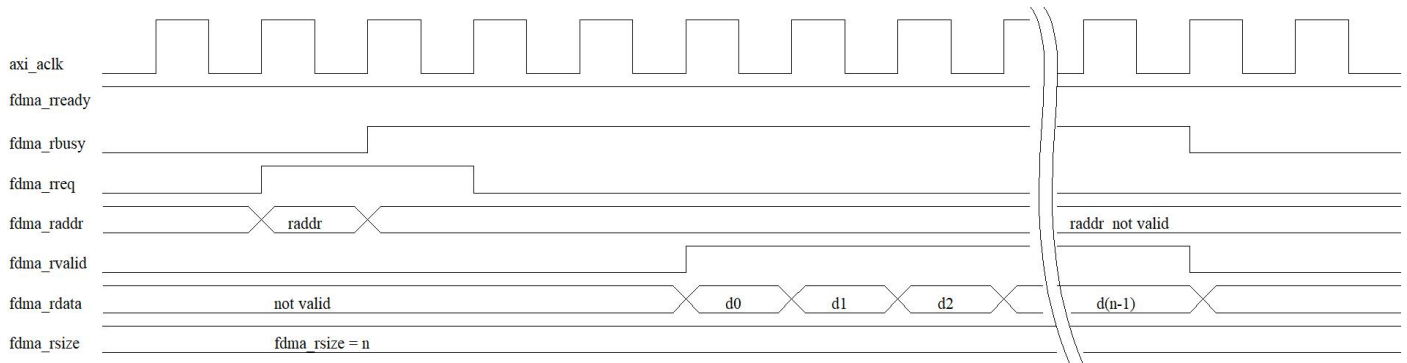
8.6 程序分析

8.6.1 FDMA 的写时序



fdma_wready 设置为 1，当 fdma_wbusy=0 的时候代表 FDMA 的总线非忙，可以进行一次新的 FDMA 传输，这个时候可以设置 fdma_wreq=1，同时设置 fdma burst 的起始地址和 fdma_wsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_wvalid=1 的时候需要给出有效的数据，写入 AXI 总线。当最后一个数写完后，fdma_wvalid 和 fdma_wbusy 变为 0。

8.6.2 FDMA 的读时序



fdma_rready 设置为 1, 当 fdma_rbusy=0 的时候代表 FDMA 的总线非忙, 可以进行一次新的 FDMA 传输, 这个时候可以设置 fdma_rreq=1, 同时设置 fdma burst 的起始地址和 fdma_rsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_rvalid=1 的时候需要给出有效的数据, 写入 AXI 总线。当最后一个数写完后, fdma_rvalid 和 fdma_rbusy 变为 0。

8.6.3 读写状态机

```

case (T_S)
WRITE1:begin
    if (fdma_waddr_r==TEST_MEM_SIZE) fdma_waddr_r<=0; 当所有的数据测试完成, 设置起始地址0, 循环测试
    if (!fdma_wbusy)begin 等待AXI总线不忙
        fdma_wareq <= 1'b1; 请求写数据
        t_data <= 0;
    end
    if (fdma_wareq&&fdma_wbusy)begin 当wbusy忙说明FDMA已经接收到请求命名
        fdma_wareq <= 1'b0;
        T_S <= WRITE2; 进入写数据阶段
    end
end
WRITE2:begin
    if (!fdma_wbusy) begin 写完数据, 并且AXI总线空闲, 进入WAIT状态
        T_S <= WAIT; FDMA是可以支持同时读写的, 这里只是为了测试方便, 先写后读
        t_data <= 32'd0;
    end
    else if (fdma_rvalid) begin 只要rvalid=1就可以一直写入数据
        t_data <= t_data + 1'b1; 这里写入加计数
    end
end
WAIT:begin//not needed
    T_S <= READ1; 进入读状态, 这个状态也可以不要
end
READ1:begin
    if (!fdma_rbusy)begin 等待AXI总线不忙
        fdma_rareq <= 1'b1; 请求读数据
        t_data <= 0;
    end
    if (fdma_rareq&&fdma_rbusy)begin 当rbusy忙说明FDMA已经接收到请求命名
        fdma_rareq <= 1'b0;
        T_S <= READ2; 进入读数据阶段
    end
end
READ2:begin
    if (!fdma_rbusy) begin 当非忙, 代表读完成了
        T_S <= WRITE1;
        t_data <= 32'd0;
        fdma_waddr_r <= fdma_waddr_r + ADDR_INC;//128/8=16 一次FDMA传输完成, 重新计算下一次FDMA的起始地址
    end
    else if (fdma_rvalid) begin 只要rvalid=1就可以一直读数据
        t_data <= t_data + 1'b1; 读状态里面也有一个计数器, 用于比较读出的数据是否正确
    end
end
default:
    T_S <= WRITE1;
endcase

```

09 使用 fdma 读写 DDR

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

9.1 概述

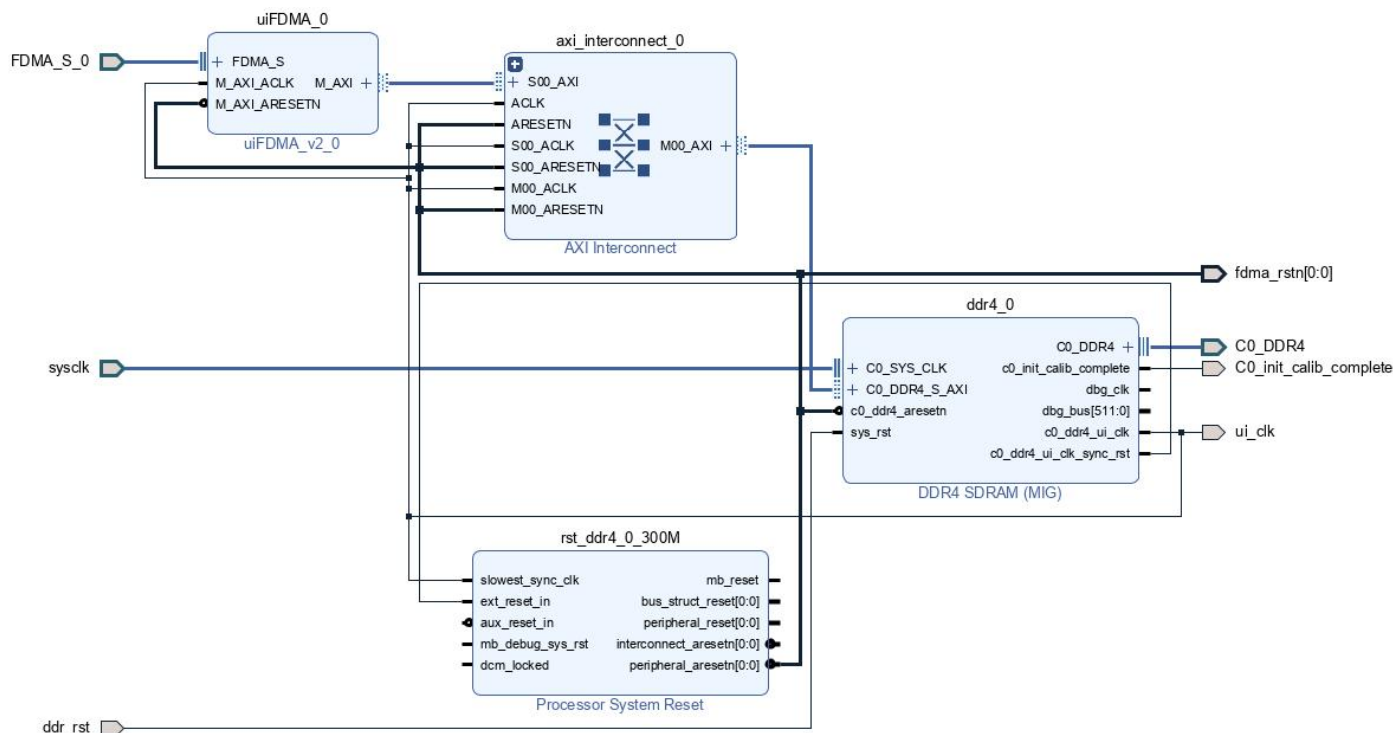
在前文的实验中我们详细介绍了 FDMA 的使用方法, 以及使用了 AXI-BRAM 演示了 FDMA 的使用, 现在我们已经掌握了 FDMA 的使用, 本文我们继续使用 FDMA 实现对 AXI-MIG 的读写, 以此读写 DDR。由于 FDMA 的读写操作都是基于 AXI 总线, 所以用户代码部分一致性也非常好, 我们的状态机都不需要做修改, 基本上只要把前文的 BRAM IP 换成 MIG IP 即可。

本文实验目的:

- 1: 利用 uiFDMA2.0/3.0 提供的接口, 编写 DDR 测试程序
- 2: 对 MIG 接口读写仿真和测试

9.2 PL 图形化设计

搭建过程我们不再详细描述, 不清楚的可以参考前“01 使用 FDMA 读写 AXI-BRAM”, 搭建好的工程如下



我们主要看下 MIG 的配置

Component Name

ddr4_0

Basic

AXI Options

Advanced Clocking

Advanced Options

I/O Planning and Design Checklist

Choose the appropriate AXI Parameter Options for highest bandwidth efficiency. For more information see the Memory Solutions User Guide [PG150](#)

Data Width

512

Arbitration Scheme

RD PRI REG

AUTO

ID Width

1

Address Width

32

AUTO

☒ AXI Narrow Burst

Component Name ddr4_0

Basic AXI Options Advanced Clocking Advanced Options I/O Planning and Design Checklist

Specify M and D

These options are only enabled if **Specify MMCM M and D on Advanced Clocking Page** option is checked on the **Basic** page

Formulae used for calculation

MMCM_CLKOUT(MHz) = tCK/4 , where tCK is Memory Device Interface Speed(MHz) selected in basic page

$CLKIN(MHz) = (MMCM_CLKOUT(MHz) * D0) / M$

$VCO(MHz) = (CLKIN(MHz) * M) / D$

$PFD(MHz) = CLKIN(MHz) / D$

CLKFBOUT_MULT (M)	15
DIVCLK_DIVIDE (D)	1
CLKOUT0_DIVIDE (D0)	5
VCO (MHz)	1500.600
Reference Input Clock Speed (ps)	9996 (100.040 MHz)
PFD (MHz)	100.040

System Clock Option

Reference Input Clock Configuration Differential ▼

Additional Clock Outputs

DDR4 can generate up to 4 additional clocks to be used in Fabric logic. This will be generated from the same MMCM which is used for generation of UI CLK. All the values in the additional clocks drop downs are calculated considering the selected MMCM VCO frequency in Mhz. For complete details on clocking of DDR4, refer to [DDR4 product Guide](#)

Clock 1 (MHz) None ▼ D = 0

Clock 2 (MHz) None ▼ D = 0

Clock 3 (MHz) None ▼ D = 0

Clock 4 (MHz) None ▼ D = 0

Component Name ddr4_0

Basic AXI Options Advanced Clocking Advanced Options I/O Planning and Design Checklist

Debug Signals for controller

☒ Disable

☐ Enable

Microblaze MCS ECC option

☐ Microblaze MCS ECC

Simulation Options

Simulation Mode BFM ▼

Example Design Options

Example Design Test Bench SIMPLE TG ▼

Advance Memory Options

☐ Enable Self Refresh

☐ Enable Save-Restore

☐ Disable OBUF on reset_n

☐ Enable Restore CRC

Migration Options

☐ Enable Migration

Component Name ddr4_0

Basic	AXI Options	Advanced Clocking	Advanced Options	I/O Planning and Design Checklist
-------	-------------	-------------------	------------------	-----------------------------------

I/O Planning

The methodology for assigning I/O pins for DDR4 IP interfaces has changed. Rather than assign I/Os within the IP, they are now assigned in the main Vivado I/O Planner along with the I/Os for the rest of the design. To access the Vivado I/O Planning environment, open the Elaborated RTL Design or Synthesized Design and select the I/O Planning layout option from the top menu bar. The DDR4 IP I/O byte groups are available for bank assignment using a similar dialog as used previously within the IP.

Refer to the Vivado Design Suite User Guide: [I/O and Clock Planning](#) or the Using Memory Controller IP with UltraScale Devices [QuickTake video tutorial](#) for more information. Performing I/O planning for the Memory IP I/Os, requires the IP to be instantiated into a top-level design or example design.

Design Checklist: Usage Information

A DDR4 Design Methodology Checklist is available to plan, design, and debug with DDR4. The checklist organizes information that is critical to successful DDR4 operation especially at top supported data rates. It includes information on core definition and generation, pin, clocking, and board planning, simulation, design flow, and hardware debug. The checklist is provided as an Excel spreadsheet that organizes all information needed when working with DDR4, where to go for more information, a check of whether the item has been reviewed, and sections to add notes should there be follow on questions or items to check at a later time. The checklist should be used throughout the design, bring-up, and debug of DDR4 cores.

Checklist can be found [here](#)

9.3 编写 FDMA 的 DDR 测试代码

以下代码除来 DDR 的引脚定义, 以及增加来 1 按键输入复位, 其他内容都和“01 使用 FDMA 读写 AXI-BRAM”一样。读写状态机完全一样, 对 PL DDR 整个内存地址空间进行测试。

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: milianke
Technical forum:uisrc.com
taobao: https://milianke.taobao.com    https://osrc.taobao.com
jd:https://milianke.jd.com
Create Date: 2021/04/26
Module Name: fdma_ddr_test
Description:
Copyright: Copyright (c) milianke
Revision: 1.0
Signal description:
1) _i input
2) _o output
3) _n activ lowpai
4) _dg debug signal
5) _r delay or register
6) _s state mechine
*/
////////////////////////////////////////////////////////////////
module fdma_ddr_test(
output          C0_DDR4_act_n,
output [16:0]C0_DDR4_adr,
output [1 :0]C0_DDR4_ba,
output [0 :0]C0_DDR4_bg,
output [0 :0]C0_DDR4_ck_c,
output [0 :0]C0_DDR4_ck_t,
output [0 :0]C0_DDR4_cke,
output [0 :0]C0_DDR4_cs_n,
inout  [7 :0]C0_DDR4_dm_n,
inout  [63:0]C0_DDR4_dq,
inout  [7 :0]C0_DDR4_dqs_c,
inout  [7 :0]C0_DDR4_dqs_t,
```

```

output [0:0]C0_DDR4_odt,
output      C0_DDR4_reset_n,
output      C0_init_calib_complete,
input       key_i,
input       sysclk_p,
input       sysclk_n
);
wire ddr_rst = ~key_i;
wire [31:0]  fdma_raddr;
reg         fdma_rareq;
wire        fdma_rbusy;
wire [127:0] fdma_rdata;
wire [15:0]  fdma_rsize;
wire         fdma_rvalid;
wire [31:0]  fdma_waddr;
reg         fdma_wareq;
wire        fdma_wbusy;
wire [127:0] fdma_wdata;
wire [15:0]  fdma_wsize;
wire         fdma_wvalid;
wire [0:0]   fdma_rstn;
wire         ui_clk;

parameter TEST_MEM_SIZE    = 32'd1024*1024*1024;//4GB
parameter FDMA_BURST_LEN   = 16'd512;
parameter ADDR_MEM_OFFSET = 0;
parameter ADDR_INC = FDMA_BURST_LEN * 16;

parameter WRITE1 = 0;
parameter WRITE2 = 1;
parameter WAIT    = 2;
parameter READ1   = 3;
parameter READ2   = 4;

reg [31:0] t_data;
reg [31:0] fdma_waddr_r;
reg [2 :0] T_S = 0;

assign fdma_waddr = fdma_waddr_r + ADDR_MEM_OFFSET;
assign fdma_raddr = fdma_waddr;

assign fdma_wsize = FDMA_BURST_LEN;
assign fdma_rsize = FDMA_BURST_LEN;
assign fdma_wdata = {t_data,t_data,t_data,t_data};

reg [8:0] rst_cnt = 0;

always @(posedge ui_clk)
    if((fdma_rstn & C0_init_calib_complete)==1'b0)begin
        rst_cnt <=0;
    end
    else begin
        if(rst_cnt[8] == 1'b0)
            rst_cnt <= rst_cnt + 1'b1;
        else
            rst_cnt <= rst_cnt;
    end
end

always @(posedge ui_clk)begin
    if(rst_cnt[8] == 1'b0)begin
        T_S <=0;
        fdma_wareq <= 1'b0;
    end
end

```

```

    fdma_rareq  <= 1'b0;
    t_data<=0;
    fdma_waddr_r <=0;
end
else begin
    case(T_S)
    WRITE1:begin
        if(fdma_waddr_r>TEST_MEM_SIZE) fdma_waddr_r<=0;
        if(!fdma_wbusy)begin
            fdma_wareq  <= 1'b1;
            t_data  <= 0;
        end
        if(fdma_wareq&&fdma_wbusy)begin
            fdma_wareq  <= 1'b0;
            T_S          <= WRITE2;
        end
    end
    WRITE2:begin
        if(!fdma_wbusy) begin
            T_S <= WAIT;
            t_data  <= 32'd0;
        end
        else if(fdma_wvalid) begin
            t_data <= t_data + 1'b1;
        end
    end
    WAIT:begin//not needed
        T_S <= READ1;
    end
    READ1:begin
        if(!fdma_rbusy)begin
            fdma_rareq  <= 1'b1;
            t_data  <= 0;
        end
        if(fdma_rareq&&fdma_rbusy)begin
            fdma_rareq  <= 1'b0;
            T_S          <= READ2;
        end
    end
    READ2:begin
        if(!fdma_rbusy) begin
            T_S <= WRITE1;
            t_data  <= 32'd0;
            fdma_waddr_r  <= fdma_waddr_r + ADDR_INC;//128/8=16
        end
        else if(fdma_rvalid) begin
            t_data <= t_data + 1'b1;
        end
    end
    default:
        T_S <= WRITE1;
    endcase
end
end

wire test_error = (fdma_rvalid && (t_data[15:0] != fdma_rdata[15:0]));

ila_0 ila_dbg (
    .clk(ui_clk),
    .probe0({fdma_wdata[15:0],fdma_wareq,fdma_wvalid,fdma_wbusy}),
    .probe1({fdma_rdata[15:0],t_data[15:0],fdma_rvalid,fdma_rbusy,T_S,test_error})
);

```

```

system system_i
(
    .C0_DDR4_act_n(C0_DDR4_act_n),
    .C0_DDR4_adr(C0_DDR4_adr),
    .C0_DDR4_ba(C0_DDR4_ba),
    .C0_DDR4_bg(C0_DDR4_bg),
    .C0_DDR4_ck_c(C0_DDR4_ck_c),
    .C0_DDR4_ck_t(C0_DDR4_ck_t),
    .C0_DDR4_cke(C0_DDR4_cke),
    .C0_DDR4_cs_n(C0_DDR4_cs_n),
    .C0_DDR4_dm_n(C0_DDR4_dm_n),
    .C0_DDR4_dq(C0_DDR4_dq),
    .C0_DDR4_dqs_c(C0_DDR4_dqs_c),
    .C0_DDR4_dqs_t(C0_DDR4_dqs_t),
    .C0_DDR4_odt(C0_DDR4_odt),
    .C0_DDR4_reset_n(C0_DDR4_reset_n),
    .FDMA_S_0_fdma_raddr(fdma_raddr),
    .FDMA_S_0_fdma_rareq(fdma_rareq),
    .FDMA_S_0_fdma_rbusy(fdma_rbusy),
    .FDMA_S_0_fdma_rdata(fdma_rdata),
    .FDMA_S_0_fdma_rready(1'b1),
    .FDMA_S_0_fdma_rsize(fdma_rsize),
    .FDMA_S_0_fdma_rvalid(fdma_rvalid),
    .FDMA_S_0_fdma_waddr(fdma_waddr),
    .FDMA_S_0_fdma_wareq(fdma_wareq),
    .FDMA_S_0_fdma_wbusy(fdma_wbusy),
    .FDMA_S_0_fdma_wdata(fdma_wdata),
    .FDMA_S_0_fdma_wready(1'b1),
    .FDMA_S_0_fdma_wsize(fdma_wsize),
    .FDMA_S_0_fdma_wvalid(fdma_wvalid),
    .C0_init_calib_complete(C0_init_calib_complete),
    .ddr_rst(ddr_rst),
    .fdma_rstn(fdma_rstn),
    .sysclk_clk_p(sysclk_p),
    .sysclk_clk_n(sysclk_n),

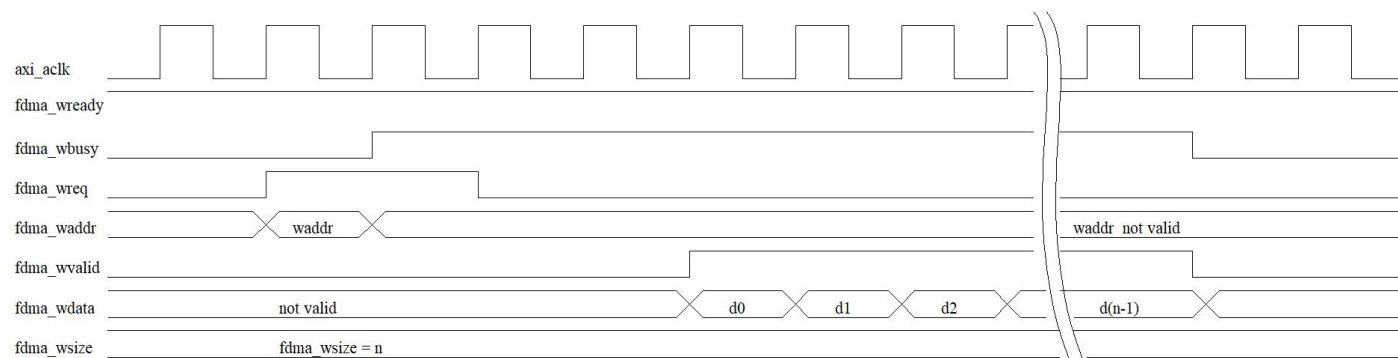
    .ui_clk(ui_clk));
endmodule

```

9.4 程序分析

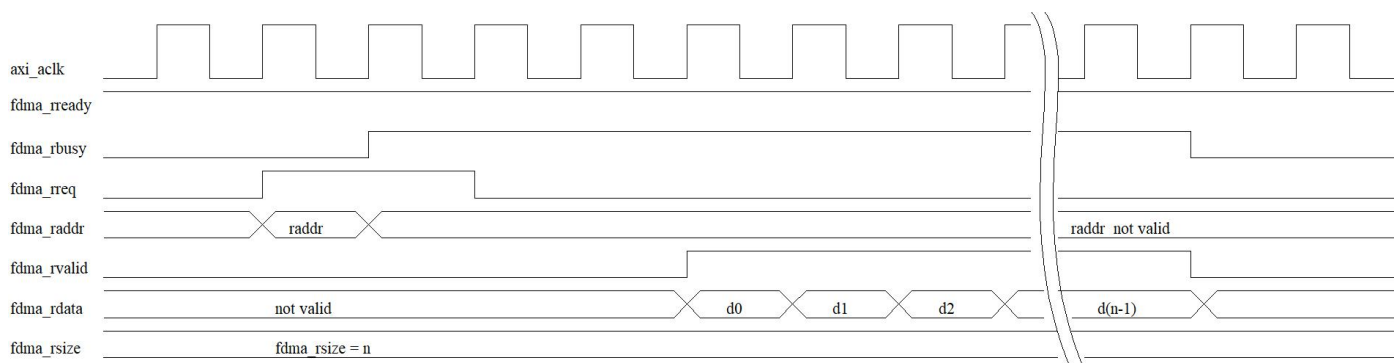
以下关于程序的分析，“01 使用 FDMA 读写 AXI-BRAM”一样，重复是为了读者不会因此遗漏这部分重要信息。

9.4.1 FDMA 的写时序



fdma_wready 设置为 1，当 fdma_wbusy=0 的时候代表 FDMA 的总线非忙，可以进行一次新的 FDMA 传输，这个时候可以设置 fdma_wreq=1，同时设置 fdma burst 的起始地址和 fdma_wsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_wvalid=1 的时候需要给出有效的数据，写入 AXI 总线。当最后一个数写完后，fdma_wvalid 和 fdma_wbusy 变为 0。

9.4.2 FDMA 的读时序



fdma_rready 设置为 1, 当 fdma_rbusy=0 的时候代表 FDMA 的总线非忙, 可以进行一次新的 FDMA 传输, 这个时候可以设置 fdma_rreq=1, 同时设置 fdma burst 的起始地址和 fdma_rsize 本次需要传输的数据大小(以 bytes 为单位)。当 fdma_rvalid=1 的时候需要给出有效的数据, 写入 AXI 总线。当最后一个数写完后, fdma_rvalid 和 fdma_rbusy 变为 0。

9.4.3 读写状态机

```

case (T_S)
WRITE1:begin
    if (fdma_waddr_r==TEST_MEM_SIZE) fdma_waddr_r<=0; 当所有的数据测试完成, 设置起始地址0, 循环测试
        if (!fdma_wbusy)begin 等待AXI总线不忙
            fdma_wareq <= 1'b1; 请求写数据
            t_data <= 0;
        end
        if (fdma_wareq&&fdma_wbusy)begin 当wbusy忙说明FDMA已经接收到请求命名
            fdma_wareq <= 1'b0;
            T_S <= WRITE2; 进入写数据阶段
        end
    end
WRITE2:begin
    if (!fdma_wbusy) begin 写完数据, 并且AXI总线空闲, 进入WAIT状态
        T_S <= WAIT; FDMA是可以支持同时读写的, 这里只是为了测试方便, 先写后读
        t_data <= 32'd0;
    end
    else if (fdma_rvalid) begin 只要rvalid=1就可以一直写入数据
        t_data <= t_data + 1'b1; 这里写入加计数
    end
end
WAIT:begin//not needed
    T_S <= READ1; 进入读状态, 这个状态也可以不要
end
READ1:begin
    if (!fdma_rbusy)begin 等待AXI总线不忙
        fdma_rareq <= 1'b1; 请求读数据
        t_data <= 0;
    end
    if (fdma_rareq&&fdma_rbusy)begin 当rbusy忙说明FDMA已经接收到请求命名
        fdma_rareq <= 1'b0;
        T_S <= READ2; 进入读数据阶段
    end
end
READ2:begin
    if (!fdma_rbusy) begin 当非忙, 代表读完成了
        T_S <= WRITE1;
        t_data <= 32'd0;
        fdma_waddr_r <= fdma_waddr_r + ADDR_INC;//128/8=16 一次FDMA传输完成, 重新计算下一次FDMA的起始地址
    end
    else if (fdma_rvalid) begin 只要rvalid=1就可以一直读数据
        t_data <= t_data + 1'b1; 读状态里面也有一个计数器, 用于比较读出的数据是否正确
    end
end
default:
    T_S <= WRITE1;
endcase

```

9.5 实验结果

9.5.1 仿真结果

1: 仿真 tb 文件

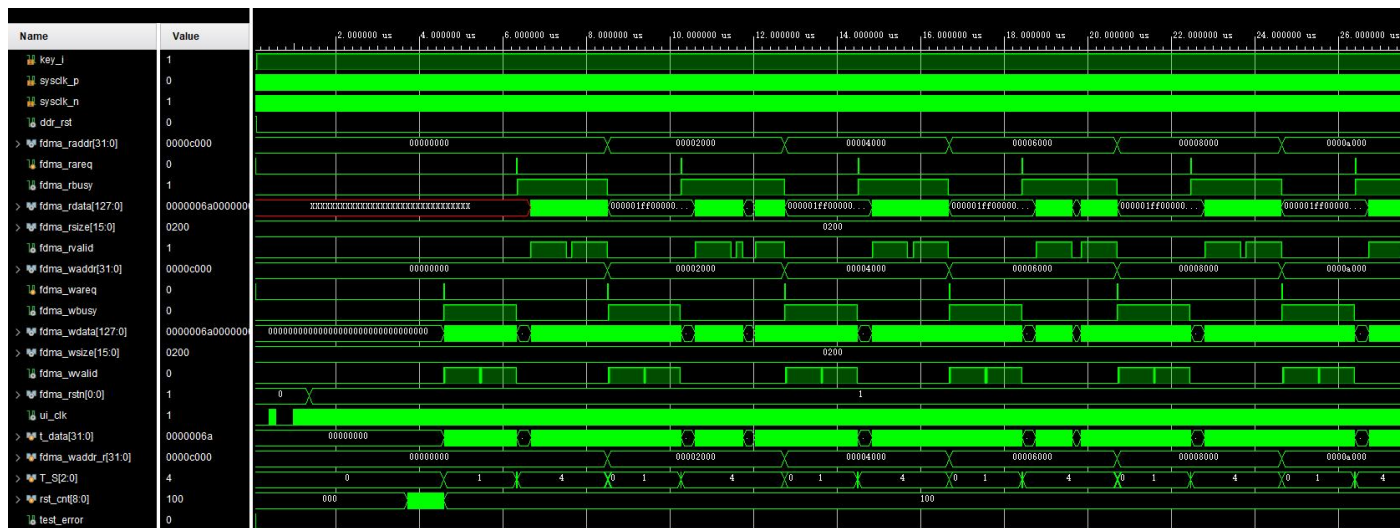
DDR4 的仿真文件在对应的 FPGA 工程路径 uisrc/02_sim 路径下，把仿真文件添加进来

ZU07A > src > fpga > u2_fdma_ddr_test > uisrc > 02_sim				▼
名称	修改日期	类型	大小	
arch_defines.v	2020/11/4 12:43	Verilog File	1 KB	
arch_package.sv	2020/11/4 12:43	SV 文件	54 KB	
ddr4_model.sv	2020/11/4 12:43	SV 文件	184 KB	
ddr4_sdram_model_wrapper.sv	2020/11/4 12:43	SV 文件	4 KB	
interface.sv	2020/11/4 12:43	SV 文件	1 KB	
MemoryArray.sv	2020/11/4 12:43	SV 文件	25 KB	
proj_package.sv	2020/11/4 12:43	SV 文件	18 KB	
sim_tb_top.sv	2021/4/28 8:30	SV 文件	22 KB	
StateTable.sv	2020/11/4 12:43	SV 文件	9 KB	
StateTableCore.sv	2020/11/4 12:43	SV 文件	205 KB	
timing_tasks.sv	2020/11/4 12:43	SV 文件	23 KB	

The screenshot displays the simulation environment. On the left, the 'Sources' panel shows the project hierarchy under 'Simulation Sources (2)', including 'sim_1 (2)' and 'sim_tb_top (sim_tb_top.sv) (95)'. The 'sim_tb_top.sv' file is selected, showing its contents in the main editor. The code defines a testbench for the DDR4 model, including a loop for writing data to memory and a loop for reading data back. The code is written in SystemVerilog and includes various assertions and delays.

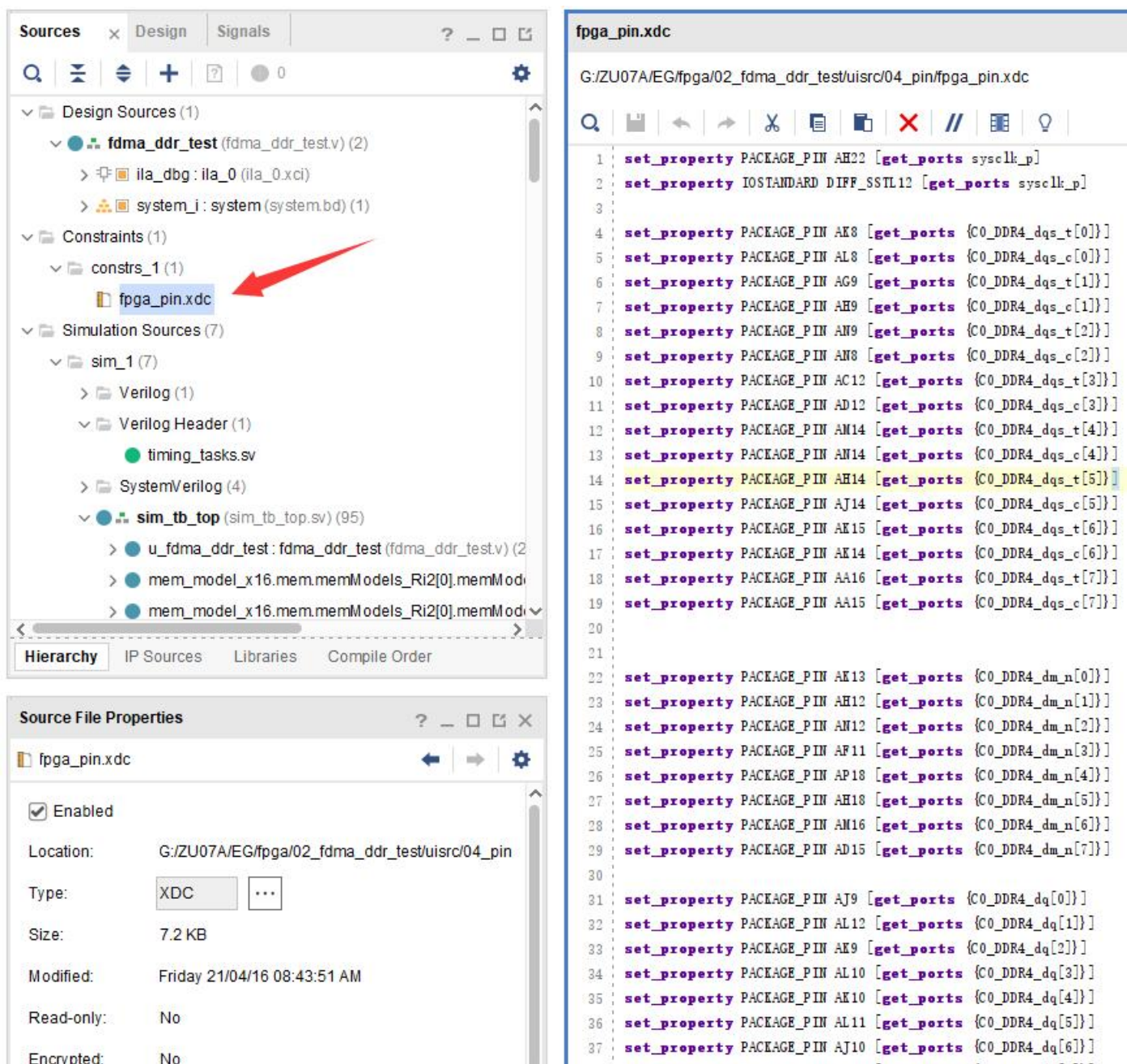
2: 仿真测试

可以看到 DDR4 的仿真速度还是很快，在 6us 左右就可以得到仿真结果了，可以看到 test_error 信号一直是低电平。

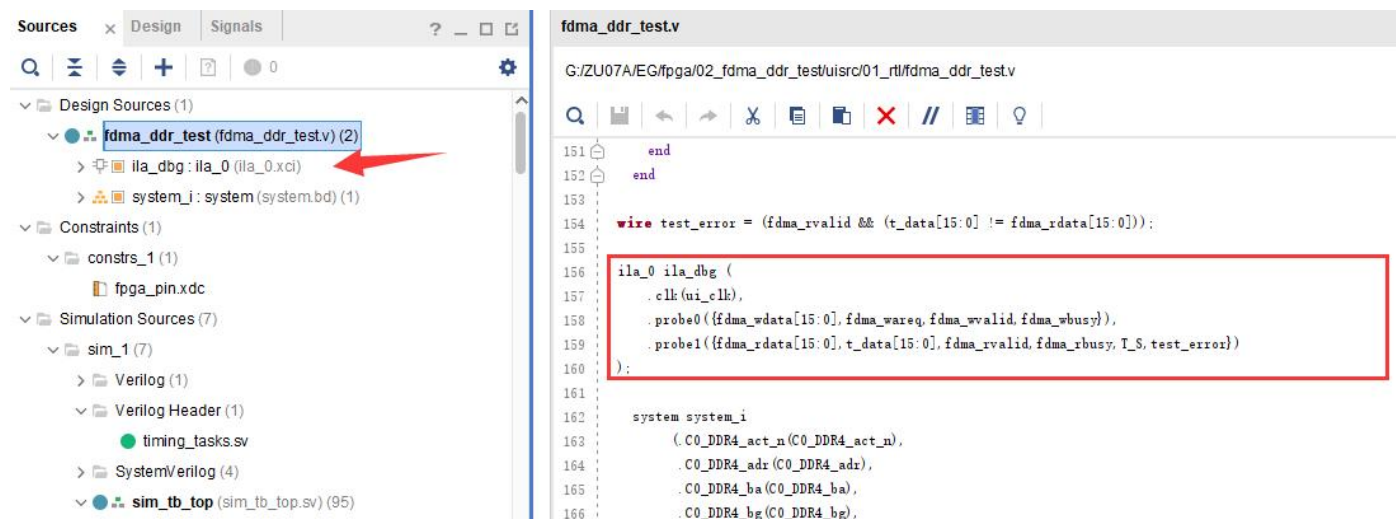


9.5.2 板级测试

添加 `fpga_pin.xdc` 文件，不同的开发板 IO 的约束定义一样，下图仅提供演示，具体以自己拿到的配套资料工程路径下的 `uisrc/04_pin/fpga_pin.xdc` 为准。当然读者也可以根据原理图自己定义约束。



通过 ila 在线逻辑分析仪 IP-CORE, 在线观察波形



通过在线逻辑分析仪查看结果, 可以看到 `test_error` 信号一直低电平



10PL 读写 PS 端 DDR(FDMA)

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

10.1 概述

FDMA 是米联客的基于 AXI4 总线协议定制的一个 DMA 控制器。有了这个 IP 我们可以统一实现用 FPGA 代码直接读写 PL 的 DDR 或者 PS 的 DDR。本文中 FDMA 的 IP 是开源的, 在配套 FPGA 工程的 uisrc/ip 路径下可以找到源码。本文的 IP 已经利用 VIVADO 做了图形化的封装, 所以可以直接通过图形化连线设计, 使用非常方便。

本文实验目的:

- 1: 利用米联客自定一定 FDMA2.0/3.0 版本搭建 SOC 工程(最新发布的版本是 3.0)
- 2: 编写 FPGA 测试代码实现, PL 写入数据到 PS DDR 然后再读出 PS DDR 中的数据, 对比是否正确。

为了让 PS 的 DDR 可以运行, 必须新建一个 vitis-sdk 工程, 这个工程主要是为了初始化 PS DDR, 我们可以简单新建一个自带的 hello 工程。

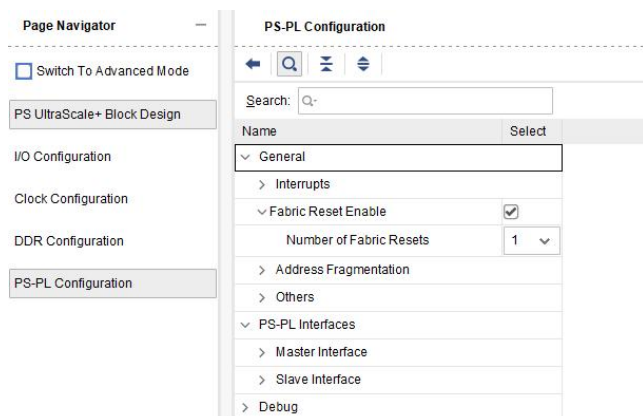
10.2 搭建 SOC 系统工程

新建一个名为 soc_prj 的工程, 之后创建一个 BD 文件, 并命名为 system, 添加并且配置好 ZYNQ Ultrascale+ MPSOC IP。读者需要根据自己的硬件类型配置好输入时钟频率、内存型号、串口, 连接时钟等。新手不清楚这些内容个, 请参考“3-2-01_ex_soc_base_07a-eg.pdf” “01 HelloWorld/DDR/网口测试” 这篇文章。

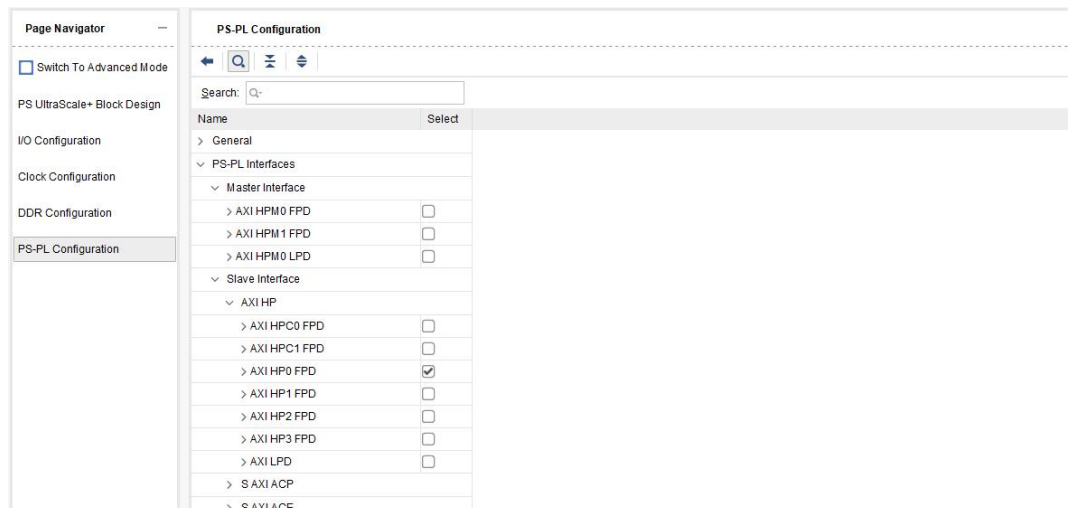


10.2.1 PS 部分设置

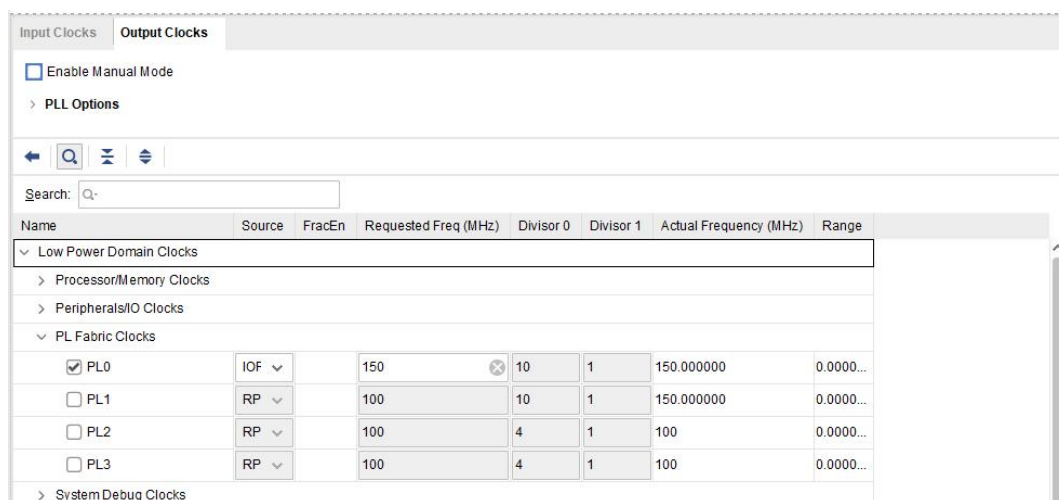
1: PS 复位设置



2: PS AXI HP0 FPD 设置



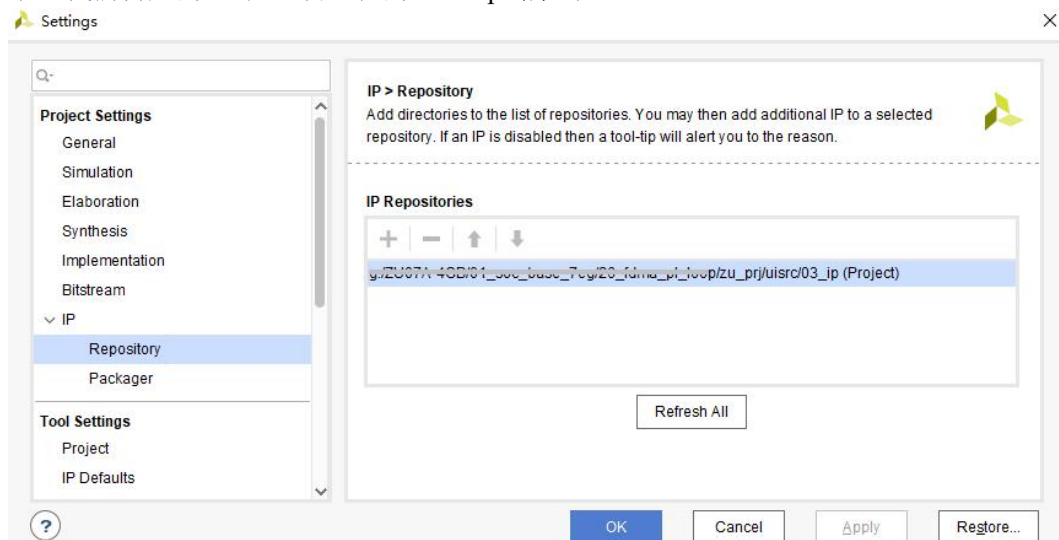
3:PL 输出时钟设置



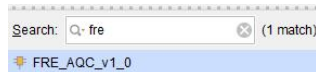
10.2.2 添加自定义 IP

1:设置 IP 路径

本文中是我们第一次在 BD 图形化设计中添加自定义的 IP，自定义的 IP 需要设置 IP 路径才能被识别到。默认情况下，我们自定的 IP 在配套工程的 uisrc/ip 路径下：

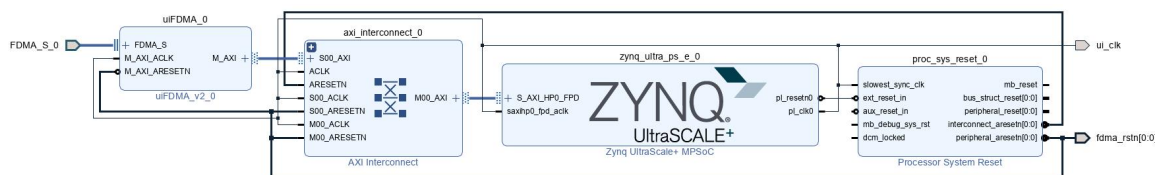


2:添加 IP

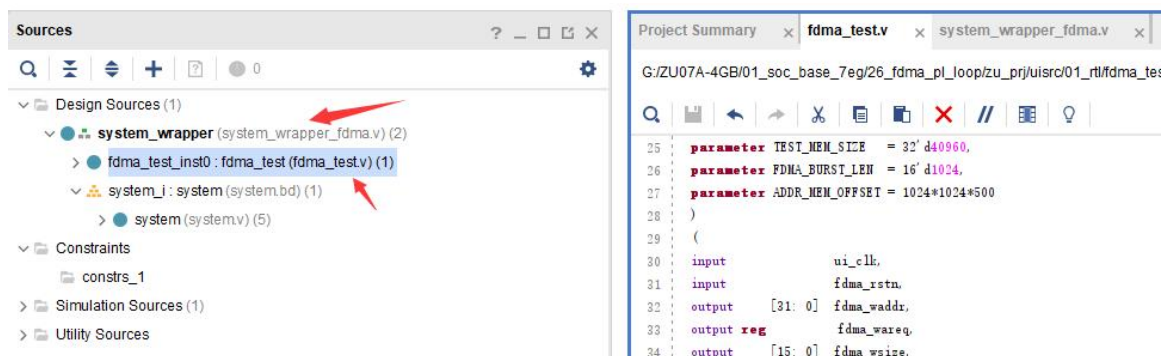


ENTER to select, ESC to cancel, Ctrl+Q for IP details

10.2.3PL 图像化编程



10.2.4 添加 FPGA FDMA 读写代码



上图中源码在配套 fpga 工程的 uisrc/rtl 路径中，本文中我们修改了默认的系统_wrapper.v 文件名为 system_wrapper_fdma.v 这样的好处是对默认文件和手动修改过的文件加以区分，以免辛苦修改的代码一不小心，被软件自动更新替换掉。

关键的代码为 fdma_test.v。在这个程序中，写入一定数据到 DDR 中，然后再读出，对比是否有错误，几个关键参数：

TEST_MEM_SIZE: 定义了测试内从空间的大小，以 byte 为单位,是整数倍的 FDMA_BURST_LEN *(fdma_wdata/8)。

FDMA_BURST_LEN:定义每次 FDMA 传输的长度，这个长度是整数倍的 fdma_wdata 或者 fdma_rdata。

ADDR_MEM_OFFSET:代码了内从访问的起始地址。

```
`timescale 1ns / 1ns
/*
Company : Liyang Milian Electronic Technology Co., Ltd.
Brand: 米联客(milianke)
Technical forum:uisrc.com
taobao: https://milianke.taobao.com   https://osrc.taobao.com
jd:https://milianke.jd.com
Create Date: 2019/12/17
Module Name: fdma_test
Description:
Copyright: Copyright (c) milianke
Revision: 1.0
Signal description:
1) _i input
2) _o output
3) _n activ lowpai
4) _dg debug signal
5) _r delay or register
6) _s state mechine
*/
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module fdma_test#
(
parameter TEST_MEM_SIZE    = 32'd40960,
parameter FDMA_BURST_LEN   = 16'd1024,
parameter ADDR_MEM_OFFSET = 1024*1024*500
)
(
input          ui_clk,
input          fdma_rstn,
```

```

output      [31: 0]  fdma_waddr,
output reg          fdma_wareq,
output      [15: 0]  fdma_wsize,
input        fdma_wbusy,
output reg [127:0]  fdma_wdata,
input        fdma_wvalid,
output  reg          fdma_wready,

output      [31: 0]  fdma_raddr,
output reg          fdma_rareq,
output      [15: 0]  fdma_rsize,
input        fdma_rbusy,
input      [127:0]  fdma_rdata,
input        fdma_rvalid,
output reg          fdma_rready
);

parameter WRITE1 = 0;
parameter WRITE2 = 1;
parameter READ1  = 2;
parameter READ2  = 3;

reg [31: 0] fdma_waddr_r;
reg [16 :0] fdma_rcnt = 0;
reg [2  :0] T_S = 0;

assign fdma_waddr = fdma_waddr_r + ADDR_MEM_OFFSET;
assign fdma_raddr = fdma_waddr;

assign fdma_wsize = FDMA_BURST_LEN;
assign fdma_rsize = FDMA_BURST_LEN;

reg [10:0] rst_cnt = 0;

always @(posedge ui_clk)
    if(fdma_rstn == 1'b0)begin
        rst_cnt <= 0;
    end
    else begin
        if(rst_cnt[10] == 1'b0)
            rst_cnt <= rst_cnt + 1'b1;
        else
            rst_cnt <= rst_cnt;
        end
    end

always @(posedge ui_clk)begin
    if(rst_cnt[10] == 1'b0)begin
        T_S <= 0;
        fdma_wareq <= 1'b0;
        fdma_rareq <= 1'b0;
        fdma_wready <= 1'b0;
        fdma_rready <= 1'b0;
        fdma_wdata <= 0;
        fdma_waddr_r <= 0;
    end
    else begin
        case(T_S)
            WRITE1:begin
                if(fdma_waddr_r>TEST_MEM_SIZE) fdma_waddr_r<=0;
                if(!fdma_wbusy)begin
                    fdma_wareq <= 1'b1;
                    fdma_wready <= 1;
                    fdma_wdata <= 0;
                end
                if(fdma_wareq&&fdma_wbusy)begin
                    fdma_wareq <= 1'b0;
                    T_S <= WRITE2;
                end
            end
            WRITE2:begin
                if(!fdma_wbusy) begin
                    T_S <= READ1;
                    fdma_wready <= 0;
                    fdma_wdata <= 32'd0;
                end
            end
            else if(fdma_wvalid) begin
                fdma_wdata <= fdma_wdata + 1'b1;
            end
        endcase
    end
end

```

```

        end
    end

    READ1:begin
        if(!fdma_rbusy)begin
            fdma_rareq  <= 1'b1;
            fdma_rready <= 1;
            fdma_rcnt   <= 0;
        end
        if(fdma_rareq&&fdma_rbusy)begin
            fdma_rareq  <= 1'b0;
            T_S         <= READ2;
        end
    end

    READ2:begin
        if(!fdma_rbusy) begin
            T_S <= WRITE1;
            fdma_rready <= 0;
            fdma_rcnt   <= 32'd0;
            fdma_waddr_r <= fdma_waddr_r + FDMA_BURST_LEN*128/8;
        end
        else if(fdma_rvalid) begin
            fdma_rcnt <= fdma_rcnt + 1'b1;
        end
    end

    default:
        T_S <= WRITE1;
    endcase
end
end

wire test_error = ((fdma_rready&&fdma_rvalid) && (fdma_rcnt[15:0] != fdma_rdata[15:0]));

ila_0 ila_dbg (
    .clk(ui_clk),
    .probe0({fdma_waddr[15:0],fdma_wdata[15:0],fdma_wareq,fdma_wvalid,fdma_wready,fdma_wbusy}),
    .probe1({fdma_rdata[15:0],fdma_rcnt[15:0],fdma_rvalid,fdma_rready,fdma_rbusy,T_S,test_error})
);

endmodule

```

10.2.5 设置地址分配

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/uiFDMA_0					
/uiFDMA_0/M_AXI (32 address bits : 4G)					
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_GIC	0xF900_0000	1M	0xF90F_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_LPS_OCM	0xFF00_0000	16	0xFFFF_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_DDR_LOW	0x0000_0000	2G	0x7FFF_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_QSPI	0xC000_0000	51	0xDFFF_FFFF
Excluded (1)					
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_DDR_HIGH	0x8_0000_0000	4G	0x8_FFFF_FFFF

10.2.6 编译并导出平台文件

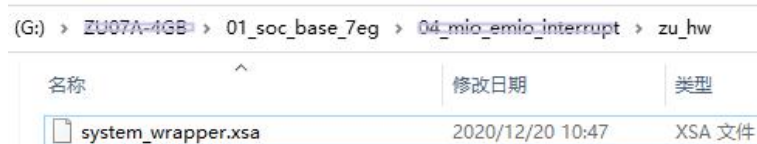
1:单击 Block 文件→右键→Generate the Output Products→Global→Generate。

2:单击 Block 文件→右键→ Create a HDL wrapper(生成 HDL 顶层文件)→Let vivado manager wrapper and auto-update(自动更新)。

3:生成 Bit 文件。

4:导出到硬件: File→Export Hardware→Include bitstream

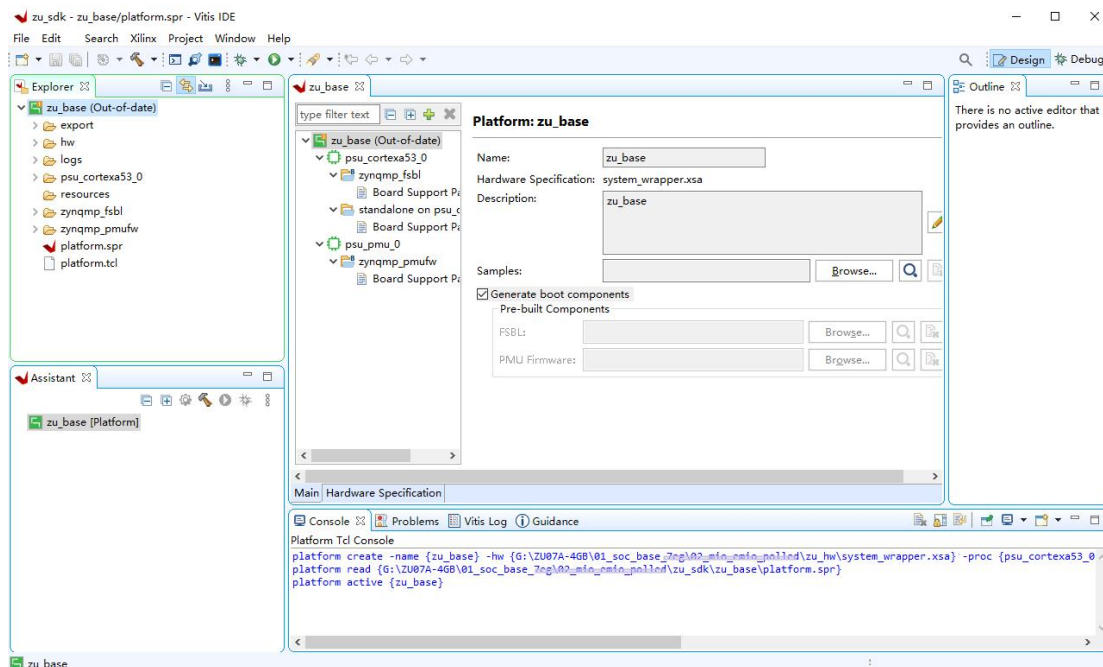
5:导出完成后, 对应工程路径的 zu_hw 路径下有硬件平台文件: system_wrapper.xsa 的文件。根据硬件平台文件 system_wrapper.xsa 来创建需要 Platform 平台。



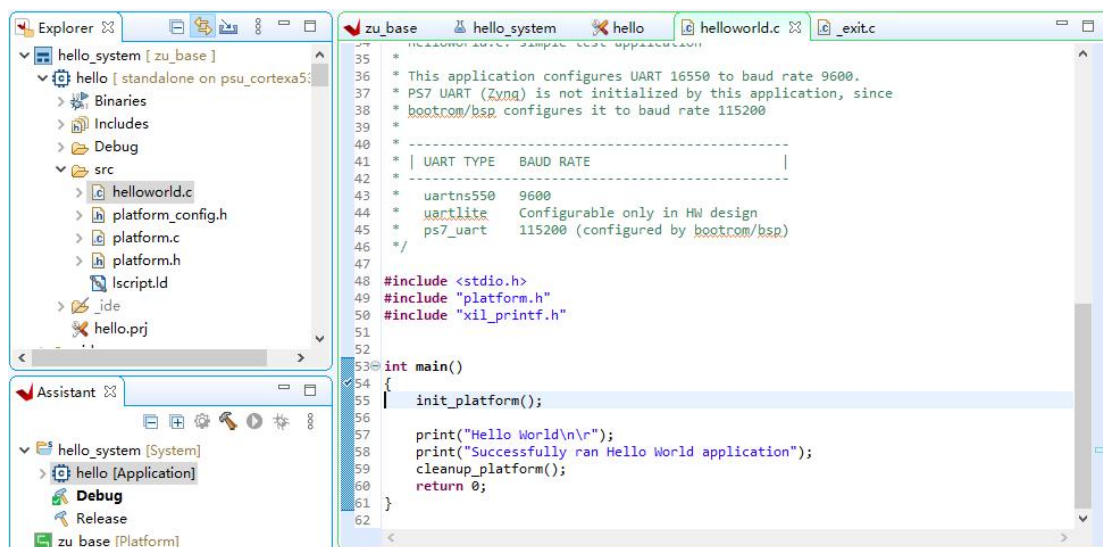
10.3 搭建 Vitis-sdk 工程

创建 zu_base sdk platform 和 APP 工程的过程不再重复，可以阅读本章节 01~05 相关 demo。以下给出创建好 zu_base sdk platform 的截图和对应工程 APP 的截图。

10.3.1 创建 SDK Platform 工程

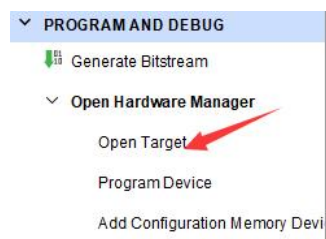


10.3.2 创建 hello APP 工程

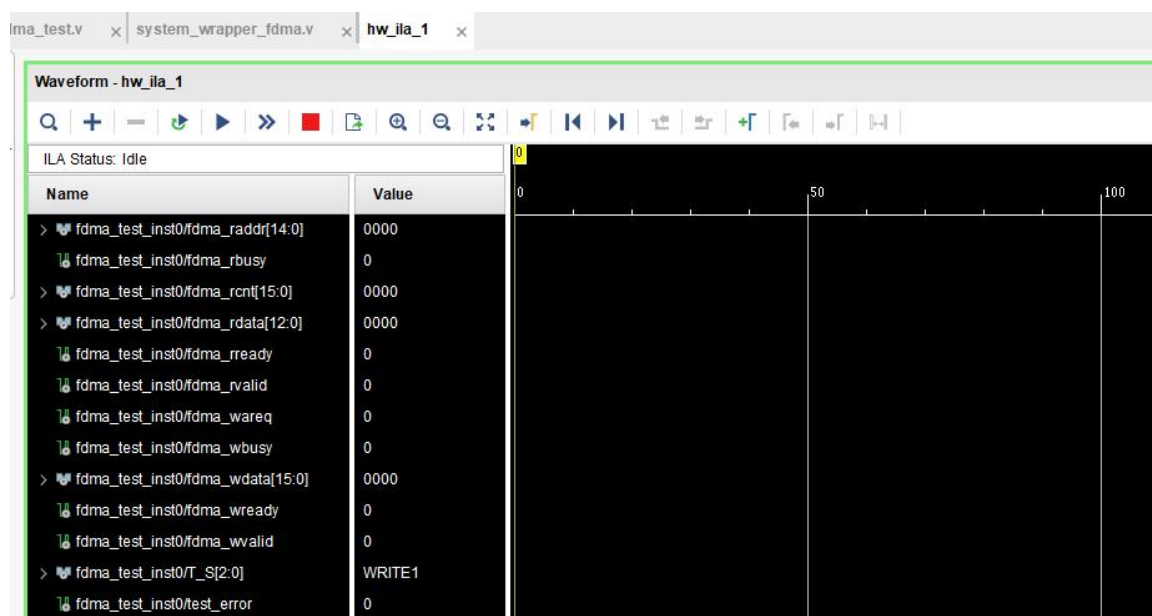
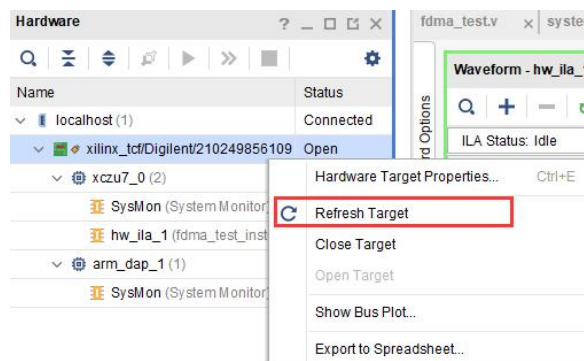


10.4 实验结果

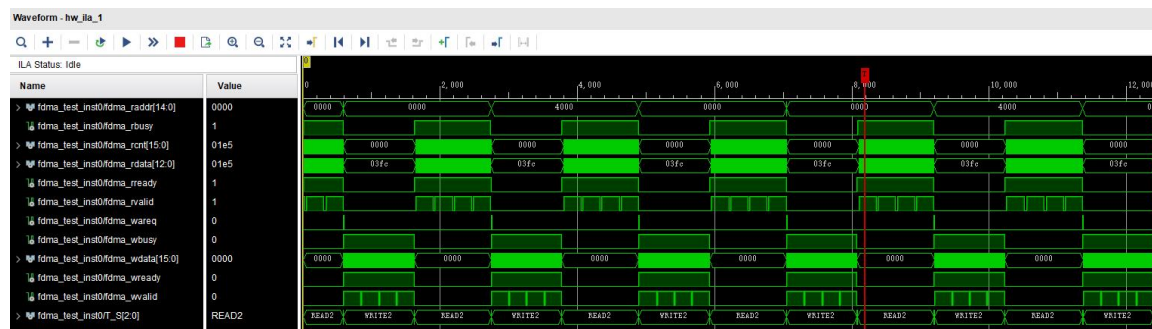
- 1:先运行 hello app
- 2:打开设备



3:如果没有出来下图的 hw_ila 波形窗口，右击刷新



4:观察在线逻辑分析结果



5:写入过程



6:读出过程

Name	Value	1, 044	1, 046	1, 048	1, 050	1, 052	1, 054	1, 056	1, 058	1, 060	1, 062								
> fdma_test_inst0fdma_raddr[14:0]	0000	0000																	
fdma_test_inst0fdma_rbusy	1																		
> fdma_test_inst0fdma_rcnt[15:0]	0013	0000	0001	0002	0003	0004	0005	0006	0007	0008	0009	000a	000b	000c	000d	000e	000f	0010	
> fdma_test_inst0fdma_rdata[12:0]	0013	03fc	0000	0001	0002	0003	0004	0005	0006	0007	0008	0009	000a	000b	000c	000d	000e	000f	0010
fdma_test_inst0fdma_ready	1																		
fdma_test_inst0fdma_rvalid	1																		

11AXI-Lite 自定义 AXI_GPIO

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频教程、答疑解惑!

11.1 概述

在前文中我们学习了 AXI 总线协议, 而且通过 VIVADO 自定义了 AXI-LITE 总线协议的 IP CORE, 并且实现了寄存器的读写。

那么在实际的应用中, 如果我们 ARM 的 IO 不够用了, 除了在前文中使用官方自带的 AXI-GPIO, 我们自己也可以定义 AXI-GPIO IP CORE。

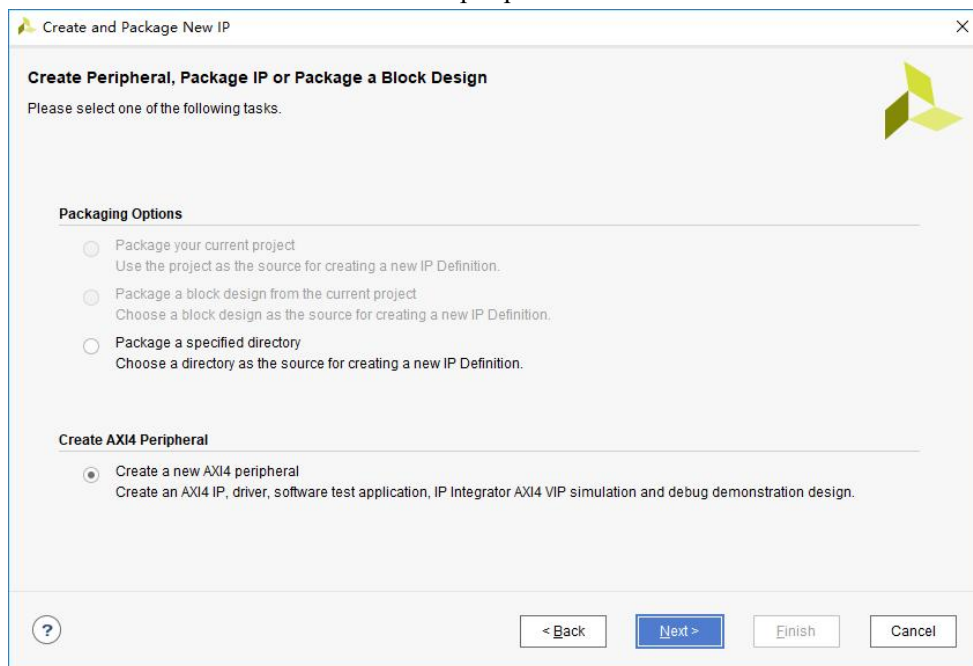
本文实验目的:

- 1: 通过前文的学习, 把掌握的自定义 AXI-LITE-SLAVE 寄存器读写方法, 用于引出扩展 PL 的 IO
- 2: 通过 VITIS-SDK 实现对自定义 IP 中寄存器的读写访问, 以此实现 PL IO 的控制。

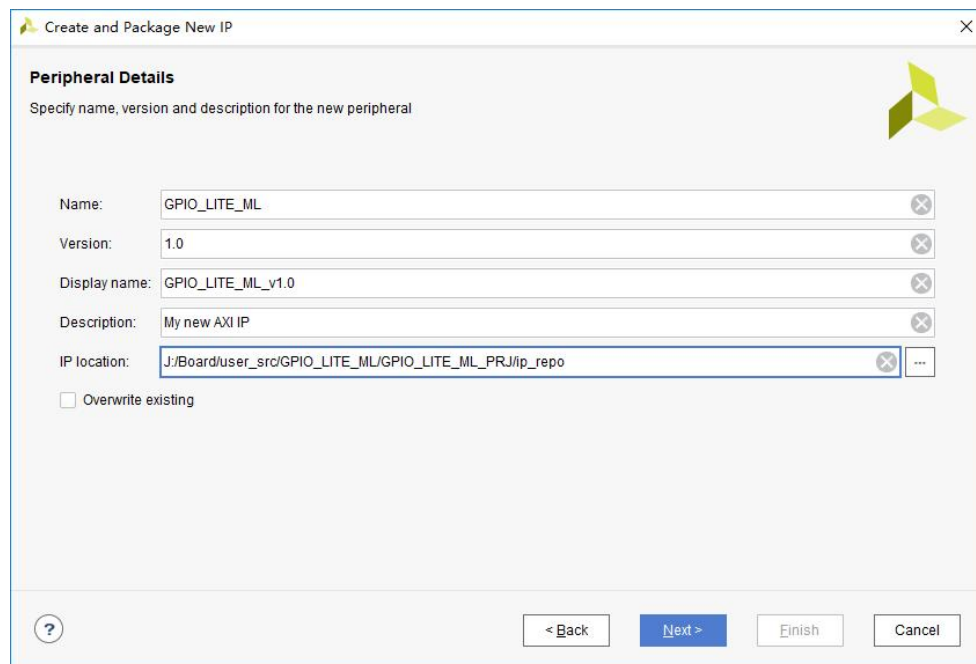
11.2 创建 IP

11.2.1 利用模板创建 AXI-Lite IP

- 1: 打开 VIVADO 软件, 新建一个工程。
- 2: 单击 Tools → Create and Package NEW IP。
- 3: 单击 Next, 选择 Create a new AXI4 peripheral, 单击 Next。



- 4: 输入要创建的 IP 名字, 此处命名为 GPIO_LITE_ML, 选择保存路径, 单击 Next。



Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

Name: GPIO_LITE_ML

Version: 1.0

Display name: GPIO_LITE_ML_v1.0

Description: My new AXI IP

IP location: J:/Board/user_src/GPIO_LITE_ML/GPIO_LITE_ML_PRJ/ip_repo

☐ Overwrite existing

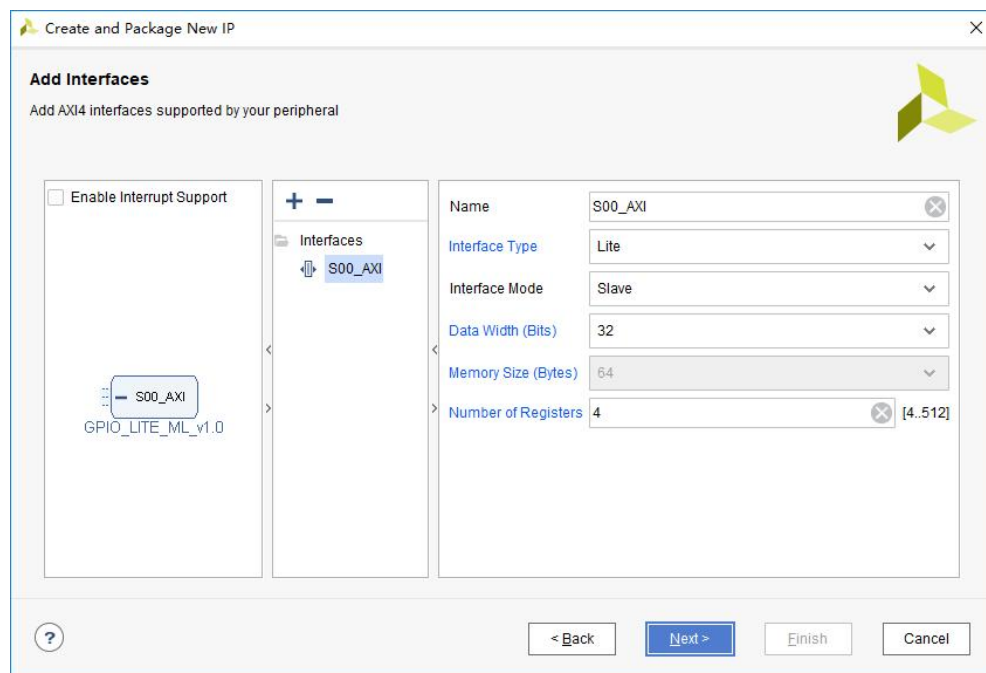
< Back Next > Finish Cancel

5: Name → S00_AXI;

Interface Type(接口类型) → Lite;

Data Width(Bits)(数据位宽) → 32 位;

Number of Registers(寄存器数量) → 4 ; 单击 next。



Create and Package New IP

Add Interfaces
Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces

- S00_AXI

Name: S00_AXI

Interface Type: Lite

Interface Mode: Slave

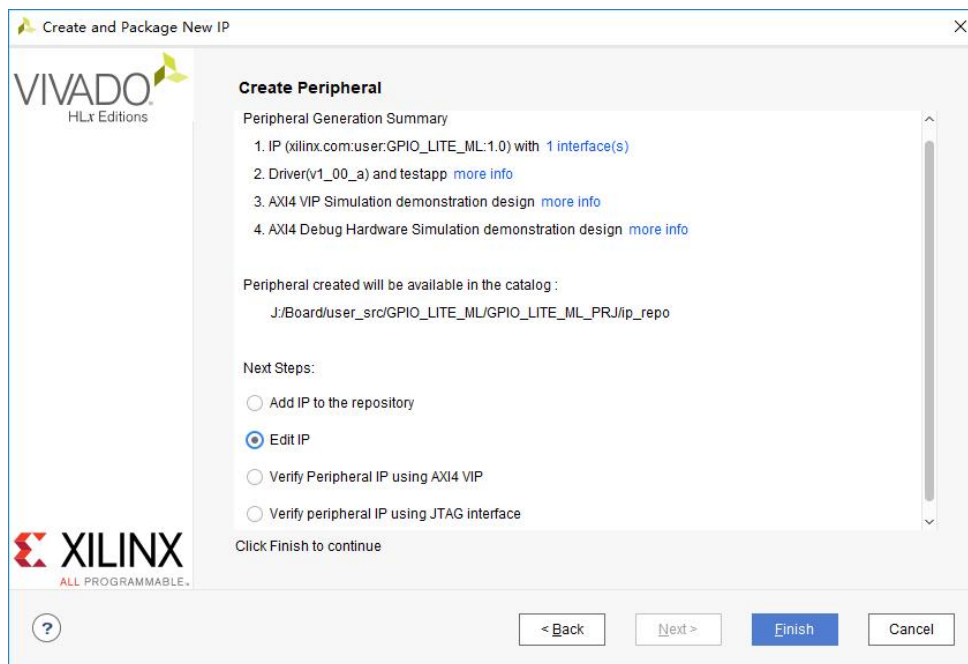
Data Width (Bits): 32

Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

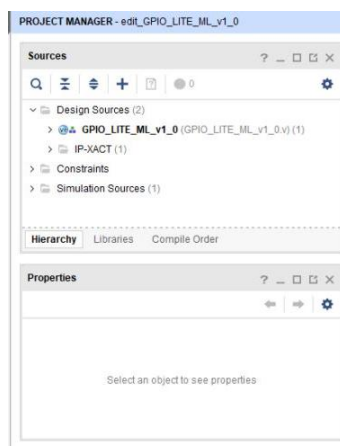
< Back Next > Finish Cancel

6: 选择 Edit IP, 点击 Finish 按钮。软件自动打开一个编辑 IP 的工程, 即 edit_GPIO_LITE_ML_V1_0.xpr 工程。

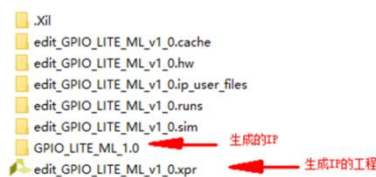


11.2.2 修改 IP 源码

1:打开的 edit_GPIO_LITE_ML_V1_0.xpr 工程界面如下。



查看生成 IP 的文件夹



2: 现在生成的 IP 需要进行修改才能满足我们使用需求。选中 Project Manager，双击 GPIO_LITE_ML_v1_0_S00_AXI_inst，做如下更改。

修改 1:

```
parameter integer C_S_AXI_ADDR_WIDTH = 4
)
(
// Users to add ports here
output wire [3:0]GPIO_LED,
// User ports ends
// Do not modify the ports beyond this line
```

修改 2:

```
// Add user logic here
assign GPIO_LED[3:0] = slv_reg0[3:0];
// User logic ends

endmodule
```

将 slv_reg0 的值赋值给了用户输出逻辑，当我们向 slv_reg0 写入数据的时候，也就相当于向 GPIO_LED 赋值。更改后的文件如下所示。

```
`timescale 1 ns / 1 ps

module GPIO_LITE_ML_v1_0_S00_AXI #
(
    // Users to add parameters here

    // User parameters ends
    // Do not modify the parameters beyond this line

    // Width of S_AXI data bus
    parameter integer C_S_AXI_DATA_WIDTH  = 32,
    // Width of S_AXI address bus
    parameter integer C_S_AXI_ADDR_WIDTH  = 4
)
(
    // Users to add ports here
    output wire [3:0]GPIO_LED,
    // User ports ends
    // Do not modify the ports beyond this line

    // Global Clock Signal
    input wire  S_AXI_ACLK,
    // Global Reset Signal. This Signal is Active LOW
    input wire  S_AXI_ARESETN,
    // Write address (issued by master, accepted by Slave)
    input wire [C_S_AXI_ADDR_WIDTH-1 : 0] S_AXI_AWADDR,
    // Write channel Protection type. This signal indicates the
    // privilege and security level of the transaction, and whether
    // the transaction is a data access or an instruction access.
    input wire [2 : 0] S_AXI_AWPROT,
    // Write address valid. This signal indicates that the master signaling
    // valid write address and control information.
    input wire  S_AXI_AWVALID,
    // Write address ready. This signal indicates that the slave is ready
    // to accept an address and associated control signals.
    output wire  S_AXI_AWREADY,
    // Write data (issued by master, accepted by Slave)
    input wire [C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_WDATA,
    // Write strobes. This signal indicates which byte lanes hold
    // valid data. There is one write strobe bit for each eight
    // bits of the write data bus.
    input wire [(C_S_AXI_DATA_WIDTH/8)-1 : 0] S_AXI_WSTRB,
    // Write valid. This signal indicates that valid write
    // data and strobes are available.
    input wire  S_AXI_WVALID,
    // Write ready. This signal indicates that the slave
    // can accept the write data.
    output wire  S_AXI_WREADY,
    // Write response. This signal indicates the status
    // of the write transaction.
    output wire [1 : 0] S_AXI_BRESP,
    // Write response valid. This signal indicates that the channel
    // is signaling a valid write response.
    output wire  S_AXI_BVALID,
    // Response ready. This signal indicates that the master
    // can accept a write response.
    input wire  S_AXI_BREADY,
    // Read address (issued by master, accepted by Slave)
    input wire [C_S_AXI_ADDR_WIDTH-1 : 0] S_AXI_ARADDR,
    // Protection type. This signal indicates the privilege
    // and security level of the transaction, and whether the
    // transaction is a data access or an instruction access.
    input wire [2 : 0] S_AXI_ARPROT,
    // Read address valid. This signal indicates that the channel
    // is signaling valid read address and control information.
    input wire  S_AXI_ARVALID,
```

```

// Read address ready. This signal indicates that the slave is
// ready to accept an address and associated control signals.
output wire S_AXI_ARREADY,
// Read data (issued by slave)
output wire [C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_RDATA,
// Read response. This signal indicates the status of the
// read transfer.
output wire [1 : 0] S_AXI_RRESP,
// Read valid. This signal indicates that the channel is
// signaling the required read data.
output wire S_AXI_RVALID,
// Read ready. This signal indicates that the master can
// accept the read data and response information.
input wire S_AXI_RREADY
);

// AXI4LITE signals
reg [C_S_AXI_ADDR_WIDTH-1 : 0] axi_awaddr;
reg axi_awready;
reg axi_wready;
reg [1 : 0] axi_bresp;
reg axi_bvalid;
reg [C_S_AXI_ADDR_WIDTH-1 : 0] axi_araddr;
reg axi_arready;
reg [C_S_AXI_DATA_WIDTH-1 : 0] axi_rdata;
reg [1 : 0] axi_rresp;
reg axi_rvalid;

// Example-specific design signals
// local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
// ADDR_LSB is used for addressing 32/64 bit registers/memories
// ADDR_LSB = 2 for 32 bits (n downto 2)
// ADDR_LSB = 3 for 64 bits (n downto 3)
localparam integer ADDR_LSB = (C_S_AXI_DATA_WIDTH/32) + 1;
localparam integer OPT_MEM_ADDR_BITS = 1;
//-----
//-- Signals for user logic register space example
//-----
//-- Number of Slave Registers 4
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg0;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg1;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg2;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg3;
wire slv_reg_rden;
wire slv_reg_wren;
reg [C_S_AXI_DATA_WIDTH-1:0] reg_data_out;
integer byte_index;
reg aw_en;

// I/O Connections assignments

assign S_AXI_AWREADY = axi_awready;
assign S_AXI_WREADY = axi_wready;
assign S_AXI_BRESP = axi_bresp;
assign S_AXI_BVALID = axi_bvalid;
assign S_AXI_ARREADY = axi_arready;
assign S_AXI_RDATA = axi_rdata;
assign S_AXI_RRESP = axi_rresp;
assign S_AXI_RVALID = axi_rvalid;
// Implement axi_awready generation
// axi_awready is asserted for one S_AXI_ACLK clock cycle when both
// S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
// de-asserted when reset is low.

always @(posedge S_AXI_ACLK)
begin
if (S_AXI_ARESETN == 1'b0)
begin
axi_awready <= 1'b0;
aw_en <= 1'b1;
end
else
begin
if (~axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en)
begin
// slave is ready to accept write address when

```

```

        // there is a valid write address and write data
        // on the write address and data bus. This design
        // expects no outstanding transactions.
        axi_awready <= 1'b1;
        aw_en <= 1'b0;
    end
    else if (S_AXI_BREADY && axi_bvalid)
        begin
            aw_en <= 1'b1;
            axi_awready <= 1'b0;
        end
    else
        begin
            axi_awready <= 1'b0;
        end
    end
end

// Implement axi_awaddr latching
// This process is used to latch the address when both
// S_AXI_AWVALID and S_AXI_WVALID are valid.

always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
        begin
            axi_awaddr <= 0;
        end
    else
        begin
            if (~axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en)
                begin
                    // Write Address latching
                    axi_awaddr <= S_AXI_AWADDR;
                end
            end
        end
    end
end

// Implement axi_wready generation
// axi_wready is asserted for one S_AXI_ACLK clock cycle when both
// S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
// de-asserted when reset is low.

always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
        begin
            axi_wready <= 1'b0;
        end
    else
        begin
            if (~axi_wready && S_AXI_WVALID && S_AXI_AWVALID && aw_en)
                begin
                    // slave is ready to accept write data when
                    // there is a valid write address and write data
                    // on the write address and data bus. This design
                    // expects no outstanding transactions.
                    axi_wready <= 1'b1;
                end
            else
                begin
                    axi_wready <= 1'b0;
                end
            end
        end
    end
end

// Implement memory mapped register select and write logic generation
// The write data is accepted and written to memory mapped registers when
// axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted. Write strobes are used to
// select byte enables of slave registers while writing.
// These registers are cleared when reset (active low) is applied.
// Slave register write enable is asserted when valid address and data are available
// and the slave is ready to accept the write address and write data.
assign slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;

always @(posedge S_AXI_ACLK)

```

```

begin
  if ( S_AXI_ARESETN == 1'b0 )
    begin
      slv_reg0 <= 0;
      slv_reg1 <= 0;
      slv_reg2 <= 0;
      slv_reg3 <= 0;
    end
  else begin
    if (slv_reg_wren)
      begin
        case ( axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
          2'h0:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 0
                slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
              end
          2'h1:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 1
                slv_reg1[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
              end
          2'h2:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 2
                slv_reg2[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
              end
          2'h3:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 3
                slv_reg3[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
              end
          default : begin
            slv_reg0 <= slv_reg0;
            slv_reg1 <= slv_reg1;
            slv_reg2 <= slv_reg2;
            slv_reg3 <= slv_reg3;
          end
        endcase
      end
    end
  end

  // Implement write response logic generation
  // The write response and response valid signals are asserted by the slave
  // when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
  // This marks the acceptance of address and indicates the status of
  // write transaction.

  always @( posedge S_AXI_ACLK )
  begin
    if ( S_AXI_ARESETN == 1'b0 )
      begin
        axi_bvalid <= 0;
        axi_bresp <= 2'b0;
      end
    else
      begin
        if (axi_awready && S_AXI_AWVALID && ~axi_bvalid && axi_wready && S_AXI_WVALID)
          begin
            // indicates a valid write response is available
            axi_bvalid <= 1'b1;
            axi_bresp <= 2'b0; // 'OKAY' response
          end
          // work error responses in future
        else
          begin
            if (S_AXI_BREADY && axi_bvalid)
              //check if bready is asserted while bvalid is high)

```

```

        //(there is a possibility that bready is always asserted high)
        begin
            axi_bvalid <= 1'b0;
        end
    end
end

// Implement axi_arready generation
// axi_arready is asserted for one S_AXI_ACLK clock cycle when
// S_AXI_ARVALID is asserted. axi_arready is
// de-asserted when reset (active low) is asserted.
// The read address is also latched when S_AXI_ARVALID is
// asserted. axi_araddr is reset to zero on reset assertion.

always @(posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
    begin
        axi_arready <= 1'b0;
        axi_araddr  <= 32'b0;
    end
    else
    begin
        if (~axi_arready && S_AXI_ARVALID)
        begin
            // indicates that the slave has accepted the valid read address
            axi_arready <= 1'b1;
            // Read address latching
            axi_araddr  <= S_AXI_ARADDR;
        end
        else
        begin
            axi_arready <= 1'b0;
        end
    end
end

// Implement axi_rvalid generation
// axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
// S_AXI_ARVALID and axi_arready are asserted. The slave registers
// data are available on the axi_rdata bus at this instance. The
// assertion of axi_rvalid marks the validity of read data on the
// bus and axi_rresp indicates the status of read transaction. axi_rvalid
// is deasserted on reset (active low). axi_rresp and axi_rdata are
// cleared to zero on reset (active low).
always @(posedge S_AXI_ACLK )
begin
    if ( S_AXI_ARESETN == 1'b0 )
    begin
        axi_rvalid <= 0;
        axi_rresp  <= 0;
    end
    else
    begin
        if (axi_arready && S_AXI_ARVALID && ~axi_rvalid)
        begin
            // Valid read data is available at the read data bus
            axi_rvalid <= 1'b1;
            axi_rresp  <= 2'b0; // 'OKAY' response
        end
        else if (axi_rvalid && S_AXI_RREADY)
        begin
            // Read data is accepted by the master
            axi_rvalid <= 1'b0;
        end
    end
end

// Implement memory mapped register select and read logic generation
// Slave register read enable is asserted when valid address is available
// and the slave is ready to accept the read address.
assign slv_reg_rden = axi_arready & S_AXI_ARVALID & ~axi_rvalid;
always @(*)
begin
    // Address decoding for reading registers

```

```

        case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
            2'h0 : reg_data_out <= slv_reg0;
            2'h1 : reg_data_out <= slv_reg1;
            2'h2 : reg_data_out <= slv_reg2;
            2'h3 : reg_data_out <= slv_reg3;
            default : reg_data_out <= 0;
        endcase
    end

    // Output register or memory read data
    always @( posedge S_AXI_ACLK )
    begin
        if ( S_AXI_ARESETN == 1'b0 )
            begin
                axi_rdata <= 0;
            end
        else
            begin
                // When there is a valid read address (S_AXI_ARVALID) with
                // acceptance of read address by the slave (axi_arready),
                // output the read data
                if (slv_reg_rden)
                    begin
                        axi_rdata <= reg_data_out;    // register read data
                    end
                end
            end
        end

        // Add user logic here
        assign GPIO_LED[3:0] = slv_reg0[3:0];
        // User logic ends

    endmodule

```

3:双击 GPIO_LITE_ML_v1_0 文件，做如下修改。

修改 1:

```

parameter integer C_S00_AXI_ADDR_WIDTH = 4
)
(
    // Users to add ports here
    output wire [3:0]GPIO_LED,
    // User ports ends
    // Do not modify the ports beyond this line

```

修改 2:

```

.S_AXI_RVALID(s00_axi_rvalid),
.S_AXI_RREADY(s00_axi_rready),
.GPIO_LED(GPIO_LED)
);

```

修改后的文件如下。

```

`timescale 1 ns / 1 ps

module GPIO_LITE_ML_v1_0 #
(
    // Users to add parameters here

    // User parameters ends
    // Do not modify the parameters beyond this line

    // Parameters of Axi Slave Bus Interface S00_AXI
    parameter integer C_S00_AXI_DATA_WIDTH = 32,
    parameter integer C_S00_AXI_ADDR_WIDTH = 4
)
(
    // Users to add ports here
    output wire [3:0]GPIO_LED,
    // User ports ends
    // Do not modify the ports beyond this line

```

```

// Ports of Axi Slave Bus Interface S00_AXI
input wire s00_axi_aclk,
input wire s00_axi_aresetn,
input wire [C_S00_AXI_ADDR_WIDTH-1 : 0] s00_axi_awaddr,
input wire [2 : 0] s00_axi_awprot,
input wire s00_axi_awvalid,
output wire s00_axi_awready,
input wire [C_S00_AXI_DATA_WIDTH-1 : 0] s00_axi_wdata,
input wire [(C_S00_AXI_DATA_WIDTH/8)-1 : 0] s00_axi_wstrb,
input wire s00_axi_wvalid,
output wire s00_axi_wready,
output wire [1 : 0] s00_axi_bresp,
output wire s00_axi_bvalid,
input wire s00_axi_bready,
input wire [C_S00_AXI_ADDR_WIDTH-1 : 0] s00_axi_araddr,
input wire [2 : 0] s00_axi_arprot,
input wire s00_axi_arvalid,
output wire s00_axi_arready,
output wire [C_S00_AXI_DATA_WIDTH-1 : 0] s00_axi_rdata,
output wire [1 : 0] s00_axi_rresp,
output wire s00_axi_rvalid,
input wire s00_axi_rready

);
// Instantiation of Axi Bus Interface S00_AXI
GPIO_LITE_ML_v1_0_S00_AXI # (
    .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
    .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
) GPIO_LITE_ML_v1_0_S00_AXI_inst (
    .S_AXI_ACLK(s00_axi_aclk),
    .S_AXI_ARESETN(s00_axi_aresetn),
    .S_AXI_AWADDR(s00_axi_awaddr),
    .S_AXI_AWPROT(s00_axi_awprot),
    .S_AXI_AWVALID(s00_axi_awvalid),
    .S_AXI_AWREADY(s00_axi_awready),
    .S_AXI_WDATA(s00_axi_wdata),
    .S_AXI_WSTRB(s00_axi_wstrb),
    .S_AXI_WVALID(s00_axi_wvalid),
    .S_AXI_WREADY(s00_axi_wready),
    .S_AXI_BRESP(s00_axi_bresp),
    .S_AXI_BVALID(s00_axi_bvalid),
    .S_AXI_BREADY(s00_axi_bready),
    .S_AXI_ARADDR(s00_axi_araddr),
    .S_AXI_ARPROT(s00_axi_arprot),
    .S_AXI_ARVALID(s00_axi_arvalid),
    .S_AXI_ARREADY(s00_axi_arready),
    .S_AXI_RDATA(s00_axi_rdata),
    .S_AXI_RRESP(s00_axi_rresp),
    .S_AXI_RVALID(s00_axi_rvalid),
    .S_AXI_RREADY(s00_axi_rready),
    .GPIO_LED(GPIO_LED)
);

// Add user logic here

// User logic ends

endmodule

```

4:进一步修改。去掉“_v1_0”。

1、GPIO_LITE_ML_v1_0_S00_AXI.v 中：

module GPIO_LITE_ML_v1_0_S00_AXI #修改为→ module GPIO_LITE_ML_S00_AXI #

module GPIO_LITE_ML_v1_0_S00_AXI # 修改后 module GPIO_LITE_ML_S00_AXI #

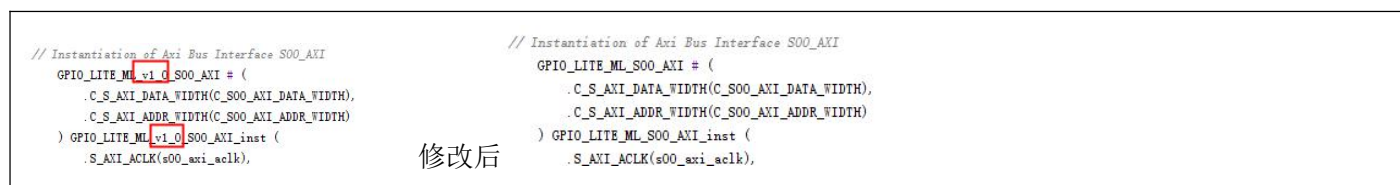
2、GPIO_LITE_ML_v1_0.v 中：

module GPIO_LITE_ML_v1_0 # 修改为→ module GPIO_LITE_ML #

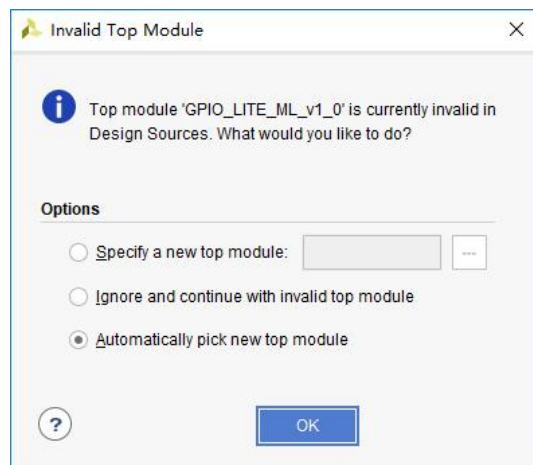
module GPIO_LITE_ML_v1_0_S00_AXI # 修改后 module GPIO_LITE_ML_S00_AXI #
(

GPIO_LITE_ML_v1_0_S00_AXI # 修改为→ GPIO_LITE_ML_S00_AXI #

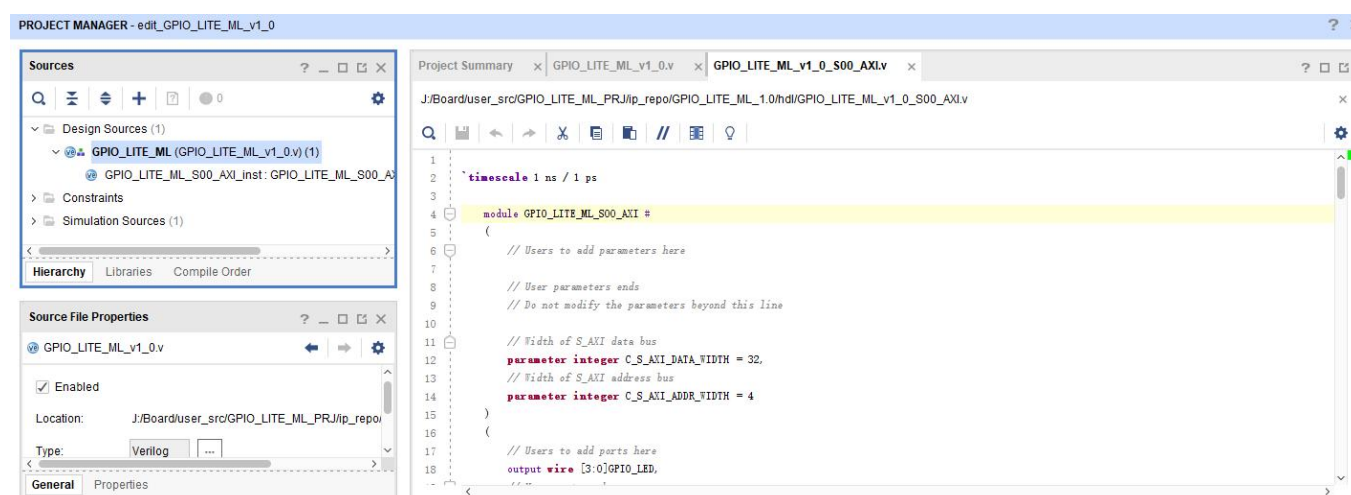
GPIO_LITE_ML_v1_0_S00_AXI_inst 修改为→ GPIO_LITE_ML_S00_AXI_inst



5:修改后，保存。出现如下界面，选择 Automatically pick new top module。

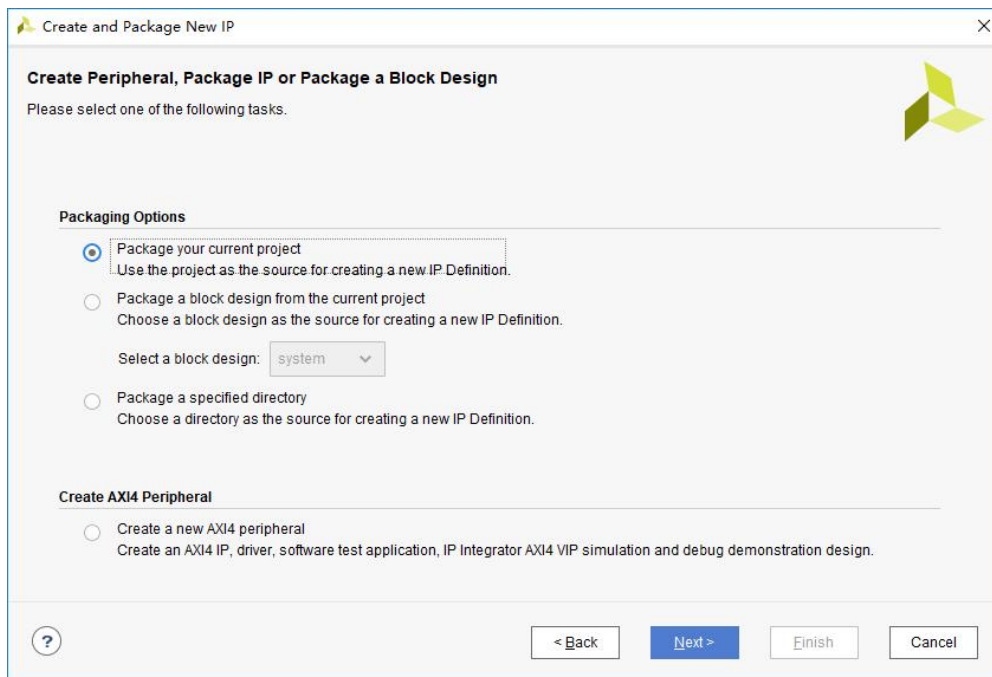


更新后界面

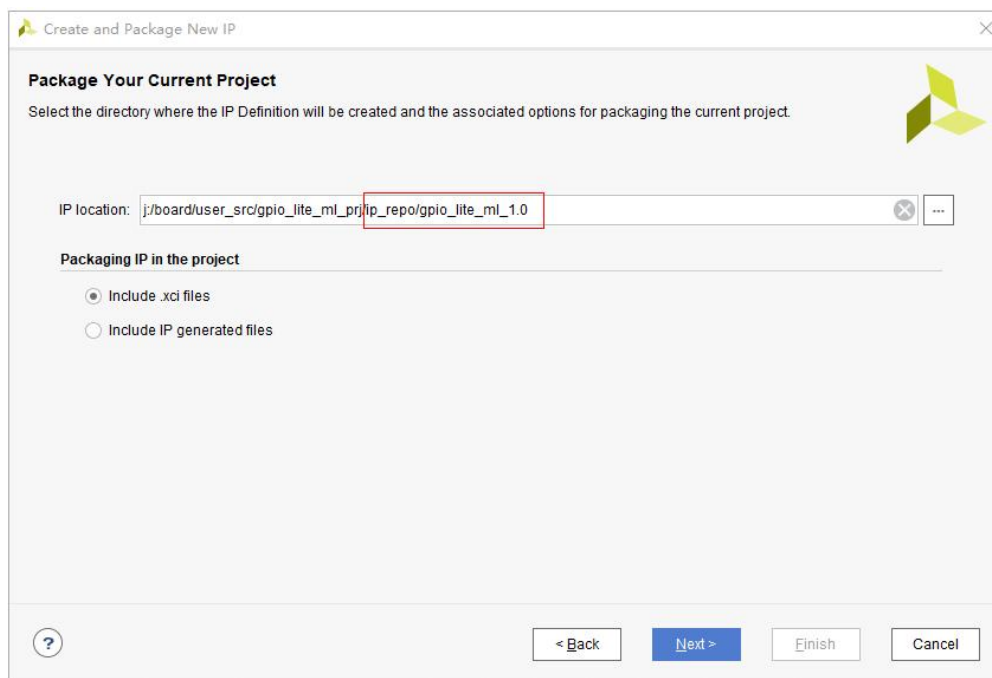


6:重新封装。选择 Tools→Creat and Pakage New IP，单击 Next。

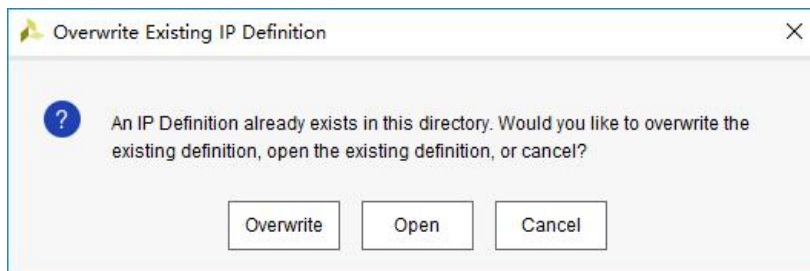
7:选择 Package your current project，单击 Next。



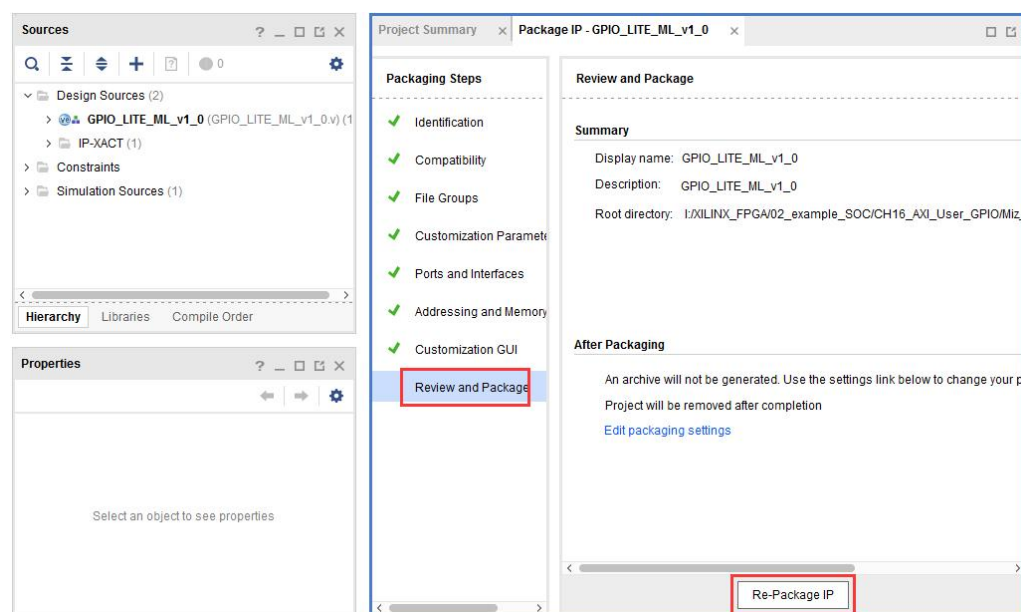
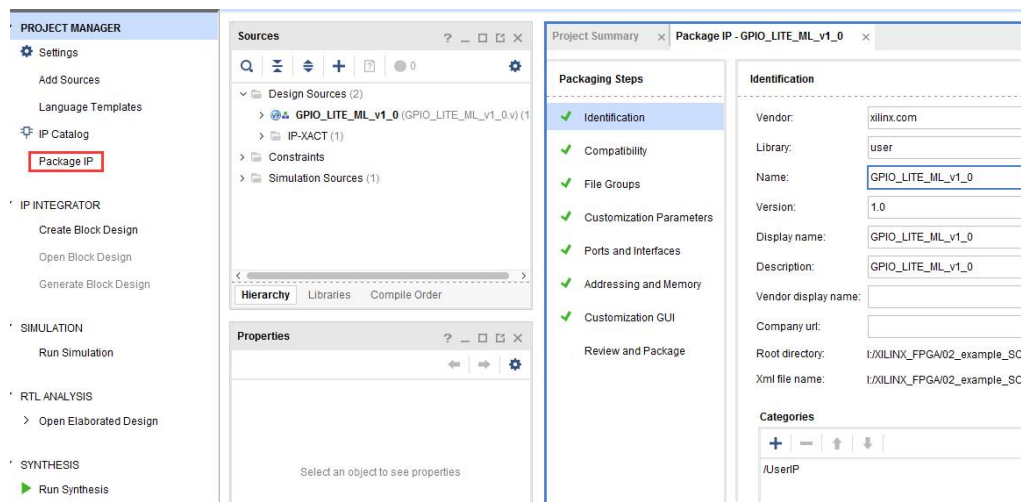
路径选择原来 IP 所在的位置，覆盖原来的文件。



8:选择 Overwrite。



9:选择 Package IP→Rewiew and Package →Re-Package IP

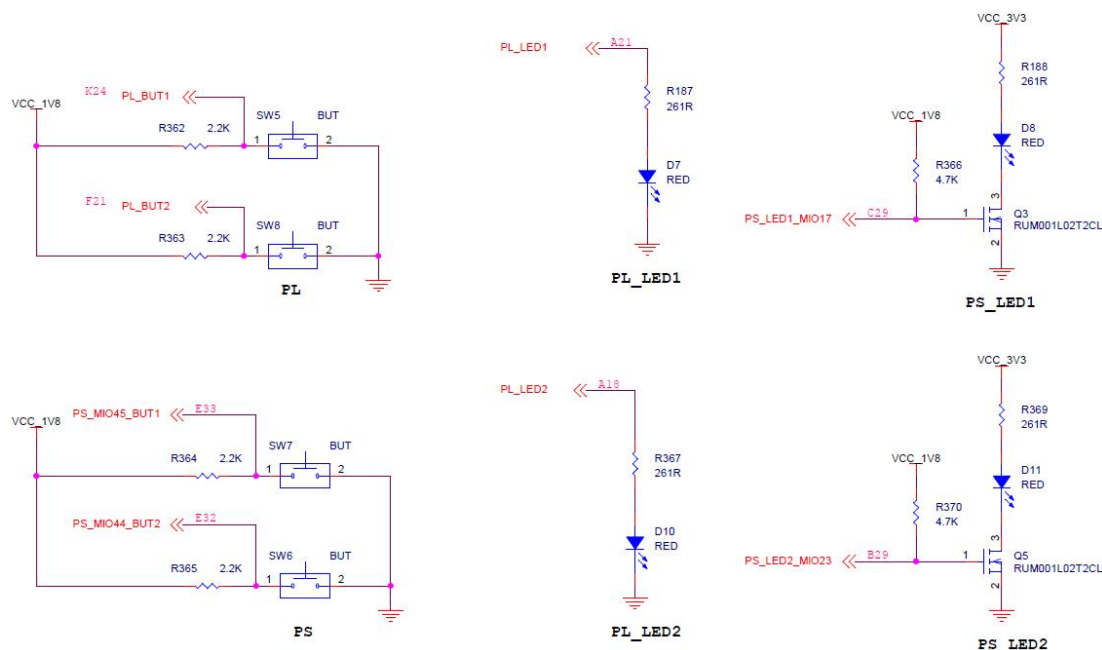


自定义 IP 生成完毕。

11.3 硬件电路分析

在功能底板上，PL 部分 FPGA 引脚 A21 和 A18 接到了 LED 上，本文实验用到这个 2 个 PL GPIO。

11.3.1 原理图



11.3.2 fpga_pin.xdc 中 IO 约束

最后的 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]是为了对 fpga 的 bit 进行压缩, 减少 bit 大小, 提高加载速度。

```
set_property PACKAGE_PIN A21 [get_ports {GPIO_LED[0]}]
set_property IOSTANDARD LVCMOS18 [get_ports {GPIO_LED[0]}]
set_property PACKAGE_PIN A18 [get_ports {GPIO_LED[1]}]
set_property IOSTANDARD LVCMOS18 [get_ports {GPIO_LED[1]}]
```

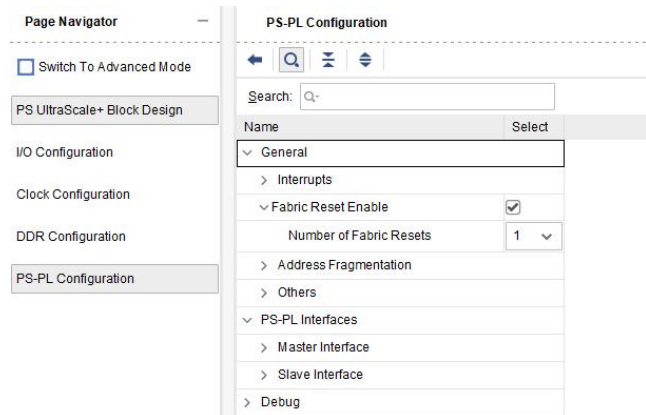
11.4 搭建 SOC 系统工程

新建一个名为为 soc_prj 的工程,之后创建一个 BD 文件, 并命名为 system, 添加并且配置好 ZYNQ Ultrascale+ MPSOC IP。读者需要根据自己的硬件类型配置好输入时钟频率、内存型号、串口, 连接时钟等。新手不清楚这些内容个, 请参考“3-2-01_ex_soc_base_07a-eg.pdf” “01 HelloWold/DDR/网口测试” 这篇文章。

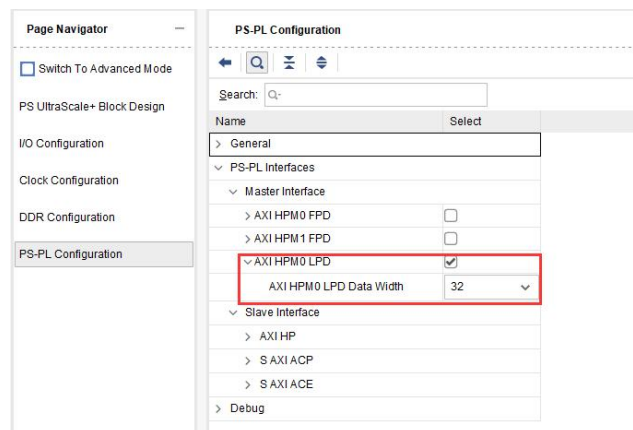


11.4.1 PS 部分设置

1:PS 复位设置



2:PS LPD 设置



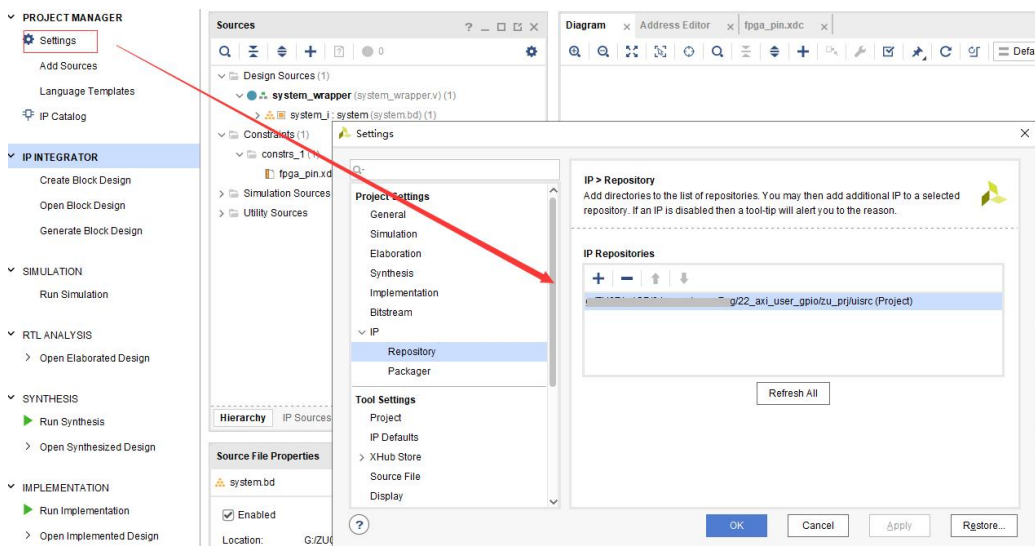
3:PL 输出时钟设置

Name	Source	FracEn	Requested Freq (MHz)	Divisor 0	Divisor 1	Actual Frequency (MHz)	Range
Low Power Domain Clocks							
Processor/Memory Clocks							
Peripherals/IO Clocks							
PL Fabric Clocks							
☒ PL0	RP		100	15	1	100.000000	0.0000...
☐ PL1	RP		100	4	1	100	0.0000...
☐ PL2	RP		100	4	1	100	0.0000...
☐ PL3	RP		100	4	1	100	0.0000...
System Debug Clocks							
Full Power Domain Clocks							
Advance Clocks							

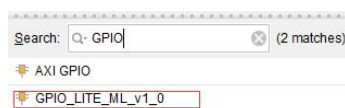
11.4.2 添加自定义 AXI-Lite IP

1:设置 IP 路径

本文中是我们第一次在 BD 图形化设计中添加自定义的 IP，自定义的 IP 需要设置 IP 路径才能被识别到。默认情况下，我们自定的 IP 在配套工程的 uisrc/ip 路径下：

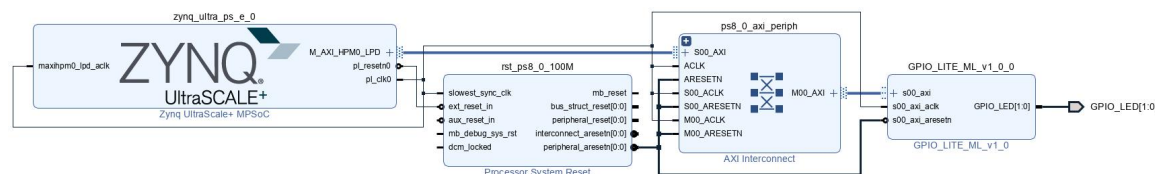


2: 添加 IP



ENTER to select, ESC to cancel, Ctrl+Q for IP details

11.4.3 PL 图像化编程



11.4.4 设置地址分配

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
zynq_ultra_ps_e_0					
zynq_ultra_ps_e_0/Data (39 address bits: 0x0080000000 [512M])					
/GPIO_LITE_ML_v1_0_0/s00_axi	s00_axi	reg0	0x00_8000_0000	64K	0x00_8000_FFFF

11.4.5 编译并导出平台文件

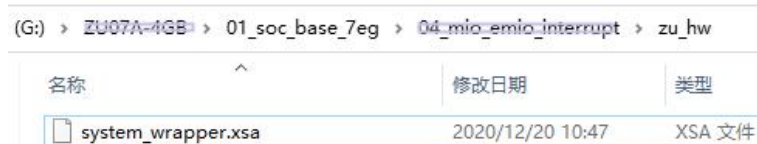
1: 单击 Block 文件 → 右键 → Generate the Output Products → Global → Generate。

2: 单击 Block 文件 → 右键 → Create a HDL wrapper(生成 HDL 顶层文件) → Let vivado manager wrapper and auto-update(自动更新)。

3: 生成 Bit 文件。

4: 导出到硬件: File → Export Hardware → Include bitstream

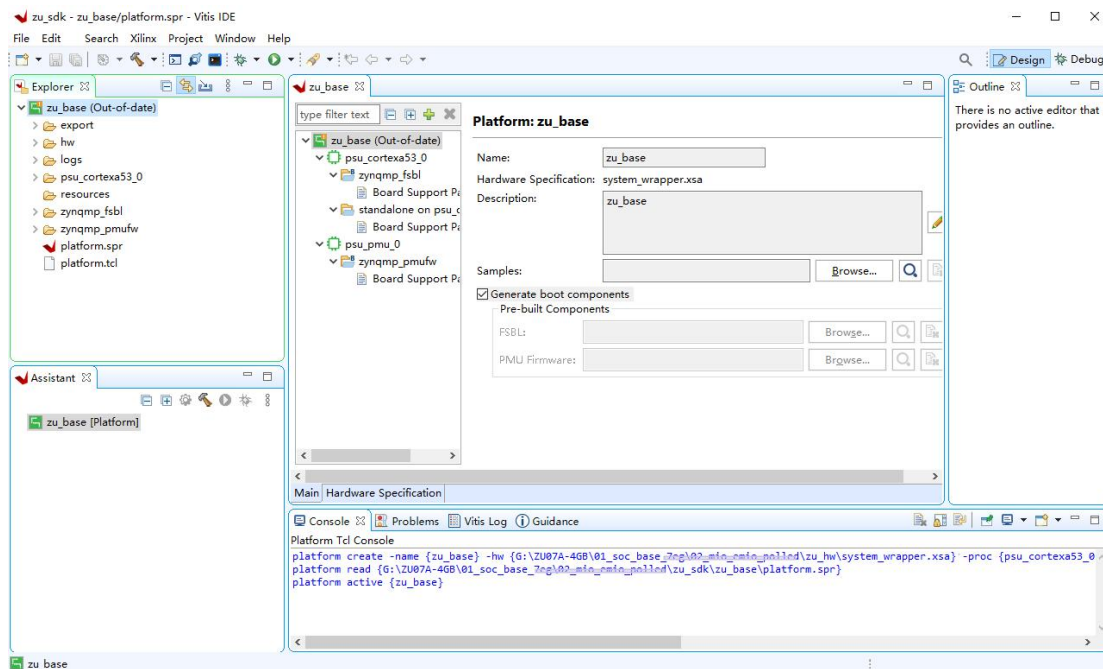
5: 导出完成后, 对应工程路径的 zu_hw 路径下有硬件平台文件: system_wrapper.xsa 的文件。根据硬件平台文件 system_wrapper.xsa 来创建需要 Platform 平台。



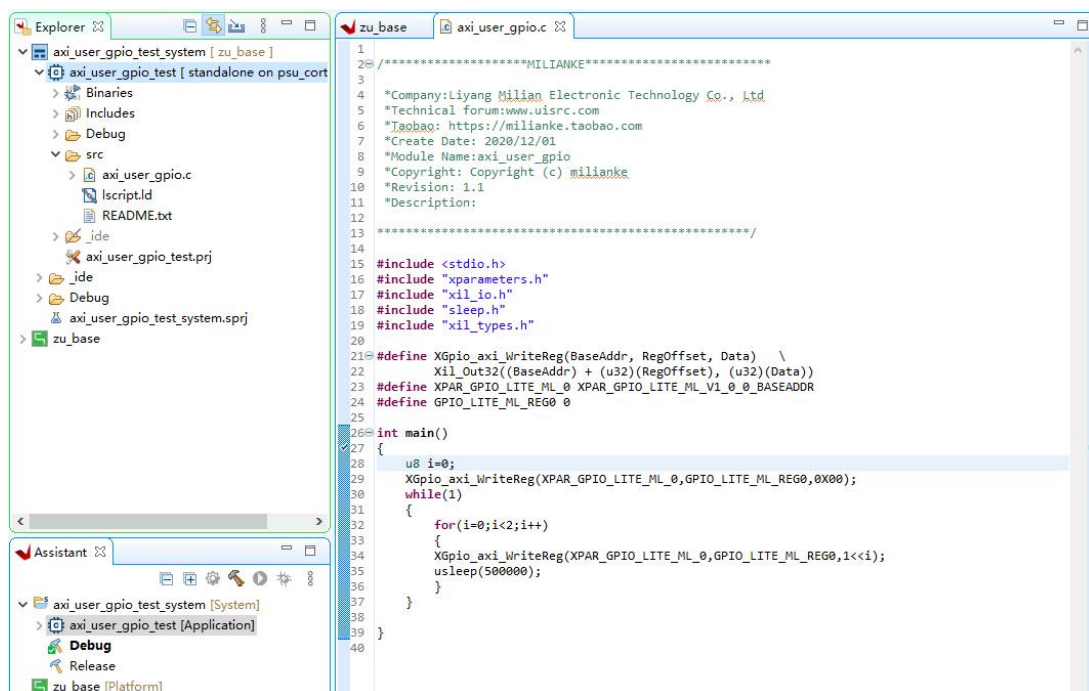
11.5 搭建 Vitis-sdk 工程

创建 zu_base sdk platform 和 APP 工程的过程不再重复，可以阅读本章节 01~05 相关 demo。以下给出创建好 zu_base sdk platform 的截图和对应工程 APP 的截图。

11.5.1 创建 SDK Platform 工程



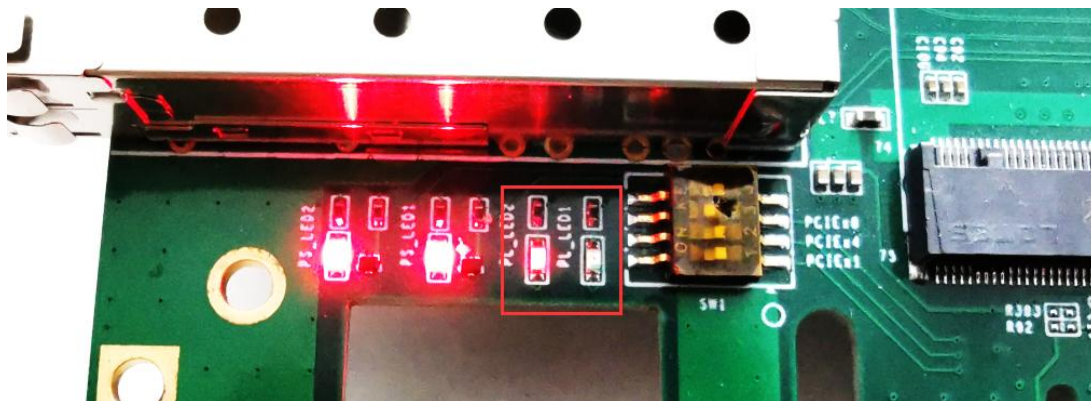
11.5.2 创建 axi_lite_gpio_test APP 工程



11.6 程序分析

XGpio_axi_WriteReg()函数实现的是向 AXI 的寄存器中写入数据，它的三个参数分别为基地址，偏移量和数据。需要注意的是此处的偏移量，AXI 的相邻寄存器偏移量相差 4 个字节，默认 slv_reg0 的偏移量是 0，因此，可以推导出 slv_reg1，slv_reg2 的偏移量分别为 4 和 8，本课中，我们只用到了 slv_reg0，所以偏移量为 0。

11.7 实验结果



12AXI-Stream 发数据到 PS(DMA)

软件版本: vitis2021.1(vivado2021.1)

操作系统: WIN10 64bit

硬件平台: 适用 XILINX A7/K7/Z7/ZU/KU 系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

12.1 概述

本文在 AXI_DMA_LOOP 环路测试架构的基础上,在 DATA FIFO 端加入 FPGA 代码,对 FIFO 写,实现将 PL 端数据 通过 DMA 发送给 PS 功能。

本文实验目的:

- 1:掌握编程 PL 代码,以 AXI-Stream 协议把数据通过 DMA 发送到 PS DDR
- 2:通过 VITIS-SDK 编程实现数据的接收
- 3:通过 VITIS-SDK 观察 PS 内存中接收到的数据是否正确。

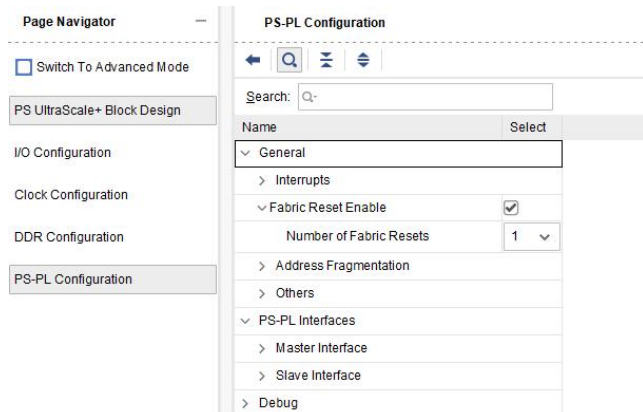
12.2 搭建 SOC 系统工程

新建一个名为 soc_prj 的工程,之后创建一个 BD 文件,并命名为 system,添加并且配置好 ZYNQ Ultrascale+ MPSOC IP。读者需要根据自己的硬件类型配置好输入时钟频率、内存型号、串口,连接时钟等。新手不清楚这些内容的,请参考“3-2-01_ex_soc_base_07a-cg.pdf” “01 HelloWorld/DDR/网口测试”这篇文章。

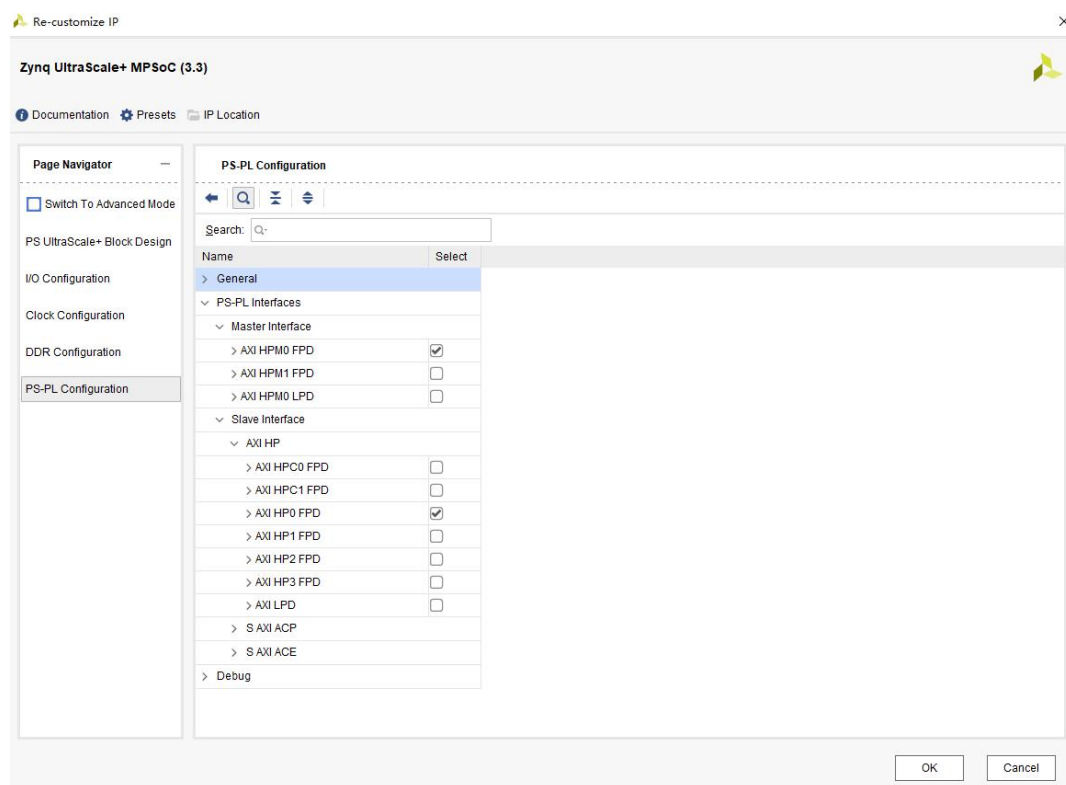


12.2.1PS 部分设置

1:PS 复位设置

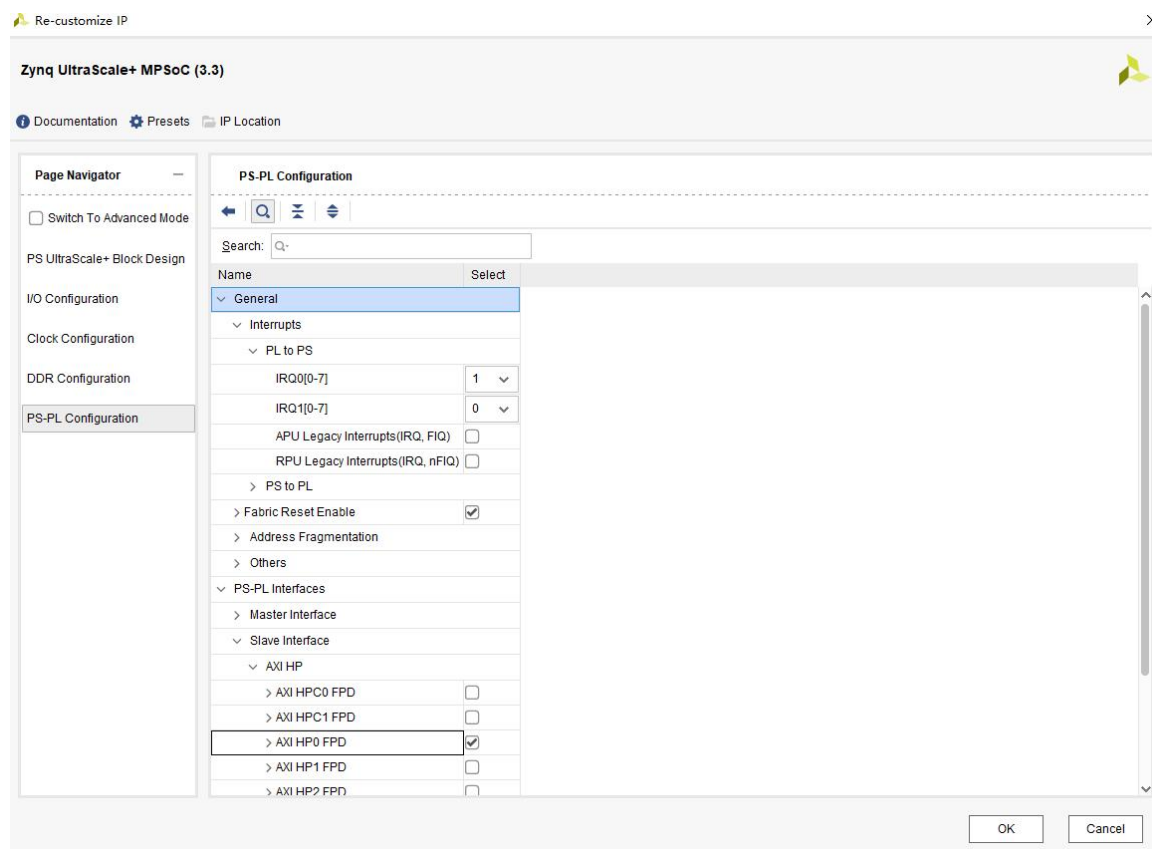


2:设置 PS FPD 接口和 HP 接口



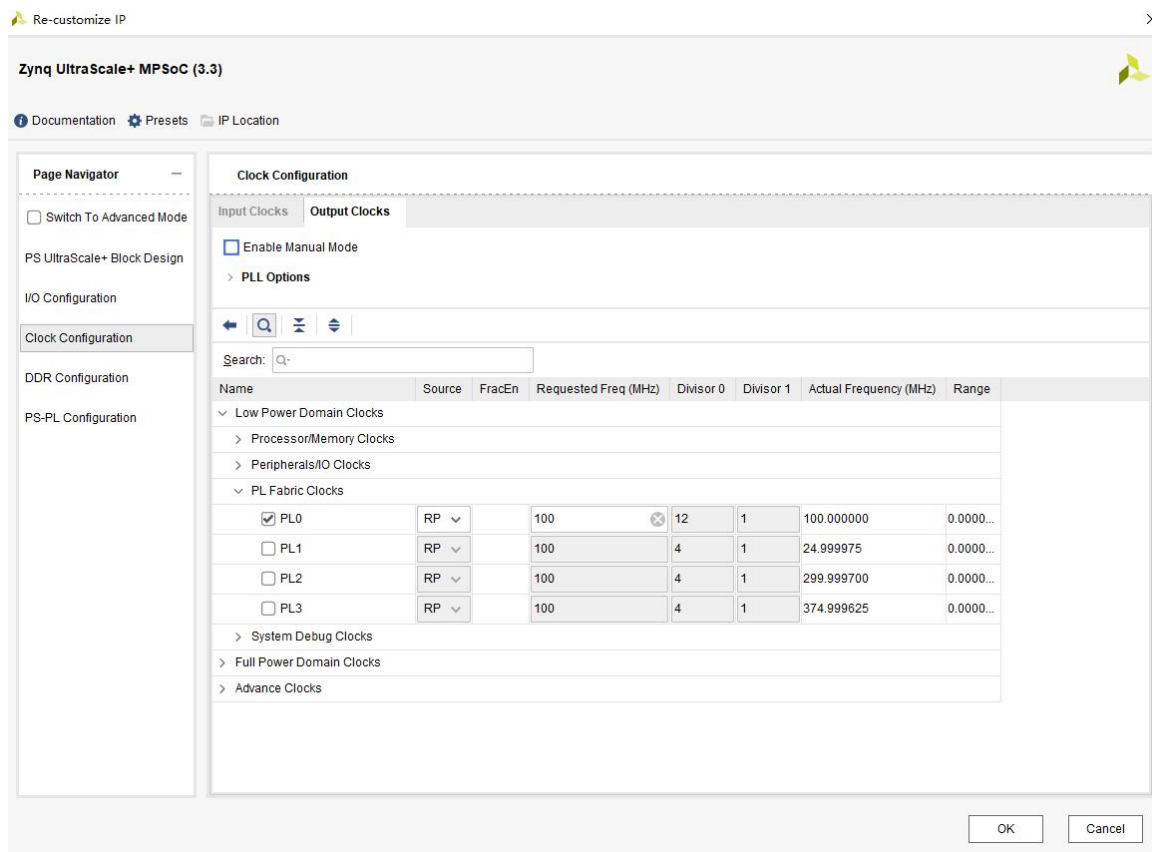
3:设置 PL 到 PS 的中断

Interurpts→勾选 Fabric Interrupt, 勾选 IRQ0[0:7]。



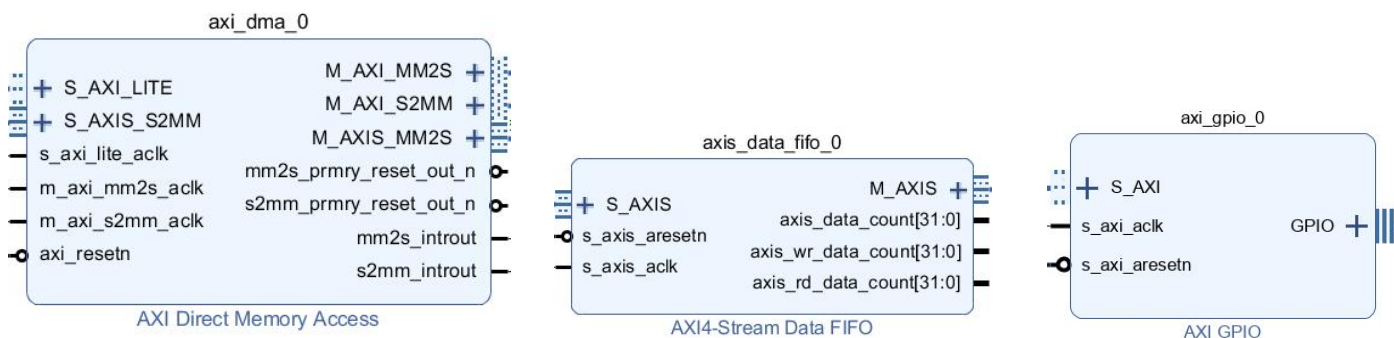
4:设置 PL 的时钟

勾选 PL0, 设置为 100, 即 PS 的 PLL 提供本系统的时钟 100MHZ。



12.2.2 添加 IP

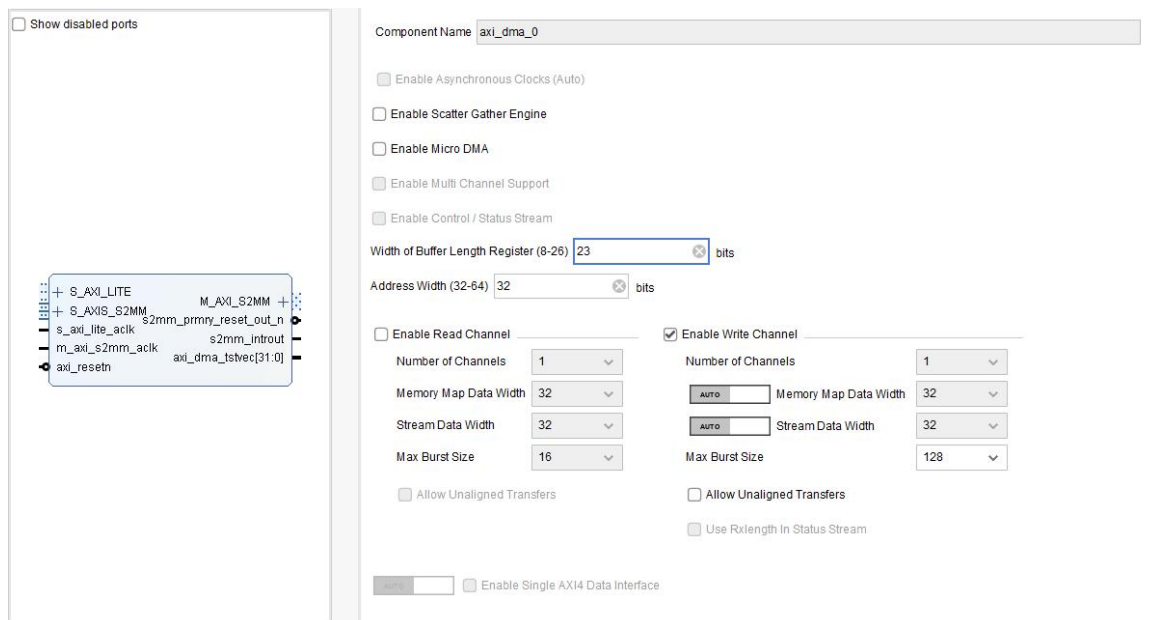
单击添加 IP 按钮“+”，输入如下模块 IP 名字的关键词，并双击添加。



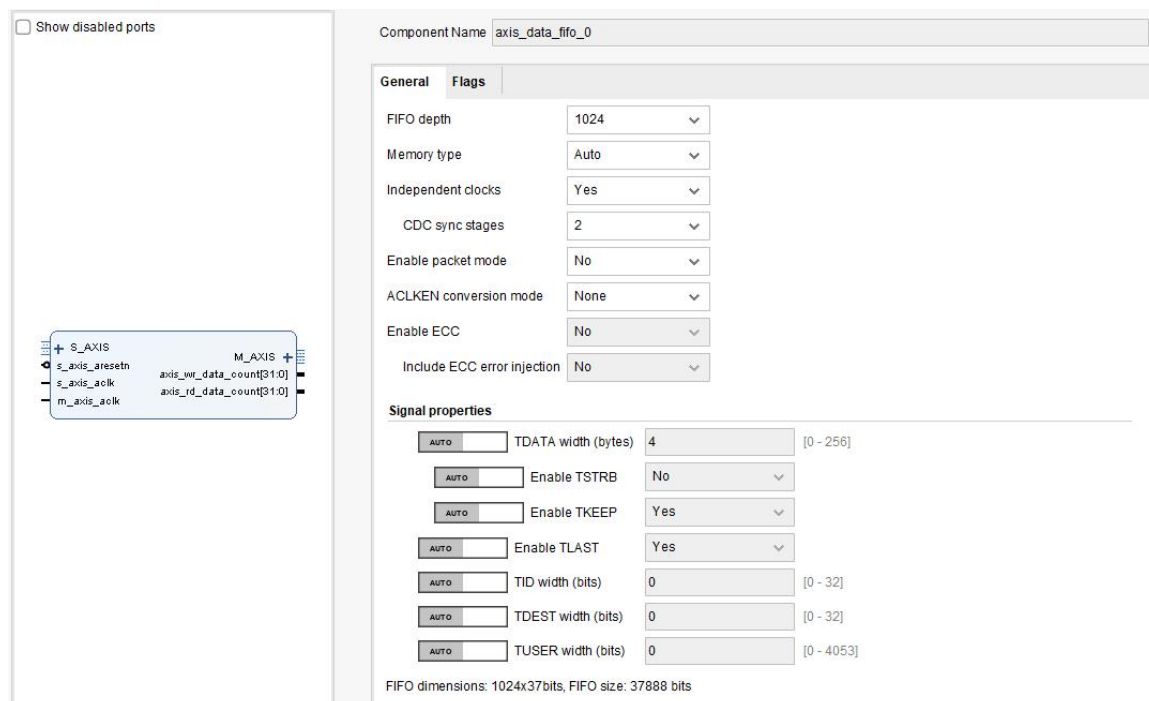
12.2.3 IP 设置

设置 Width of buffer length register : 23。(寄存器设置最大为 26，即 2 的 26 次方 64M 大小，这里设置 23bit 就够用了，长度越大，需要的资源也就越多)

这里也只要使用 DMA 写通道，

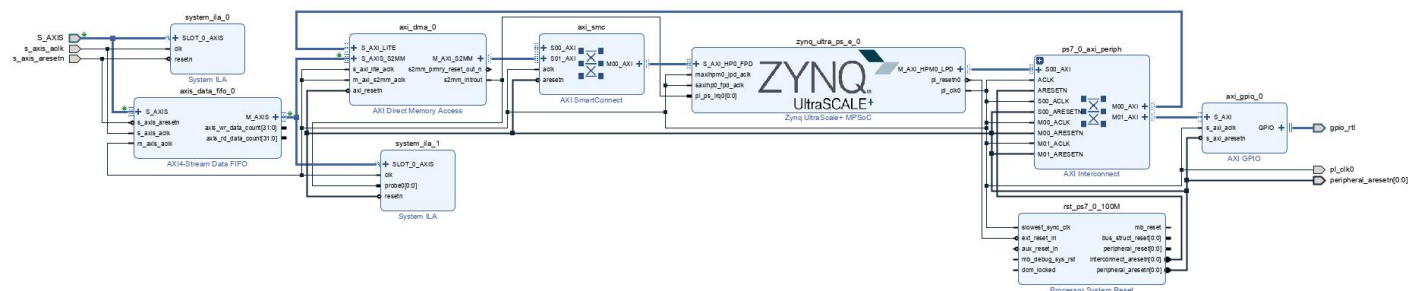


Data FIFO 设置 TDATA Width 为 4。



12.2.4PL 图形编程

下图中可以看到 FIFO 的 S_AXIS 接口引出到了外部的 FPGA 代码中，所以后面我们需要编写合适的 AXI-Stream FPGA 代码来写 FIFO。

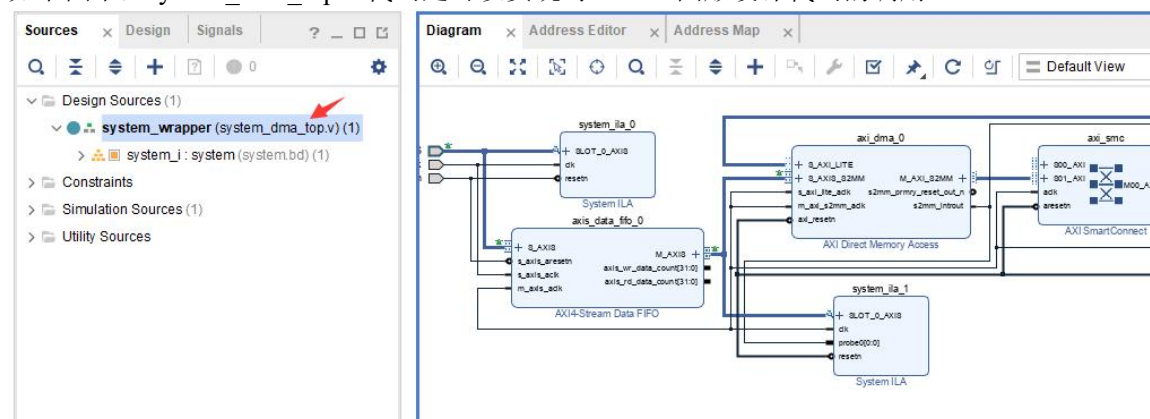


12.2.5 地址空间分配

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/axi_dma_0					
/axi_dma_0/Data_S2MM (32 address bits : 4G)					
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_DDR_LOW	0x0000_0000	2G	0x7FFF_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_QSPI	0xC000_0000	512M	0xDFFF_FFFF
Excluded (3)					
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_GIC	0xF900_0000	1M	0xF90F_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_LPS_OCM	0xFF00_0000	16M	0xFFFF_FFFF
/zynq_ultra_ps_e_0/SAXIGP2	S_AXI_HP0_FPD	HP0_DDR_HIGH	0x8_0000_0000	4G	0x8_FFFF_FFFF
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (39 address bits : 0x00800000000 [512M])					
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0x00_8000_0000	64K	0x00_8000_FFFF
/axi_gpio_0/S_AXI	S_AXI	Reg	0x00_8001_0000	64K	0x00_8001_FFFF

12.2.6 编写 AXI-Stream 写代码

如下图中，system_dma_top.v 代码是可以实现对 FPGA 图形设计代码的调用。



现在我们看看 system_dma_top.v 中如何利用 axi-stream 协议完成对 fifo 的写操作。

```

module system_wrapper();

    reg [31:0] S_AXIS_tdata;
    wire S_AXIS_tlast;
    wire S_AXIS_tvalid = 1'b1;
    wire FCLK_CLK0;
    wire s_axis_aclk;
    wire s_axis_aresetn;
    wire [3:0] S_AXIS_tkeep;
    wire S_AXIS_tready;
    wire [0:0] gpio_rtl_tri_o;
    wire [0:0] peripheral_aresetn;

    assign S_AXIS_tkeep = 4'b1111;
    assign s_axis_aclk = pl_clk0;
    assign s_axis_aresetn = peripheral_aresetn && gpio_rtl_tri_o;

    always@(posedge pl_clk0) begin
        if(s_axis_aresetn == 1'b0) begin
            S_AXIS_tdata <= 0;
        end
        else begin
            if(S_AXIS_tready && S_AXIS_tdata < 511)
                S_AXIS_tdata <= S_AXIS_tdata + 1'b1;
            else if(S_AXIS_tready)
                S_AXIS_tdata <= 0;
        end
    end
end

```

```

assign S_AXIS_tlast = (S_AXIS_tdata == 511)&&S_AXIS_tready&&S_AXIS_tvalid;

system system_i
(
    .S_AXIS_tdata(S_AXIS_tdata),
    .S_AXIS_tkeep(S_AXIS_tkeep),
    .S_AXIS_tlast(S_AXIS_tlast),
    .S_AXIS_tready(S_AXIS_tready),
    .S_AXIS_tvalid(S_AXIS_tvalid),
    .gpio_rtl_tri_o(gpio_rtl_tri_o),
    .peripheral_aresetn(peripheral_aresetn),
    .pl_clk0(pl_clk0),
    .s_axis_aclk(s_axis_aclk),
    .s_axis_aresetn(s_axis_aresetn));
endmodule

```

以上代码非常简洁，本身 axi-stream 协议就非常简洁。

以上代码中设置 S_AXIS_tvalid = 1'b1 始终为 1 代表，任何时候只要 S_AXIS_tready 有效，数据计数器 S_AXIS_tdata 就加 1，每次发送 512 个 32bit 数据，也就是 1024 个字节数据。每发送 1024 个数据产生一个中断。中断的产生是靠 axi-stream 协议的 S_AXIS_tlast 信号决定。S_AXIS_tlast = (S_AXIS_tdata == 511)&&S_AXIS_tready&&S_AXIS_tvalid。

12.2.7 编译并导出平台文件

1:单击 Block 文件→右键→Generate the Output Products→Global→Generate。

2:单击 Block 文件→右键→ Create a HDL wrapper(生成 HDL 顶层文件)→Let vivado manager wrapper and auto-update(自动更新)。

3:生成 Bit 文件。

4:导出到硬件: File→Export Hardware→Include bitstream

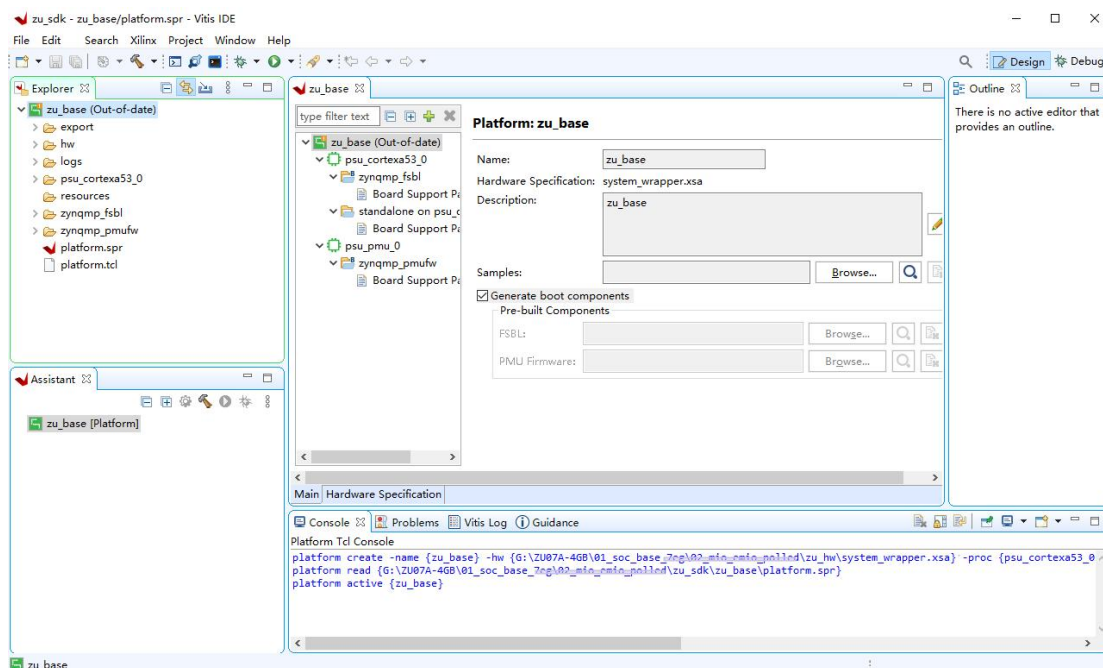
5:导出完成后，对应工程路径的 zu_hw 路径下有硬件平台文件：system_wrapper.xsa 的文件。根据硬件平台文件 system_wrapper.xsa 来创建需要 Platform 平台。

(G:) > ZU07A-4GB > 01_soc_base_7eg > 04_mio_emio_interrupt > zu_hw		
名称	修改日期	类型
system_wrapper.xsa	2020/12/20 10:47	XSA 文件

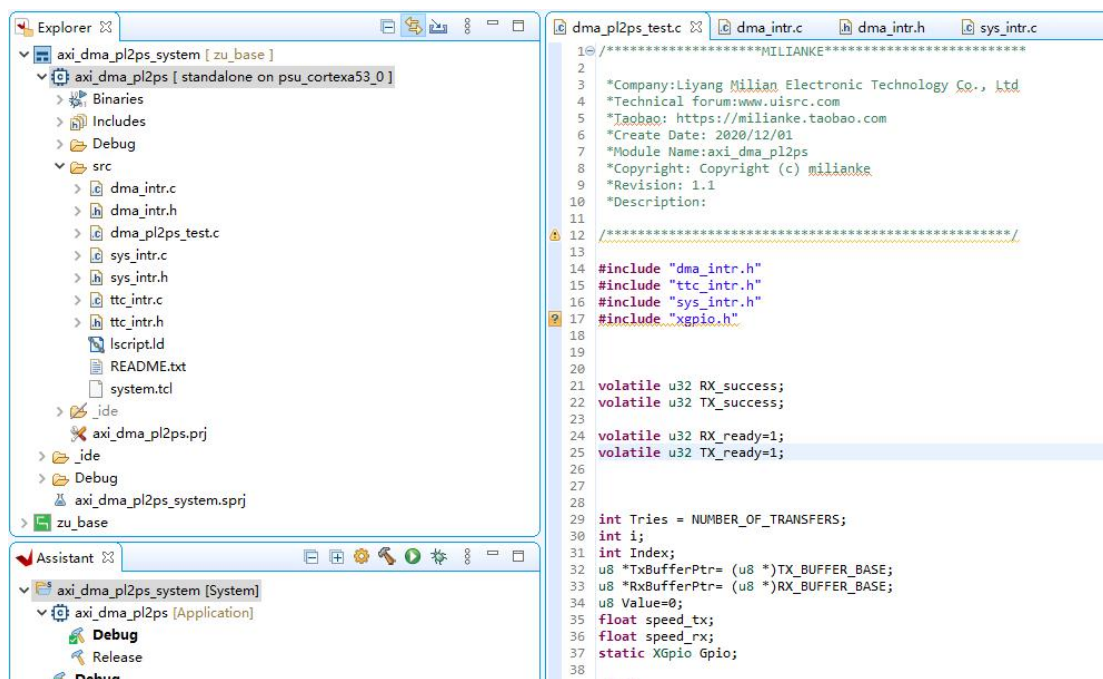
12.3 搭建 Vitis-sdk 工程

创建 zu_base sdk platform 和 APP 工程的过程不再重复，可以阅读本章节 01~05 相关 demo。以下给出创建好 zu_base sdk platform 的截图和对应工程 APP 的截图。

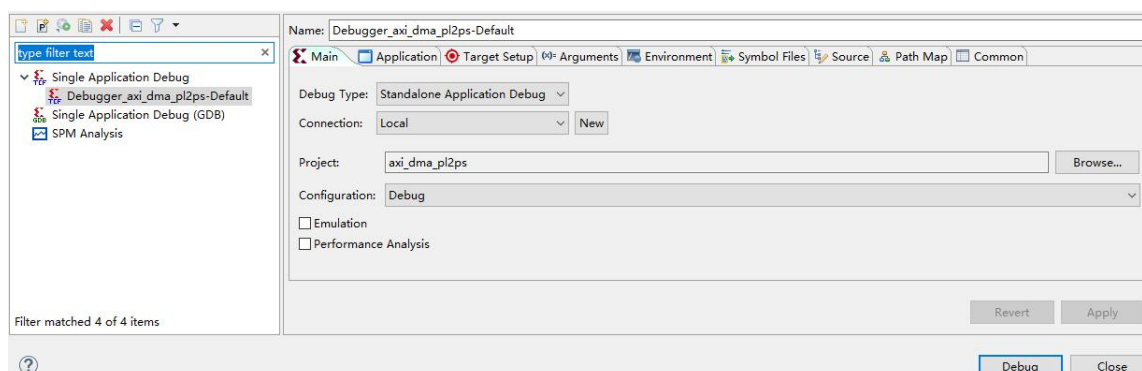
12.3.1 创建 SDK Platform 工程



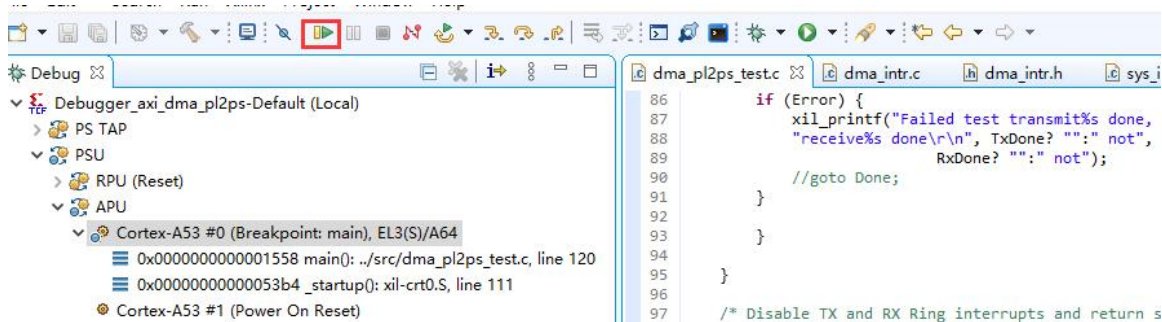
12.3.2 创建 axi_dma_pl2ps APP 工程



12.4 实验结果



Debug 程序，单程序停止 main 函数处，打开 VIVADO，扫描芯片，这个时候会自动之前添加的在线逻辑分析仪 IP 核。如下红框的按键先不要单击。



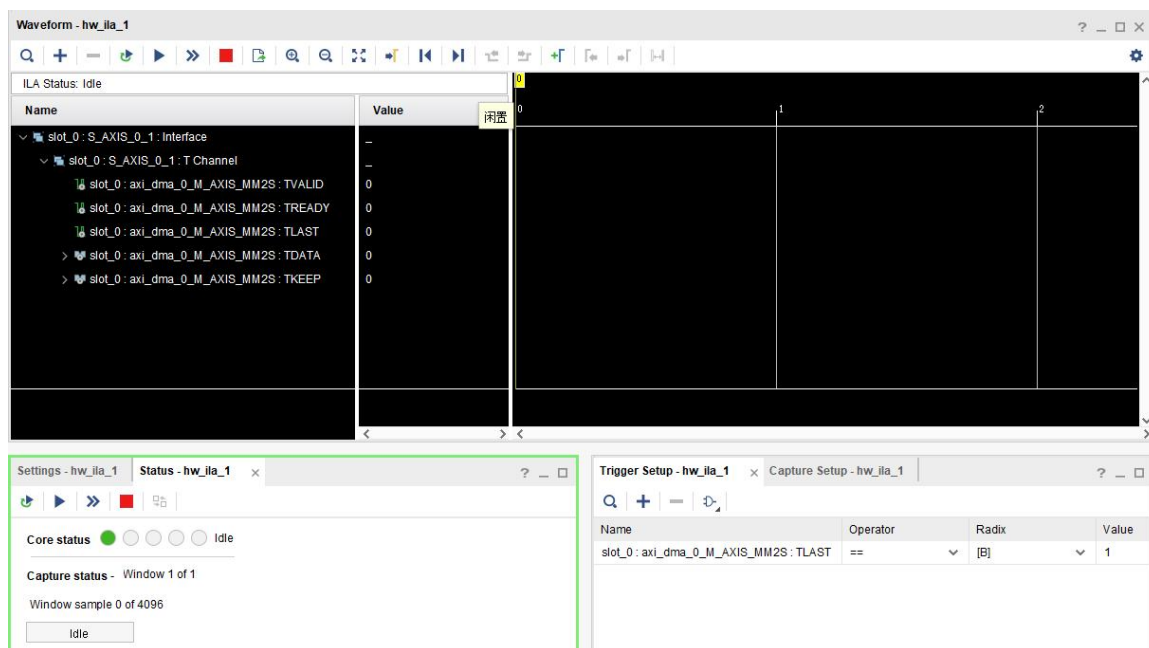
具体步骤如下：

在 VIVADO 工程中点击 Open Target 然后点击 Auto Connect

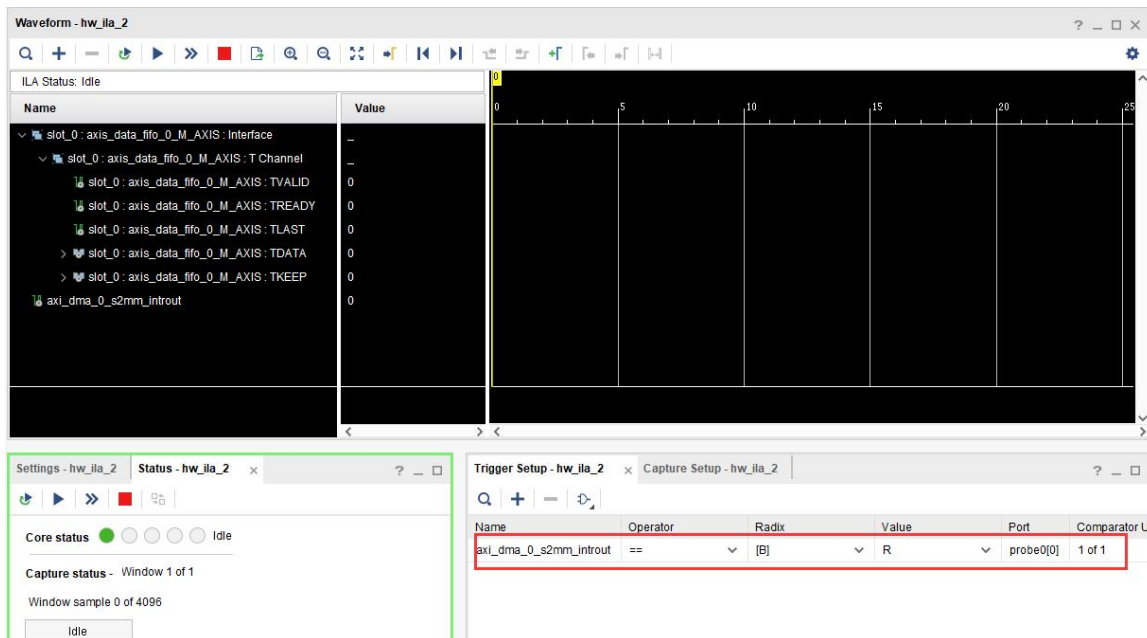


连接成功后入下图

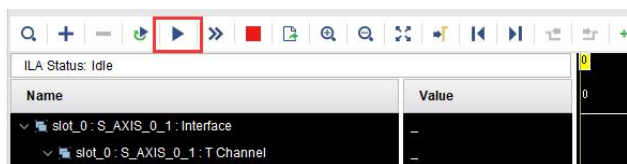
下图中，我们利用 axi-tlast 信号作为触发信号



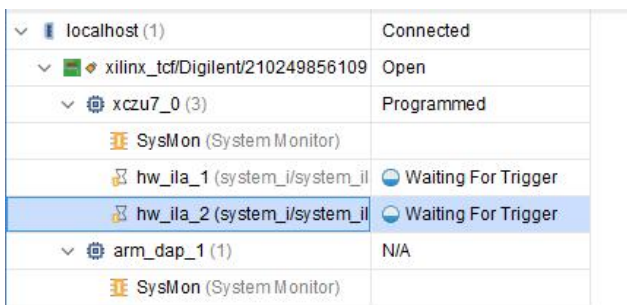
下图中我们使用 DMA 产生的中断信号作为触发信号



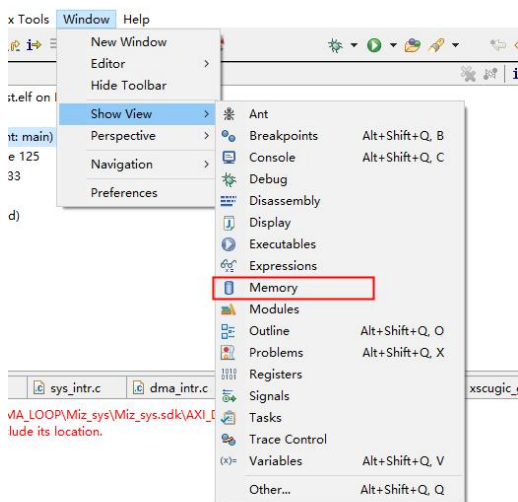
单击如下图片，让在线逻辑分析仪的 2 个窗口都处于等待触发状态



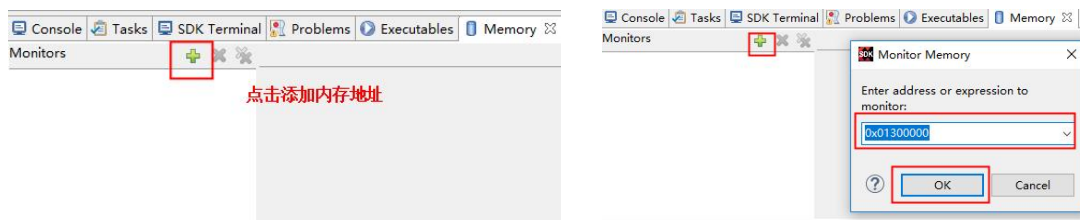
如下图所示是等待触发状态



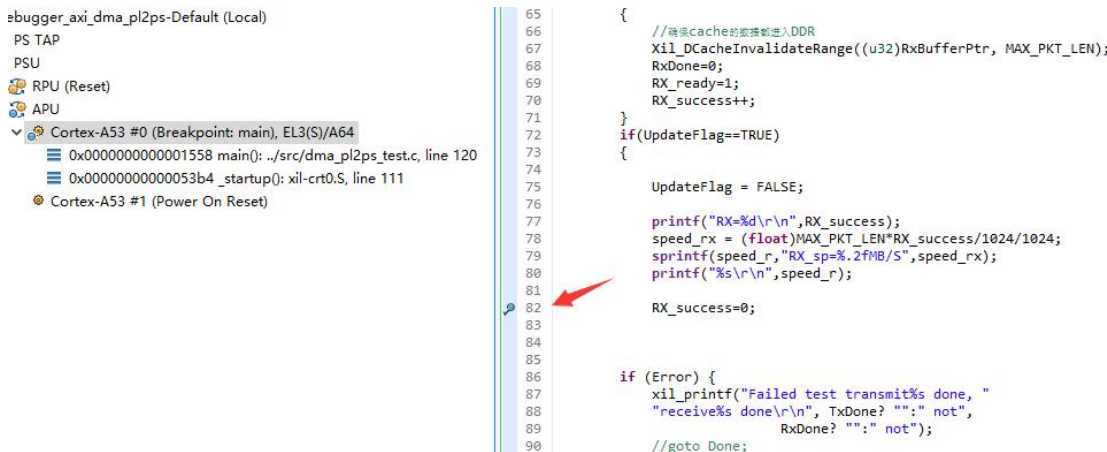
回到 SDK,继续设置,打开 Memory: Window->Show View->Memory



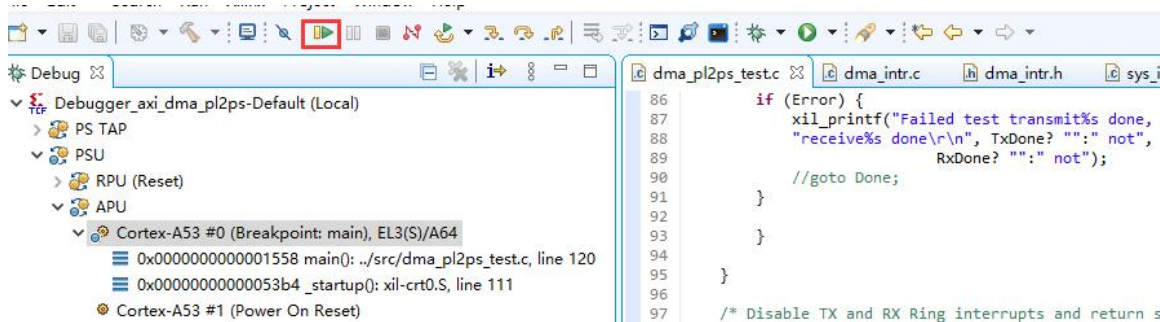
点击添加接收内存部分地址用于观察内存中的数据 地址为 0x01300000



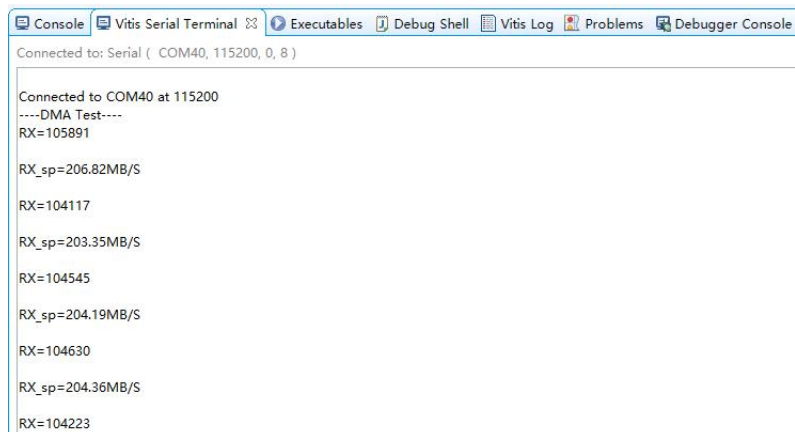
为了观察一次收发数据：设置断点，重新让收发程序跑一次。双击以下程序处可以设置断点。



设置完成后，单击如下可以单击如下红框的按钮



串口打印结果，通过设置 TTC 定时器，每间隔 1S 打印一次计算的测速。实际上也就是计算单位时间内，DMA 了多少次。可以通过增加数据位宽，以及增加 PL 时钟的频率，提高速度。读者可以去尝试下。

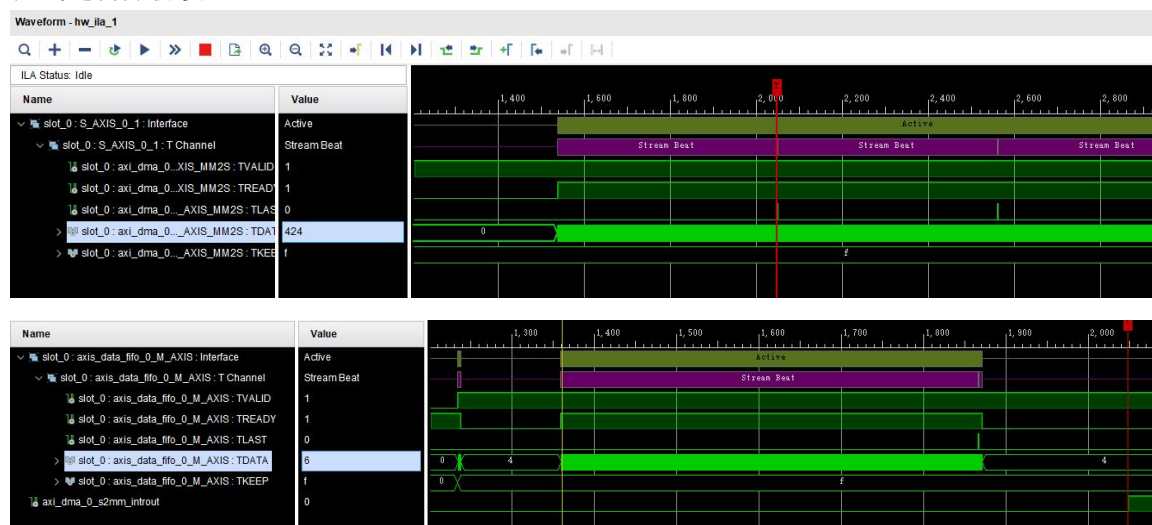


内存中观察到的数据，可以看到地址 0x01300000 地址的开始并不是 00 而是 AB 和我们的预期有些差异，不过数据是非常有规律的，说明 DMA 是正常工作的，但是我们的用户逻辑实际上没有能够确保发送数据的时候第一数据是 00，这个大家可以思考下为什么，有什么办法可以改进。我们后面 lwip 的 demo 就对此问题必须做改进。

Memory 0x01300000 : 0x1300000 <Hex Integer> New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
000000001300000	AEADACAB	B2B1B0AF	B6B5B4B3	BAB9B8B7
000000001300010	BE8DBCB8	C2C1C0BF	C6C5C4C3	CAC9C8C7
000000001300020	CECDCCCB	D2D1D0CF	D6D5D4D3	DAD9D8D7
000000001300030	DEDDDCDB	E2E1E0DF	E6E5E4E3	EAE9E8E7
000000001300040	EEDECECB	F2F1F0EF	F6F5F4F3	FAF9F8F7
000000001300050	FEFDFCFB	020100FF	06050403	0A090807
000000001300060	0E0D0C0B	1211100F	16151413	1A191817
000000001300070	1E1D1C1B	2221201F	26252423	2A292827
000000001300080	2E2D2C2B	3231302F	36353433	3A393837
000000001300090	3E3D3C3B	4241403F	46454443	4A494847
0000000013000A0	4E4D4C4B	5251504F	56555453	5A595857
0000000013000B0	5E5D5C5B	6261605F	66656463	6A696867
0000000013000C0	6E6D6C6B	7271706F	76757473	7A797877
0000000013000D0	7E7D7C7B	8281807F	86858483	8A898887
0000000013000E0	8E8D8C8B	9291908F	96959493	9A999897
0000000013000F0	9E9DBC9B	A2A1A09F	A6A5A4A3	AAA9A8A7

在线逻辑分析仪



附录 1：常见问题

1 联系方式

米联客官方社区: www.uisrc.com

米联客社区 FPGA/SOC QQ 群:

群 1:516869816

群 2:543731097

群 3:86730608

群 4:34215299

技术微信: 18951232035

技术电话: 18951232035

官方微信公众号:



2 售后

1、7 天无理由退货(人为原因除外)

2、质保期限: 本公司产品自快递签收之日起, 提供一年质保服务(主芯片, 比如 FPGA 或者 CPU 等除外)。

3、维修换货, 需提供淘宝订单编号或合同编号, 联系销售/技术支持安排退回事宜。

售后维修请登录工单系统: https://www.uisrc.com/plugin.php?id=x7ree_service

4、以下情形不属于质保范畴。

A: 由于用户使用不当造成板子的损坏: 比如电压过高造成的开发板短路, 自行焊接造成的焊盘脱落、铜线起皮 等

B: 用户日常维护不当造成板子的损坏: 比如放置不当导致线路板腐蚀、基板出现裂纹等

5、质保范畴外(上方第 4 条)及质保期限以外的产品, 本公司提供有偿维修服务。维修仅收取器件材料成本, 往返运费全部由客户承担。

6、寄回地址, 登录网页获取最新的售后地址: <https://www.uisrc.com/t-1982.html>

3 销售

天猫米联客旗舰店: <https://milianke.tmall.com>

京东米联客旗舰店: <https://milianke.jd.com/>

官方淘宝 1 店: <https://milianke.taobao.com>

官方淘宝 2 店: <https://osrc.taobao.com>

销售电话: 18921033576

公司地址: 常州溧阳总部: 常州溧阳市中关村吴潭渡路雅创高科智造谷 10-1 幢楼

南京研发基地: 南京市栖霞区仙林大道 181 号 5 幢 1820 室

4 在线视频

<https://www.uisrc.com/video.html>

5 软件下载

<https://www.uisrc.com/f-download.html>

6 经验分享

<https://www.uisrc.com/f-390.html>

7 官方博文

<https://www.uisrc.com/portal.php?mod=list&catid=35>